

Dissertation: Analyse Patterns im Softwareentwicklungsprozeß

Mit Beispielen für Informationsmanagement und deren
Anwendungen für die Virtuelle Universität der
Wirtschaftsuniversität Wien

Mag. Michael Hahsler
Abteilung für Informationswirtschaft
Wirtschaftsuniversität Wien

24. Jänner 2001

DOKTORAT DER SOZIAL- UND
WIRTSCHAFTSWISSENSCHAFTEN

1. Beurteiler: a.o.Univ.Prof.Dr. Andreas Geyer-Schulz

2. Beurteiler: o.Univ.Prof.Dr. Wolfgang Janko

Eingereicht am: 24. Jänner 2001

Analyse Patterns im Softwareentwicklungsprozeß

Mit Beispielen für Informationsmanagement und deren Anwendungen für
die Virtuellen Universität der Wirtschaftsuniversität Wien

Dissertation

zur Erlangung des akademischen Grades eines

Doktors

der Sozial- und Wirtschaftswissenschaften

an der Wirtschaftsuniversität Wien

eingereicht bei

Erster Beurteiler: a.o.Univ.Prof.Dr. Andreas Geyer-Schulz

Zweiter Beurteiler: o.Univ.Prof.Dr. Wolfgang Janko

Fachgebiet: Informationswirtschaft

von

Mag. Michael Hahsler

Wien, 24. Jänner 2001

Ich versichere:

1. daß ich die Dissertation selbständig verfaßt, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und mich auch sonst keiner unerlaubten Hilfe bedient habe.
2. daß ich diese Dissertation bisher weder im In- noch im Ausland (einer Beurteilerin / einem Beurteiler zur Begutachtung) in irgendeiner Form als Prüfungsarbeit vorgelegt habe.
3. daß dieses Exemplar mit der beurteilten Arbeit übereinstimmt.

24. Jänner 2001

Datum

Michael Hahsler

Unterschrift

Inhaltsverzeichnis

Zusammenfassung	v
Einleitung	1
I Softwareentwicklung mit Analyse Patterns	6
1 Der Software-Lebenszyklus	8
1.1 Vorgehensweisen der Softwareentwicklung	9
1.2 Die Analysephase	13
1.3 Übergang zur Designphase	16
1.4 Qualität von Softwarearchitektur	19
1.5 Refactoring	21
2 Der Pattern-Ansatz	22
2.1 Der Begriff Pattern	22
2.2 Ein Beispiel Pattern: University as a Marketplace	24
3 Design Patterns	27
3.1 Die Anwendung von Patterns in der Designphase	28
3.2 Aufbau von Design Patterns	29
3.3 Kategorisierung von Design Patterns	32
3.4 AntiPatterns	34
3.5 Application Frameworks	36
3.6 Nutzen von Design Patterns	38
4 Analyse Patterns	42
4.1 Literaturüberblick zu Patterns in der Analysephase	43
4.2 Anwendungsbereich von Analyse Patterns	45
4.3 Aufbau von Analyse Patterns	46
4.4 Abgrenzung zu Design Patterns	50
4.5 Nutzen und Wirkung von Analyse Patterns	53

5	Beispiele: Analyse Patterns für Informationsmanagement	58
5.1	Die Einfache Pinwand	60
5.2	Die Pinwand mit strukturierten Nachrichten	65
5.3	Die Virtuelle Bibliothek	70
5.4	Die Virtuelle Bibliothek mit aktiven Agenten	75
6	Zusammenfassung	80

II Eine Case Study zu Analyse Patterns: Die Virtuelle Universität der Wirtschaftsuniversität Wien 83

7	Rahmenbedingungen der Case Study	86
7.1	Informationstechnologie und Universitäten	87
7.2	Auswirkungen der Virtualisierung von Universitäten	88
7.3	Die Initiative an der Wirtschaftsuniversität Wien	89
8	Das Analyse Pattern Virtuelle Bibliothek als Informationsbroker für die Virtuelle Universität	92
8.1	Nutzen eines Brokers auf einem Informationsmarkt	93
8.1.1	Definition von Informationsgütern	93
8.1.2	Netzwerke und Netzwerkeffekte	95
8.1.3	Der Einfluß eines Brokers auf Netzwerkeffekte	98
8.2	Umsetzung des Informationsbrokers	100
8.2.1	Identifikation der Informationsgüter	100
8.2.2	Der Informationsbroker als eine Virtuelle Bibliothek .	104
8.2.3	Design und Implementierung des Informationsbrokers	106
8.3	Konsequenzen der Verwendung der Virtuellen Bibliothek . . .	110
9	Erweiterung des Informationsbrokers durch das Analyse Pattern Virtuelle Bibliothek mit aktiven Agenten	115
9.1	Analyse der Transaktionskosten auf Informationsmärkten .	116
9.1.1	Transaktionskosten	117
9.1.2	Kostenstruktur eines Informationsmarktes	120
9.1.3	Der Einfluß eines Brokers auf Transaktionskosten . . .	120
9.2	Architektur des Informationsbrokers	125
9.2.1	Metadaten Management Ebene	127
9.2.2	Broker Management Ebene	128
9.2.3	Interface Ebene	128
9.3	Agenten der Metadaten Management Ebene	128

9.3.1	Eintragungsagenten	128
9.3.2	Beobachtungsagenten für Informationsgüter	132
9.4	Agenten der Interface und Broker Management Ebene	136
9.4.1	Benutzerbeobachtungsagenten	141
9.4.2	Analyseagenten	143
9.4.3	Empfehlungsagenten	144
9.5	Konsequenzen der Verwendung von aktiven Agenten	146
10	Ergebnisse	150
10.1	Einsparungen bei der Softwareentwicklung durch Analyse- patterns	150
10.1.1	Vorgehensweise	151
10.1.2	Ein zusätzlicher Katalog des Informationsbrokers	154
10.1.3	Eine neue Applikation: Die Digitale Bibliothek	156
10.2	Soziale Aspekte der Verwendung von Analyse Patterns	159
10.3	Zusammenfassung	164
A	Analysen zum Informationsbroker der Virtuellen Univer- sität	165
A.1	Verweildauer von Informationsgütern im Broker	165
A.2	Vermittlungshäufigkeit von Informationsgütern	169
A.3	Analyse der Sessionlängen	176
B	Untersuchung des Code Reuse	181
B.1	Liste der Dateien für die Einrichtung des Katalogs für For- schungsveranstaltungen	182
B.2	Liste der Dateien für die Digitale Bibliothek	184
	Abbildungsverzeichnis	189
	Literaturverzeichnis	192

Diese Arbeit ist allen Personen gewidmet, die mich auf meinem bisherigen Weg ermutigt und unterstützt haben.

Besonderer Dank gilt

Kristina, meinen Eltern und meinem Bruder, die immer an mich geglaubt haben.

Meinen Freunden, für die ich während der Arbeit an dieser Dissertation leider viel zu wenig Zeit hatte.

Und nicht zuletzt Andreas Geyer-Schulz, dem Erstbetreuer dieser Arbeit, der mir über die letzten Jahre durch seine Erfahrung und mit unendlicher Geduld bei meiner wissenschaftlichen Weiterentwicklung half.

Zusammenfassung

Diese Arbeit beschäftigt sich mit *Analyse Patterns*, der Anwendung von Patterns in der Analysephase der Softwareentwicklung. In der Designphase werden Patterns seit einigen Jahren eingesetzt, um Expertenwissen und Wiederverwendbarkeit in den Designprozeß einfließen zu lassen. Es existiert bereits eine Fülle an solchen Design Patterns. Die Analysephase ist ein neuer Anwendungsbereich für Patterns, der bisher in der Literatur noch nicht ausreichend behandelt wurde.

In dieser Arbeit wird die Anwendung des Pattern-Ansatzes in der Analysephase aufgearbeitet und konkretisiert. Analyse Patterns unterstützen den gesamten Softwareentwicklungsprozeß und helfen bekannte Probleme während der Analysephase zu lösen. Dadurch können Zeit und Kosten bei der Entwicklung neuer Softwaresysteme eingespart werden. Die Hauptvorteile von Analyse Patterns sind:

- Analyse Patterns bringen in den gesamten Entwicklungsprozeß Expertenwissen in Form von konkreten Beispielen und abstrakten Modellen ein, und beschleunigen damit den Erstellungsprozeß von Software durch die Möglichkeit der Wiederverwendung von Analysemodellen und den damit verbundenen Designvorschlägen. Somit wird der Software-Lebenszyklus massiv beschleunigt, wodurch weniger Kosten anfallen.
- Analyse Patterns tragen durch ihren starken Fokus auf Erfahrungen mit organisatorischen, wirtschaftlichen und sozialen Konsequenzen der Pattern-Lösung bei, mögliche Probleme in diesen Bereichen bereits in der Analyse Phase zu erkennen und zu berücksichtigen. Beispielsweise können Akzeptanzprobleme eines neuen Softwaresystems in einer Organisation zu Konflikten und Widerständen führen, wodurch die Einführung massiv verzögert werden kann, oder das Projekt sogar ganz scheitert. Werden solche Probleme in den Analyse Patterns angesprochen, können sie frühzeitig erkannt werden, und es können rechtzeitig geeignete Maßnahmen zur Akzeptanzerhöhung geplant und ergriffen werden.

Diese Eigenschaften von Analyse Patterns werden anhand konkreter Beispiele in einer Case Study nachgewiesen. Diese Case Study beschreibt

den Einsatz von in dieser Arbeit entwickelten Analyse Pattern für Informationsmanagement anhand des Projekts *Virtuelle Universität der Wirtschaftsuniversität Wien*, in dem ein Internet-Informationbroker zur Unterstützung von Lehre und Forschung realisiert wird. Die Erfahrungen aus diesem Projekt werden untersucht, und die Auswirkungen der Analyse Patterns auf Wiederverwendung bei der Softwareentwicklung und auf die Akzeptanz des resultierenden Systems werden präsentiert.

The Sheathed Sword

To fight and conquer in all your battles is not supreme excellence; supreme excellence consists in breaking the enemy's resistance without fighting. In practical art of war, the best thing of all is to take the enemy's country whole and intact; to shatter and destroy it is not so good. So, too, it is better to capture an army entire than to destroy it, to capture a regiment, a detachment, or a company entire than to destroy them.

Thus the highest form of generalship is to balk the enemy's plans; the next best is to prevent the junction of the enemy's forces; the next in order is to attack the enemy's army in the field; and the worst policy of all is to besiege walled cities, because the preparation of mantlets, movable shelters, and various implementations of war will take up three whole months; and piling up of mounds over against the wall will take three months more.

*A Pattern by Sun Tzu
The Art of War, 500 B.C.*

Einleitung

Motivation für diese Arbeit. Wiederverwendung (*Reuse*) ist eines der wichtigsten Konzepte zur Verkürzung der benötigten Entwicklungszeit von Software. Schnelle Entwicklung von vermarktbaren Produkten (Reduktion von *Time to Market*) ist gerade in der schnelllebigen Softwarebranche eines der ausschlaggebenden Kriterien für Erfolg. Weiters trägt Wiederverwendung zur Verbesserung der Softwarequalität bei, da mit jedem Einsatz die wiederverwendeten Komponenten aufs neue geprüft werden.

Patterns, eine Technik zur Wiederverwendung von erfolgreichen Lösungen, wurden durch den Architekturprofessor Christopher Alexander bekannt [Alexander, 1979]. Vereinfacht ausgedrückt beschreiben in Patterns Experten regelmäßig auftretende Probleme, geben Lösungsvorschläge und diskutieren deren Vor- und Nachteile. Durch das schriftliche Festhalten von Expertenwissen und Erfahrungen in Form von Patterns können diese leichter gesammelt, weitergegeben und verglichen werden.

Patterns werden in der Designphase seit einigen Jahren eingesetzt, um Expertenwissen und Wiederverwendbarkeit in den Designprozeß einfließen zu lassen. Es wurden bereits eine Fülle solcher Design Patterns niedergeschrieben. Die bekanntesten Patternsammlungen sind die Bücher von Gamma et al. [Gamma et al., 1993], Buschmann und Meunier [Buschmann und Meunier, 1995] und die Serie *Pattern Languages of Program Design* [Coplien und Schmidt, 1995] [Vlissides et al., 1996] [Martin et al., 1998] [Harrison et al., 2000].

Auf der Analyseebene wurden von mehreren Autoren (zum Beispiel [Fowler, 1997] [Hay, 1996] [Coad et al., 1997] [Fernandez, 1998, Fernandez und Yuan, 1999] [Gaertner und Thirion, 1999]) einzelne Patterns vorgestellt, jedoch wurden *Analyse Patterns* als eigener Ansatz in der Literatur bisher noch nicht ausreichend untersucht und beschrieben. Folgende Fragen sind offen:

- Wo und wie können Analyse Patterns im Software-Lebenszyklus eingesetzt werden?

- Wie sollen Analyse Patterns im Vergleich zu Design Patterns aufgebaut sein?
- Wie unterscheiden sich Analyse Patterns von Design Patterns?
- Welchen Nutzen bringt der Einsatz von Analyse Patterns für die Softwareentwicklung?

Diese Fragen sollen in der folgenden Arbeit untersucht werden.

Beitrag dieser Arbeit. In dieser Arbeit werden die offenen Fragen zum Einsatz von Analyse Patterns untersucht und beantwortet. Es wird für Analyse Patterns der Anwendungsbereich identifiziert, ein geeignetes Formular zur Beschreibung entwickelt und ihr Unterschied zu Design Patterns geklärt. Dabei wird gezeigt, daß Analyse Patterns folgende zwei Hauptvorteile bieten:

1. Analyse Patterns unterstützen die Wiederverwendung von Expertenwissen im gesamten Softwareentwicklungsprozeß, somit können flexiblere Lösungen in einer kürzeren Zeit realisiert werden. Dies führt erstens zu einer bedeutenden Kostenreduktion für die Softwareentwicklung, und zweitens kann auch die Marktposition des Entwicklers durch eine schnellere Entwicklung (kürzere *Time to Market*) verbessert werden.
2. Analyse Patterns helfen, die sozialen Konsequenzen, die durch Entscheidungen in der Analysephase entstehen, besser zu berücksichtigen. Durch die Beschreibung in Analyse Patterns können Probleme, die auf negativen sozialen Konsequenzen für Betroffene beruhen, leichter frühzeitig erkannt und berücksichtigt werden. Dies führt zu einer höheren Akzeptanz des entwickelten Softwareprodukts, und damit zu weniger Widerstand durch die Betroffenen bei der Einführung. Dies bedeutet wiederum, daß das Produkt schneller und damit kostengünstiger eingeführt werden kann.

Diese zwei Hauptvorteile werden in der Arbeit nachgewiesen. Dazu werden vier neue Analyse Patterns für das Informationsmanagement in Gruppen entwickelt und in einer Case Study zur Realisierung des Internet-Informationsbroker des Projekts *Virtuelle Universität der Wirtschaftsuniversität Wien* (<http://vu.wu-wien.ac.at>) eingesetzt. Anhand dieser Case Study wird der Nutzen der Analyse Patterns in den Bereichen, Einsparungen aufgrund von Wiederverwendung, sowie Akzeptanz durch die Berücksichtigung von sozialen Konsequenzen, nachgewiesen.

Diese Arbeit dient als Basis für die Weiterentwicklung von Analyse Patterns als eigenständige Anwendung von Patterns, die die Möglichkeit bietet, die Softwareentwicklung durch Wiederverwendung in Analyse, Design und Implementierung zu beschleunigen, und wirtschaftliche, organisatorische und soziale Probleme bereits während der Analyse zu erkennen und zu berücksichtigen.

Aufbau der Arbeit. In Teil I dieser Arbeit wird die Verwendung von Analyse Patterns untersucht, und es werden Beispiele für den Bereich Informationsmanagement vorgestellt.

In Kapitel 1 werden die Grundlagen des Software-Lebenszykluses zusammengefaßt, und es werden Probleme im Bereich der Analyse identifiziert.

In den Kapiteln 2 und 3 werden der Pattern-Ansatz und seine Anwendung für die Designphase des Softwareentwicklungsprozesses beschrieben.

In Kapitel 4 wird der Einsatz von Patterns für die Analysephase mit Hilfe der bisher beschriebenen Erkenntnisse und unter Berücksichtigung der bereits existierenden Literatur zu Analyse Patterns entwickelt, und es wird aufgezeigt, wie durch diese Patterns Probleme während der Analysephase behoben werden können:

- Der Anwendungsbereich von Analyse Patterns wird beschrieben. Dieser Anwendungsbereich geht weit über die reine Analysephase hinaus, und umfaßt auch den Abstraktionsprozeß des realen Problems, sowie die Transformation des Analysemodells in ein Designmodell.
- Eine strukturierte Beschreibung für Analyse Patterns wird entwickelt, und es wird eine Abgrenzung zu Design Patterns gegeben. Patterns müssen schriftlich festgehalten werden. Die in dieser Arbeit für Analyse Patterns entwickelte strukturierte Beschreibungsform basiert auf der Struktur von bekannten Formularen für Design Patterns. Im Unterschied zu diesen hat sich jedoch der Inhalt beträchtlich verändert. Die Hauptveränderungen sind der starke wirtschaftliche, organisatorische und soziale Bezug im Abschnitt für Konsequenzen und ein eigener Abschnitt für Designvorschläge, der zu bereits ausgearbeiteten Design Patterns überleitet und damit auch die Designphase wesentlich verkürzt.
- Die Verwendung und der potentielle Nutzen von Analyse Patterns wird untersucht. Es wird gezeigt, wie die einzelnen Abschnitte der Analyse Patterns eingesetzt werden, um die Vorgänge, von der Analyse des realen Problems bis hin zur Transformation des Analysemodells in ein flexibles Design, zu unterstützen.

In Kapitel 5 werden Beispiele für die in Kapitel 4 beschriebenen Analyse Patterns gegeben. Dazu werden vier aufeinander aufbauende Analyse Patterns für den Bereich Informationsmanagement entwickelt:

1. Das Analyse Pattern *Einfache Pinwand* ermöglicht die Speicherung von Nachrichten für asynchrone Kommunikation innerhalb von kleinen Gruppen. Es zeigt, wie Informationen in einer Gruppe effizient verarbeitet werden können, und wie sich daraus eine gemeinsame Wissensbasis aufbaut.
2. Das Analyse Pattern *Pinwand mit strukturierten Nachrichten* erweitert die Nachrichten der einfachen Pinwand um eine Struktur. Dies erfolgt zum Beispiel durch ein eigenes Feld für den Betreff, Absender oder eine Kategorisierung. Da Nachrichten mit Struktur übersichtlicher dargestellt werden können als freier Text, werden mit diesem Pattern Probleme durch steigende Gruppengröße und dem daraus resultierenden höheren Kommunikationsbedarf reduziert.
3. Das Analyse Pattern *Virtuelle Bibliothek* dient dazu, einen einheitlichen und effizienten Zugang zu verteilten Informationsobjekten mit zentraler Suchmöglichkeit zu bieten. Dazu wird eine Pinwand mit strukturierten Nachrichten eingesetzt, wobei die Felder zur Speicherung von Metadaten verwendet werden, und ein spezielles Feld einen Verweis auf das eigentliche Informationsobjekt beinhaltet. Die Verwendung von verteilter Information hat weitreichende organisatorische und soziale Konsequenzen in den Bereichen Besitz und Kontrolle von Information.
4. Das Analyse Pattern *Virtuelle Bibliothek mit aktiven Agenten* erweitert die virtuelle Bibliothek um aktive Komponenten, die Transaktionskosten bei der Sammlung, Klassifikation, Verwaltung, Suche und Verwendung von verteilten Informationsobjekten reduzieren. Dies hat wirtschaftliche und soziale Konsequenzen, die sich stark auf die Akzeptanz des entstehenden Systems auswirken können.

Abgeschlossen wird Teil I mit einer Zusammenfassung der Erkenntnisse in Kapitel 6.

In Teil II dieser Arbeit werden die präsentierten Analyse Patterns für eine Case Study eingesetzt, um die Vorteile der Verwendung von Analyse Patterns nachzuweisen. Dazu wird folgendermaßen vorgegangen:

In Kapitel 7 werden die Rahmenbedingungen für die Case Study, das Projekt *Virtuelle Universität der Wirtschaftsuniversität Wien* (VU-Projekt), beschrieben. Es werden die Notwendigkeit des Projekts, sowie soziale Probleme, die mit so einem Projekt verbunden sind, untersucht. Und es wird

gezeigt, wie ein Informationsbroker als Informationsplattform für das Projekt eingesetzt werden kann.

In Kapitel 8 wird das Analyse Pattern *Virtuelle Bibliothek* für den Informationsbroker des VU-Projekts angewandt. Dazu wird zuerst der Nutzen eines Informationsbrokers unter Berücksichtigung von Netzwerkeffekten auf einem Informationsmarkt analysiert. Danach wird die Umsetzung des Informationsbrokers für das Projekt beschrieben. Abgeschlossen wird das Kapitel mit einer Untersuchung der Konsequenzen durch den Einsatz des Analyse Patterns. Dabei zeigt sich, daß das Pattern einen starken positiven Einfluß auf die Akzeptanz des Informationsbrokers ausübt.

In Kapitel 9 wird der Informationsbroker um aktive Agenten des Patterns *Virtuelle Bibliothek mit aktiven Agenten* erweitert. Dazu werden zuerst Transaktionskosten auf Informationsmärkten analysiert. Mit diesen Erkenntnissen wird eine Architektur für den Informationsbroker entwickelt, und es werden einige dafür verwendete Agenten beschrieben. Abgeschlossen wird das Kapitel mit einer Untersuchung der Kostenreduktionen durch den Einsatz der aktiven Agenten.

Abgeschlossen wird die Arbeit mit der Untersuchung der in der Case Study gefundenen Auswirkungen der Verwendung von Analyse Patterns in Kapitel 10. Dabei zeigt sich, daß Analyse Patterns eine schnellere Entwicklung durch Wiederverwendung in allen Phasen bewirken, und daß dabei höhere Akzeptanz des Systems durch Berücksichtigung von sozialen Konsequenzen entsteht.

Teil I

Softwareentwicklung mit Analyse Patterns

In diesem Teil der Arbeit wird die Verwendung von Patterns in der Analysephase untersucht. Da Analyse Patterns als eigenständiger Ansatz in der Literatur bisher noch nicht ausreichend untersucht wurden, wird ihre Verwendung anhand von Erkenntnissen aus dem Lebenszyklus der Softwareentwicklung, dem Pattern-Ansatz und von Design Patterns betrachtet. Damit wird versucht, ein Gesamtbild der Verwendung von Patterns in der Softwareentwicklung (Analyse Patterns und Design Patterns) darzustellen.

Im Kapitel 1 werden die für diese Arbeit benötigten Grundlagen des Lebenszyklus der Softwareentwicklung zusammengefaßt. Besonders wird auf Objekt-Orientierung, die konzeptionelle Trennung zwischen Analyse- und Designphase sowie auf Problembereiche in diesen Phasen eingegangen.

Danach wird in Kapitel 2 der Pattern-Ansatz, eine Möglichkeit Expertenwissen wiederzuverwenden, dargestellt. Dies ist notwendig, da der Pattern-Ansatz die Grundlage für Design Patterns und auch von Analyse Patterns darstellt.

In Kapitel 3 wird Literatur zu Design Patterns und ähnliche Ansätze, wie *AntiPatterns* und *Frameworks*, zusammengefaßt und ihr Nutzen für die Softwareentwicklung beschrieben.

Mit diesen Grundlagen und einem kurzen Überblick über die existierende Literatur zu *Analyse Patterns*, wird in Kapitel 4 der Einsatz von Patterns in der Analysephase dargestellt. Dazu wird der Anwendungsbereich von Analyse Patterns in der Softwareentwicklung identifiziert, eine strukturierte Beschreibungsform entwickelt und der Unterschied zu Design Patterns und anderen Ansätzen aufgezeigt. Abgeschlossen wird das Kapitel mit einer Analyse des Nutzens von Patterns in der Analysephase.

In Kapitel 5 werden eine Reihe von Analyse Patterns für Informationsmanagement entwickelt. Diese Patterns dienen zur Illustration des in Kapitel 4 entwickelten Ansatzes und werden in Teil II der Arbeit zur Realisierung eines Informationssystems in Form einer Case Study verwendet.

Kapitel 6 enthält eine kurze Zusammenfassung dieses Teils der Arbeit.

Kapitel 1

Der Software-Lebenszyklus

In diesem Kapitel werden kurz die benötigten Grundlagen der Softwareentwicklung zusammengefaßt. Diese Grundlagen werden für das Verständnis des Pattern-Ansatzes benötigt und sollen weiters Problembereiche der Softwareentwicklung aufzeigen.

In Abschnitt 1.1 werden einige der bekanntesten Vorgehensweisen zur Softwareentwicklung vorgestellt. Eingegangen wird unter anderem auf das Wasserfallmodell, den objekt-orientierten Lebenszyklus nach Booch und auf den erst vor kurzem vorgestellten *Unified Development Process*, der zur Entwicklung von objekt-orientierten Systemen in Zusammenhang mit der weit verbreiteten Notation UML (*Unified Modeling Language*) verwendet werden kann.

In den Abschnitten 1.2 und 1.3 werden die Analyse- und die Designphase im Entwicklungsprozeß genauer betrachtet. Dabei wird besonders auf Probleme, wie die Transformation des objekt-orientierten Analysemodells in ein Designmodell, eingegangen. Da sich diese Arbeit mit Analyse Patterns beschäftigt, werden die restlichen Phasen (Implementierung, Wartung, ...) nicht weiter dargestellt.

Im Abschnitt 1.4 werden Qualitätskriterien für Softwarearchitektur beschrieben. Besonders wird dabei auf nicht-funktionale Eigenschaften (Robustheit, Anpaßbarkeit, Wiederverwendbarkeit, ...) von Software eingegangen, da diese die langfristigen Gesamtkosten einer Softwareentwicklung bestimmen und für die Untersuchung des Nutzens von Analyse Patterns wichtig sind.

Zum Abschluß des Kapitels wird in Abschnitt 1.5 noch *Refactoring*, ein Ansatz zur Verbesserung der nicht-funktionalen Eigenschaften bereits existierender Systeme, vorgestellt. Dieser Ansatz wird erwähnt, da er oft in Zusammenhang mit Design Patterns vorkommt, und auch für Analyse Patterns von Bedeutung ist.

1.1 Vorgehensweisen der Softwareentwicklung

Der gängige Ansatz, um ein großes Softwareentwicklungsprojekt zu planen und zu steuern, ist die Zerlegung des Gesamtprojekts in mehrere Teilschritte, die nacheinander durchlaufen werden. Der dadurch entstehende Ablauf wird Wasserfallmodell [Royce, 1970] genannt.

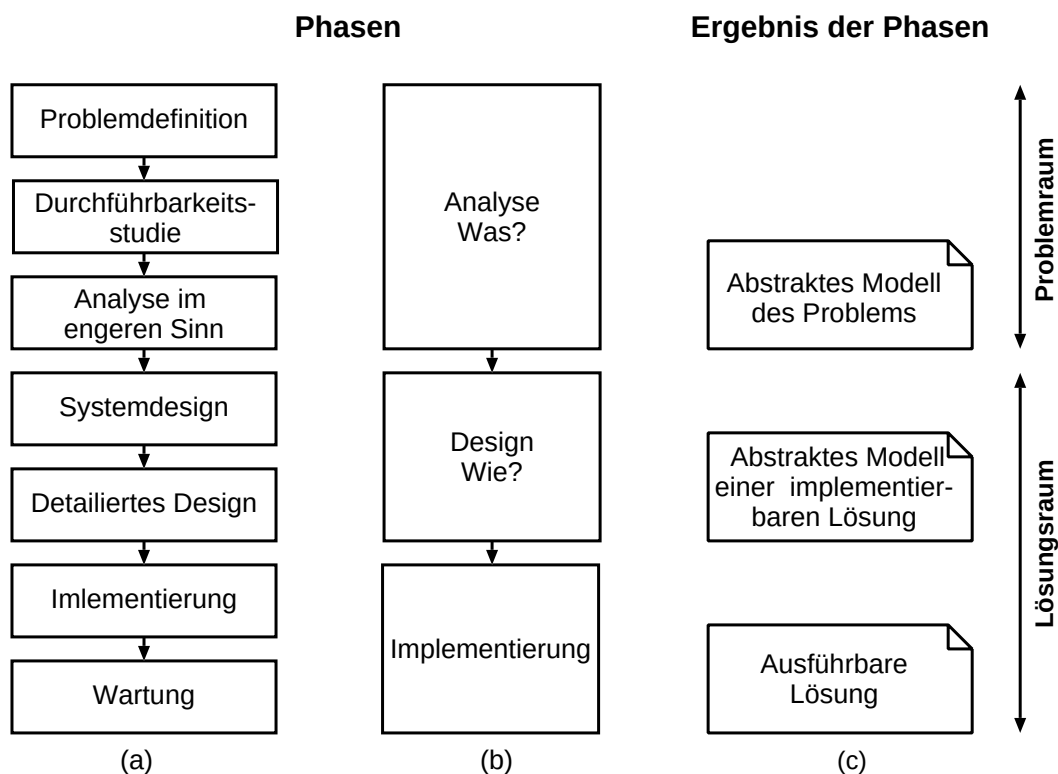


Abbildung 1.1: Traditioneller Lebenszyklus: (a) nach Davis; (b) grobe Einteilung in Analyse, Design und Implementierung; (c) Ergebnisse der Phasen

Die Schritte oder Phasen des modernen Software-Lebenszyklus sind in Abbildung 1.1 dargestellt. Neben den Einzelschritten nach Davis [Davis, 1983, S.8–18] ist auch eine grobe Einteilung in Analyse, Design und Implementieren des Systems, sowie deren Ergebnisse (siehe zum Beispiel [Peters, 1981, S.14]) in Abbildung 1.1 dargestellt (von links nach rechts). Die konzeptionelle Trennung zwischen Analyse und Design ist sehr wichtig. Durch die Analyse werden die zu lösenden Probleme erkannt und beschrieben. Die Frage, die während der Analyse beantwortet wird, ist: Was muß das zu erstellende System leisten? Ergebnis der Analyse ist

ein abstraktes Modell aller für die Lösung relevanten Aspekte im Problemraum. Einschränkungen der Umgebung (wie zum Beispiel die Einbindung in bestehende Systeme oder organisatorische Probleme) sind nicht einbezogen, sie werden erst während des Designs betrachtet. In der Designphase wird auf folgende Frage eingegangen: Wie kann das System die geforderte Leistung erbringen? Aus dem abstrakten Modell im Problemraum wird ein abstraktes Modell der umsetzbaren Lösung entwickelt, das später in Form der ausführbaren Lösung implementiert werden kann.

Für jede Phase ist ihr Platz im Zeitrahmen des Gesamtprojekts sowie ihr genauer Zweck bestimmt. Um von einer Phase in die nächste wechseln zu können, müssen vordefinierte Kriterien (*Milestones*) erfüllt sein. Diese Kriterien stellen sicher, daß die nötigen Voraussetzungen für alle folgenden Phasen geschaffen wurden. Die Sequenzialisierung der Phasen bedeutet aber nicht, daß nach der Beendigung einer Phase keine Möglichkeit mehr zur Veränderung der Ergebnisse dieser Phase besteht. Manche Autoren sehen den ganzen Prozeß vielmehr als einen iterativen Ablauf, wobei Ergebnisse aus späteren Phasen als Feedback für frühere Phasen dienen. Als problematisch ist hierbei jedoch der Kostenaspekt zu betrachten. Beispielsweise führen nachträgliche Änderungen oder Erweiterungen der Ergebnisse der Analysephase zu viel höheren Kosten, als dies bei sofortiger Berücksichtigung der Fall gewesen wäre. Der Grund dafür ist, daß eine nachträgliche Änderung in einer früheren Phase eine Lawine von notwendigen Änderungen in den Folgephasen auslösen kann. Die Notwendigkeit der Rückwirkung von Design- und Implementierungsphase auf die Analyse führte zu anderen Formen von Lebenszyklusmodellen, wie beispielsweise dem Spiralmodell [Boehm, 1988] oder dem Prototyping (siehe [Budde *et al.*, 1992]).

Der objekt-orientierte Lebenszyklus unterscheidet sich vom klassischen Lebenszyklus durch noch massivere Rückkopplungen zwischen den einzelnen Phasen. Beispielsweise werden im Springbrunnenmodell [Henderson-Sellers und Edwards, 1990] Teile der Phasen sogar überlappend dargestellt, um diesen Aspekt besonders zu betonen. Dies bedeutet aber nicht, daß die Trennung zwischen Analyse und Design dadurch aufgehoben wird. Es wird lediglich schwieriger, die Trennung zu erkennen. Die Analyse befaßt sich weiterhin mit der Erkennung und der Beschreibung der relevanten Aspekte im Problemraum (*Problem Domain*) und das Design mit dem Auffinden einer umsetzbaren Lösung im Lösungsraum (*Solution Domain*).

In Abbildung 1.2 ist der objekt-orientierte Lebenszyklus nach Booch [Booch, 1994] dargestellt. Neben der Trennung zwischen Analyse und Design gibt es noch die Phasen der Evolution und der Modifikation. Durch diese Phasen werden explizit die Veränderungen des Systems nach der

Designphase in den Lebenszyklus einbezogen. Die Evolutionsphase beinhaltet Implementierung, Tests und Integration des klassischen Software Lebenszyklus. Durch die starke Rückkopplung, angedeutet durch Pfeile zurück zur Design- und Analysephase, entsteht ein iterativer Entwicklungsprozeß, der mit der endgültigen Implementierung endet (*Evolutionary Prototyping*). Die Modifikationsphase ist gekennzeichnet durch Erweiterung und Anpassung des laufenden Systems, und geht damit viel weiter als die Wartung aus dem klassischen Lebenszyklus.

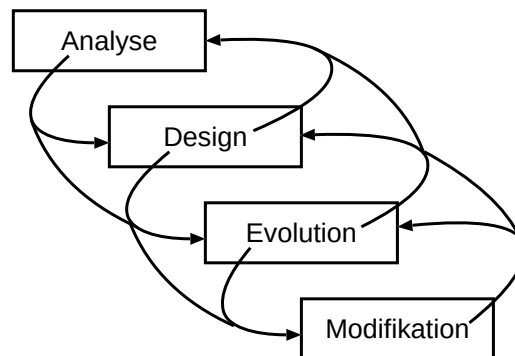


Abbildung 1.2: Objekt-orientierter Lebenszyklus nach Booch

Eine Übersicht und ein Vergleich von verschiedenen objekt-orientierten Analyse- und Designverfahren und der verwendeten Notationen findet man bei Monarchi und Puhr [Monarchi und Puhr, 1992]. Der Artikel geht sowohl auf rein objekt-orientierte Ansätze, wie dem gerade beschriebenen Ansatz von Booch, als auch auf Varianten, die traditionelle mit objekt-orientierten Techniken mischen, ein. Die Autoren analysieren die Vollständigkeit der Ansätze in den Bereichen Unterstützung von objekt-orientierter Analyse, objekt-orientiertem Design, Notation und Komplexitätsmanagement. Die drei Ansätze, die mit Abstand in den meisten Bereichen Unterstützung bieten, stammen von Booch [Booch, 1994], Coad und Yourdon [Coad und Yourdon, 1991], und Rumbaugh et al. [Rumbaugh *et al.*, 1991]. Einen weiteren, sehr anwendungsbezogenen Überblick über die gängigsten Methoden bieten Tkach und Puttick [Tkach und Puttick, 1994, S.25–50].

Die neueste Entwicklung im Bereich objekt-orientierter Softwareentwicklung ist der Einsatz der *Unified Modeling Language (UML)* [OMG, 1999] der *Object Management Group* als grafische Notation für die Modellierung von Systemen. 1997 haben diese Notation Ivar Jacobson, Grady Booch und James Rumbaugh durch die Vereinigung verschiedener Diagrammtypen entwickelt. Die Diagrammtypen stammen aus den Entwicklungsmethoden: *Object-Oriented Design (OOD)* [Booch, 1994], *Object-Oriented*

Software Engineering (OOSE) [Jacobson, 1992] und *Object-Modeling Technique (OMT)* [Rumbaugh *et al.*, 1991]. Jeder Diagrammtyp beschreibt jeweils bestimmte Aspekte des Systems. Beispielsweise werden in UML die Anforderungen an das System mittels Use Case-Diagrammen und den dazugehörigen Szenarios dargestellt. Klassendiagramme (*Class Diagrams*) werden zur Notation der statischen Sicht des Systems verwendet. Die dynamischen Aspekte werden durch Sequenzdiagramme (*Sequence Diagrams*) beziehungsweise durch Kollaborationsdiagramme (*Collaboration Diagrams*) dargestellt. Das daraus entstehende Gesamtmodell des Systems kann auf einer höheren Abstraktionsebene in der Analysephase verwendet werden und später durch schrittweise Verfeinerung in ein Designmodell umgesetzt werden. UML selbst ist nur eine integrierte Sammlung von Notationen und besitzt kein eigenes Vorgehensmodell zur Softwareentwicklung. Erst Ende 1999 wurde von den Entwicklern von UML ein eigener Entwicklungsprozeß mit dem Namen *Unified Software Development Process* [Jacobson *et al.*, 1999a, Jacobson *et al.*, 1999b] vorgestellt. Dieses Vorgehensmodell basiert auf drei Säulen:

1. Verwendung von Use Cases, um die funktionalen Anforderungen des Systems zu erfassen.
2. Konzentration auf die Architektur als Bauplan des Systems.
3. Iterationen und schrittweise Implementierung des Projekts in mehreren Stufen (*Releases*).

Ein Projekt durchläuft in diesem Entwicklungsprozeß folgende Phasen, wobei jede Phase aus mehreren Iterationen besteht:

1. **Interception.** Eine gute Idee wird zu einer Vision des fertigen Systems entwickelt. Schwerpunkt liegt auf der Analyse der Anforderungen mit Use Cases.
2. **Elaboration.** Detaillierung der Use Cases und Design der Architektur. Die wichtigsten Use Cases werden bereits implementiert.
3. **Construction.** Das gesamte System wird implementiert. Es kommt nur mehr zu geringfügigen Änderungen der Architektur.
4. **Transition.** Das System wird durch eine kleine Anzahl von erfahrenen Benutzern geprüft. Probleme werden behoben. Das Ergebnis ist das fertige System.

Der Übergang von Analyse zu Design findet in diesem Vorgehensmodell in der Phase der Elaboration statt. Der Übergang ist nicht abrupt, wie bei

anderen Modellen, sondern ist durch ein langsames Fertigstellen der Analyse und einem ebenso langsamen Anlaufen des Designs gekennzeichnet. Dies darf aber auf keinen Fall als Vermischung von Analyse und Design gedeutet werden, da die konzeptionelle Trennung weiterhin besteht.

Da sich diese Arbeit mit dem Einsatz von Patterns in der Analyse- und der Designphase des Software-Lebenszyklus beschäftigt, werden im folgenden diese Phasen genauer beschrieben.

1.2 Die Analysephase

Die am weitesten verbreitete Analysemethode ist die *Strukturierte Analyse* [DeMarco, 1979, Yourdon, 1989]. Ihr erster Vertreter, Tom DeMarco, definiert die Aufgabe der Analyse wie folgt [DeMarco, 1979, S.4]:

Analysis is the study of a problem, prior to taking some action. In the specific domain of computer systems development, analysis refers to the study of some business area or application, usually leading to the specification of a new system.

James Rumbaugh und seine Mitautoren charakterisieren objekt-orientierte Analyse mit [Rumbaugh *et al.*, 1991, S.148]:

The purpose of object-oriented analysis is to model the real-world system so that it can be understood. The successful analysis model states what must be done, without restricting how it is done, and avoids implementation decisions.

Aus diesen Aussagen lassen sich zwei wichtige Punkte für die Analyse erkennen. Erstens muß in der Analysephase das Problem genau untersucht werden, um die notwendigen Kenntnisse über das Problemumfeld (*Domain Knowledge*) zu erwerben. Und zweitens ist das Ergebnis der Analyse das Analysemodell, welches eine Abstraktion des realen Problems darstellt. Dieses Modell ist die Spezifikation des geplanten Systems, in dem keine Einschränkungen über die Art der Implementierung enthalten sind.

Um die Analysephase erfolgreich abzuschließen, müssen nach DeMarco durch den Systemanalytiker folgende Ergebnisse erzielt werden [DeMarco, 1979, S.4]:

1. Optimales Ziel wählen.
2. Detaillierte Spezifikation des Ziels, um die Implementierung auf die Erreichung des Ziels überprüfen zu können.

3. Genaue Schätzung der wichtigen Parameter, die mit dem Ziel verbunden sind. Beispielsweise Kosten, Nutzen, Zeitpläne und Leistungsmerkmale.
4. Erreichung einer Übereinstimmung über die ersten drei Punkte bei allen Beteiligten.

Die gerade beschriebenen Ergebnisse werden in der Regel durch Benutzerbefragung (*User Interviews*, *User Requirement Analysis*), Problemdefinition (*Problem Statement*, *User Requirement Specification*), Kosten-Nutzen Analyse (*Cost-Benefit Analysis*) und Durchführbarkeitsstudien (*Feasibility Studies*), sowie dem Analysemodell erreicht (siehe [DeMarco, 1979, S.4–9], [Davis, 1983], [Rumbaugh *et al.*, 1991, Kap.8]).

Die Erstellung des objekt-orientierten Analysemodells nach Rumbaugh (OMT) [Rumbaugh *et al.*, 1991, Kap.8] besteht aus mehreren Schritten, die parallel betrachtet werden müssen. Am Anfang steht die Identifikation der benötigten Objekte, ihrer Eigenschaften und ihrer Beziehung zu anderen Objekten. Die daraus entstehende statische Struktur des Systems wird im Objektmodell (*Object Model*) festgehalten. Weiters werden die Interaktionen zwischen den einzelnen Objekten betrachtet und im dynamischen Modell (*Dynamic Model*) abgebildet. Das funktionale Modell (*Functional Model*) zeigt den Fluß von Information durch das System, ohne auf den genauen zeitlichen Ablauf einzugehen. Jedes dieser Modelle beschreibt eine andere Sicht auf des Systems. Gemeinsam ergeben die Modelle die problemgetriebene Spezifikation des geplanten Systems.

Jacobson (OOSE) [Jacobson, 1992] sieht die Aufgabe der Analyse in der Erarbeitung von zwei Modellen, dem Modell der Anforderungen (*Requirements Model*) und dem eigentlichen Analysemodell (*Analysis Model*). Die Anforderungen werden in *Use Cases* festgehalten. Sie bilden einerseits die Grundlage für die Erstellung des Systems und andererseits werden sie für die Durchführung von Tests nach der Fertigstellung verwendet. Das Analysemodell spezifiziert logische Objekte und ihre Beziehungen zueinander. Es ist die Basis für die Struktur des Systems. Das Anforderungsmodell und das Analysemodell gemeinsam bilden den Ausgangspunkt für das Design und die Implementierung des Systems.

Von besonderem Interesse sind noch mögliche Schwierigkeiten und Problembereiche innerhalb der Analysephase, da diese den Softwareentwicklungsprozeß massiv verzögern können, und dadurch hohe zusätzliche Kosten verursacht werden. Eine Schwierigkeit in der Analysephase ist die Abstraktion des realen Problems bei der Modellbildung. Die Bildung eines Modells ist ein komplizierter Vorgang, der sich nicht durch einfache Regeln formalisieren läßt. Deshalb beschreibt Grady Booch [Booch, 1994,

Kap.1] die Softwareentwicklung als Ingenieurwissenschaft, die eine Mischung aus Wissenschaft und Kunst darstellt. Besonders die Abstraktion von komplexen Systemen ist dabei als künstlerischer Schaffensprozeß zu verstehen, der eine Verbindung aus Vorstellungskraft, Erfahrung und Wissen voraussetzt.

DeMarco identifiziert folgende weitere Problembereiche in der Analysephase [DeMarco, 1979, S.9–13]:

Kommunikation. Für die Durchführung eines Softwareprojekts ist zuerst eine gemeinsame Sprache notwendig. Die Benutzer des zukünftigen Systems verwenden die Fachsprache, die für ihren Berufsstand üblich ist. Programmierer haben ihre eigene Fachsprache. Daher kann es sehr leicht zu Mißverständnissen kommen. Die gleichen Worte können von beiden Gruppen für unterschiedliche Konzepte verwendet werden, wodurch die Kommunikation zwischen den Gruppen erheblich erschwert wird. Dieses Problem nennt Davis [Davis, 1983, S.5] *Communication Gap*. Die Aufgabe des Systemanalytikers ist es, die Anforderungen der Benutzer in technische Spezifikationen für die Programmierer zu übersetzen.

Weiters schlägt DeMarco die Verwendung einer einheitlichen Notation für Analyse und Design vor. Als einheitliche grafische Notation für objekt-orientierte Analyse und Design bietet sich die *Unified Modeling Language (UML)* [OMG, 1999] an. Durch die Verwendung einer grafischen Notation wird einem weiteren Kommunikationsproblem, nämlich dem hohen Aufwand bei der verbalen Beschreibung von Abläufen eines noch nicht existierenden Systems, Rechnung getragen.

Sich ständig ändernde Anforderungen an das System. Während der Analysephase muß sich der Systemanalytiker in die Welt der Benutzer einarbeiten und deren Fachsprache erlernen. Dieser Lernprozeß führt zwangsläufig am Anfang bei der Erhebung der Anforderungen zu Kommunikationsfehlern, die später korrigiert werden müssen. Weiters besteht die Möglichkeit, daß sich während der Projektlaufzeit Änderungen der Ziele oder Änderungen der Umwelt ergeben, die berücksichtigt werden müssen. Bei größeren Projekten mit langen Laufzeiten sind solche Änderungen sehr wahrscheinlich und die Möglichkeit zu Anpassungen muß explizit während der Analyse vorgesehen werden.

Dokumentation des Ziels. Die Spezifikation für große und komplexe Systeme muß in eine Vielzahl kleinerer Spezifikationen zerlegt werden. Findet keine Zerlegung statt oder sind die Einzelteile noch immer

zu umfangreich, werden die Spezifikationen unlesbar und unhandhabbar. Bei der Zerlegung des Systems in Subsysteme muß auf die genaue Abgrenzung der Funktionen, beziehungsweise Zuständigkeiten der Teilsysteme, sowie auf die exakte Definition der Schnittstellen Rücksicht genommen werden. Nur dies stellt sicher, daß die Teilsysteme unabhängig entwickelt, getestet und dann zu einem funktionsfähigen Gesamtsystem zusammengesetzt werden können.

Politik. Die Einführung eines neuen Systems bringt Unsicherheit mit sich. Die Furcht vor Veränderung von bestehenden Machtverhältnissen kann zu einem treibenden Faktor für die Verweigerung von Kooperation mit dem Systemanalytiker werden. Machtansprüche und Ängste vor einer Verschlechterung der eigenen Position können den Spielraum für die Erstellung eines Systems erheblich einschränken und im Extremfall sogar das gesamte Projekt gefährden.

Nachdem die Analyse abgeschlossen ist, und festgelegt wurde, was das System leisten soll, kann zur Designphase übergegangen werden.

1.3 Übergang zur Designphase

In der Designphase wird bestimmt, wie das Problem, das in der Analysephase beschrieben wurde, gelöst werden soll. Das Ergebnis der Analysephase war das Analysemodell, ein abstraktes Modell im Problemraum. Das Ziel der Designphase ist nun die Erstellung eines abstrakten, aber implementierbaren Modells im Lösungsraum. Im traditionellen Design (beispielsweise [Davis, 1983, S.11–15, Kap.5 und 6], [Sommerville, 1995, S.31–33]) wird dieses Ziel durch die Beschreibung einer allgemeinen Lösung für das Problem (*System Design*) gefolgt durch eine genaue Beschreibung der verwendeten Verfahren und Algorithmen (*Detailed Design*) erreicht. Dabei erfolgt eine weitere funktionale Zerlegung des Systems in Subsysteme (*Top-down Functional Decomposition*).

Im objekt-orientierten Verfahren OMT [Rumbaugh *et al.*, 1991] wird aufgrund der gleichen grafischen Notation für Analyse und Design keine Notwendigkeit für eine Transformation des Analysemodells in ein Designmodell gesehen. Durch die Analyse wurden bereits die wichtigsten Objekte, ihre Eigenschaften und ihre Beziehungen in Form verschiedener Modelle (*Object Model*, *Dynamic Model* und *Functional Model*) analysiert und beschrieben. Davon ausgehend werden folgende Arbeitsschritte durchgeführt:

1. **System Design.** In diesem Schritt werden die grundlegenden Ansätze zur Lösung der in der Analysephase identifizierten Probleme ausgewählt. Es wird die Systemarchitektur durch Zerlegung des Systems in Teilsysteme erarbeitet (*Object-oriented Decomposition*), und den Teilsystemen werden Ressourcen (Prozesse, Speicher und Kommunikationskanäle) zugeteilt. Abschließend werden, abhängig von den analysierten Erfordernissen, Prioritäten für die Teilsysteme festgelegt.
2. **Object Design.** Die in der Analyse erkannten Objekte und Klassen bilden das Gerüst für das Design. Es müssen Verfahren und Algorithmen für die einzelnen Klassen spezifiziert werden. Komplexe Operationen werden in einfachere zerlegt, wobei neue Klassen entstehen können. Abschließend wird das Design so optimiert, daß je nach den Prioritäten aus dem System Design die Ausführungszeit, der Speicherbedarf oder andere Kostenfaktoren minimiert werden.

Andere Autoren (zum Beispiel [de Champeaux *et al.*, 1992], [Kaindl, 1999]) sehen hingegen auch für objekt-orientierte Verfahren die Notwendigkeit für eine Transformation des abstrakten Modells aus dem Problemraum (Analysephase) in ein abstraktes Modell im Lösungsraum (Designphase), selbst wenn für beide Modelle dieselbe Notation verwendet wird. Kaindl [Kaindl, 1999] sieht gerade in der Verwendung einer einheitlichen Notation für Analyse und Design die Gefahr, daß der immanente Unterschied schwerer erkennbar ist. In Abbildung 1.3 wird dieser Unterschied veranschaulicht. Das Modell der Analysephase ist eine Abstraktion des realen Problems und beinhaltet nur jene Objekte aus der realen Welt, die für das Verständnis des analysierten Problems relevant sind. Dagegen stellt das Designmodell eine Abstraktion des zu implementierenden und (noch) nicht existenten Systems dar, es enthält die Objekte, die zur Lösung des Problems implementiert werden müssen. Unter diesen Gesichtspunkten erscheint ein einfaches Übernehmen der Objekte des Analysemodells in die Designphase ohne Transformation sehr zweifelhaft.

Korrekterweise muß also für das Design ein neues Modell entwickelt werden, daß die abstrahierten Anforderungen aus dem Analysemodell einhält. Kaindl [Kaindl, 1999] beschreibt Probleme existierender Methoden im Bereich dieses Übergangs vom Analysemodell zum Designmodell. Er argumentiert, wenn das Analysemodell das Designmodell bereits enthält, das heißt, das Designmodell ist einfach eine Verfeinerung des Analysemodells, dann muß sich die Analyse zwangsläufig schon mit Designfragen des Systems beschäftigen. Dies führt aber bei großen Systemen zu Problemen mit zu hoher Komplexität, da schon überlegt wird, wie das System seine Aufgaben erfüllen soll, bevor die Aufgaben vollständig definiert sind.

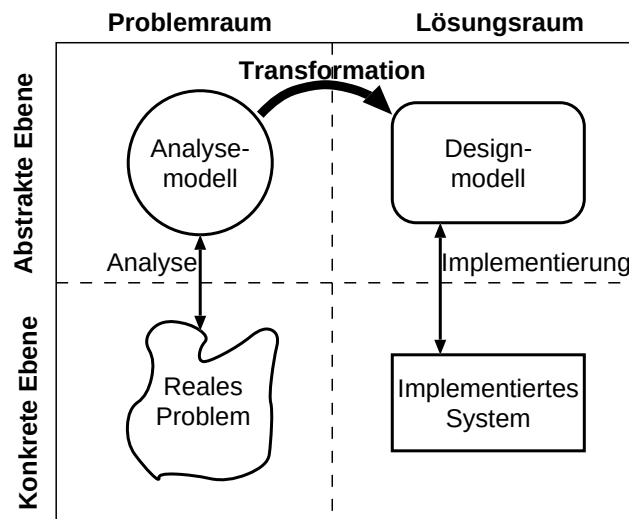


Abbildung 1.3: Transformation zwischen Analyse- und Designmodell

Beispielsweise begegnet Kerth diesem Problem durch die Verwendung einer objektfreien Analysephase und beschreibt den Übergang in Form von Patterns [Kerth, 1995].

De Champeaux et al. [de Champeaux *et al.*, 1992] haben sich eingehend mit dem Übergang von Analyse zu Design beschäftigt. Dabei wurden folgende wichtige Eigenschaften für den Transformationsprozeß identifiziert:

- **Trennung in zusammengehörige Teile.** Der gesamte Transformationsprozeß soll in sinnvoll zusammengehörige Schritte mit genau definiertem Input und Output zerlegt werden.
- **Sequenzialisierung.** Logische Abhängigkeiten zwischen den Schritten müssen im Zeitablauf beachtet werden.
- **Konservativismus.** Vorzeitige Festlegungen von unwichtigen Details sollen vermieden werden, da dies ohne Notwendigkeit Alternativen zu einem späteren Zeitpunkt beschränkt.
- **Allgemeinheit.** Die Transformation soll so allgemein wie möglich durchgeführt werden. Dadurch können Redundanzen vermindert werden, die durch die getrennte Betrachtung von vielen Einzelfällen entstehen.
- **Kontinuität.** Der Transformationsprozeß soll die gleiche Repräsentationsform (Notation) für Input und für Output verwenden.

Die Designentscheidungen, die während des Transformationsprozesses getroffen werden, bestimmen die Eigenschaften der entstehenden Software. Diese Eigenschaften werden im folgenden Abschnitt behandelt.

1.4 Qualität von Softwarearchitektur

Einen Überblick über von verschiedenen Autoren identifizierte Qualitätskriterien für Software, deren Zusammenhang und Messung bietet Gillies [Gillies, 1992]. Eine mögliche Einteilung von Eigenschaften für Software besteht in funktionale- und nicht-funktionale Eigenschaften (siehe [Buschmann *et al.*, 1996, S.404–410]). Funktionale Eigenschaften sind direkt für den Anwender sichtbar. Zu diesen Eigenschaften gehören: *Usability*, *Integrity* und *Correctness*. Sie stellen Eigenschaften von implementierten Funktionen wie zum Beispiel Algorithmen dar. Software, die diese funktionalen Erfordernisse nicht erfüllt, ist unbrauchbar.

Die nicht-funktionalen Eigenschaften beeinflussen, wie der Name schon sagt, nicht die von außen erkennbare Funktion des Systems. Sie haben aber großen Einfluß auf die Erstellung und Wartung des Systems, sowie auf spätere Erweiterungen. Solche Eigenschaften sind nach [Gillies, 1992], [Buschmann *et al.*, 1996] und [de Champeaux *et al.*, 1992]:

Changeability. Große Softwaresysteme haben oft eine relativ lange Lebensdauer. Da sich die Anforderungen in diesem langen Zeitraum verändern, ist es notwendig, das System anzupassen. Die Kosten für diese Anpassungen hängen hauptsächlich davon ab, ob das System bei der Erstellung bereits für Veränderungen konzipiert wurde. Die Eigenschaften, die ein System anpaßbar machen, sind *Extensibility*, *Flexibility*, *Performance Tunability* und *Fixability* [Fayad und Cline, 1996]. *Extensibility* bedeutet, daß das System einfach mit neuen Funktionen erweitert werden kann. *Flexibility* fordert, daß das System auch für neue Anwendungsbereiche verwendet werden kann. *Performance Tunability* beschäftigt sich mit der Skalierbarkeit des Systems, wobei auf die verteilte Ausführung von Teilfunktionen Wert gelegt wird. Und *Fixability* bezeichnet die Fähigkeit, einen Fehler im System zu beheben, ohne dadurch neue Fehler zu erzeugen.

Interoperability und Compatibility. Software Systeme arbeiten nicht total getrennt von der Umwelt. Es müssen genau definierte Schnittstellen nach außen geschaffen werden, über die die Kommunikation abläuft.

Reliability. Verlässlichkeit stellt die Fähigkeit von Software dar, mit dem Auftreten von Fehlern und mit falscher Verwendung durch den Benutzer umzugehen. Der erste Aspekt wird mit Fehlertoleranz und der zweite mit Robustheit bezeichnet. Bei beiden muß im Fehlerfall mit einer geeigneten Maßnahme durch das Programm reagiert werden, um die Integrität des Systems als ganzes nicht zu gefährden.

Reusability. Wiederverwendbarkeit teilt sich in zwei Bereiche. Erstens in den Bereich der Wiederverwendung von bereits entwickelten Komponenten, und zweitens in den Bereich der Entwicklung von wiederverwendbaren Komponenten. Lewis et al. [Lewis *et al.*, 1991] untersuchten durch ein kontrolliertes Experiment mit Studenten den Einfluß von Objekt-Orientierung und Wiederverwendung auf die Produktivität von Softwareentwicklung. Das Ergebnis zeigt, daß sowohl die Verwendung von Objekt-Orientierung als auch der Einsatz von Wiederverwendung einen deutlich positiven Einfluß auf die Produktivität ausüben.

Testability. Das Testen von großen und komplexen Softwaresystemen bringt beträchtliche Kosten mit sich. Bei gut testbaren Systemen kann die Funktion kleiner Teile des Gesamtsystems entkoppelt vom Rest überprüft werden.

Understandability. Um Eigenschaften wie *Changeability* und *Reusability* zu ermöglichen muß der Zweck, der hinter jedem Teil der Software und dessen Design steht, klar erkennbar sein. Ist der Zweck nicht erkennbar oder mißverständlich, besteht die Gefahr, daß bei Veränderungen funktionale und nicht-funktionale Eigenschaften verloren gehen.

Klassische Designmethoden konzentrieren sich hauptsächlich auf den funktionalen Bereich von Software. Für nicht-funktionale Eigenschaften muß der Designer selbst durch seine Erfahrung sorgen. Objekt-Orientierung erleichtert dies zwar durch Konzepte, wie Kapselung und Vererbung, doch müssen diese richtig eingesetzt werden, um höhere Produktivität zu bewirken. Design Patterns, wie sie in dieser Arbeit beschrieben werden, sind durch Experten schriftlich festgehaltene Lösungsvorschläge für wiederkehrende Designprobleme. Diese Lösungsvorschläge beinhalten in der Regel neben rein funktionalen Eigenschaften auch nicht-funktionale Eigenschaften, die durch die Erfahrungen der Autoren des Patterns in das Design eingebracht wurden. Insbesondere verbessert die Verwendung von gut durchdachten Patterns die Klarheit und die Flexibilität des Designs.

1.5 Refactoring

Refactoring ist die funktionserhaltende Restrukturierung von Software mit dem Ziel, die Lesbarkeit und Erweiterbarkeit des Systems zu erhöhen und die Fehleranfälligkeit zu verringern [Opdyke, 1992, S.7]. Das heißt, es wird ein bestehendes System so verändert, daß seine nicht-funktionalen Eigenschaften verbessert werden.

Vom Blickwinkel des Refactoring kann die Entwicklung von objekt-orientierter Software in die drei Phasen *Prototyping*, *Expansion* und *Consolidation* unterteilt werden (siehe [Foote und Opdyke, 1995]). Mit Prototyping wird die Erstellung der Software bezeichnet, die mit der Inbetriebnahme beendet ist. Es folgt die Expansion, dabei entstehen mit der Zeit neue Anforderungen an das System, die durch Erweiterungen der Software gelöst werden. Durch diese Erweiterungen wird die Software aber inflexibel und immer fehleranfälliger. Soll sie trotzdem noch weiterentwickelt werden, muß man sie grundlegend reorganisieren, ohne ihre Funktion zu verändern. Dieser Prozeß heißt *Refactoring* und stellt die Konsolidierungsphase dar.

Foote und Opdyke [Foote und Opdyke, 1995] haben für die drei Phasen Patterns entwickelt, die grundlegende Probleme und ihre Lösungen beschreiben. Diese Patterns zielen nicht auf die Behebung einzelner funktionaler Probleme ab, sondern rein auf die Verbesserung der Softwarearchitektur, um eine kontinuierliche Weiterentwicklung der Software zu ermöglichen.

Da Refactoring die Suche nach erweiterbaren und flexiblen Lösungen bedeutet, erhöht sich die Chance, nach dem Refactoring wiederverwendbares Design oder wiederverwendbare Komponenten zu finden. Wie wir in Kapitel 2 sehen werden, sind Patterns einerseits eine Hilfe für Refactoring, da sie flexible Lösungen für häufig vorkommende Probleme beschreiben und andererseits entstehen Patterns selbst häufig aus dem Prozeß des Refactorings.

Kapitel 2

Der Pattern-Ansatz

Dieses Kapitel ist eine kurze Einführung in den Pattern-Ansatz, eine Möglichkeit Expertenwissen niederzuschreiben und später unabhängig vom Experten wiederzuverwenden. Der Pattern-Ansatz ist die Grundlage für Design Patterns und auch für Analyse Patterns, die in dieser Arbeit untersucht werden.

In Abschnitt 2.1 wird der Begriff Pattern, und die Idee dahinter, beschrieben. Anhand von Patterns für die Planung von Gebäuden und Städten, die für den Einsatz von Patterns in der Softwareentwicklung maßgebend waren, werden die grundlegenden Eigenschaften von Patterns sowie der Prozeß ihrer Erstellung zusammengefaßt.

In Abschnitt 2.2 wird zur Illustration des Ansatzes ein Beispiel Pattern (*University as a Marketplace*) aus dem Bereich der Architektur vorgestellt.

2.1 Der Begriff Pattern

Am Anfang jeder Entwicklung, sei es im Bereich der Softwareentwicklung oder in anderen Disziplinen, steht ein Problem, das es zu lösen gilt. Personen, die bereits viele Probleme in einem Bereich gelöst haben, nennt man Experten. Die Frage ist nun, was unterscheidet Experten von anderen Menschen. Um darauf eine Antwort zu finden, muß das Problemlösungsverhalten von Experten genauer betrachtet werden. In den meisten Fällen erfindet der Experte keine neuen Lösungen, sondern er wendet ihm bekannte Lösungsmuster an. Mit der Zeit sammelt der Experte durch die praktische Arbeit Erfahrungen, die er abstrahiert und zu seiner persönlichen Wissensbasis verknüpft. Genau dieses Wissen über Problemlösungen und ihre Anwendung macht eine Person zum Experten.

Ein Pattern ist schriftlich festgehaltenes Expertenwissen für einen bestimmten Problembereich. Das Pattern beschreibt ein Problem, das immer wieder in ähnlicher Form auftritt, und seine Lösung. Das Problem und die Lösung werden im Pattern abstrakt dargestellt. Die Beschreibung konzentriert sich dabei auf die Gemeinsamkeiten einer Gruppe von gleichartigen Problemen und den Kern ihrer Lösung. Diese Abstraktion unterscheidet Patterns von einfachen Beschreibungen in Form von Fallstudien. Erst durch die Abstraktion können gut durchdachte Patterns wieder und wieder angewandt werden.

Patterns werden in vielen Bereichen – oft unbewußt – angewandt, da das Denken in einem Problem-Lösungs-Schema weit verbreitet ist. Beispiele für Bereiche, in denen Patterns bewußt verwendet werden, sind die Architektur [Alexander *et al.*, 1977], Strategie und Kriegsführung [Tzu, 1983], Jazzmusik [Coker *et al.*, 1970], das Management von Organisationen [Coplien, 1995], das Verständnis von kognitiven Prozessen [Gardner *et al.*, 1998] und auch die Entwicklung von wiederverwendbarer Software [Foote und Opdyke, 1995].

Für die Verwendung von Patterns im Bereich der Softwareentwicklung sind besonders die Arbeiten von Christopher Alexander [Alexander *et al.*, 1977] [Alexander, 1979] wichtig. Als Professor für Architektur an der Universität Berkeley erstellte er Patterns für die Planung von Gebäuden und Städten.

In *The Timeless Way of Building* [Alexander, 1979] beschreibt Alexander die wesentlichen Eigenschaften von Patterns. Diese Eigenschaften werden hier mit Seitenangaben aus diesem Buch beschrieben.

- Patterns sind eine dreiteilige Regel, sie drücken eine Beziehung zwischen einem gewissen Kontext, einem Problem und seiner Lösung aus. Ein Pattern ist die Beschreibung eines Dings und gleichzeitig die Beschreibung des Prozesses, der dieses Ding erzeugt [S.247].
- Patterns müssen ihre innere Struktur klar darstellen. Sie müssen folgende Fragen beantworten: Was genau läßt das Pattern funktionieren? Warum hilft das Pattern? Wann und Wo funktioniert es? [S.249]
- Ein Pattern zeigt die unveränderlichen Merkmale, die das Problem lösen [S.257], beziehungsweise die Merkmale, die gute von schlechten Dingen unterscheiden [S.260].
- Um Patterns zu entdecken, muß mit Beobachtung begonnen werden [S.254]. Manchmal beginnt man mit einer Reihe von positiven Beispielen [S.258]. Manchmal beginnt man mit negativen Beispielen und versucht die erkannten Probleme zu lösen [S.259]. Und gelegentlich

beginnt man nicht mit konkreten Beispielen, sondern versucht, das Pattern mit rein abstrakten Argumenten aufzubauen [S.259].

Alexander beschreibt auch eine Vorgehensweise für die Erstellung von Patterns. Die Schritte mit den Seitenangaben aus [Alexander, 1979] sind:

1. Definieren Sie die Merkmale eines "Dings" (Alexander verwendet hier "Platz", da er über Architektur schreibt), das es wert ist, abstrahiert zu werden [S.249]. Das Ziel des Patterns, die Erreichung oder Vermeidung dieser Merkmale, wird dadurch festgelegt.
2. Definieren Sie das Problem, beziehungsweise das System der Kräfte (Alexander nennt sie *Forces*), die das Pattern ins Gleichgewicht bringt [S.251]. Dies ist die Analyse, welches Problem durch das Pattern gelöst werden soll, und warum die Merkmale des betrachteten Dings gut oder schlecht sind.
3. Definieren Sie den Kontext, in dem die Kräfte auftreten, und in dem sie durch die im Pattern beschriebene Beziehung ins Gleichgewicht gebracht werden [S.252]. Dadurch wird beschrieben, in welchen Zusammenhang und unter welchen Voraussetzungen das Pattern sinnvoll angewandt werden kann und welche Konsequenzen daraus entstehen.
4. Redefinieren Sie das Pattern als Einheit, um es einsetzbar zu machen [S.265]. Dazu muß man dem Pattern einen Namen geben [S.267]. Erst dadurch wird ein Pattern mitteilbar [S.268].

Nach diesen Schritten können Patterns von verschiedenen Personen diskutiert, wiederverwendet, verbessert und mit eigenen Beobachtungen überprüft werden. Außerdem kann sich mit dieser Information jeder selbst entscheiden, ob er die Anwendung des Patterns für ein bestimmtes Problem für sinnvoll hält.

2.2 Ein Beispiel Pattern: University as a Marketplace

Ein Beispiel für ein Architekturpattern ist das Pattern *University as a Marketplace (of ideas)* [Alexander *et al.*, 1977, S.231–235] von Alexander. Er beschreibt das Problem und den Kontext folgendermaßen:

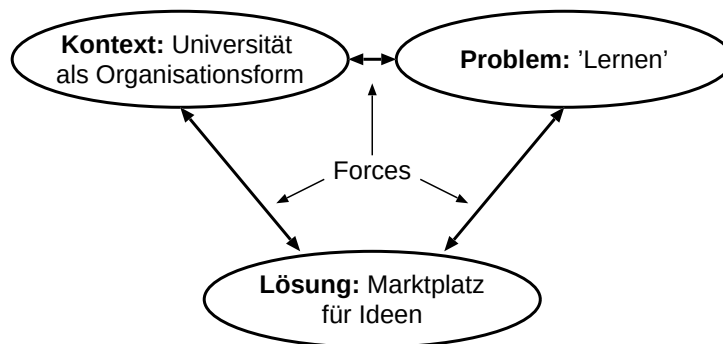


Abbildung 2.1: Das Zusammenspiel von Problem, Kontext und Lösung für das Pattern: *University as a Marketplace*

Concentrated, cloistered universities, with closed administration policies and rigid procedures which dictate who may teach a course, kill opportunities for learning.

Die Beziehungen der dreiteiligen Regel dieses Patterns sind in Abbildung 2.1 dargestellt. Das grundlegende, in diesem Pattern behandelte **Problem** ist es, den Lernprozeß im allgemeinen zu unterstützen und zu fördern. Den **Kontext** dieses Patterns stellen Universitäten als Organisationsform für das Lernen dar. Im weiteren beschreibt Alexander Kräfte (*Forces*), die die Beziehungen zwischen den drei Komponenten des Patterns bestimmen. Kräfte wirken oft in verschiedene Richtungen, was dazu führt, daß das Problem nicht leicht lösbar ist. Das Pattern soll nun eine Lösung bieten, die diese Kräfte in ein harmonisches Gleichgewicht bringt. Beispielsweise argumentiert Alexander im hier betrachteten Pattern, daß im Mittelalter Universitäten nur Ansammlungen von Lehrern waren, die rein durch ihr Lehrangebot Studenten anzogen. Sie waren Marktplätze für Ideen, auf denen sich Studenten Ideen aneignen konnten, um sich danach selbstständig ein Bild über deren Sinnhaftigkeit machen zu können. Im Gegensatz zu diesem Konzept steht, das heutige Universitätssystem. Universitäten sind meist isoliert und überverwaltet, was zu einer massiven Einschränkung der angebotenen Ideen führt. Die wirkenden Kräfte sind einerseits eine möglichst große Vielfalt an Ideen anzubieten und andererseits eine möglichst effiziente Verteilung dieser Ideen zu organisieren. Da diese Kräfte in den meisten Organisationsformen miteinander in Konflikt stehen, ist die Lösung nicht einfach.

Eine **Lösung** skizziert Alexander so (die Aspekte der Stadtplanung wurden weggelassen):

Therefore:

Establish the university as a marketplace of higher education.

As a social conception this means that the university is open to people of all ages, on a full-time, part-time, or course by course basis. Anyone can offer class. Anyone can take class. Physically, the university marketplace has a central crossroad ...

Der Grundpfeiler dieser Lösung ist, daß jeder auf dem Marktplatz für Ideen ohne Einschränkungen agieren kann. Er kann Kurse konsumieren und anbieten. Die Steuerung übernimmt der Marktmechanismus durch Angebot und Nachfrage:

- Angebot. Für Studenten, die an einem Kurs teilnehmen, wird der Lehrer bezahlt. Dadurch wird das Angebot der Kurse durch die Nachfrage gesteuert. Freiwillige Zusammenarbeit zwischen mehreren Lehrern ist jederzeit möglich.
- Nachfrage. Der Konsum von Kursen verursacht dem Studenten Kosten. Dadurch werden die Studenten gezwungen eine Auswahl zu treffen und nur Kurse, die sie wirklich interessieren, zu besuchen. Um soziale Ungerechtigkeiten auszuschließen, schlägt Alexander die Verwendung von Gutscheinen, zu denen alle Personen gleichberechtigt Zugang haben, vor.

Alexander erkannte jedoch auch Bereiche, deren Regulierung nicht ohne weiters dem Marktmechanismus überlassen werden können. Dabei handelt es sich zum Beispiel um einen sinnvollen Ausgleich zwischen Anzahl der Studenten in einem Kurs und der Bezahlung des Lehrers. Die Bezahlung ausschließlich nach der Anzahl der Studenten in einem Kurs erscheint nicht zielführend, da dies zu überfüllten Kursen führt, was eine schlechtere Betreuung des einzelnen Studenten zur Folge hat. Außerdem muß eine Art von Evaluierung eingesetzt werden, um verlässliche Information über Lehrer und Kurse für potentielle Studenten zugänglich zu machen.

Alexander faßt seine Patterns in einer Pattern Sprache (*Pattern Language*) zusammen. Innerhalb dieser Sprache wird aufgezeigt, wie einzelne Patterns mit anderen zusammenpassen oder wie komplexe Patterns auf mehreren einfachen basieren. Beispielsweise paßt zu dem eben beschriebenen Pattern, *University as a Marketplace*, das Pattern *Network of Learning* [Alexander *et al.*, 1977, S.99–103], welches einen dezentralisierten Lernprozeß beschreibt.

Natürlich bezog sich Alexander auf die Baukunst und damit verbundene organisatorische und soziale Aspekte, doch ist die Grundidee des Pattern-Ansatzes fast unverändert auch auf die Softwareentwicklung anwendbar. Der Rest dieses Kapitels beschäftigt sich mit der Verwendung von Patterns in der Design- und der Analysephase der Softwareentwicklung.

Kapitel 3

Design Patterns

In diesem Kapitel wird der Einsatz von Patterns für das Design von Software dargestellt. Design Patterns sollen existierende Entwicklungsmethoden durch die Wiederverwendung von Expertenwissen über flexibles Design erweitern und dadurch zu besserer Software führen.

In Abschnitt 3.1 wird die Anwendung von Design Patterns, als Bindeglied zwischen Analyse und Implementierung im Software-Lebenszyklus, dargestellt.

Ein wichtiger Bestandteil von Design Patterns ist ihre Beschreibung in Formularform. In Abschnitt 3.2 werden die am häufigsten verwendeten Formularbestandteile und ihre Bedeutung vorgestellt.

Danach wird in Abschnitt 3.3 auf mögliche Einteilungen und Klassifikationen für Design Patterns, sogenannte Pattern-Kataloge, eingegangen. Der Sinn der Klassifikation von Patterns ist das leichtere Auffinden von Patterns und das Aufzeigen von Zusammenhängen zwischen einzelnen Patterns.

In den Abschnitten 3.4 und 3.5 werden mit Design Patterns verwandte Ansätze, wie *AntiPatterns* und *Application Frameworks*, kurz umrissen. *AntiPatterns* identifizieren in der Praxis oft verwendete, schlechte Lösungen für Designprobleme, dadurch helfen sie mit, diese schlechten Lösungen zu vermeiden. *Application Frameworks* sind eine objekt-orientierte Technik zur Wiederverwendung von Code und Design. Sie sind fast fertige Applikationen, die nur noch angepaßt und fertiggestellt werden müssen.

Abgeschlossen wird das Kapitel mit Abschnitt 3.6, einer Zusammenfassung des Nutzens, der durch die Anwendung von Patterns in der Designphase entsteht. Dabei wird auch auf Problembereiche und Mißverständnisse im Zusammenhang mit Design Patterns eingegangen.

3.1 Die Anwendung von Patterns in der Designphase

Breite Zustimmung hat die Anwendung von Patterns in der Softwareentwicklung durch das Buch *Design Patterns: Elements of Reusable Object-Oriented Software* von Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides [Gamma *et al.*, 1993] gefunden. In diesem Buch werden mehr als zwanzig Designpatterns in einer einheitlichen Weise beschrieben.

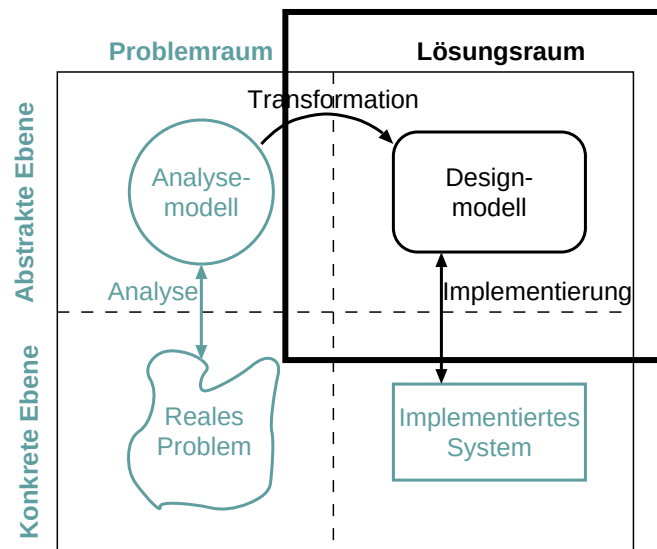


Abbildung 3.1: Einsatzbereich von Design Patterns

Design Patterns werden, wie der Name schon sagt, in der Designphase des Software-Lebenszyklus eingesetzt (siehe Abbildung 3.1). Sie repräsentieren gutes, erprobtes Design und helfen dadurch bei der Erstellung des Designmodells und geben gleichzeitig Hinweise für die Implementierung. Design Patterns stellen ein Bindeglied zwischen Analyse und Implementierung dar.

Ein Design Pattern ist die abstrakte Beschreibung einer Lösung für ein Problem des Softwaredesigns in einem bestimmten Kontext. Wichtig dabei ist, daß das Designproblem oft – wenn auch in leicht abgewandelter Form – auftritt und durch die beschriebene Lösung behoben werden kann. Für die Lösung werden die durch ihre Anwendung entstehenden Vorteile, aber auch mögliche negative Konsequenzen und Einschränkungen, beschrieben. Diese Beschreibung dient als Anhaltspunkt für die Entscheidung, ob ein Pattern angewandt werden soll. Existieren mehrere Patterns zur Lösung eines Problems, kann das am Besten geeignete ausgewählt und angewandt werden.

Jedes Pattern muß einen eindeutigen Namen besitzen, der in das Designvokabular aufgenommen wird. Kennen zwei Designer ein Pattern, kann alleine durch den Namen das abstrakte Konzept des Patterns kommuniziert werden. Die Kommunikation funktioniert einfacher, schneller und Möglichkeiten für Mißverständnisse werden reduziert.

John Vlissides hebt noch den Aspekt hervor, daß ein Pattern "lehrend" sein muß [Vlissides, 1998, S.4]. Durch die Beschreibung des Patterns muß der Benutzer selbständig in der Lage sein, zu erkennen, wann das Pattern angewandt werden kann, und wie die beschriebene abstrakte Lösung konkret umgesetzt wird.

Da für immer mehr Probleme Design Patterns gefunden und publiziert werden, wird die Auswahl eines geeigneten Patterns beziehungsweise der Vergleich mehrerer Patterns zunehmend schwieriger. Um die Auswahl zu erleichtern, wurden für Design Patterns von mehreren Autoren strukturierte Beschreibungsformen (Formulare) vorgeschlagen. Eine Auswahl der am weitesten verbreiteten Bestandteile dieser Formulare wird im folgenden Abschnitt beschrieben.

3.2 Aufbau von Design Patterns

Eine geeignete Beschreibungsform ist für Design Patterns essentiell. Vorteilhaft ist eine strukturierte Beschreibung in Form eines Formulars. Dies bietet unter anderem folgende Vorteile:

- Der Zweck eines Patterns kann schneller erfaßt werden. Es muß nicht immer das ganze Pattern gelesen werden.
- Patterns können leichter verglichen werden. Besonders ein Vergleich der Konsequenzen durch die Anwendung des Patterns ist bei der Auswahl eines geeigneten Patterns wichtig.
- Patterns können durch die Struktur leichter gelehrt und erlernt werden.
- Beim Schreiben eines neuen Patterns werden wichtige Teile des Patterns durch die Vorgabe der Felder des Formulars nicht so leicht vergessen.

Der Aufbau der ersten und bekanntesten strukturierten Beschreibung für Design Patterns wurde von Gamma et al. [Gamma *et al.*, 1993, S.6–8] entwickelt und zur Beschreibung von über zwanzig Patterns für objekt-orientiertes Design verwendet. Buschman und Meunier [Buschmann und Meunier, 1995, S.19–21] haben die Struktur leicht

abgewandelt und durch neue Abschnitte (wie zum Beispiel *Solution* und *Variants*) erweitert. Im folgenden wird die Beschreibungsform nach Buschman et al. vorgestellt und die korrespondierenden Bezeichnung von Gamma et al. in Klammern angegeben.

Name (Pattern Name and Classification). Der Name und eine kurze Beschreibung des Patterns. Der Name soll das Problem und seine Lösung bezeichnen. Er muß sehr sorgfältig gewählt werden, da er in das Designvokabular aufgenommen wird. Das heißt, der Name wird später stellvertretend für alle wichtigen Aspekte des gesamten Patterns verwendet. Dies erleichtert die Kommunikation zwischen Designern. Schlägt einer die Verwendung eines bestimmten Patterns vor, weiß der andere sofort, was gemeint ist. Die Beschreibung soll am besten in einem Satz darstellen, welche Probleme mit diesem Pattern angesprochen werden.

Also Known As (Also Known As). Ein alternativer Name, unter dem dieses Design Pattern bekannt ist.

Example (Motivation). Es wird ein Beispiel beschrieben, das die Existenz des Designproblems zeigt. Das Beispiel zeigt eine konkrete Verwendung des Patterns und hilft die abstrakte Beschreibung des Patterns leichter zu verstehen.

Context (Applicability). Beschreibung der Situationen, in denen das Pattern angewandt werden kann.

Problem (Intent). Abstrakte Beschreibung des Problems und der Kräfte (*Forces*), die durch das Pattern ins Gleichgewicht gebracht werden. Es wird zuerst der Kern des Problems allgemein beschrieben. Danach werden Erfordernisse, Einschränkungen und wünschenswerte Eigenschaften des Ergebnisses aufgezählt und beschrieben.

Solution (nicht vorgesehen). Abstrakte Beschreibung des grundsätzlichen Prinzips der Lösung.

Structure (Structure + Participants). Hier werden die Konfiguration der beteiligten Komponenten und ihre Beziehungen zueinander beschrieben. Es wird in der Regel eine grafische Repräsentation zur Beschreibung verwendet. Als Notation für objekt-orientiertes Design bietet sich zum Beispiel die *Unified Modeling Language* [OMG, 1999] an.

Dynamics (Collaborations). Beschreibt, wie die Komponenten zusammenarbeiten. Es handelt sich dabei um die dynamischen Aspekte des Systems, die das Laufzeitverhalten bestimmen.

Implementation (Implementation + Sample Code). Hinweise auf Probleme und Besonderheiten bei der Implementierung des Patterns. Es handelt sich um Vorschläge und keine Regeln, da die Implementierung an die speziellen Bedürfnisse jedes Einzelfalls angepaßt werden muß. Es können auch Programmteile und Pseudocode als Beispiele zur Illustration einer möglichen Implementierung angegeben werden.

Solved Example (nicht vorgesehen). Eine Beschreibung wichtiger Aspekte für die Lösung des Beispiels aus dem Abschnitt *Example*, die bisher noch nicht erklärt wurden. Beispielsweise kann aufgezeigt werden, wie die abstrakte Lösung aus den Abschnitten *Solution*, *Structure* und *Dynamics* konkret in dem Beispiel umgesetzt wurde.

Variants (nicht vorgesehen). Eine kurze Beschreibung von Varianten, Erweiterungen oder Spezialisierungen des Patterns.

Known Uses (Known Uses). Beispiele von existierenden Systemen, in denen das Pattern verwendet wird.

Consequences (Consequences). Beschreibt die Ergebnisse und Einschränkungen, die sich aus der Anwendung des Patterns ergeben. Es ist unwahrscheinlich, daß ein Pattern alle Aspekte eines Problems vollständig lösen kann. Darum ist es wichtig auf vom Pattern nicht oder nur teilweise gelöste Aspekte des Problems einzugehen. Die Beschreibung der Konsequenzen ist ein wichtiges Kriterium für die Auswahl des passenden Patterns.

See Also (Related Patterns). Patterns, die ähnliche Probleme lösen oder die helfen, das eben beschriebene Pattern zu verfeinern.

Andere Autoren fassen Abschnitte aus dieser Struktur zusammen oder verwenden zusätzliche, um bestimmte Aspekte zu betonen. Im großen und ganzen sind sie der Darstellung hier sehr ähnlich.

3.3 Kategorisierung von Design Patterns

Da man nicht alle Patterns ständig im Kopf haben kann, benötigt man eine Übersicht über verfügbare Patterns. Dazu muß eine Ordnung in die Patterns gebracht werden, die es erlaubt, geeignete Patterns für ein bestimmtes Problem rasch zu finden und zu vergleichen. Mehrere Autoren haben ihre Patterns nach gewissen Kriterien geordnet, oder ein allgemeines Ordnungsschema für Patterns vorgeschlagen.

Erich Gamma et al. [Gamma *et al.*, 1993, Gamma *et al.*, 1995] teilen Design Patterns nach zwei Klassifikationskriterien ein. Dies sind der Umfang der Patterns und ihr Zweck.

- Nach dem Umfang werden nur Patterns für *Klassen* und für *Objekte* unterschieden, da ausschließlich objekt-orientierte Patterns beschrieben werden.
- Nach dem Zweck werden Patterns in die Kategorien *Creational*, *Structural* und *Behavioral* eingeteilt. *Creational Patterns* beschäftigen sich mit der Erzeugung von Objekten. *Structural Patterns* beschreiben die statische Zusammensetzung von Objekten und Klassen. *Behavioral Patterns* charakterisieren das dynamische Verhalten von Objekten und Klassen.

Erich Gamma und seine Mitautoren haben über zwanzig Patterns beschrieben und in dieses Schema eingefügt. Beispiele sind die Patterns *Abstract Factory*, *Adapter* und *Strategy*. Alternativ zu der gerade beschriebenen Einteilung bieten Gamma et al. auch eine Art Landkarte in Form eines Graphs für ihre Patterns an. Die Patterns sind darauf als Knoten dargestellt. Die Beziehungen zwischen den Patterns sind als Kanten eingezeichnet, wobei die Kanten mit der Art der Beziehung beschriftet sind. Zum Beispiel stehen die Patterns *Interpreter* und *Composite* in Beziehung, da *Interpreter* das Pattern *Composite* verwendet, um seine Grammatik zu definieren.

Frank Buschman und seine Kollegen [Buschmann und Meunier, 1995, Buschmann *et al.*, 1996] teilen Patterns nach ihrem Abstraktionsgrad ein.

1. Die Patterns der höchsten Abstraktionsstufe nennen sie *Architectural Patterns*. Diese Patterns erklären die fundamentale strukturelle Organisation von Softwaresystemen. Sie geben die nötigen Subsysteme an und spezifizieren ihre Funktionen und Beziehungen zueinander.
2. In der mittleren Abstraktionsebene werden *Design Patterns* verwendet. Diese zeigen, wie Subsysteme oder Komponenten verfeinert werden. Sie lösen wiederkehrende Designprobleme.

3. Die geringste Abstraktion weisen *Idiome* auf. Sie lösen Probleme, die im Zusammenhang mit einer bestimmten Programmiersprache auftreten. Wichtige Idiome für eine Sprache können für eine andere nutzlos sein. Beispiele von Idiomen für C++ findet man in [Coplien, 1992, Coplien, 2000], und über 600 Idiome für die Programmiersprache APL sind in der *FinnAPL Idiom Library* [Ali *et al.*, 1982] zusammengefaßt.

Als zweites Klassifikationskriterium wird der zu lösende Problembereich verwendet. Einige solcher Problembereiche sind zum Beispiel *Communication*, *Access Control* und *Distributed Systems*. Die Liste der Problembereiche ist erweiterbar und kann bei Bedarf ergänzt werden. In dieses Schema wurden rund zwanzig Patterns eingeordnet. Die Patterns decken sich größtenteils mit den schon von Gamma *et al.* präsentierten.

Walter Zimmer [Zimmer, 1995] geht von den Patterns in [Gamma *et al.*, 1995] aus. Da Patterns nur selten isoliert voneinander existieren, versucht Zimmer die Beziehungen zwischen den einzelnen Patterns aufzuzeigen. Er identifiziert drei verschiedene Arten von Beziehungen zwischen zwei Patterns (mit X und Y bezeichnet), nämlich:

- X verwendet Y in seiner Lösung.
- Eine Variante von X verwendet Y in seiner Lösung.
- X ist ähnlich wie Y.

Diese Beziehungen werden als Graphen zwischen den Patterns dargestellt. Danach führt Zimmer drei Schichten ein, denen er die Patterns zuordnet.

1. Die unterste Schicht nennt er *Basic Design Patterns and Techniques*. Sie beinhaltet Patterns, die für sehr generelle Probleme angewandt werden. Sie werden regelmäßig durch Patterns der höheren Schichten verwendet.
2. *Design Patterns for Typical Software Problems* heißt die mittlere Schicht. Patterns dieser Schicht werden für speziellere Probleme verwendet.
3. Die oberste Schicht nennt Zimmer *Design Patterns Specific to an Application Domain*. Sie beinhaltet Patterns, die für einen oder mehrere Bereiche des Software Designs verwendet werden können.

Durch die hierarchische Einordnung der Patterns und der grafischen Darstellung der Beziehungen zwischen ihnen wird die Orientierung in einem System von Patterns sehr erleichtert. Die oft schwer erkennbaren Zusammenhänge und Abhängigkeiten werden explizit dargestellt.

Von einer *Pattern Language*, wie sie Alexander [Alexander *et al.*, 1977] für die Architektur vorstellt, ist man bei Design Patterns für Software noch weit entfernt. Um komplett zu sein, müßte eine solche Sprache Lösungen für alle Design Probleme eines bestimmten Bereiches bieten. Dies wurde bisher nur bedingt und auch nur für sehr beschränkte Teilbereiche erreicht. Beispiele sind:

- CHECKS [Cunningham, 1995] für den Bereich Informationsintegrität.
- RAPPeL [Whitenack, 1995] für den Prozeß der Anforderungsanalyse bei objekt-orientierter Entwicklung.
- Caterpillar's Fate [Kerth, 1995] für die Transformation von objektfreier Analyse in Design.

Die Frage, die sich in diesem Zusammenhang stellt, ist, ob eine vollständige Pattern Language für Softwaredesign überhaupt sinnvoll ist. Möglicherweise ist es ausreichend, die bisher vorgestellten Patternssysteme weiter auszubauen und zu verfeinern.

3.4 AntiPatterns

Design Patterns gehen von oft vorkommenden Problemen aus und beschreiben Lösungen, die sich als brauchbar erwiesen haben. AntiPatterns [Brown *et al.*, 1998] versuchen den umgekehrten Weg zu gehen. Abbildung 3.2 zeigt einen Vergleich zwischen Patterns und AntiPatterns. AntiPatterns identifizieren schlechte Lösungen (AntiPattern Lösungen), die in der Praxis oft vorkommen, und versuchen Auswege für diese schlechten Lösungen zu finden. AntiPattern Lösungen sind dadurch gekennzeichnet, daß ihre negativen Konsequenzen die entstehenden Vorteile bei weitem übertreffen. Diese Konsequenzen beruhen auf der Tatsache, das wichtige Kräfte (*Forces*) übersehen oder ignoriert wurden. Unter Berücksichtigung dieser neuen Kräfte wird die AntiPattern Lösung in eine neue Lösung (*Refactored Solution*, siehe *Refactoring* in Abschnitt 1.5) umgewandelt, die negative Konsequenzen zu Gunsten der Vorteile verringert, ohne die Funktion des Systems zu verändern.

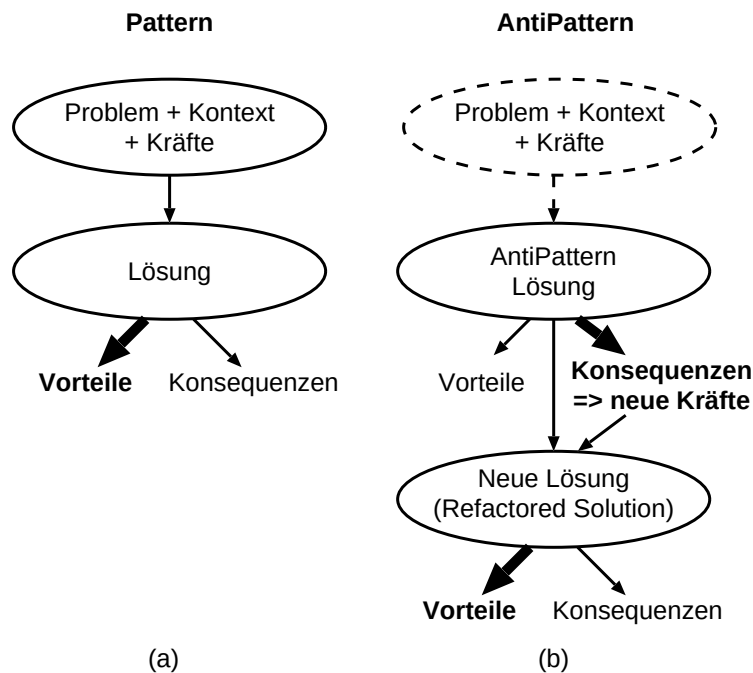


Abbildung 3.2: Vergleich zwischen Patterns (a) und AntiPatterns (b)

Brown et al. stellen AntiPatterns auf folgenden drei Ebenen vor [Brown et al., 1998, S.18]:

Development AntiPatterns. Diese AntiPatterns behandeln Probleme, die während des Programmierens auftreten. Beispiele sind: *Spaghetti Code*, *Cut-and-Paste Programming* und *Mushroom Management*.

Architectural AntiPatterns. Hier werden Probleme mit der Struktur von Systemen, also Probleme des Systemdesigns behandelt. Beispiele sind: *Vendor Lock-In*, *Swiss Army Knife* und *Reinvent the Wheel*.

Management AntiPatterns. Beschreiben Probleme organisatorischer Natur, die während der Planung und Erstellung von Systemen auftreten. Beispiele sind: *Analysis Paralysis*, *Death by Planning* und *Irrational Management*.

Als grundlegende Probleme für das Auftreten von AntiPatterns, wurden eine Anzahl von Hauptgründen identifiziert [Brown et al., 1998, S.19–26]:

- Übereilte Entscheidungen führen zu schlechtem Design.
- Gleichgültigkeit gegenüber der Qualität der Lösung.

- Engstirnigkeit. Weigerung, allgemein anerkannte Praktiken zu verwenden.
- Faulheit führt zu der Lösung mit dem geringsten Aufwand.
- Gier führt zu einem Übermaß an Detailliertheit und Komplexität.
- Ignoranz führt zu fehlendem Verständnis.
- Stolz beeinträchtigt Wiederverwendung von Komponenten, die jemand anderer bereits entwickelt hat.

In einem weiteren Buch [Brown *et al.*, 1999] stellen Brown et al. AntiPatterns (und auch Patterns) für *Software Configuration Management*, *Project Management* und *Requirements and Testing* vor.

AntiPatterns sind eine reaktive Variante von Patterns. Sie setzen da an, wo negative Konsequenzen bereits aufgetreten sind. Es geht darum, die negativen Konsequenzen durch Veränderungen in den Griff zu bekommen. Aber natürlich können AntiPatterns auch aktiv dazu beitragen, daß es erst gar nicht zu den problematischen Lösungen kommt. AntiPatterns können vor Designentscheidungen konsultiert werden und damit zur Vermeidung von AntiPattern Lösungen beitragen.

3.5 Application Frameworks

Application Frameworks sind eine objekt-orientierte Technik zur Wiederverwendung von Code und Design [Johnson, 1997]. Durch den zunehmenden Zeitdruck bei der Entwicklung neuer Systeme gewinnt Wiederverwendung und Wiederverwendbarkeit immer mehr an Bedeutung. Wiederverwendung führt außerdem zu einer Vereinheitlichung der entwickelten Systeme und dadurch auch zu einer Reduktion der Wartungskosten. Beispiele für Frameworks, die zu einer Vereinheitlichung der grafischen Bedienungsoberfläche führen, sind das kommerzielle Framework *Microsoft Foundation Classes* [Prosise, 1999] für Windows und das frei verfügbare Framework *Qt* [Dalheimer, 1999], auf dem die KDE-Bedienungsoberfläche für Linux basiert.

Ein Framework besteht aus fertigen und halbfertigen Teilsystemen, wobei die Architektur des Softwaresystems durch diese Teilsysteme vordefiniert ist. Wiederverwendbar wird das System durch die halbfertigen Teilsysteme, deren Funktionen erweitert und verändert werden können. Wolfgang Pree [Pree, 1995, S.106] nennt diese flexiblen Bereiche eines Frameworks *Hot Spots*. Ihre Festlegung ist der schwierigste Teil beim Design eines

neuen Frameworks. Erstens muß man den Anwendungsbereich des Frameworks genau kennen, um festzulegen welche Bereiche flexibel gestaltet werden sollen, und zweitens muß man zwischen der Flexibilität und dem späteren Anpassungsaufwand einen Kompromiß finden. Cline und Girou [Cline und Girou, 2000] argumentieren, daß bei der Spezifikation von leicht anpaßbarer Software (Frameworks) statt der flexiblen Bereiche, die stabilen Bereiche identifiziert werden sollen. Diese Bereiche stellen die grundlegenden und gleichbleibenden Abläufe und Zusammenhänge in bestimmten Wirtschaftsbereichen dar und werden *Enduring Business Themes* genannt.

Frameworks werden von Experten hergestellt, angewandt werden sie hingegen von normalen Entwicklern. Aus diesem Grund ist die Beschreibung eines Frameworks mindestens genauso wichtig, wie seine technischen Eigenschaften. Eine Möglichkeit, die Anwendung von Frameworks zu beschreiben, sind *Cookbooks* [Pree, 1995, S.79–87]. Sie beinhalten verschiedene Rezepte und beschreiben die Verwendung des Frameworks auf informale Weise. Die Beschreibung bezieht sich auf die Anwendung und nicht auf das Design und die Implementation des Frameworks, da man davon ausgeht, daß sich ein Entwickler nur für die Lösung seines Problems interessiert und nicht für das genaue Design, das dahinter steht.

Eine Schwäche von Cookbooks ist, daß sie nur beschreiben, wie ein Framework auf typische Weise angewendet wird. Dieser Aspekt ist zwar wichtig, jedoch nicht ausreichend, um ein Framework vollständig zu beschreiben. Ein gutes Framework kann auf Arten angewendet werden, an die der Designer bei der Erstellung nicht gedacht hat. Um solche Anwendungen zu ermöglichen, muß auch das Design des Frameworks genau beschrieben werden. Außerdem fehlt in Cookbooks die Beschreibung des Zwecks des Frameworks. Dies ist wichtig, damit sich der Anwender schnell ein Bild machen kann, ob das Framework für sein Problem überhaupt geeignet ist.

Für die Beschreibungen von Frameworks gibt es also zwei entgegengesetzte Anforderungen. Einerseits soll die Beschreibung möglichst kurz sein, um den Zeitaufwand bei der Auswahl und der Anwendung des Frameworks kurz zu halten, und andererseits muß die Beschreibung vollständig sein, um alle möglichen Anwendungen zu erlauben. Diese konkurrierenden Ziele können durch die Verwendung eines *Satzes von Patterns* erreicht werden [Johnson, 1992]. Für einfache Probleme, die regelmäßig auftauchen, müssen nur die ersten paar Patterns gelesen werden. Speziellere Probleme werden dann durch weiterführende Patterns beschrieben, die erst bei Bedarf gelesen werden müssen.

Johnson [Johnson, 1997] vergleicht Frameworks mit anderen Techniken für Wiederverwendung. Er kommt zum Schluß, daß Frameworks in ihrer Abstraktion und Flexibilität zwischen der Wiederverwendung von Kompo-

nenten (*Code Reuse*) und der Wiederverwendung von reinem Design (zum Beispiel Design Patterns) liegen. Er identifiziert auch einige Probleme, die bei Frameworks auftreten können. Je leistungsfähiger und komplexer ein Framework wird, desto schwerer wird es für den Anwender erlernbar und handhabbar. Außerdem ist die Entwicklung von Frameworks sehr schwer und kostenintensiv.

Pree beschäftigt sich mit der Erstellung von Frameworks. Er beschreibt *Metapatterns* [Pree, 1995, Kap.4], die zeigen, wie man Frameworks konstruiert, die unabhängig von einem speziellen Anwendungsbereich verwendet werden können. Er sieht diese Metapatterns als eine Ansammlung von Erfahrungen aus bereits bestehenden Frameworks, die für die Entwicklung neuer Frameworks genutzt werden sollen. Pree schlägt vor, seine Metapatterns zu einem *hot-spot-driven Approach* weiterzuentwickeln.

3.6 Nutzen von Design Patterns

Patterns werden von erfahrenen Softwaredesignern schon sehr lange verwendet. Vor dem Aufkommen des Pattern-Ansatzes hat jedoch jeder Designer "seine" Patterns selbst entwickelt, da keine geeignete Möglichkeit bestand, um Erfahrungen in geordneter Form weiterzugeben. Patterns bieten diese Möglichkeit. Ihre Verwendung zur Beschreibung der Beziehungen zwischen einem Problem und möglicher Lösungen bringt laut Gamma et al. [Gamma et al., 1995, S.351–355] folgende vier große Vorteile:

Ein gemeinsames Design Vokabular. Patterns bieten durch ihren Namen eine einfache Möglichkeit zur Kommunikation und Dokumentation. Kennen mehrere Designer bestimmte Patterns, reicht ihr Name aus, um über Designalternativen zu sprechen. Die Namen der Patterns bilden ihr gemeinsames Vokabular.

Eine Dokumentations- und Lernhilfe. Bei großen objekt-orientierten Systemen werden oft unbewußt Design Patterns verwendet. Beschreibt man so ein System mit Design Patterns, die im System gefunden werden, kann seine Funktion viel schneller und leichter verstanden werden.

Da Design Patterns für allgemeine Probleme, die immer wieder vorkommen, Lösungen bieten, liegt es nahe, sie für Lehrzwecke zu verwenden. Goldfedder und Rising [Goldfedder und Rising, 1996] beschreiben Erfahrungen über den Unterricht mit Patterns.

Eine Erweiterung zu existierenden Methoden. Existierende Designmethoden beschreiben das Vorgehen bei der Erstellung eines Designs.

Sie unterstützen jedoch nicht die Möglichkeit zur Sammlung und Weitergabe von Erfahrungen. Design Patterns erweitern existierende Methoden um folgende Eigenschaften:

- Design Patterns helfen bei der Sammlung und Weitergabe von Erfahrung von Experten.
- Design Patterns beschreiben, warum eine bestimmte Designalternative (ein Pattern) verwendet werden soll, und die sich daraus ergebenden Konsequenzen. Daher können Designentscheidungen später leichter nachvollzogen werden.
- Design Patterns erleichtern den Übergang vom Analysemodell zu einem implementierbaren Designmodell durch Beispiele für wiederverwendbares und flexibles Design.

Ein Ziel für Refactoring. Software, die lange verwendet wird, hat das Problem, daß sich mit der Zeit die Anforderungen, die sie zu erfüllen hat, ändern. Oft werden zusätzliche Anforderungen durch Erweiterungen der Software befriedigt. Ungeplante Erweiterungen führen aber zu einem uneinheitlichen und unflexiblen System, das für spätere Anpassungen ungeeignet ist. Um die Software weiter zu entwickeln, muß sie umorganisiert werden. Dieser Prozeß heißt *Refactoring* (siehe Abschnitt 1.5).

Design Patterns beschreiben Strukturen, die das Resultat von Refactoring sind. Dadurch zeigen Design Patterns einen Weg für die Umwandlung eines unflexiblen Systems in ein flexibles, auch zukünftigen Anforderungen gewachsenes System vor. Geht man noch einen Schritt weiter, und verwendet Patterns im Design von neuer Software, wird der Bedarf für Refactoring zu einem späteren Zeitpunkt verringert oder sogar ganz vermieden.

Marschall Cline [Cline, 1996] beschreibt neben den Vorteilen von Design Patterns auch folgende Problembereiche:

Überbewertung des Ansatzes. Der Anwendungsbereich, in dem Design Patterns effizient eingesetzt werden können, ist schwer zu erkennen. Durch die Verwendung von Design Patterns entstehen zusätzliche Kosten (Lernaufwand, Aufwand bei der Erstellung neuer Patterns, ...), die auch gerechtfertigt sein müssen. Design Patterns ersparen hauptsächlich dann Kosten, wenn die Software später verändert, angepaßt oder wiederverwendet werden soll. Ist dies nicht der Fall, bringt die Verwendung von Patterns nicht alle erwarteten Vorteile und der Einsatz muß genau überlegt werden.

Einige Design Patterns sind unnötig schwer zu erlernen. Da viele Designer Patterns entwickeln oder Patterns von anderen verändern, geht oft die Klarheit des Designs verloren. Weiters werden durch einen zu geringen Abstraktionsgrad unnötigerweise zu viele Details berücksichtigt. Daraus entstehen zu komplexe Patterns, die schwer verständlich und nur sehr beschränkt anwendbar sind.

Noch unbrauchbare Kategorisierungen von Design Patterns. Design Patterns werden nach verschiedenen Kriterien, wie Zweck oder Abstraktionsgrad, eingeteilt. Diese Einteilung hilft Personen, die bereits Design Patterns verstehen. Jedoch sind viele dieser Kriterien für den durchschnittlichen Designer, der gerade Patterns lernt, nicht hilfreich.

Weiters wurden von Vlissides [Vlissides, 1998, S.3–10] einige weit verbreitete Mißverständnisse zum Thema Design Patterns beschrieben. Die Mißverständnisse betreffen einerseits das Wesen von Design Patterns und andererseits ihr Leistungsvermögen. Einige dieser Mißverständnisse sind:

Patterns benötigen Tools und Methoden, um effektiv zu sein. Der Nutzen von Patterns kommt aus ihrer Anwendung. Das Erlernen der abstrakten Konzepte, die den Patterns zugrunde liegen, und der Art, wie sie Probleme lösen, ist ihr größter Nutzen. Die Verwendung von Tools zur automatischen Umsetzung in Code ist nur bei Patterns mit relativ geringem Abstraktionsgrad sinnvoll möglich und bringt daher für die meisten Design Patterns nur sehr geringen Nutzen.

Patterns garantieren wiederverwendbare Software, hohe Produktivität etc. Alleine die Verwendung von Patterns sichert nicht den Erfolg eines Projekts. Der kreative Prozeß des Designs von Systemen ist weiterhin vom Entwickler abhängig. Patterns können nur das Problemverständnis des Entwicklers unterstützen und gute Beispiele für die Problemlösung geben.

Patterns sind nur für Design und Implementierung von Software verwendbar. Patterns sind niedergeschriebenes Expertenwissen. Sie sind nicht auf bestimmte Bereiche beschränkt. Expertenwissen wurde und wird in vielen Bereichen in Form von Patterns niedergeschrieben. Beispiele sind, wie in der Einführung bereits erwähnt, Architektur [Alexander *et al.*, 1977], Strategie und Kriegsführung [Tzu, 1983], Jazzmusik [Coker *et al.*, 1970], das Management von Organisationen [Coplien, 1995], das Verständnis von kognitiven Prozessen [Gardner *et al.*, 1998] und auch die Entwicklung von wiederverwendbarer Software [Foote und Opdyke, 1995].

Die beschriebenen Probleme und Mißverständnisse sind großteils darauf zurückzuführen, daß der Pattern-Ansatz im ersten Moment simpel erscheint. Jedoch liegen zwischen dem Verstehen des Konzepts von Patterns und seiner erfolgreichen Anwendung viele Hürden. Der Entwickler von Patterns benötigt ausgezeichnete Kenntnisse des Problemraums, aus dem das Pattern stammt. Er benötigt die Fähigkeit, diese Kenntnisse so zu abstrahieren, daß eine wiederverwendbare Lösung entsteht. Auch an den Benutzer von Patterns werden hohe Anforderungen gestellt. Er muß genügend Patterns kennen, um das richtige Pattern für sein Problem auswählen zu können. Dabei reicht es nicht aus, nur die Namen verschiedener Patterns zu kennen, sondern er muß das abstrakte Modell dahinter verstehen. Erst dadurch wird es möglich, zu erkennen, wo und wie das Pattern eingesetzt werden kann. Diese relativ hohen Anforderungen erschweren den Einsatz von Patterns.

Kapitel 4

Analyse Patterns

Nachdem die Vorteile der Verwendung von Patterns in der Designphase breite Akzeptanz finden, stellt sich die Frage, ob sich Patterns auch für die Analysephase eignen, und welchen Nutzen sie in dieser Phase bieten können.

In Abschnitt 4.1 wird ein Literaturüberblick zu Patterns in der Analysephase gegeben. Bisher wurden ausschließlich Beispiele für Analyse Patterns veröffentlicht, auf Analyse Patterns als eigener Ansatz wurde noch nicht ausführlich eingegangen. Daher wird im Rest dieses Kapitels versucht, diesen Ansatz anhand der Erkenntnisse aus dem Software-Lebenszyklus und aus dem Pattern-Ansatz klar darzustellen.

In Abschnitt 4.2 wird der breite Anwendungsbereich von Analyse Patterns dargestellt. Analyse Patterns unterstützen die Softwareentwicklung vom Erkennen des realen Problems, über die Analyse, bis hin zur Umsetzung in ein Designmodell.

Wie für Design Patterns ist auch für Analyse Patterns die geordnete Aufzeichnung von zentraler Bedeutung. Daher wird in Abschnitt 4.3 ein geeignetes Formular für Analyse Patterns entwickelt, und seine Verwendung wird an einem Beispiel verdeutlicht.

In Abschnitt 4.4 werden Analyse Patterns von Design Patterns abgegrenzt. Analyse Patterns und Design Patterns sind zwar ähnlich, da beide auf dem Pattern-Ansatz aufbauen, doch ist ihr Anwendungsbereich in der Softwareentwicklung unterschiedlich, da ihre Schwerpunkte in verschiedenen Phasen liegen.

Zum Abschluß werden in Abschnitt 4.5 der Nutzen und die Wirkung von Analyse Patterns untersucht. Dabei wird auf die Hauptvorteile von Analyse Patterns, nämlich die Beschleunigung des Softwareentwicklungsprozesses und die bessere Berücksichtigung von sozialen Aspekten in der Entwicklung, eingegangen.

4.1 Literaturüberblick zu Patterns in der Analysephase

Das Thema Analyse Patterns wurde als eigener Ansatz in der Literatur noch nicht ausreichend behandelt. Bisher wurden nur einige Beispiele von Analyse Patterns publiziert. Das erste Buch zu diesem Thema, *Analysis Patterns: Reusable Object Models* [Fowler, 1997], stammt von Martin Fowler. Er definiert darin Analyse Patterns folgendermaßen [Fowler, 1997, S.8]:

Analysis patterns are groups of concepts that represent a common construction in business modeling. It may be relevant to only one domain, or it may span many domains.

Fowler beschreibt in diesem Buch eine Reihe von Analyse Patterns zu Bereichen, wie beispielsweise *Accountability, Observations and Measurements, Inventory and Accounting*, die er im Laufe seiner Tätigkeit als Berater für verschiedene Projekte entwickelt hat. Für jeden Bereich präsentiert er Diagramme, die Objekte aus dem Problemraum und deren Beziehungen untereinander abbilden. Er beschreibt die Funktionsweise der Diagramme informal und gibt Beispiele zur Illustration an.

Ähnliche Patterns, wie Fowler, beschreibt David Hay im Buch *Data Model Patterns* [Hay, 1996]. Er verwendet Datenmodelle in Form von *Entity-Relationship Modellen*, deren Zweck er, wie Fowler, informal beschreibt. Er schreibt über die Patterns in seinem Buch [Hay, 1996, S.4]:

This book is about "conventions of thought." It identifies common situations that are present in a variety of businesses and government agencies, and which can be modeled in a standardized way.

Einen weiteren interessanten Ansatz für die Verwendung von Patterns in der Analysephase verwenden Peter Coad, David North und Mark Maryfield in ihrem Buch *Object Models: Strategies, Patterns, and Applications* [Coad et al., 1997]. Sie entwickeln anhand von fünf Fallstudien Objektmodelle und beschreiben diese Entwicklung in einer strukturierten Form. Sie beginnen mit der Identifikation des Zwecks und der Eigenschaften des Systems, führen eine Auswahl der Objekte im Problemraum durch und analysieren ihre Beziehungen. Im Laufe dieser Fallstudien werden verschiedene Strategien und Patterns aufgedeckt. Strategien (*Strategies*) sind Vorgehensweisen für den Entwicklungsprozeß, zum Beispiel wie bei der Identifikation des Zwecks vorgegangen werden soll. Die gefundenen

Patterns beschreiben Beziehungen zwischen Objekten im Problemraum, wie sie typischerweise vorkommen. Für die Beschreibung wird eine Struktur mit grafischer Notation und Feldern für *Typical Object Interactions*, *Examples* und *Combinations* verwendet. Ein Beispiel ist das *Participant-Transaction Pattern*. Es beschreibt die Beziehung und typische Interaktionen zwischen einem Teilnehmer (*Participant*) und einer Transaktion.

Fernandez [Fernandez, 1998] betrachtet Analyse Patterns mehr von der technischen Seite. Er schreibt in seinem Kurzbeitrag zu einem Workshop für Softwarearchitektur:

An analysis pattern is a set of classes and associations that have some meaning in the context of an application; that is, it is a conceptual model of a part of the application. However, the same structure may be valid for other applications, and this is the aspect that makes them very valuable for reuse and composition.

Patterns in der Analysephase beinhalten Analysemodelle, also abstrakte Modelle von Beziehungen zwischen Objekten im Problemraum. Patterns zeichnen sich, aufgrund der Abstraktion, durch ihre Wiederverwendbarkeit aus. Daher können verschiedene reale Probleme mit den gleichen abstrakten Modellen eines Analyse Patterns beschrieben werden. Beispielsweise kann das Analyse Pattern *Accountability* von Fowler nicht nur für die Struktur der Verantwortungsverhältnisse in einer Organisation verwendet werden, sondern es kann unverändert ebenso für die Abbildung des Verhältnisses zwischen Arzt und Patient eingesetzt werden (siehe [Fowler, 1997, S.24–33]).

Beispiele für Analyse Patterns werden unter anderem bei den *Pattern Languages of Programs* Konferenzen [PLoP, 2000] vorgestellt. Gaertner und Thirion [Gaertner und Thirion, 1999] beschreiben *Grafcet*, eine Notationsform für ereignisgesteuerte Echtzeitsysteme in Form eines Analyse Patterns. Fernandez und Yuan [Fernandez und Yuan, 1999] präsentieren ein Analyse Pattern für die Reservierung von wiederverwendbaren Einheiten, wie beispielsweise Hotelzimmer oder Mietautos. Im Gegensatz zu der unstrukturierten Beschreibungsform von Fowler und Hay, verwenden beide Patterns ein Formular zur Strukturierung.

Die Literaturrecherche führte zu folgenden offenen Fragen:

1. Für welchen Bereich in der Softwareentwicklung können Analyse Patterns verwendet werden?
2. Was soll ein Formular für Analyse Patterns beinhalten, um den Anwendungsbereich zu unterstützen?

3. Wie unterscheiden sich Analyse Patterns von Design Patterns?
4. Welchen Nutzen bringt der Einsatz von Analyse Patterns?

Auf diese Fragen wird in den folgenden Abschnitten eingegangen.

4.2 Anwendungsbereich von Analyse Patterns

Abbildung 4.1 zeigt den Anwendungsbereich von Analyse Patterns. Analyse Patterns unterstützen hauptsächlich folgende zwei Vorgänge des Software-Lebenszyklus:

- **Analyse.** Das Erkennen der Zusammenhänge des realen Problems und ihre Abbildung in einem abstrakten Analysemodell.
- **Transformation.** Der Übergang vom Problemraum in den Lösungsraum, also die Transformation des Analysemodells in ein Designmodell.

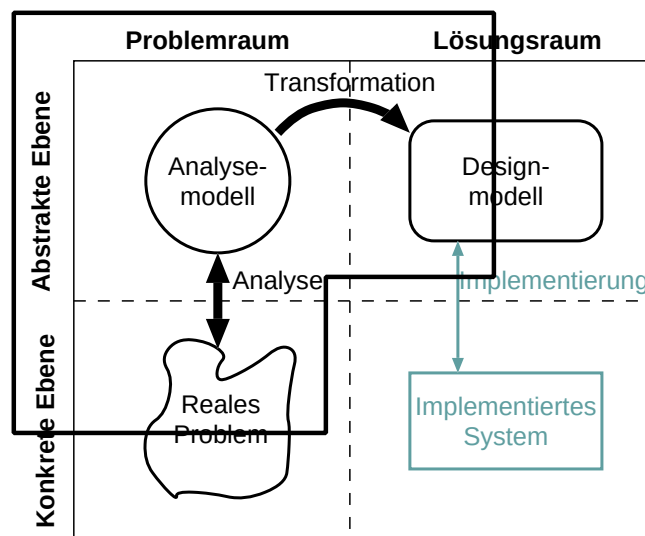


Abbildung 4.1: Einsatzbereich von Analyse Patterns

In der Analysephase wird ein abstraktes Modell im Problemraum (das Analysemodell) erstellt. Im ersten Schritt werden Ziele als Anforderungen an das System modelliert und als *Requirements Model* dokumentiert. Zum Beispiel durch *Use Cases* in OOSE [Jacobson, 1992, S.126–130]. Parallel dazu müssen die Zusammenhänge zwischen den realen Objekten im

Problemraum geklärt werden. Diese Zusammenhänge werden im Analysemodell, zum Beispiel mit dem *Object Model*, dem *Dynamic Model* und dem *Functional Model* aus OMT [Rumbaugh *et al.*, 1991, Kap.8], dargestellt. Wie in Abschnitt 1.2 beschrieben, ist Modellierung ein komplexer Vorgang, in dem Erfahrung sehr wichtig ist. Mit der Erstellung der abstrakten Modelle im Problemraum ist die Analysephase abgeschlossen, doch bereitet der Übergang von der objekt-orientierten Analyse zum Design weitere Probleme (siehe Abschnitt 1.3). Fehler in der Modellierung können hohe Kosten und große Verzögerungen verursachen.

Analyse Patterns, wie sie hier verwendet werden, sprechen explizit die Probleme der Modellbildung und der Transformation an. Dies geschieht durch die Beschreibung von Expertenwissen in Form von abstrakten Modellen mit konkreten Beispielen. Dabei helfen die konkreten Beispiele erstens beim Verständnis des abstrakten Modells und zweitens bei der Umsetzung dieses Modells in eine konkrete Lösung.

4.3 Aufbau von Analyse Patterns

Fowler [Fowler, 1997] und Hay [Hay, 1996] beschreiben ihre Analyse Patterns, ohne eine vorgegebene Struktur einzuhalten. Dennoch gilt für Analyse Patterns, genauso wie für Design Patterns, daß sie in einer Form präsentiert werden sollen, die schnell erfaßbar und leicht vergleichbar ist [Vlissides, 1998, S.147–148] (siehe Abschnitt 3.2 für die Struktur von Design Patterns). Daher bietet sich auch für Analyse Patterns eine strukturierte Beschreibung in Form eines Formulars an.

Folgendes Formular wird für Analyse Patterns vorgeschlagen. Die Herkunft der einzelnen Abschnitte ist jeweils in Klammer angegeben:

Pattern Name [Gamma *et al.*, 1995, Buschmann *et al.*, 1996] Der Name des Patterns drückt die wesentlichen Aspekte des Patterns aus. Er wird Teil der Fachsprache in der Analysephase.

Also known as [Gamma *et al.*, 1995, Buschmann *et al.*, 1996] Weitere Synonyme für dieses Pattern.

Intent [Gamma *et al.*, 1995] Was leistet das Pattern und welches Problem löst es? Eine sehr kurze, abstrakte Darstellung des Problems.

Motivation [Gamma *et al.*, 1995] Ein Beispiel, das die Wirkung des Patterns anhand eines konkreten Problems zeigt. Es handelt sich um ein Szenario, in dem die wirkenden Kräfte und die Lösungskraft des Patterns illustriert werden. Es unterstützt das Verständnis der abstrakten Darstellung des Problems und der Lösung in den Abschnitten *Intent*, *Forces and Context* und *Solution*. Außerdem dient es als Beispiel, wie das Pattern konkret angewandt werden kann.

Forces and Context [Alexander, 1979] Eine Aufzählung und Diskussion der Kräfte, die im Problemraum wirken. Die Aufgabe des Patterns ist es, diese Kräfte in ein harmonisches Gleichgewicht zu bringen. Nur dadurch entsteht eine Lösung, die negative Konsequenzen minimiert.

Solution [Buschmann *et al.*, 1996] Beschreibung der abstrakten Lösung. Die Lösung ist ein Analysemodell, das alle relevanten strukturellen Aspekte sowie das Verhalten beschreibt. Außerdem muß in diesem Abschnitt klar gezeigt werden, wie das Pattern die wirkenden Kräfte ins Gleichgewicht bringt.

Consequences [Gamma *et al.*, 1995, Buschmann *et al.*, 1996] Beschreibung der Konsequenzen, die durch die Anwendung des Patterns entstehen. Es müssen alle bekannten Vorteile und negativen Konsequenzen beschrieben werden. Konsequenzen können technischer, wirtschaftlicher, organisatorischer oder sozialer Natur sein.

Design [Neu] Dieser Abschnitt ist neu. Er zeigt, wie das im Pattern vorgeschlagene Analysemodell beispielhaft in ein Design umgesetzt werden kann. Es handelt sich dabei um Designvorschläge (zum Beispiel die Verwendung von Design Patterns) und entdeckte Fallen (zum Beispiel die Gefahr des Auftretens von AntiPatterns).

Known Uses [Gamma *et al.*, 1995, Buschmann *et al.*, 1996] Beispiele von Systemen, in denen das Pattern entdeckt oder bereits erfolgreich angewandt wurde. Diese Beispiele können auch zu Case Studies erweitert werden, damit die genaue Wirkung des Analyse Patterns leichter erkennbar wird.

Als Beispiel für die Verwendung dieses Formulars werden in Kapitel 5 vier aufeinander aufbauende Analyse Patterns für das Management von ver-

teilter Information in Organisationen dargestellt. Das einfachste Pattern, die *Einfache Pinwand*, wird hier kurz zusammengefaßt:

Pattern Name Einfache Pinwand

Intent Die *Einfache Pinwand* dient zur effizienten Verteilung von Information innerhalb von Gruppen und zum Aufbau einer gemeinsamen Wissensbasis.

Motivation Die Forschungsgruppe eines multinationalen Unternehmens ist auf mehrere Niederlassungen verteilt. Der auftretende Kommunikationsbedarf kann durch bestehende Kommunikationsmedien, wie beispielsweise persönliche Treffen, Post, Telefon, Fax oder E-Mail nur unzureichend gedeckt werden. Regelmäßige, direkte und synchrone Kommunikation (Treffen, Telefon, ...) ist in diesem Szenario durch zu hohe Reisekosten und Zeitunterschiede unmöglich. Asynchrone Kommunikation (E-Mail, Fax, ...) kann dagegen zu Informationsüberlastung der Gruppenmitglieder führen. Weiters bedingt diese Kommunikationsart Ineffizienz, da jedes Gruppenmitglied alle verteilten Informationen sammeln und ordnen muß, damit diese, falls benötigt, verwendet werden können.

Forces and Context Aus dem Beispiel können folgende Kräften identifiziert werden:

- Eine effiziente 1:n Kommunikation innerhalb einer Gruppe ist notwendig.
- Persönliche, direkte Kommunikation ist nicht möglich.
- Das Risiko von Informationsüberlastung besteht.
- Eine gemeinsame Wissensbasis wird benötigt.

Solution Als Lösung wird eine Pinwand vorgeschlagen. Diese Pinwand stellt ein Informationssystem dar, das wie ein Schwarzes Brett funktioniert. Man kann Nachrichten auf der Pinwand anbringen und bereits vorhandene Nachrichten lesen.

Consequences Daraus ergeben sich mehrere Konsequenzen:

- Die Kommunikation ist asynchron. Die Arbeit wird nicht durch Telefonate unterbrochen, und Probleme mit verschiedenen Zeitzonen treten nicht auf.
- Die Informationsüberlastung wird reduziert. Der Benutzer sucht sich die benötigte Information bei Bedarf. Er wird nicht ständig mit Information, die er gerade nicht benötigt, konfrontiert.
- Durch die Pinwand entsteht automatisch eine gemeinsame Wissensbasis als Summe aller Nachrichten.

Diese Konsequenzen haben soziale Auswirkungen, da die Reduktion von Informationsüberlastung die Akzeptanz des Informationssystems erhöht. Weiters hat eine Pinwand wirtschaftliche Auswirkungen in Form einer Kostenreduktion aufgrund des geringeren Zeitverlusts für Kommunikation (weniger Unterbrechungen, individuelles Sammeln von Nachrichten entfällt). Und schließlich wird mit der gemeinsamen Wissensbasis ein Wert für die Organisation geschaffen.

Design Für ein flexibles Design der Pinwand gibt es einige wichtige Punkte. Eine Pinwand besteht aus drei Hauptkomponenten, der Datenbasis zur Speicherung der Nachrichten, dem Interface zur Darstellung der Nachrichten und der Steuereinheit, die alle Abläufe der Pinwand koordiniert. Diese Komponenten sollten auch im Design und in der Implementierung klar getrennt sein, um später Anpassungen des Systems zu erleichtern. Weiters gibt es für die einzelnen Komponenten Vorschläge für Design Patterns, beispielsweise kann das *Command Processor Pattern* [Sommerlad, 1996] für die Steuereinheit eingesetzt werden, um sie flexibel und erweiterbar zu gestalten.

Known Uses Gefunden wurde dieses Analyse Pattern in Anwendungen wie Newsgroups, Schwarze Bretter von Groupware Anwendungen und Gästebüchern auf Web-Seiten.

Das hier kurz vorgestellte Analyse Pattern wird gemeinsam mit drei weiteren Patterns (*Pinwand mit strukturierten Nachrichten*, *Virtuelle Bibliothek* und *Virtuelle Bibliothek mit aktiven Agenten*) in Kapitel 5 beschrieben. Teil II dieser Arbeit enthält eine Case Study für diese Analyse Patterns. Die Patterns werden zur Entwicklung eines wissenschaftlichen Informationsbrokers verwendet, und die Auswirkungen der Anwendung der

Patterns werden analysiert. Diese Case Study könnte auch als Beispiel in dem Abschnitt *Known Uses* das Analyse Patterns angegeben werden.

4.4 Abgrenzung zu Design Patterns

Obwohl die eben beschriebene Struktur großteils aus Abschnitten besteht, die von Gamma et al. [Gamma *et al.*, 1995, S.6–8] für Design Patterns verwendet wurden, gibt es fundamentale Unterschiede zwischen Analyse Patterns und Design Patterns. Design Patterns bieten die Möglichkeit zur Wiederverwendung von Designmodellen, Analyse Patterns dagegen unterstützen Wiederverwendung im Bereich der Analyse. Das heißt, es werden abstrakte Analysemodelle für unterschiedliche reale Probleme wiederverwendet. Dadurch wird die für die Analyse benötigte Zeit reduziert, und aufgrund der Erfahrung aus der Wiederverwendung eines Analysemodells können (technische, soziale, organisatorische und wirtschaftliche) Konsequenzen besser abgeschätzt werden. Um Wiederverwendung auf der Analyseebene zu ermöglichen, wurden die Bedeutung und der Inhalt der von Design Patterns bekannten Abschnitte an die Bedürfnisse der Analysephase angepaßt, und die Beschreibung wurde um neue Abschnitte, wie *Forces and Context*, *Solution* und *Design*, erweitert. Die wichtigsten Veränderungen und Erweiterungen sind:

- Der zusätzliche Abschnitt *Forces and Context* spiegelt den Gedanken Alexanders [Alexander, 1979, S.41–54] wider, daß ein Pattern ein harmonisches Zusammenwirken verschiedener Kräfte erreichen soll. Die Kräfte und ihr Umfeld werden in diesem Abschnitt identifiziert, und ihre Wirkung wird erläutert.
- *Solution* wurde von Buschmann et al. [Buschmann *et al.*, 1996, S.19–21] übernommen und beinhaltet die abstrakte Lösung des in *Intent* dargestellten Problems. Diese Lösung ist eine Abstraktion der in *Motivation* beschriebenen konkreten Lösung.
- Der Abschnitt *Consequences* wurde bereits von Gamma et al. [Gamma *et al.*, 1995, S.6–8] für Design Patterns verwendet, sein Inhalt hat sich jedoch bei Analyse Patterns so stark verändert, daß darauf besonders hingewiesen werden muß. Da sich die Design Patterns von Gamma et al. auf die Designphase beziehen, werden in den *Consequences* ausschließlich technische Aspekte berücksichtigt. Bei Analyse Patterns wird dieser Abschnitt jedoch hauptsächlich für wirtschaftliche, organisatorische und soziale Aspekte verwendet.

- Wirtschaftliche Aspekte gehen auf den direkten Nutzen für die Organisation, meist durch Kostenreduktion, ein.
 - Organisatorische Aspekte beschäftigen sich mit Innovationen und der Veränderung von Abläufen (Geschäftsprozessen) in der Organisation. Der damit verbundene organisatorische Wandel (siehe [Rogers, 1995], [Davenport, 1993], [Harrington, 1991]) hat meist auch starke wirtschaftliche und soziale Aspekte.
 - Die sozialen Aspekte sind für Analyse Patterns der wichtigste Punkt. Alexander et al. gehen in ihrem Buch *A Pattern Language: Towns, Buildings, Construction* [Alexander et al., 1977] regelmäßig auf soziale Aspekte von Patterns ein. Diese bestimmen, ob und wie stark die Verwendung eines Patterns, zum Beispiel die Verwendung unterschiedlicher Analyse Patterns für ein einzuführendes Informationssystem, auf Widerstand durch die Benutzer stößt, und welche Gegenmaßnahmen ergriffen werden können.
- Der Abschnitt *Design* ist ein gänzlich neuer Abschnitt. In diesem Abschnitt werden Vorschläge zur Umsetzung des Analysemodells in ein Design gegeben, und es werden Probleme und Fallen aufgezeigt. Der Abschnitt enthält beispielsweise Design Patterns oder AntiPatterns, die zu den im Pattern beschriebene Lösungen passen. Das Ziel dieses Abschnittes ist, durch Designbeispiele die benötigte Zeit in der Designphase zu reduzieren und die Probleme mit der Transformation zu verringern.

Durch diese Veränderungen kann mit Analyse Patterns die Softwareentwicklung vom Abstraktionsprozeß in der Analyse bis hin zur Transformation in ein umsetzbares Designmodell unterstützt werden. In Abbildung 4.2 wird die Zuordnung der einzelnen Abschnitte eines Analyse Patterns zu den Vorgängen im Lebenszyklus bei der Softwareentwicklung dargestellt. Die Vorgänge sind:

1. Die Abschnitte *Motivation* und *Known Uses* sind konkrete Beispiele für die Verwendung des Analyse Patterns zur Lösung eines realen Problems. Sie helfen beim Verständnis der abstrakten Eigenschaften des Patterns und seines Problembezugs.
2. *Intent, Forces and Context* und *Consequences* beinhalten abstrakte Beschreibungen des Zwecks, der im Problemraum wirkenden Kräfte und der durch das Pattern entstehenden Konsequenzen. Diese Beschreibungen werden für die Auswahl eines geeigneten Analyse Patterns verwendet. Damit diese Auswahl getroffen werden kann, muß

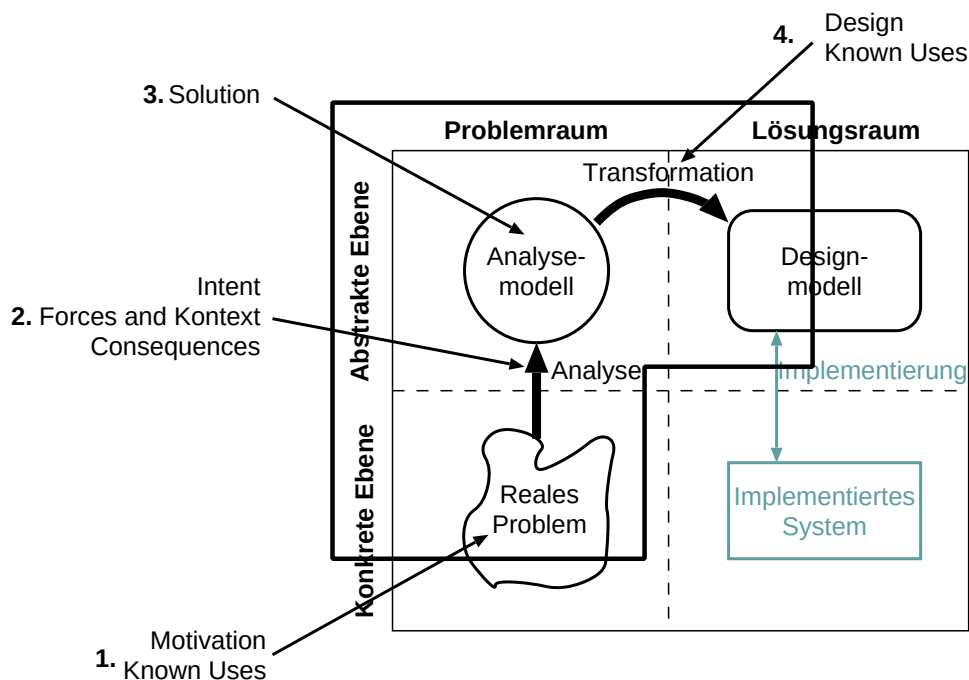


Abbildung 4.2: Die Verwendung der einzelnen Abschnitte von Analyse Patterns

zuerst der Zusammenhang zwischen dem realen Problem und den abstrakten Abschnitten *Intent* und *Forces and Context* des Patterns erkannt werden. Dabei helfen die Abschnitte *Motivation* und *Known Uses*. Danach kann anhand der im Abschnitt *Consequences* beschriebenen organisatorischen und sozialen Aspekte entschieden werden, ob das Pattern verwendet werden soll.

3. Der Abschnitt *Solution* stellt das abstrakte Analysemodell für das Pattern dar. Es wird als Teil des Gesamtmodells in das Analysemodell des Systems integriert und beschreibt dort einen Teilaspekt des Systems.
4. Für die Transformation des abstrakten Analysemodells in ein abstraktes und implementierbares Designmodell wird der Abschnitt *Design* verwendet. Dieser Abschnitt beinhaltet Designvorschläge, die die Umsetzung erleichtern. Weiters kann die Untersuchung des Designs bereits existierender Software (aus Abschnitt *Known Uses*) wertvolle Hinweise für das Design liefern.

Analyse Patterns beinhalten in ihrer Beschreibung Unterstützung für den Gesamtprozess vom Erkennen des realen Problems bis zur Umsetzung in ein flexibles Design. Oft existieren zum vorgeschlagenen Design bereits

implementierte Teile, beispielsweise bei der Verwendung von Design Patterns. Dadurch kann der gesamte Entwicklungsprozeß durch die Wahl eines geeigneten Analyse Patterns massiv beschleunigt werden.

4.5 Nutzen und Wirkung von Analyse Patterns

Analyse Patterns sollen helfen, die benötigte Zeit und damit die Kosten für die Erstellung eines Softwareprodukts zu reduzieren. Verzögerungen in der Analysephase werden oft durch Probleme im Umfeld des Projekts bedingt. Solche Probleme wurden von DeMarco [DeMarco, 1979, S.9–13] aufgezeigt und in dieser Arbeit in Abschnitt 1.2 beschrieben. Diese Problembereiche sind:

- Das Kommunikationsproblem zwischen Systemanalytikern und Benutzern.
- Sich ständig ändernde Anforderungen an das System.
- Die Dokumentation des Ziels.
- Politik, der Umgang mit den sozialen Aspekten von Unsicherheit und Veränderung.

Folgende weitere problematische, beziehungsweise bisher in der Literatur nicht ausreichend behandelte Bereiche wurden in dieser Arbeit identifiziert:

- Problematisch sind die Umsetzung des realen Problems in das am besten geeignete abstrakte Analysemodell in der Analysephase (siehe Abschnitt 1.2) und die Transformation des Analysemodells in ein flexibles Design (siehe Abschnitt 1.3).
- Die explizite Wiederverwendung von Erfahrungen in der Analysephase wurde in der Literatur noch kaum beschrieben. Fowler [Fowler, 1997] und Hay [Hay, 1996] (siehe Abschnitt 4.1) präsentieren zwar kleine Beispiele für die Wiederverwendung von Analysemodellen, sie beschreiben diesen Vorgang aber nicht explizit.

Im folgenden wird die Wirkung von Analyse Patterns in diesen Problemreichen beschrieben, und darauf eingegangen, wie damit Verzögerungen in der Softwareentwicklung entgegengewirkt werden kann. Gemeinsam mit

Design Patterns haben Analyse Patterns folgende positive Eigenschaften in den Bereichen Kommunikation, Erlernen von Problemlösungen, Flexibilität und Wiederverwendung (siehe auch Abschnitt 3.6):

Ein gemeinsames Vokabular für die Kommunikation in der Analysephase. Die Namen der Analyse Patterns können für die effiziente Kommunikation zwischen mehreren Systemanalytikern verwendet werden. Keinen oder nur geringen Nutzen bringt dieses Vokabular jedoch bei der Kommunikation mit den zukünftigen Benutzern des Systems, da diese das Fachvokabular von Patterns nicht kennen. Das Kommunikationsproblem zwischen Systemanalytikern und Benutzern in der Analysephase [DeMarco, 1979, S.10] bleibt erhalten, jedoch helfen die Beispiele in den Abschnitten *Motivation* und *Known Uses* bei der Veranschaulichung, was die Verwendung des abstrakten Modells eines Analyse Patterns bedeutet.

Eine Dokumentations- und Lernhilfe. Hier liegen die Stärken von Patterns. Besteht ein komplexes Modell aus vielen Einzelteilen (Patterns), deren Zweck, Funktion und Eigenschaften bekannt sind, ist das Verständnis des gesamten Systems bei weitem einfacher, als würde man sich erst jeden Einzelteil überlegen müssen. Die explizite Kennzeichnung von Patterns bei der Dokumentation von Analysemodellen reduziert daher die benötigte Zeit, um das Modell zu verstehen, und man kann sich mehr auf die Zusammenhänge konzentrieren. Durch die Verwendung von Analyse Patterns, deren Zweck und Ziel im Abschnitt *Intent* genau beschrieben sind, wird das Problem der Dokumentation des Ziels [DeMarco, 1979, S.12] vermindert.

Flexibilität bei sich ständig ändernden Anforderungen an das System. Patterns beschreiben flexible, abstrakte Lösungen, die oft angewandt werden können. Aufgrund dieser Flexibilität kann bei kleineren Veränderungen in den Anforderungen, das abstrakte Analysemodell mit minimalen Anpassungen des Patterns weiter verwendet werden. Bei größeren Veränderungen können Analyse Pattern durch andere Patterns, welche die Anforderungen besser lösen, ersetzt werden.

Wiederverwendung von Teilen des Analysemodells als Erweiterung zu existierenden Methoden. Wie alle Patterns, bringen auch Analyse Patterns Erfahrungen von Experten in die Modellierung ein. Es werden die Gemeinsamkeiten der hilfreichen Teile von Analysemodellen schriftlich festgehalten, und diese können damit vom Entwickler, unabhängig von der verwendeten Entwicklungsmethode, wiederverwendet werden.

Zusätzlich zu diesen allgemein für Patterns gültigen Eigenschaften, weisen Analyse Patterns weitere positive Eigenschaften auf, die aus der in Abschnitt 4.3 entwickelten Struktur für ihre Beschreibung stammen. Vor allem durch die Abschnitte *Motivation*, *Consequences*, *Design* und *Known Uses* entstehen folgende Eigenschaften:

Beschleunigung des Softwareentwicklungsprozesses durch flexible Analysemodelle mit Designvorschlägen. Eine schwierige und daher zeitaufwendige Aufgabe bei der Softwareentwicklung ist die Modellbildung in der Analysephase. Modellbildung im allgemeinen ist ein komplizierter Abstraktionsvorgang, der nicht durch einfache Regeln formalisierbar ist. Hamdy A. Taha [Taha, 1982, S.4–6] stellt in seinem Einführungsbuch zu *Operations Research* fest, daß er zwar keine festen Regeln für die Bildung eines Modells geben kann, daß aber die Beschreibung von häufig vorkommenden Modelltypen mit ihrer allgemeinen Struktur und ihren Charakteristika hilfreich sein kann. Dies ist genau, was Analyse Patterns leisten sollen, häufig vorkommende Modelle vorzustellen (in Abschnitt *Solution*), und ihre Konsequenzen zu beschreiben.

Weiters bringt die Verwendung von Beispielen in Analyse Patterns Vorteile. John Vlissides [Vlissides, 1998, S.148–149] argumentiert, daß konkrete Beispiele für Patterns äußerst wichtig sind, da sie beim Verständnis ihrer abstrakten Eigenschaften helfen, und damit die Anwendung beschleunigen. Um dies zu unterstützen, werden die Lösungen in Analyse Patterns immer durch Beispiele in den Abschnitten *Motivation* und *Known Uses* konkretisiert.

Booch bedauert im Zusammenhang mit Komplexität von Software und Design, daß keine umfassenden Bibliotheken mit bewährten Abstraktionen existiert [Booch, 1994, Kap.1]. So eine Bibliothek könnte als umfassender Pattern-Katalog oder als vollständige Pattern-Sprache für Analyse Patterns gesehen werden (siehe Abschnitt 3.3 für Bestrebungen im Bereich von Design Patterns). Dadurch könnte der Abstraktionsprozeß eines konkreten Problems auf die Auswahl von geeigneten Analyse Patterns, ihre Anpassung und Zusammensetzung reduziert werden. Dies würde zu einer enormen Zeitersparnis in der Analysephase führen. Pattern-Kataloge können aber aufgrund der Komplexität, der in der Realität vorkommenden Probleme, nur einen Teil der benötigten Analyse eines Systems abdecken. Für diesen Teil tritt dann die erwähnte Zeit- und Kostenersparnis auf.

Ein weiterer zeitaufwendiger Bereich in der Softwareentwicklung ist die Transformation des Analysemodells in ein flexibles Designmodell. Analyse Patterns beschleunigen dies durch die Verwendung des Ab-

schnittes *Design* in ihrer Beschreibung. Dieser Abschnitt beinhaltet Designvorschläge (Design Patterns, AntiPatterns), für die in der Regel Implementierungsvorschläge existieren.

Wurde für ein Problem ein Analyse Pattern entwickelt, verkürzt sich die benötigte Zeit für die Softwareentwicklung massiv. Das Analyse Pattern muß nur noch ausgewählt werden. Es liefert das abstrakte Analysemodell und Designvorschläge, die wiederum Design Patterns mit Vorschlägen für die Implementierung enthalten können. In dieser Wiederverwendung in allen Phasen (Analyse, Design und Implementierung) steckt ein enormes Potential für Kosteneinsparungen, und es kann die Zeit bis zur Vermarktung des fertigen Softwareprodukts (*Time to market*) erheblich verkürzt werden.

Ein Ziel für die Verbesserung von Geschäftsprozessen unter Berücksichtigung von sozialen Aspekten. Analyse Patterns stellen regelmäßig vorkommende Teile von Analysemodellen dar, nämlich die Abstraktion von realen Objekten und ihrer typischen Beziehungen. Sie wurden von Experten bei der Analyse von vielen Systeme entwickelt und beinhalten daher die Abstraktion von regelmäßig vorkommenden konkreten Problemen (Geschäftsprozessen), die sich in der Praxis bewährt haben (*Best Practices*). Es liegt daher nahe, existierende Geschäftsprozesse (das konkrete Problem) so neu zu gestalten, daß sie auf ein Analyse Pattern (die Abstraktion der *Best Practices*) mit den wünschenswertesten Eigenschaften und Konsequenzen passen. Im Abschnitt *Consequences* von Analyse Patterns wird explizit neben technischen und wirtschaftlichen hauptsächlich auf organisatorische und soziale Konsequenzen des Patterns eingegangen. Dies hilft bei der Abschätzung, wie sich die organisatorischen Veränderungen durch die Verwendung des Patterns im sozialen Umfeld auswirken, und mit welchen Reaktionen durch die Betroffenen zu rechnen ist.

Abbildung 4.3 zeigt die Rückwirkung von Analyse Patterns auf das reale Problem. Den Vorgang der Abänderung von Geschäftsprozessen nennt man *Business Process Improvement* [Harrington, 1991]. Entsteht dabei ein vollkommen neuer Geschäftsprozeß, wird der Vorgang als *Process Innovation* [Davenport, 1993] bezeichnet. Innovationen führen zwangsläufig zu Veränderungen in der Organisation (*Organizational Change*) mit unter anderem auch sozialen Problemen. Damit diese Veränderungen durchgeführt werden können, muß der Innovationsprozeß verstanden und richtig betrieben werden (siehe [Rogers, 1995, Kap.10] [Davenport, 1993, S.167–198]). Durch fehlendes Problembewußtsein kann enormer wirtschaftlicher Schaden ver-

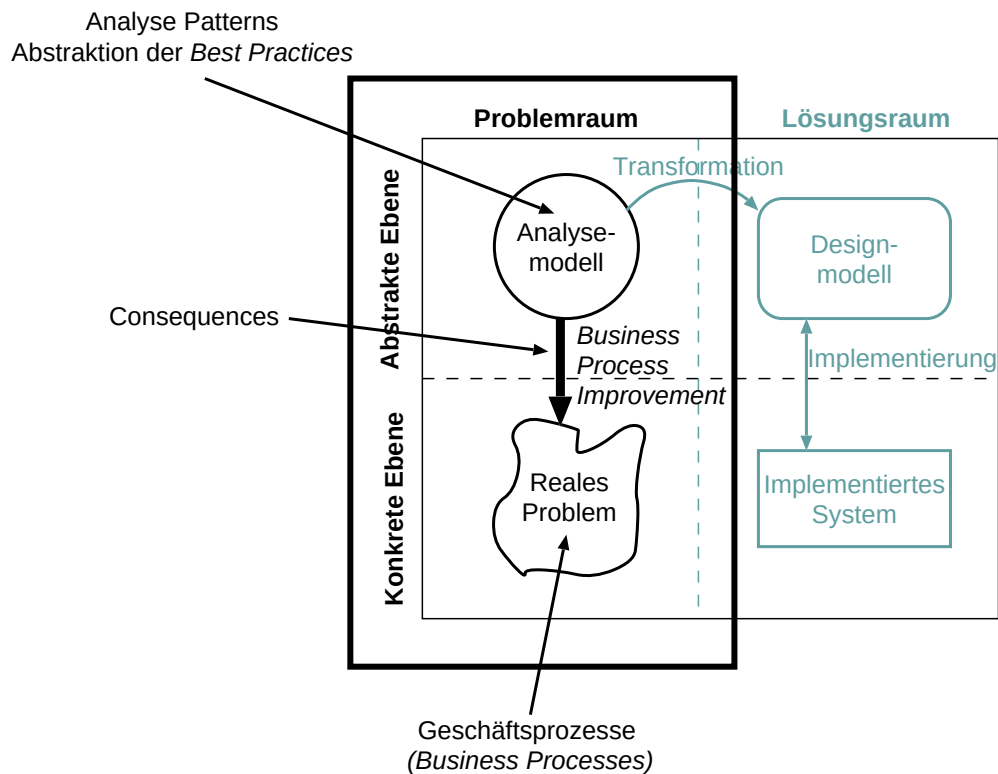


Abbildung 4.3: Business Process Improvement: Rückwirkung von Analyse Patterns auf das reale Problem

ursacht werden [Rogers, 1995, Kap.11]. Damit dies bei der Entwicklung von Informationssystemen verhindert wird, wird im Abschnitt *Consequences* jedes Analyse Patterns auf mögliche Problem aufgrund von organisatorischen Veränderungen eingegangen.

Beispiele für Analyse Patterns, die diesen Nutzen bieten, werden im folgenden Kapitel vorgestellt.

Kapitel 5

Beispiele: Analyse Patterns für Informationsmanagement

In diesem Kapitel werden Beispiele für den in Kapitel 4 entwickelten Ansatz gegeben. Dazu wird eine Familie von Analyse Patterns für das Management von Information in Gruppen entwickelt und mit dem Formular aus Abschnitt 4.3 beschrieben. Die hier dargestellten Analyse Patterns bauen aufeinander auf und wurden in ihren Grundzügen bereits in *Software Patterns: Pinwände* [Hahsler, 1997] vorgestellt und implementiert.

Das erste Pattern ist die *Einfache Pinwand* (Abschnitt 5.1). Einfache Pinwände, wie zum Beispiel das Schwarze Brett im Vorraum eines Universitätsinstitutes, werden oft als informale Kommunikationskanäle genutzt. Für kleine Gruppen, mit mäßigem Kommunikationsbedarf, stellt eine Pinwand ein effizientes Kommunikationsmittel dar.

Steigt die Gruppengröße und damit auch der Kommunikationsbedarf (Anzahl der Nachrichten auf der Pinwand), wird es für die Suchenden immer aufwendiger, die benötigten Nachrichten aus dem Pool aller Nachrichten herauszufinden. Die Weiterentwicklung der einfachen Pinwand zu einer *Pinwand mit strukturierten Nachrichten* (Abschnitt 5.2) reduziert dieses Problem der Skalierbarkeit. Durch eine festgelegte Struktur für alle Nachrichten (zum Beispiel Einführung einer Zeile Betreff für jede Nachricht), muß der Suchende nicht mehr die gesamte Nachricht lesen, um eine Auswahl treffen zu können.

Oft ist es notwendig, die Information an der Pinwand regelmäßig zu aktualisieren. Ist es aber sehr aufwendig, diese Aktualisierung durchzuführen (zum Beispiel existieren mehrere Pinwände parallel), entstehen Zeitverzögerungen und Inkonsistenzen. Diese Probleme werden durch das Pattern *Virtuelle Bibliothek* (Abschnitt 5.3) angesprochen. Durch die Verwendung von verteilten Informationsobjekten wird die Verteilung der

geänderten Information unnötig. Ein Beispiel aus dem Universitätsbereich ist die Verteilung der aktuellen Information einzelner Institute. Jedes Institut hat sein eigenes Schwarzes Brett für die aktuellen Termine im eigenen Institutsbereich. Die Universität selbst bietet nur mehr ein zentrales Verzeichnis der Institute mit dem Ort, wo sich die aktuellen Informationen befinden, an. Durch Informationstechnologie, wie das Internet, können verteilte Informationsobjekte (zum Beispiel Informationsseiten der Institute) einfach durch ein zentrales Verzeichnis, einer virtuellen Bibliothek, zugänglich gemacht werden.

Die Verteilung von Information bringt aber auch eine Menge von Problemen mit sich. Beispielsweise ist die Verfügbarkeit und Konsistenz problematisch. Was passiert, wenn ein Institut in ein anderes Gebäude umzieht, beziehungsweise wenn sich die Internetadresse der Web-Seite mit der Information ändert? Durch diese Probleme entstehen Kosten bei der Administration der zentralen virtuellen Bibliothek, die mit der Anzahl der verteilten Informationsobjekte steigen. Wie diese und weitere Kosten durch den Einsatz von Agententechnologie reduziert werden können, zeigt das letzte hier vorgestellte Analyse Pattern, die *Virtuelle Bibliothek mit aktiven Agenten* (Abschnitt 5.4).

5.1 Die Einfache Pinwand

Pattern Name Einfache Pinwand

Also Known As -

Intent Wie kann eine Gruppe Information effizient teilen und eine gemeinsame Wissensbasis aufbauen?

Motivation Stellen Sie sich vor, Sie arbeiten in der Forschungsabteilung eines großen, multinationalen Unternehmens. Das Unternehmen hat mehrere Arbeitsgruppen, die sich jeweils aus Forschern in verschiedenen Ländern zusammensetzen. Regelmäßige Treffen zwischen den Mitgliedern einer Arbeitsgruppe sind aufgrund der Distanz und der Abstimmung der Termine nicht möglich. Für die Zusammenarbeit innerhalb der Arbeitsgruppen wird eine effiziente Methode für die Weitergabe und Speicherung von Information benötigt. Um Informationsüberlastung entgegenzuwirken, muß eine einfache Möglichkeit geboten werden, die für die Arbeit benötigte Information schnell aufzufinden. Die wichtigsten Punkte sind daher asynchrone 1:n Kommunikation und die Möglichkeit zum Aufbau einer gemeinsamen Wissensbasis. Leider weisen bestehende Kommunikationsmedien, wie Post, Telefon (mit Konferenzschaltung), Fax oder E-Mail (mit Verteilerliste) diese Eigenschaften nicht in ausreichendem Maß auf.

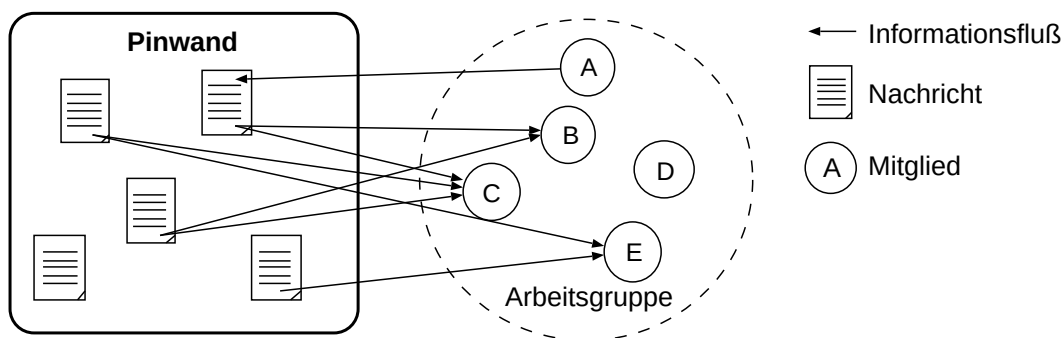


Abbildung 5.1: Beispiel einer einfachen Pinwand zur Kommunikation innerhalb einer Arbeitsgruppe

Die Arbeitsgruppen verwenden das in Abbildung 5.1 dargestellte Kommunikationssystem, das einem Schwarzen Brett entspricht. Wichtige Informationen werden von Gruppenmitgliedern in Form von Nachrichten auf der Pinwand für alle sichtbar angebracht. In Abbildung 5.1 schreibt das

Gruppenmitglied A eine Nachricht und die Mitglieder B, C und E lesen Nachrichten (jeweils symbolisiert durch Pfeile).

Forces and Context

- Effiziente 1:n Kommunikation innerhalb einer Gruppe ist notwendig.
- Direkte persönliche Kommunikation ist nicht möglich oder reicht nicht aus.
- Es besteht das Risiko von Informationsüberlastung.
- Wann und von wem Information benötigt wird, ist bei der Erstellung der Information nicht genau bekannt.
- Eine gemeinsame Wissensbasis wird benötigt.

Solution Verwenden Sie eine Pinwand für asynchrone 1:n Kommunikation innerhalb einer Gruppe. Die Pinwand speichert Nachrichten von Gruppenmitgliedern und macht sie für alle Mitglieder verfügbar. Die Nachrichten bestehen aus unstrukturiertem Text und bilden die gemeinsame Wissensbasis der Gruppe. Um benötigte Nachrichten zu finden, kann der Suchende den Text aller Nachrichten auf der Pinwand durchsuchen lassen. Abbildung 5.2 zeigt die Struktur der Pinwand.

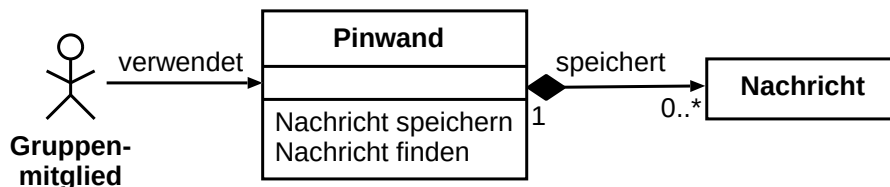


Abbildung 5.2: Klassendiagramm der Analyse einer einfachen Pinwand

Die Gruppenmitglieder arbeiten mit der Pinwand. Je nach Rolle, die sie bei einer Interaktion mit der Pinwand einnehmen, unterscheiden sich folgende Akteure:

1. Benutzer: Sucht und liest Nachrichten auf der Pinwand.
2. Informationsanbieter: Schreibt neue Nachrichten und speichert sie auf der Pinwand.
3. Administrator: Ist für den Betrieb der Pinwand verantwortlich. Er löscht zum Beispiel nicht mehr aktuelle Nachrichten.

Consequences Das Pattern bietet folgenden Nutzen und hat folgende Einschränkungen:

- **Asynchrone Kommunikation.** Die Arbeit wird nicht ständig durch eintreffende Telefonate oder E-Mails unterbrochen, und die Zusammenarbeit ist trotz Zeitverschiebung möglich.
- **Reduktion von Informationsüberlastung.** Der Benutzer sucht aktiv nach benötigten Informationen und wird nicht durch Unmengen an Informationen, wie zum Beispiel bei E-Mail Listen, überlastet.
- **Automatisches Entstehen einer gemeinsamen Wissensbasis.** Nachrichten können gesammelt und archiviert werden. Dadurch entsteht eine gemeinsame Wissensbasis (*Collective Dynabase*) [Press, 1992].
- **Vertrauliche Informationen.** Um vertrauliche Informationen über eine Pinwand zu verwalten, müssen Sicherheitsmaßnahmen für den Zugriff getroffen werden. Diese Maßnahmen können als Rechte einzelner Mitglieder oder als Recht von Teilgruppen zu einer Nachricht organisiert werden. Dazu muß ein geeignetes System zum Management von Rechten eingesetzt werden.
- **Lebenszeit von Nachrichten.** Was passiert, wenn der Inhalt einer Nachricht nicht mehr aktuell ist? Wer löscht oder archiviert diese Nachricht? Dies kann manuell durch den Administrator der Pinwand geschehen, bei großen Pinwänden mit vielen kurzlebigen Nachrichten entsteht dadurch aber ein enormer Aufwand. Ein pragmatischer Ansatz dafür ist, jeder Nachricht nur eine bestimmte Lebenszeit einzuräumen und sie nach Ablauf dieser Zeit automatisch zu entfernen oder zu archivieren.

Design Obwohl das Pinwand Pattern nicht komplex erscheint, sollte man sich beim Design doch einige Gedanken über Flexibilität, Erweiterbarkeit und Wiederverwendbarkeit machen. Zu bedenken ist:

1. **Trennung zwischen den Hauptkomponenten des Systems.** Wie im Klassendiagramm in Abbildung 5.3 dargestellt, besteht die Pinwand aus drei Hauptkomponenten. Diese sind die Datenbasis zur Speicherung der Nachrichten, das Interface für die Interaktionen mit den Benutzern und die Steuereinheit, die sämtliche Aktivitäten der Pinwand regelt. Die klare Trennung der Komponenten und die Festlegung ihrer Schnittstellen bedeuten zwar mehr Aufwand im Design und in der Implementierung, sie macht sich aber später

bei Änderungen und Erweiterungen des Systems bezahlt. Beispielsweise ist es durch die Trennung einfach möglich, ein proprietäres Interface durch ein Web-basiertes zu ersetzen, ohne den gesamten Programmcode zu verändern. Im *Model-View-Controller Pattern* [Buschmann *et al.*, 1996, S.125–143] für interaktive Systeme wird diese Eigenschaft als *Plugable Views and Controllers* beschrieben.

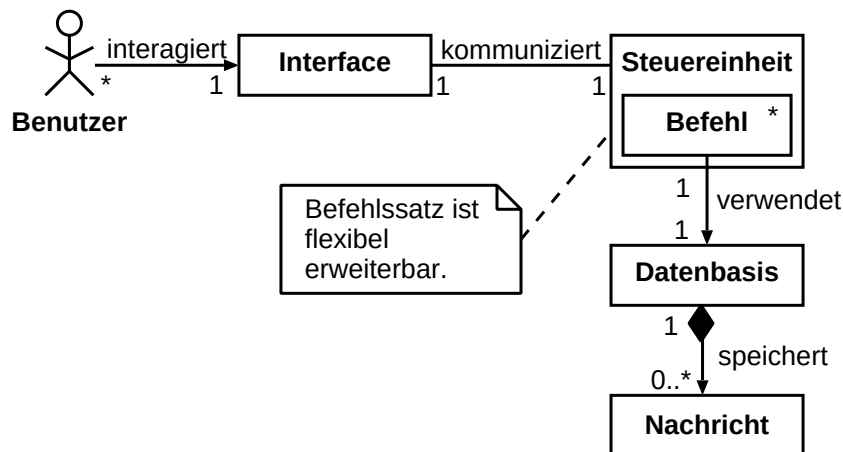


Abbildung 5.3: Klassendiagramm des Designs einer einfachen Pinwand

2. Erweiterbarkeit der Steuereinheit. Die Steuereinheit regelt alle Abläufe innerhalb der Pinwand. Neue Funktionen müssen einfach zu der Pinwand hinzugefügt werden können. Dies kann dadurch erreicht werden, daß die Steuereinheit eine erweiterbare Liste an Befehlen beinhaltet, die vom Benutzer über das Interface oder automatisch ausgelöst werden können. Ein Implementierungsvorschlag ist das *Command Processor Pattern* [Sommerlad, 1996].
3. Auswahl der Datenbasis. Die Möglichkeiten, Nachrichten zu speichern, reicht von der Speicherung als Dateien im Dateisystem bis zur Verwendung von Datenbank Management Systemen (objektorientiert oder relational). Die Auswahl sollte aufgrund von Leistungserfordernissen, Lizenzkosten und Implementierungsaufwand getroffen werden. Da die Datenbasis eine getrennte Komponente darstellt, kann sie nachträglich ausgetauscht werden. Als Anbindung unterschiedlicher Datenbasen an die Steuereinheit können beispielsweise das *Facade Pattern* [Gamma *et al.*, 1995, S.185–193] oder das *Adaptor Pattern* [Gamma *et al.*, 1995, S.139–150] eingesetzt werden.
4. Auswahl des Interfaces. Über das Interface, interagiert der Benutzer mit der Pinwand. Als Interface kann entweder eine selbst im-

plementierte Benutzeroberfläche verwenden werden, oder es kann ein standardisiertes Protokoll, wie das *Hypertext Transfer Protocol* (HTTP) [Fielding *et al.*, 1997] für den Einsatz von Web-Browsern unterstützt werden. Der große Vorteil bei der Verwendung von Web-Browsern als Interface ist ihre weite Verbreitung als grafische Benutzeroberfläche. Web-Browser sind für alle modernen Betriebssysteme verfügbar und werden meist mit den Betriebssystemen mitinstalliert. Außerdem sind die meisten Benutzer mit der Bedienung eines Web-Browsers bereits vertraut.

Known Uses

- Das *News Network Transfer Protocol* [Kantor und Lapsley, 1986], das für Newsgroups im Internet eingesetzt wird.
- Schwarze Bretter von Groupware Anwendungen, wie zum Beispiel im *BSCW*-System [Bentley *et al.*, 1997].
- *Guestbooks* auf Web-Seiten, dazu gibt es viele Implementierungen, wie zum Beispiel das System *Guestserver* [Ellingsen, 2000].

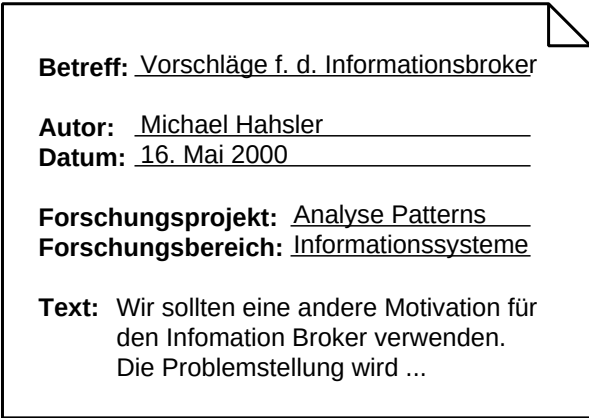
5.2 Die Pinwand mit strukturierten Nachrichten

Pattern Name Pinwand mit strukturierten Nachrichten.

Also Known As -

Intent Sammeln, Organisieren und Darstellen von strukturierter Information innerhalb einer Gruppe.

Motivation Ihre wissenschaftliche Arbeitsgruppe besteht aus sehr vielen Forschern und produziert daher eine große Anzahl an Nachrichten zu sehr unterschiedlichen Themen. Die Nachrichten enthalten häufig sehr viel Text. Das Sammeln von reinen Textnachrichten auf der Pinwand reicht nicht mehr aus, um benötigte Informationen schnell zu finden. Es wird notwendig, daß der Benutzer schnell entscheiden kann, ob eine Nachricht die von ihm benötigte Information enthält. Da auf einer einfachen Pinwand die Nachrichten aus unstrukturiertem Text bestehen, kann nur nach Schlagworten im Text gesucht werden. Die Suche wird zusätzlich dadurch erschwert, daß verschiedene Personen verschiedene Begriffe für dasselbe Konzept verwenden. Zur Verbesserung der Suchergebnisse können Techniken aus *Information Retrieval*, wie *Relevance Feedback* oder *Semantic Networks* verwendet werden (siehe [Grossman und Frieder, 1998, S.85]).



Betreff: Vorschläge f. d. Informationsbroker

Autor: Michael Hahsler

Datum: 16. Mai 2000

Forschungsprojekt: Analyse Patterns

Forschungsbereich: Informationssysteme

Text: Wir sollten eine andere Motivation für den Information Broker verwenden.
Die Problemstellung wird ...

Abbildung 5.4: Beispiel eines Formulars für strukturierte Nachrichten

Um die Suche zu vereinfachen, verwendet die Arbeitsgruppe das in Abbildung 5.4 dargestellte Formular für Nachrichten. Die Organisation der

Nachrichten nach bestimmten Kriterien, wie zum Beispiel nach Forschungsprojekten oder Forschungsbereichen, hilft, um die Suche auf eine kleinere Anzahl von Nachrichten einzuschränken. Damit kann die Suchzeit erheblich verkürzt werden. Es ist auch möglich für die Anzeige der Pinwand, Teile des Formulars auszublenden. Zum Beispiel wird die Pinwand übersichtlicher, wenn nur Betreff und Forschungsbereich jeder Nachricht angezeigt werden. Außerdem wird durch die Vorgabe eines einheitlichen Formulars das Risiko von unvollständigen Nachrichten reduziert. Beispielsweise ist es nicht mehr so leicht, den Namen des Autors zu vergessen, da dafür das Feld absichtlich freigelassen werden muß.

Forces and Context

- Gruppen mit vielen Mitgliedern, vielen Nachrichten oder Nachrichten zu vielen verschiedenen Themen machen einfache Pinwände unübersichtlich und schwer zu durchsuchen.
- Es ist aufwendig, den Zweck von unstrukturierten Nachrichten zu erkennen. Meist muß ein großer Teil der Nachricht gelesen werden.
- Die automatische Verarbeitung von unstrukturiertem Text ist aufwendig.
- Menschen neigen dazu, unvollständige Nachrichten zu verfassen. Sie vergessen, wichtige Aspekte anzugeben.

Solution Verwenden Sie eine Pinwand mit strukturierten Nachrichten. Die Nachrichten bestehen aus mehreren Feldern, denen jeweils eine bestimmte Bedeutung zugeordnet ist. Felder können dazu verwendet werden, Nachrichten zu einem bestimmten Themenbereich eindeutig zu identifizieren. Für die Organisation nach Themenbereichen bieten sich fachbezogene hierarchische Klassifikationssysteme an. Zum Beispiel gibt es für die Einteilung nach Branchen das *North American Industry Classification System* (NAICS) [NTIS, 1997] oder im Fachbereich Informatik das *ACM Computing Classification System* [ACM, 1998].

Unterstützt die Pinwand mehrere verschiedene, halb-strukturierte Typen von Nachrichten, wird *Intelligent Information Sharing* möglich [Malone *et al.*, 1987]. Abbildung 5.5 zeigt eine mehrstufige Hierarchie von Nachrichtentypen, die aus verschiedenen Feldern aufgebaut sind. Den einzelnen Feldern ist jeweils eine bestimmte Bedeutung zugeordnet.

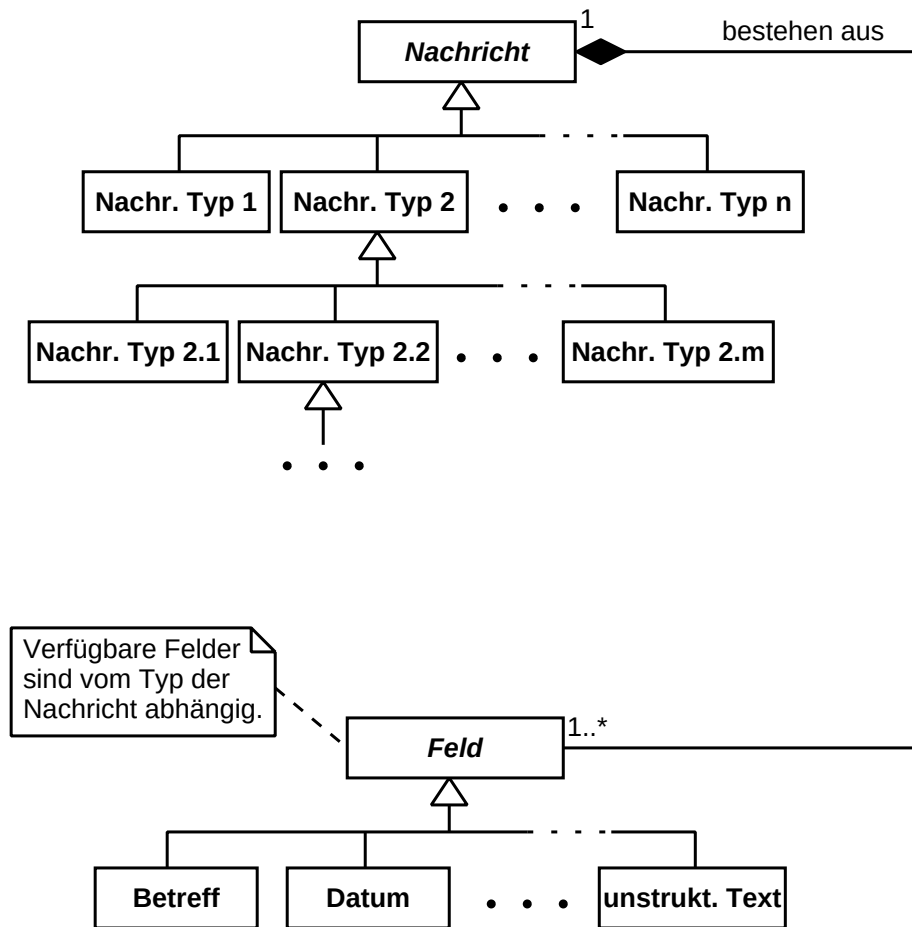


Abbildung 5.5: Klassendiagramm einer Hierarchie von polymorphen Nachrichtentypen

Consequences

- **Struktur.** Durch die gleiche oder sehr ähnliche Struktur für alle Nachrichten, kann der Zweck der einzelnen Nachricht schneller erfaßt werden. Strukturiertes Text kann leichter aufgenommen werden.
- **Automatische Verarbeitung.** Felder, die nur einen vordefinierten Inhalt haben können (zum Beispiel Datum, eine Klassifikation, Ja/Nein-Feld), sind leicht automatisiert verarbeitbar. Es werden dazu keine komplexen Erkennungstechniken benötigt.
- **Vollständigkeit der Nachrichten.** Die Struktur des Formulars mit der Kennzeichnung von Feldern, die unbedingt ausgefüllt werden müssen, verhindert, daß wichtige Teile der Information vergessen

werden. Es ist auch möglich vorzusehen, daß vom System nur vollständige Nachrichten akzeptiert werden.

- Inkompatibilität zwischen Nachrichten mit unterschiedlicher Struktur. Die Verwendung einer einzigen, fixen Struktur für alle Nachrichten auf der Pinwand hat den Nachteil, daß alle möglichen Felder vordefiniert werden müssen. Es entstehen dabei für viele Nachrichten unnötige Felder, die leer bleiben oder zu Verwechslungen führen können. Es ist außerdem nicht möglich, während des Schreibens einer Nachricht ein neues Feld, das zur Lesbarkeit der Nachricht beitragen würde, hinzuzufügen. Werden andererseits mehrere verschiedene Typen von Formularen mit unterschiedlicher logischer Struktur angeboten, muß eine Lösung für die Verarbeitung von polymorphen Nachrichten gefunden werden.

Design Zusätzlich zu den für die einfache Pinwand beschriebenen Punkten ist für das Design der Pinwand mit strukturierten Nachrichten noch folgendes zu bedenken:

1. Entwicklung des Formulars für die Nachricht. Die Erstellung eines geeigneten Formulars benötigt eine detaillierte Analyse des Informationsbedarfs der einzelnen Benutzer. Dazu müssen die verschiedenen Benutzergruppen und ihre Aktivitäten, die durch die Pinwand unterstützt werden sollen, identifiziert werden. Das daraus entstehende Formular muß intuitiv, ohne in einem Leitfaden nachschlagen zu müssen, verwendbar sein, sonst verliert der Benutzer zuviel Zeit mit Überlegungen, welche Information in welches Feld gehört. Ist die Bezeichnung eines Felds nicht ausreichend, um Mißverständnisse auszuschließen, muß der Zweck des Felds direkt im Formular erklärt werden.
2. Erweiterung des Interface. Durch strukturierte Nachrichten ist es möglich, das Interface beträchtlich zu verbessern. Beispielsweise können die Nachrichten auf der Pinwand durch ausgewählte Felder repräsentiert werden. Blendet man alle Felder außer dem Betreff aus, lassen sich viel mehr Nachrichten übersichtlich auf dem Bildschirm anzeigen. Eine weitere Möglichkeit ist die Visualisierung der Beziehungen zwischen Nachrichten aufgrund ihrer hierarchischen Kategorisierung. Alle Erweiterungen des Interface zielen darauf ab, die Suchzeit zu reduzieren und einen besseren Überblick über die Zusammenhänge zwischen einzelnen Nachrichten zu bekommen. In Abbildung 5.6 ist die Verwendung von mehreren Interfaces parallel dargestellt. Die Aufbereitung der Daten für die Darstel-

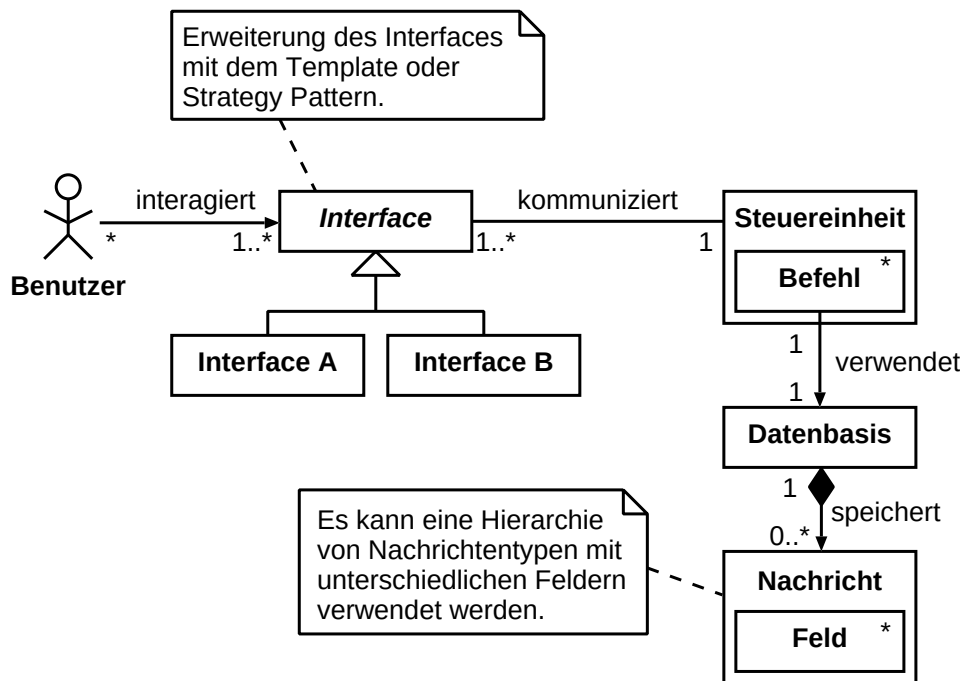


Abbildung 5.6: Klassendiagramm des Designs der Pinwand für strukturierte Nachrichten

lung wird an verschiedene Implementierungen des Interface (Interface A, Interface B, ...) weitergeleitet. Ein flexibles und erweiterbares Design dazu bieten das *Strategy Pattern* und das *Template Pattern* [Gamma *et al.*, 1995].

Known Uses Die einfache Pinwand ist ein Spezialfall der Pinwand mit strukturierten Nachrichten. Bei der einfachen Pinwand besteht die Nachricht nur aus einem einzigen Feld für unstrukturierten Text. Daher können alle Anwendungen für die einfache Pinwand auch mit der Pinwand für strukturierte Nachrichten realisiert werden.

- Das System *Information Lens* [Malone *et al.*, 1987].
- Brokersysteme im Internet. Zum Beispiel Online Jobbörsen von Arbeitsvermittlern.

5.3 Die Virtuelle Bibliothek

Pattern Name Virtuelle Bibliothek

Also Known As Verteilte Pinwand, Verteilte Bibliothek

Intent Einheitlicher und effizienter Zugang zu verteilten Informationsquellen und Diensten mit zentraler Suchmöglichkeit.

Motivation Stellen Sie sich vor, Sie sind an einer Universität verantwortlich für die Unterstützung von Forschung und Lehre mit neuen Medien, wie zum Beispiel der Verwendung von Web-Seiten im Internet. Im Netzwerk der Universität befindet sich bereits eine Menge Forschungsmaterial und Kursunterlagen, die Mitarbeiter der Universität erstellt haben. Diese Unterlagen sind aber über das gesamte Netzwerk verstreut und sind daher sehr schwer zu finden. Einige Unterlagen sind über die Startseiten der Institute erreichbar, andere über die persönlichen Seiten der Mitarbeiter und einige sind ohne den genauen Link zu kennen, gar nicht erreichbar. Obwohl Ihre Universität eine Suchmaschine verwendet, verbessert eine organisierte Darstellung aller verfügbaren Unterlagen den Zugang und damit auch die Verwendung und den Nutzen der Unterlagen.

Die Unterlagen werden in sehr unterschiedlichen Formaten angeboten. Beispiele sind:

- HTML-Seiten mit Text und Grafiken.
- Dokumente in proprietären Formaten von verschiedenen Textverarbeitungs-, Tabellenkalkulations- und Präsentationsgrafikprogrammen.
- Druckbare Formate wie *Postscript* und das im Internet weit verbreitete *Portable Document Format (PDF)* der Firma Adobe.
- Programme auf dem Server.
- Programme für den Web-Browser.
- Web-basierte *Computer Based Training* Systeme.

Alle Formate müssen unterstützt werden. Eine weitere wichtige Eigenschaft der Unterlagen ist, daß sie regelmäßig überarbeitet, verändert und erweitert werden.

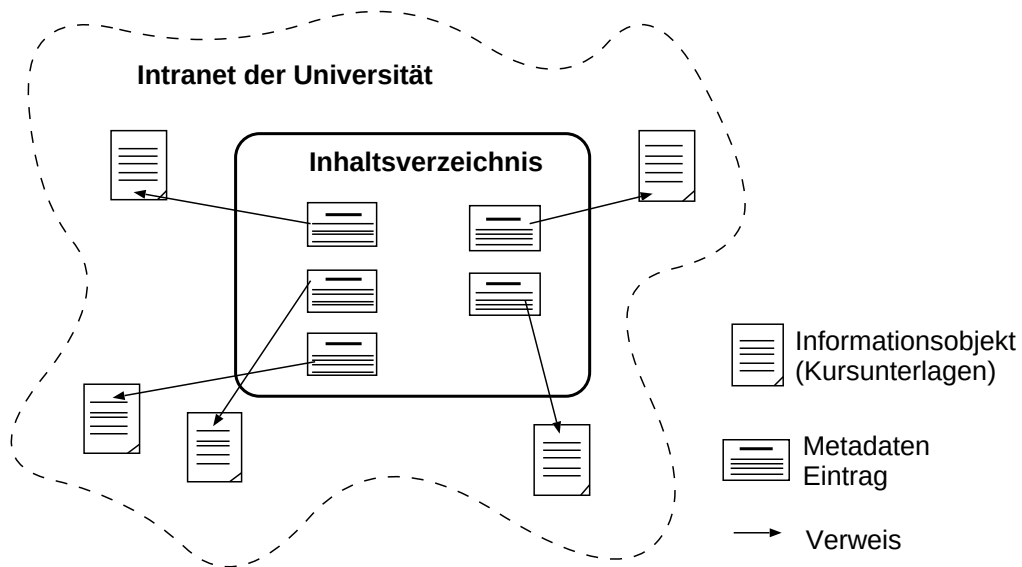


Abbildung 5.7: Beispiel eines Inhaltsverzeichnisses für verteilte Kursunterlagen in einem Universitätsnetzwerk

Abbildung 5.7 zeigt ein Inhaltsverzeichnis für Kursunterlagen im Netzwerk der Universität. Die Kursunterlagen (Informationsobjekte) bleiben physisch so verteilt, wie bisher. Zum Beispiel bleiben Lehrveranstaltungsunterlagen im persönlichen Web-Bereich des Vortragenden. Zentral wird für jedes Informationsobjekt nur eine Art elektronische Karteikarte geführt. Diese Karteikarte beinhaltet eine kurze Beschreibung des Informationsobjekts (Metadaten) und einen Verweis (URL) zu dem Informationsobjekt. Damit kann im zentralen Inhaltsverzeichnis gesucht werden, und an den Kursunterlagen selbst muß aber nichts geändert werden.

Forces und Context

- Es wird ein zentraler Zugangspunkt zu verteilten Informationen benötigt.
- Informationsobjekte werden oft verändert.
- Informationsobjekte sind im Besitz ihrer Erzeuger und sollen auch dort verbleiben.
- Informationsobjekte in einer Vielzahl von Formaten müssen unterstützt werden. Dies beinhaltet auch online ausführbare Programme und andere interaktive Informationsobjekte.

- Informationsobjekte sollen einfach in das System eingetragen, oder aus dem System entfernt werden können.

Solution Verwenden Sie eine Pinwand mit strukturierten Nachrichten als virtuelle Bibliothek. Abbildung 5.8 zeigt die Struktur. Die Nachrichten, in der virtuellen Bibliothek als Einträge bezeichnet, werden verwendet, um Metadaten über verteilte Informationsobjekte zu verwalten. Die Metadaten umfassen typischerweise die Bezeichnung, eine Kategorisierung, eine kurze Beschreibung, Schlüsselworte für das Informationsobjekt, sowie den Verweis (URL) zum Informationsobjekt.

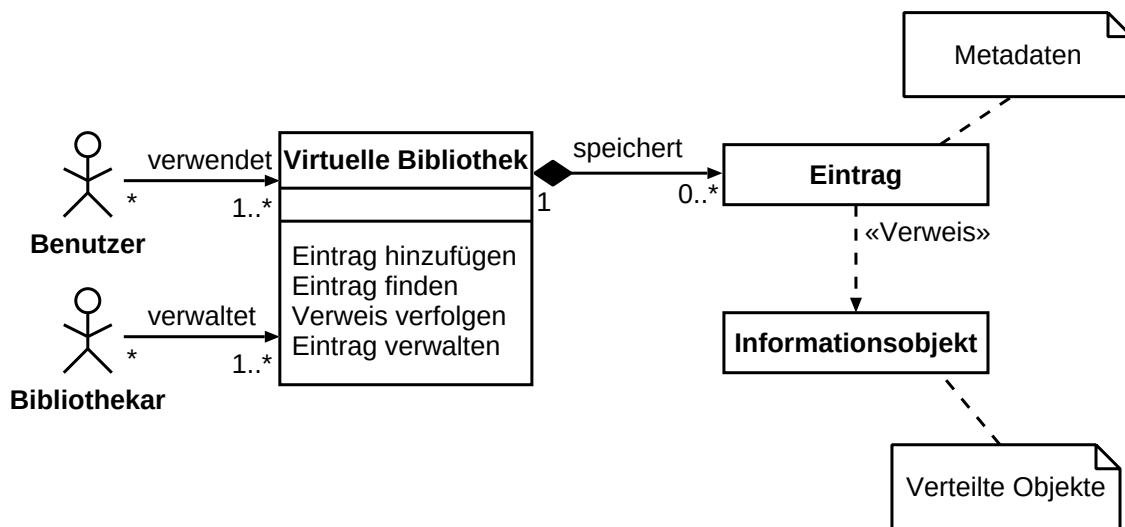


Abbildung 5.8: Klassendiagramm der Analyse einer Virtuellen Bibliothek

Durch die Verwendung von verteilten Objekten werden anstatt der gesamten Informationsobjekte nur Metadaten zentral gespeichert. Dadurch bleibt die physische und organisatorische Zuordnung der Informationsobjekte auch nach der Einführung einer virtuellen Bibliothek unverändert. Trotzdem können Benutzer durch die virtuelle Bibliothek in den zentral gespeicherten Metadaten nach passenden Einträgen suchen. Nachdem ein interessanter Eintrag gefunden wurde, muß nur noch der Verweis verfolgt werden, um zum Informationsobjekt zu gelangen. Die Einträge werden von Bibliothekaren verwaltet und aktuell gehalten.

Consequences

- Der zentrale Zugangspunkt zu vielen Informationsobjekten gewinnt durch eine große Auswahl an Attraktivität für die Benutzer, womit

sich die Verwendung und damit auch der Nutzen der einzelnen Objekte erhöht.

- Organisatorisch bedeutet die Verteilung der Informationsobjekte, daß der Besitzer sein Informationsobjekt jederzeit, ohne Beschränkung und unabhängig vom zentralen Verzeichnis verändern kann. Der Besitzer kann jede, in seinem Bereich (auf seinem Server) installierte Software verwenden, um beispielsweise interaktive Informationsobjekte anzubieten. Dazu ist keine zentrale Installation der Software notwendig.
- Eine soziale Konsequenz ist, daß der Besitzer weiterhin die vollkommene Kontrolle über sein Informationsobjekt besitzt. Es steht ihm jederzeit frei, den Zugang zu seinem Informationsobjekt oder zu Teilen seines Informationsobjekts zu überwachen oder einzuschränken. Außerdem hat der Besitzer die Möglichkeit sein Informationsobjekt uneingeschränkt so zu gestalten wie er möchte.
- Aus der verteilten Kontrolle über die Informationsobjekte ergibt sich ein weiterer organisatorischer Aspekt. Durch die Verteilung wird es für Organisationen schwieriger, eine konsistente *Corporate Identity* über alle Informationsobjekte zu schaffen. Dazu müssen geeignete organisatorische Maßnahmen getroffen werden. Beispielsweise können Empfehlungen, oder Vorlagen für das Layout zur Verfügung gestellt werden.
- Ein organisatorisches Problem sind ungültige Verweise. Die Verbindung zwischen der virtuellen Bibliothek (dem zentralen Verzeichnis) und den verteilten Informationsobjekten besteht nur aus einem Verweis (URL). Wird nun ein Informationsobjekt gelöscht oder verschoben, ohne daß das zentrale Verzeichnis benachrichtigt wird, entsteht ein ungültiger Verweis. Die fehlende Benachrichtigung ist ein Problem, daß beispielsweise durch eine regelmäßig Überprüfung der Informationsprodukte verringert werden kann.
- Ein technischer Aspekt ist die Verlässlichkeit der Netzinfrastruktur. Sind Teile des Netzwerks oder einzelne Server für einige Zeit funktionsunfähig, werden dadurch Informationsobjekte in der virtuellen Bibliothek unerreichbar. Dafür wird durch die Verteilung das Gesamtsystem robuster gegen Totalausfälle, da von technischen Problemen in der Regel nur ein begrenzter Teil (ein Server oder ein Netzwerksegment) betroffen ist.

Design

1. Die virtuelle Bibliothek verwendet das Design der Pinwand mit strukturierten Nachrichten, wobei die strukturierten Nachrichten als Einträge der Bibliothek verwendet werden. Da die Pinwand eine erweiterbare Steuereinheit besitzt, können leicht die benötigten Funktionen, wie zum Beispiel zur Verfolgung des Verweises in den Metadaten, hinzugefügt werden.
2. Für Metadaten gibt es einige Bestrebungen zur Standardisierung. Die *Dublin Core Initiative (DC)* [Weibel *et al.*, 1998] beschäftigt sich mit der Frage, welche Metadaten notwendig sind, um elektronische Quellen auffinden zu können. Für einzelne Verwendungszwecke existieren bereits Standards für Metadaten, beispielsweise gibt es für Lernobjekte den *Draft Standard for Learning Object Metadata* [Hodgins, 2000]. Und es gibt Ansätze, wie das *Resource Description Framework (RDF)* [Brickley und Guha, 2000] oder das *Knowledge Interchange Format (KIF)* [Genesereth, 1998], die sich mit der Repräsentation von Metadaten beschäftigen.

Known Uses

- Verzeichnisse im Internet, wie zum Beispiel die *WWW Virtual Library* (<http://vlib.org/>), *Yahoo!* (<http://www.yahoo.com>), oder das *Open Directory Project* (<http://www.dmoz.org>).
- Das *Virtual Library System* [Hahsler, 1997] ist eine Implementierung dieses Analyse Patterns.

5.4 Die Virtuelle Bibliothek mit aktiven Agenten

Pattern Name Virtuelle Bibliothek mit aktiven Agenten

Also Known As -

Intent Reduktion von Transaktionskosten bei der Sammlung, Klassifikation, Verwaltung, Suche und der Verwendung von verteilten Informationsquellen und Diensten.

Motivation Stellen Sie sich vor, Sie arbeiten für eine bekannte und sehr große Beratungsfirma im Bereich E-Commerce. Ihr Tätigkeitsbereich umfaßt unter anderem die Beobachtung der aktuellen Entwicklungen im Bereich E-Commerce (zum Beispiel Marktentwicklung, Entwicklung neuer Standards und neuer Produkte) und die Bereitstellung der Ergebnisse dieser Beobachtung für die Berater in Ihrem Unternehmen.

Die besten Quellen für aktuelle Information sind die Web-Seiten von Unternehmen, Standardisierungskomitees, Zeitschriften und Forschungsgruppen im Internet. Sie legen also eine Datenbank mit den Links auf die für E-Commerce relevanten Web-Seiten an. Diese Datenbank verwenden Sie als Basis für Ihre Beobachtungen. Sie kann auch von den Beratern direkt zur Suche von bestimmten Informationen verwendet werden.

Durch das exponentielle Wachstum und die enorme Änderungsrate des Internet, wird die Beobachtung der Entwicklungen im Bereich E-Commerce und die Verwaltung der Datenbank sehr zeitaufwendig und teuer. Um dieses Problem zu verringern setzen Sie Programme ein, die die Links in der Datenbank auf Gültigkeit überprüfen, die Web-Seiten nach Veränderungen durchsuchen, Überschriften extrahieren und weitere Analysen automatisiert durchführen. Dadurch ersparen Sie sich regelmäßig alle Web-Seiten händisch zu durchsuchen, und Sie können gezielt Web-Seiten aufsuchen, die interessante Veränderungen aufweisen.

Forces and Context

- Steigende Anzahl von verteilten Informationsobjekten oder Wachstum der Größe der Informationsobjekte.
- Veränderungen von Informationsobjekten müssen schnell erkannt werden.

- Verwaltungs- und Transaktionskosten sind von zentraler Bedeutung für die Wirtschaftlichkeit, die Verwendbarkeit und die Akzeptanz des Systems.
- Flexibilität bei der Erweiterung. Neue Funktionen zur Reduktion von Transaktionskosten müssen schnell und einfach integrierbar sein, ohne den Rest des Systems zu gefährden.

Solution Verwenden Sie eine virtuelle Bibliothek zur Verwaltung der verteilten Informationsobjekte, und erweitern Sie diese um aktive Agenten [Russell und Norvig, 1995], die eingesetzt werden, um Transaktionskosten für die beteiligten Personen zu senken.

Für die Bestimmung für welche Aufgaben Agenten eingesetzt werden sollen, müssen die Struktur und die Aufteilung der Transaktionskosten zwischen den Beteiligten genau analysiert werden. Milgrom und Roberts [Milgrom und Roberts, 1992, Part I-III] bieten eine Einführung in die Arten und Wirkungen von Transaktionskosten.

Die eingesetzten Agenten sind jeweils auf eine bestimmte Aufgabe oder Teilaufgabe spezialisiert und agieren voneinander unabhängig und asynchron. Die Agenten kommunizieren nicht direkt miteinander, sondern benutzen nur gemeinsam die Datenbasis der virtuellen Bibliothek. Sie lesen Einträge und können diese auch verändern.

Abbildung 5.9 zeigt wie zwei, von vielen möglichen Agenten zur Reduktion der Transaktionskosten für die Benutzer beitragen. Die Umwelt, die von den Agenten durch ihre Sensoren wahrgenommen wird, besteht aus der virtuellen Bibliothek mit ihren Einträgen, den verteilten Informationsobjekten und den Benutzern. Die dargestellten Agenten beeinflussen ihre Umwelt durch die Aktualisierung der Einträge in der virtuellen Bibliothek (Beobachtungsagenten) und durch die Interaktion mit den Benutzern bei der Informationssuche (Interfaceagenten). Die Aufteilung des Gesamtprozesses (Beobachtung und Benutzerinteraktion) auf zwei unabhängige Agenten ist in Abbildung 5.9 durch die zwei Nachrichtensequenzen A1-A3 und B1-B3 dargestellt. Wie man durch die Trennung erkennt, besteht keine direkte Kommunikation zwischen den Agenten. Diese Unabhängigkeit erleichtert die Umsetzung einzelner Agenten und vereinfacht ihre Verteilung auf mehrere Maschinen.

Consequences

- Die technischen Aspekte sind:

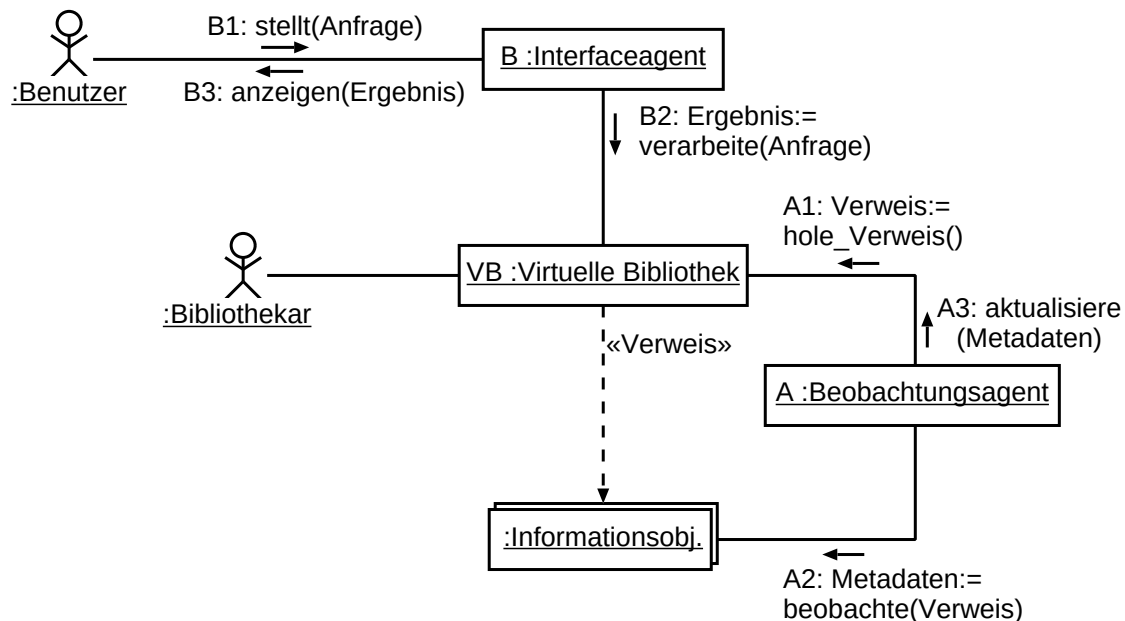


Abbildung 5.9: Kollaborationsdiagramm für die Zusammenarbeit von zwei Agenten zur Unterstützung einer virtuellen Bibliothek

- Das System kann leicht verteilt werden, da die Agenten voneinander unabhängig und asynchron agieren.
- Einfache Agenten können schnell implementiert und in das System integriert werden.
- Die Synchronisierung mehrerer Agenten ist problematisch, es ist nur schwache Konsistenz möglich.
- Die Hauptkonsequenz ist aber wirtschaftlich und sozial motiviert. Die Agenten helfen die Transaktionskosten der Benutzer durch ihre automatisierten Informationsdienstleistungen zu reduzieren. Beispielsweise hilft der in Abbildung 5.9 dargestellte Beobachtungsagent die Metadaten über die Informationsprodukte in der virtuellen Bibliothek aktuell zu halten. Dadurch werden Kosten reduziert, die aufgrund von veralteter Information entstehen würden. Dies führt zu höherem Nutzen und in weiterer Folge auch zu besserer Akzeptanz des Systems.

Design Die virtuelle Bibliothek mit aktiven Agenten verwendet eine virtuelle Bibliothek (siehe Analyse Pattern Virtuelle Bibliothek 5.3) zur Sammlung von Metadaten zu verteilten Informationsobjekten und eine Gemeinschaft von unabhängigen Agenten, die jeweils auf die Reduktion von bestimmten Teilen der Transaktionskosten spezialisiert sind.

Beispielsweise kann die Auswahl von Informationsobjekten durch die Anzeige von zusätzlichen, aktuellen Informationen erleichtert werden. Die in *Solution* angesprochenen Beobachtungsagenten können dazu Sensoren für verschiedene Veränderungen und Eigenschaften der Informationsobjekte haben. Geyer-Schulz und Hahsler [Geyer-Schulz und Hahsler, 2000] beschreiben einige Eigenschaften von Informationsobjekten, teilen sie nach Dimensionen ein und stellen dafür automatische Klassifikationsmethoden vor. Weiters wird vorgeschlagen, wie diese Eigenschaften in Form von Markierungen (*Labels*) für die Benutzer visualisiert werden können (zum Beispiel durch Interfaceagenten der virtuellen Bibliothek). Die vorgeschlagenen Markierungen, ihre Bedeutung und ihre Wirkung auf die Transaktionskosten sind:

1. Dimension Zeit:

- Neu - Das Informationsobjekt wurde kürzlich zur virtuellen Bibliothek hinzugefügt. Neue Einträge werden schneller erkannt.
- Verändert - Das Informationsobjekt wurde geändert. Die Verfolgung von Neuigkeiten wird vereinfacht.
- Veraltet - Das Informationsobjekt ist nicht mehr verfügbar. Der Benutzer braucht den Verweis nicht mehr zu verfolgen, beziehungsweise der Eintrag kann vom Bibliothekar gelöscht werden.

2. Dimension Region und Sprache:

- Region - Das Informationsobjekt enthält Informationen, die für eine gewisse geographische Region relevant sind. Der Benutzer kann die Suche geografisch einschränken.
- Sprache - Sprache, in der die Information angeboten wird. Der Benutzer kann die Informationsobjekte auswählen, die für ihn verständlich sind.

3. Dimension Bewertung:

- Verwendung - Welche Informationsobjekte werden oft verwendet? Der Benutzer findet populäre Informationsobjekte.
- Infrastruktur - Wie zuverlässig ist die Quelle eines Informationsobjekts technisch gesehen? Der Benutzer erfährt die Wahrscheinlichkeit, daß er für den Zugriff auf das Informationsobjekt viel Zeit benötigt oder daß ein solcher Zugriff regelmäßig gar nicht möglich ist.

Weitere Agenten, wie zum Beispiel ein Agent, der bei der Eintragung der Metadaten eines neuen Informationsobjekts hilft, können jederzeit in das System integriert werden.

Known Uses

- Die *Virtuelle Universität der Wirtschaftsuniversität Wien* [Geyer-Schulz *et al.*, 1999] (<http://vu.wu-wien.ac.at>).
- *News Wire Services* [Oki *et al.*, 1992].
- *Recommender Systeme* [Resnick und Varian, 1997].

Kapitel 6

Zusammenfassung

In diesem Teil der Arbeit wurde der Einsatz von Patterns in der Analysephase der Softwareentwicklung untersucht. Da dieser Ansatz in der Literatur bisher nicht ausreichend behandelt wurde, mußte der Ansatz mit Hilfe von Literatur über den Software-Lebenszyklus (Kapitel 1), über den Pattern-Ansatz (Kapitel 2), zu Design Patterns (Kapitel 3) und mit bereits existierenden Beispielen zu Analyse Patterns (Abschnitt 4.1) weiterentwickelt werden.

Die wichtigsten Unterschiede zu den in der Literatur gefundenen Beispielen von Analyse Patterns und anderen Ansätzen, wie Design Patterns und Application Frameworks sind:

- Analyse Patterns werden von Fowler [Fowler, 1997] und Hay [Hay, 1996] ohne einheitliche Struktur präsentiert. In dieser Arbeit wurde für Analyse Patterns eine strukturierte Beschreibungsform entwickelt. Diese Beschreibungsform unterstützt, im Gegensatz zu den Beispielen der anderen Autoren, den breiten Anwendungsbereich von Analyse Patterns, von der Erstellung des Analysemodells (Abschnitte *Motivation* und *Solution*) bis hin zur Transformation in ein flexibles Designmodell (Abschnitt *Design*). Durch diese durchgängige Unterstützung wird eine Verkürzung der benötigten Entwicklungszeit erreicht.
- Von Design Patterns, wie sie beispielsweise Gamma et al. [Gamma et al., 1995] und Buschmann et al. [Buschmann et al., 1996] beschreiben (siehe auch Kapitel 3), unterscheiden sich die hier vorgestellten Analyse Patterns vor allem in ihrem Anwendungsbereich. Die bedeutendsten Unterschiede spiegeln sich direkt in der strukturierten Beschreibungsform wider. Die Beschreibungform basiert zwar auf dem Formular für Design Patterns nach Gamma et al.

[Gamma *et al.*, 1995, S.6–8], der Inhalt der einzelnen Abschnitte wurde jedoch von einer rein technischen zu einer ganzheitlichen Sichtweise verändert. Besonders auffallend ist dies durch die Verschiebung des Schwerpunkts des Abschnittes *Consequences* zu wirtschaftlichen, organisatorischen und sozialen Aspekten. Weiters unterstützt der neue, von Alexanders [Alexander, 1979, S.41–54] geliehene Abschnitt *Forces and Context* diese Sichtweise. Damit werden auch soziale Probleme, die mit einer bestimmten organisatorischen Veränderung verbunden sind, automatisch in den Softwareentwicklungsprozeß einbezogen. Dies führt zu einer Erhöhung der Akzeptanz und damit zu einer schnelleren Einführung der Software.

- Bei Application Frameworks wird Code und Design wiederverwendet, indem gekennzeichnete Teile von vorgefertigtem Code nur noch angepaßt werden müssen (siehe Abschnitt 3.5). Analyse Patterns enthalten keinen vorgefertigten Code, sondern nur Vorschläge für flexibles Design und in weiterer Folge für eine flexible Implementierung. Damit bieten Analyse Patterns eine viel größere Flexibilität, sie benötigen aber auch einen höheren Implementierungsaufwand. Es besteht jedoch die Möglichkeit für ein Analyse Pattern ein Application Framework zu erstellen, und dann das Pattern als Beschreibung dafür einzusetzen (siehe Johnson [Johnson, 1992] für die Beschreibung von Application Frameworks mit Patterns).

Nach der Untersuchung von Patterns in der Analysephase wurden folgenden zwei Hauptvorteile des Einsatzes von Analyse Patterns für die Softwareentwicklung identifiziert (siehe Abschnitt 4.5):

1. Durch die Zusammenführung von konkreten Beispielen mit wiederverwendbaren Analysemodellen und Designvorschlägen in einem Analyse Pattern, wird der gesamte Entwicklungsprozeß, vom Erkennen des konkreten Problems bis zur Entwicklung eines flexiblen Designs, und in manchen Fällen sogar bis zur Implementierung, beschleunigt. Die konkreten Beispiele in Analyse Patterns helfen beim Erkennen von Problemen, auf die das Pattern anwendbar ist. Wurde das zu lösende Problem erkannt, liefert das Analyse Pattern ein Analysemodell, für dessen Umsetzungen Designvorschläge beschrieben werden. In der Regel beinhalten diese Vorschläge Design Patterns, die wiederum häufig Beispiele für die Implementierung enthalten.

Damit kann die Entwicklung einer Lösung eines Problems, für das ein Analyse Pattern existiert, stark verkürzt werden, da in allen Phasen Expertenwissen, oder sogar bereits fertige Teile, wiederverwendet werden können. Somit können die Entwicklungszeit und damit

auch die Entwicklungskosten durch den Einsatz von Analyse Patterns deutlich reduziert werden.

2. In Analyse Patterns werden als ein zentraler Punkt soziale Konsequenzen der vorgeschlagenen Lösung beschrieben, die bei der Verwendung berücksichtigt werden müssen. Die oft technisch orientierte Softwareentwicklung wird dadurch um die soziale Komponente erweitert, womit Probleme, wie fehlende Akzeptanz des fertigen Systems bei den Betroffenen, verringert werden können. Wie in Abschnitt 1.2 bereits beschrieben, ist das Management von Unsicherheit und Ängsten vor Verschlechterung der eigenen Position durch ein neues System, ein wichtiger Problembereich in der Analysephase. Die daraus entstehenden sozialen Probleme können Projekte stark verzögern, was mit hohen zusätzlichen Kosten verbunden ist, oder im Extremfall kann sogar das gesamte Projekt gefährdet werden. Analyse Patterns helfen mit dies zu verhindern.

Um diese Punkte zu untersuchen, wird in Teil II dieser Arbeit eine Case Study zum Einsatz der in Kapitel 5 entwickelten Analyse Patterns beschrieben. Die Analyse Patterns werden zur Realisierung eines Informationsbrokers für das Projekt *Virtuelle Universität*, das an der Wirtschaftsuniversität Wien seit 1997 durchgeführt wird, verwendet. Anhand dieser Case Study wird der positive Einfluß der Verwendung von Analyse Patterns auf die Entwicklungskosten und auf die Akzeptanz des daraus entstehenden Systems nachgewiesen.

Teil II

Eine Case Study zu Analyse Patterns: Die Virtuelle Universität der Wirtschaftsuniversität Wien

Die folgenden Kapitel enthalten eine Case Study zum Einsatz von Analyse Patterns. Dazu werden die in Kapitel 5 vorgestellten Analyse Patterns für Informationsmanagement verwendet, um den Informationsbroker des Projekts *Virtuelle Universität der Wirtschaftsuniversität Wien (VU)* (<http://vu.wu-wien.ac.at>) zu realisieren. Anhand dieser Case Study wird die Wirkung von Analyse Patterns in folgenden Bereichen untersucht und nachgewiesen:

- Höhere Akzeptanz des entwickelten Systems durch die Betroffenen, da durch Analyse Patterns die sozialen Konsequenzen des organisatorischen Wandels automatisch in den Entwicklungsprozeß einbezogen werden.
- Beschleunigung des Entwicklungsprozesses durch ein hohes Maß an Wiederverwendung in Analyse-, Design-, und der Implementierungsphase des Software-Lebenszyklus.

In Kapitel 7 werden die Rahmenbedingungen für die Case Study beschrieben, und es wird gezeigt, warum ein Informationsbroker als Zugangsplattform für wissenschaftliche Informationen im VU-Projekt zum Einsatz kommt.

In Kapitel 8 wird die Anwendung des Analyse Pattern *Virtuelle Bibliothek* (siehe Abschnitt 5.3) für den Informationsbroker auf einem Informationsmarkt dargestellt. Es wird gezeigt, daß ein Informationsbroker auf einem unübersichtlichen Informationsmarkt (zum Beispiel das Internet) einen positiven Einfluß auf Netzwerkeffekte hat, und damit Informationsanbieter und -konsumenten anzieht. Danach wird die Umsetzung eines Teils des Informationsbrokers für die *Virtuelle Universität* beschrieben. Abgeschlossen wird das Kapitel mit einer Analyse der konkreten positiven Konsequenzen und der noch ungelösten Probleme durch den Einsatz des Analyse Patterns für den Broker.

In Kapitel 9 wird der Informationsbroker um aktive Agenten aus dem Analyse Pattern *Virtuelle Bibliothek mit aktiven Agenten* erweitert. Durch diese Agenten werden Kosten für den Betrieb des Brokers, sowie für seine Verwendung durch Anbieter und Konsumenten von Information, reduziert. Um dies zu untersuchen, werden Transaktionskosten auf Informationsmärkten betrachtet, und es werden die positiven Auswirkungen eines Informationsbrokers mit aktiven Agenten auf diese Kosten analysiert. Danach werden mehrere konkrete Agenten für den Informationsbroker des VU-Projekts vorgestellt, und die Konsequenzen ihres Einsatzes werden zusammengefaßt.

In Kapitel 10 werden schließlich die Auswirkungen der verwendeten Analyse Patterns in der Case Study identifiziert und untersucht. Anhand von

zwei Teilprojekten des in der Case Study beschriebenen Informationsbrokers wird eine deutliche Reduktion des Entwicklungsaufwandes durch Wiederverwendung nachgewiesen. Diese Aufwandsreduktion wirkt sich direkt in Form einer Reduktion der Softwareentwicklungszeit und damit auch der Entwicklungskosten aus. Weiters werden deutliche Hinweise für eine hohe Akzeptanz des entwickelten Systems aufgrund einer besseren Berücksichtigung von sozialen Aspekten durch die Anwendung der Analyse Patterns präsentiert.

Kapitel 7

Rahmenbedingungen der Case Study

In diesem Kapitel werden die Rahmenbedingungen für das Projekt *Virtuelle Universität der Wirtschaftsuniversität Wien (VU)* kurz vorgestellt, da diese zum Verständnis der für den Einsatz von Analyse Patterns wichtigen sozialen Auswirkungen des Projekts notwendig sind.

In Abschnitt 7.1 werden als Motivation für das VU-Projekt einige Ergebnisse des *Dearing Reports* [Dearing, 1997] vorgestellt, welche die Notwendigkeit des Einsatzes moderner Informationstechnologie für Universitäten hervorheben.

Danach werden in Abschnitt 7.2 positive und negative Eigenschaften beschrieben, die mit dem Einsatz moderner Informationstechnologien und der Virtualisierung von Universitäten verbunden sind. Virtualisierung in der Lehre führt beispielsweise zu Flexibilisierung, kann aber auch zu sozialer Verarmung aufgrund von fehlender Interaktion mit anderen Lernenden führen.

Abgeschlossen wird das Kapitel mit einer Darstellung der Ziele des VU-Projekts an der Wirtschaftsuniversität Wien im Bereich der Unterstützung von Lehre, Forschung und Verwaltung durch moderne Informationstechnologien und Medien. Dabei wird aufgezeigt, wo und wie der in dieser Case Study entwickelte, wissenschaftliche Informationsbroker diese Ziele unterstützt.

7.1 Informationstechnologie und Universitäten

1997 wurde der *Dearing Report* [Dearing, 1997] publiziert. Dieser in Großbritannien durch das *National Committee of Inquiry into Higher Education* erstellte Bericht beschäftigt sich mit dem Einfluß von Kommunikations- und Informationstechnologie auf alle Aspekte des universitären Lebens. Laut diesem Report sind einige wichtige Auswirkungen dieser Technologien [Dearing, 1997, Kap.13]:

- In den nächsten 10 Jahren wird die Mehrzahl der Studierenden Computer verwenden, um einen Teil des Lehrmaterials zu beziehen, und sie werden den Großteil der nötigen Organisation und Kommunikation für ihr Studium mit dem Computer erledigen.
- Forscher bekommen derzeit bereits Zugang zu Ausrüstung und riesigen Datenbanken. Der Bedarf dafür wird in Zukunft stark ansteigen.
- In der nächsten Dekade wird höhere Bildung als Gut, in einem globalen Markt mit steigendem Wettbewerb, handelbar sein.
- In den nächsten 10 Jahren sollen und werden alle höheren Bildungsorganisationen wichtige Teile der Verwaltung und der Lehre auf den Computer verlagern.

Weiters enthält der Report einige Empfehlungen. Eine davon ist:

Recommendation 41: We recommend that all higher education institutions in the UK should have in place overarching communications and information strategies by 1999/2000.

Besonders im Bereich der Fernuniversitäten ist das Internet frühzeitig als ideales Kommunikations- und Transportmedium für Lehrzwecke erkannt worden. Beispiele dazu sind die Web-Sites der *Fernuniversität der Gesamthochschule Hagen* (<https://vu.fernuni-hagen.de/>) und der *Open University* (<http://www.open.ac.uk/>). Aber auch klassische Universitäten beginnen Informationstechnologie für Verwaltung, Forschung und Lehre einzusetzen, ein Beispiel von vielen ist das personalisierte Portal der *University of California at Los Angeles* (<http://my.ucla.edu/>).

Das bedeutet, alle Universitäten werden in Zukunft neue Informationstechnologien einsetzen müssen, um konkurrenzfähig zu bleiben. Wie und in welchem Ausmaß dies geschieht, hängt von den Zielsetzungen und der Strategie der jeweiligen Organisation ab.

7.2 Auswirkungen der Virtualisierung von Universitäten

In den letzten Jahren wurde das Schlagwort *Virtuelle Universität* auch im deutschsprachigen Raum sehr bekannt. Schumann und Hagenhoff [Schumann und Hagenhoff, 1998] charakterisieren eine Virtuelle Universität durch folgende Eigenschaften:

- Flexibilisierung durch räumliche und zeitliche Unabhängigkeit der Studierenden und den Verzicht auf universitätsspezifische Lernumgebungen, wie Hörsäle.
- Multimediale Aufbereitung von Lehrveranstaltungen (Audio, Video).
- Umfassende interaktive Kommunikationsmöglichkeiten (von E-Mail bis Videokonferenzen).
- Direkten Zugang zu Lehrmaterial und Hintergrundmaterial.

Diese vorteilhaften Eigenschaften unterscheiden eine Virtuelle Universität von klassischen Fernuniversitäten und dem Einsatz von Telelearning. Als nachteilige Aspekte sehen Schumann und Hagenhoff [Schumann und Hagenhoff, 1998] die soziale Verarmung von Studierenden durch fehlende Gruppendynamik. Dies kann zu erhöhten Abbrecherquoten führen. Außerdem ist es dadurch möglich, daß wichtige Fähigkeiten, wie zum Beispiel Rhetorik und Präsentationstechnik, nicht mehr vermittelbar sind.

Natürlich gibt es zum Thema Virtuelle Universität auch viele warnende Stimmen. Noble [Noble, 1998] beschreibt, wie verschiedene Gruppen neue Technologien ohne Rücksicht auf die Betroffenen (Studierende und Lehrer) forcieren, und nur an ihren eigenen wirtschaftlichen Nutzen denken. Diese Gruppen sind:

1. Software-, Hardware- und Inhaltsanbieter. Sie wollen in den riesigen Bildungsmarkt eindringen.
2. Verantwortliche für Fortbildung in Unternehmen. Sie wollen billige "just-in-time"-Weiterbildung.
3. Universitätsverwaltungen. Sie sehen in der Verwendung von computergestütztem Unterricht ein Mittel zur Reduktion ihrer Personalkosten.

Aufgrund dieser wirtschaftlichen Zwänge wird Bildung zu einem handelbaren Gut werden, das von den Lehrern (die Kurse entwickeln) unabhängig existiert. Margolis [Margolis, 1998] beschreibt basierend auf diesen Annahmen ein extremes Szenario in dem nur wenige, global agierende Universitäten überleben. Lehrer und die physische Lehrinfrastruktur werden abgebaut, Forschungsausgaben gekürzt, und die akademische Freiheit gehört der Vergangenheit an.

Turoff [Turoff, 1999] beschreibt im Gegensatz zu Margolis zwei Zukunftsszenarios für Ausbildung. Das eine baut auf maximaler Effizienz für den Ausbildungsprozeß auf, wobei das Ziel eine Minimierung der Ausbildungskosten ist. Es werden den einzelnen Lernenden nur die benötigten "Skills" vermittelt. Das daraus entstehende Szenario ähnelt sehr stark den Beschreibungen von Noble [Noble, 1998] und Margolis [Margolis, 1998] mit allen negativen Effekten.

Als alternatives Szenario beschreibt Turoff was passiert, falls sich maximale Effektivität durchsetzt. Mit maximaler Effektivität meint Turoff einen optimalen Lernprozeß für den Lernenden, der als Ziel das Aneignen von kognitiven Prozessen, wie zum Beispiel die Fähigkeit, selbständig neue Probleme zu lösen, hat. In diesem Szenario entwickeln sich Universitäten in fast allen Aspekten genau umgekehrt zu dem vorigen Szenario. Es gibt viele Universitäten, es wird gemeinsames Lernen in kleinen Gruppen unterstützt, und der Lehrer ist Berater, der die Kontrolle über den Lernprozeß hat. Im Vordergrund stehen Technologien zur Unterstützung von Gruppen. Beispielsweise Werkzeuge für kollaborative Kommunikation und die Sammlung von Wissen.

Welches Szenario sich für Universitäten durchsetzen wird, hängt sicher von ökonomischen Zwängen ab, aber auch von der Art, wie verantwortungsvoll mit neuen Technologien in der Ausbildung umgegangen wird.

7.3 Die Initiative an der Wirtschaftsuniversität Wien

1997 wurde an der Wirtschaftsuniversität Wien das Projekt *Virtuelle Universität (VU)* durch die Initiative von Andreas Geyer-Schulz, Dozent an der Abteilung für Informationswirtschaft, gestartet. Wie im *Dearing Report* [Dearing, 1997, Kap.13] gefordert, sollte damit für die Wirtschaftsuniversität Wien eine Kommunikations- und Informationsstrategie entwickelt werden, die einen optimalen Lernprozeß für den Lernenden (analog zum Szenario mit maximaler Effektivität von Turoff [Turoff, 1999], siehe Abschnitt 7.1) unterstützt.

Die Mittel für das Projekt waren eine minimale finanzielle Unterstützung durch die Universität und freie Verwendung der Ausrüstung der Abteilung für Informationswirtschaft.

Die Zielsetzung des noch immer laufenden Projekts war und ist der Einsatz moderner Kommunikationsmedien, wie dem Internet, zur Unterstützung von Lehre und Forschung. Dieses Ziel soll durch freiwillige Mitarbeit aller beteiligter Gruppen, innerhalb und außerhalb der Universität, erreicht werden.

Für die Umsetzung des Projekts wurde 1997 folgender Stufenplan entwickelt:

1. Bewußtsein für "Neue Medien" erhöhen.
2. Vorteile des Einsatzes dieser Medien für Lehre und Forschung kommunizieren.
3. Unterstützung der Lehrenden und Forscher bei der Erstellung von online Information, wie Lehrmaterial, Informationsseiten, ...
4. Zugang zu online Informationen vereinfachen.
5. Qualität der angebotenen Information erhöhen.

Die Punkte 1 und 2 dienen dazu, die Betroffenen zu freiwilliger Mitarbeit zu motivieren. Hauptsächlich werden Lehrende auf die Vorteile der Verwendung des Internets für ihre konventionellen Lehrveranstaltungen aufmerksam gemacht (siehe auch Turoff [Turoff, 1999]). Einige Beispiele sind:

- Lehrunterlagen können online angeboten werden. Sie können von den Studenten bei Bedarf jederzeit online verwendet oder ausgedruckt werden. Dadurch erspart sich der Lehrende die Verwaltung von Kopiervorlagen, er kann Änderungen in den Unterlagen leichter durchführen, und der Student bekommt immer die aktuellste Version der Unterlagen.
- Im Internet findet man eine Fülle an Informationsquellen (Lehrunterlagen anderer Lehrer, Datenbanken, ...). Diese Informationsquellen können einfach in die Unterlagen und den Unterricht eingebunden werden.
- Die Kommunikation zwischen Studenten und Lehrenden über E-Mail bietet viele Vorteile gegenüber dem Telefon oder der Sprechstunde. Beispielsweise wird der Lehrende nicht bei wichtigen Tätigkeiten unterbrochen, und der Student ist nicht an bestimmte Zeiten oder Orte gebunden.

- Studentenarbeiten können leicht online publiziert werden. Dies wirkt für Studenten als zusätzlicher Anreiz bessere Arbeiten zu erstellen.

Zur Umsetzung der Punkte 3 bis 5 des entwickelten Stufenplans muß der Zugang zu online Informationen vereinfacht werden. Häufig wird dazu ein elektronischer Lehrveranstaltungskatalog verwendet. Dafür gibt es viele Beispiele, da fast jede Universität ein elektronisches Vorlesungsverzeichnis mit Lehrveranstaltungen für ihre Studenten betreibt. Die Lehrveranstaltungen werden dabei meist durch die Verwaltung der Universität im Katalog aktuell gehalten. Die in solchen Katalogen enthaltenen Informationen beschränken sich oft auf Bezeichnung der Lehrveranstaltung, Vortragende, Termine und eine Zusammenfassung der Inhalte der Veranstaltung (siehe <http://www.wu-wien.ac.at/infocenter/vvz/> für das kommentierte Vorlesungsverzeichnis der Wirtschaftsuniversität Wien).

Ein bekanntes Beispiel für einen elektronischen Katalog, der eine andere, globale Strategie verfolgt, ist die *World Lecture Hall* (siehe <http://www.utexas.edu/world/lecture/>). Dieses Informationssystem wird von der *University of Texas at Austin* betrieben, und ist ein Katalog, in dem Internet-Adressen von Lehrunterlagen gesammelt und nach Kategorien angeboten werden, die Lehrende weltweit freiwillig eingetragen.

Für die Virtuelle Universität der Wirtschaftsuniversität Wien wird ein ähnlicher Ansatz wie bei der *World Lecture Hall* gewählt, doch wird dabei das Internet als riesiger Marktplatz für Informationsgüter (Web-Seiten mit Lehr- oder Forschungsmaterial) gesehen. Auf diesem Marktplatz handeln Lehrende, Lernende und Forscher als Akteure mit Information. Sie bieten Informationsgüter an, indem sie Web-Seiten öffentlich zugänglich machen, beziehungsweise suchen und konsumieren (verwenden) sie Informationsgüter, die jemand anderer zur Verfügung stellt. Aufgrund der Größe und der fehlenden Struktur des Internets ist jedoch das Auffinden von benötigten Informationsgütern auf diesem Marktplatz (zum Beispiel mit Internet-Suchmaschinen) zeitaufwendig und dadurch mit hohen Suchkosten verbunden. Um diesem Problem entgegen zu wirken, soll ein online Informationsbroker für im Internet verfügbare Lehrunterlagen und wissenschaftliche Materialien eingesetzt werden. Der Broker bietet den Anbietern eine Plattform, auf der sie freiwillig ihre Informationsgüter anbieten können. Den Konsumenten bietet er eine einfache, zentrale Suchmöglichkeit.

Mit der Case Study in den folgenden Kapiteln wird dargestellt, wie unter Berücksichtigung der hier beschriebenen Rahmenbedingungen die Analyse Patterns für Informationsmanagement aus Kapitel 5 eingesetzt werden, um den Informationsbroker des VU-Projekts zu realisieren.

Kapitel 8

Das Analyse Pattern Virtuelle Bibliothek als Informationsbroker für die Virtuelle Universität

In diesem Kapitel wird der Informationsbroker der Virtuellen Universität beschrieben. Es wird gezeigt, wie dafür das Analyse Pattern *Virtuelle Bibliothek* aus Abschnitt 5.3 implementiert wurde und welche Auswirkungen dies hat.

Begonnen wird in Abschnitt 8.1 mit einer Analyse des Nutzens eines Brokers auf einem Informationsmarkt. Dazu werden zuerst Informationsgüter genauer betrachtet, da diese spezielle Eigenschaften (zum Beispiel vernachlässigbar geringe Reproduktionkosten) aufweisen, die sie von normalen, physischen Gütern unterscheiden. Mit Hilfe von Netzwerkeffekten wird gezeigt, wie ein wachsender Informationsbroker auf einem unübersichtlichen Informationsmarkt Informationsanbietern und -konsumenten Nutzen bringt, und dadurch weiter wächst.

In Abschnitt 8.2 wird die Umsetzung des Informationsbrokers als Konkretisierung des Analyse Patterns *Virtuelle Bibliothek* beschrieben. Es werden die Informationsgüter für den Broker identifiziert, und das Design und der für das VU-Projekt eingesetzte Prototyp des Informationsbrokers mit verteilter Informationshaltung werden präsentiert.

Abgeschlossen wird das Kapitel in Abschnitt 8.3 mit einer Untersuchung der Konsequenzen aus der Anwendung des Analyse Patterns. Positive Konsequenzen sind die Verringerung der Widerstände durch Lehrende, da die Lehrenden durch den Informationsbroker die volle Kontrolle über das von ihnen erstellte Lehrmaterial behalten, und ihnen keine bestimmten

Autorenwerkzeuge vorgeschrieben werden. Wie jedes Pattern kann auch das Analyse Pattern *Virtuelle Bibliothek* nicht alle Bereiche gleich gut abdecken. Die Behandlung von ungültigen Verweisen zu den verteilten Informationen am Informationsmarkt, und die Eigenschaften, die der Informationsbroker haben soll, um seine Akzeptanz bei Anbietern und Konsumenten zu erhöhen, werden vom Pattern nicht angesprochen. Dies sind Fragen, die im Kapitel 9 behandelt werden.

8.1 Nutzen eines Brokers auf einem Informationsmarkt

Für die Analyse der Auswirkungen der Einführung eines Informationsbrokers auf einen Informationsmarkt wird in diesem Abschnitt der Begriff *Informationsgüter* definiert. Es werden Netzwerkeffekte auf einem Informationsmarkt beschrieben, und es wird gezeigt, daß die Einführung eines Informationsbrokers in einen unübersichtlichen Informationsmarkt die effektiv wirkenden positiven Netzwerkeffekte verstärkt, und damit den Nutzen aller Beteiligten erhöht.

8.1.1 Definition von Informationsgütern

Varian [Varian, 1999] definiert ein Informationsgut als die grundlegende, gehandelte Einheit in einer Informationswirtschaft. Ein Informationsgut kann alles sein, was digitalisiert werden kann. Beispiele für Informationsgüter sind ein Buch, ein Film oder ein Telefongespräch. Er beschreibt folgende drei Eigenschaften von Informationsgütern, die sie von normalen, physischen Gütern unterscheiden, und die daher Transaktionen auf Märkten für Informationsgüter massiv beeinflussen:

Informationsgüter sind Erlebnismgüter (Experience Goods). Damit ein potentieller Konsument feststellen kann, welchen Inhalt ein Informationsgut hat, und ob es den eigenen Erwartungen und Bedürfnissen entspricht, müßte er es zuerst konsumieren. Danach hat der potentielle Konsument das Informationsgut aber bereits und braucht es nicht mehr zu erwerben. Dies stellt ein Problem für den Handel mit Informationsgütern dar. Mögliche Lösungen dafür sind: Previews von Teilen des Guts, Besprechungen und Kritiken durch Experten, oder Vertrauen auf die Reputation des Produzenten.

Kostenstruktur von Informationsgütern. Nach Shapiro und Varian [Shapiro und Varian, 1999, S.3] treten für Informationsgüter hohe

Fixkosten (Produktionskosten), aber nur sehr geringe Grenzkosten (Reproduktionskosten) auf. Beispielsweise kostet die Herstellung eines Kinofilms mehrere 100 Millionen US-Dollar, aber für eine Kopie benötigt man nur eine leere Videokassette.

Im folgenden werden Produktionskosten, Lagerungskosten und Reproduktionskosten von Informationsgütern beschrieben.

- **Produktionskosten.** Die Produktionskosten von Informationsgütern sind abhängig von den Eigenschaften des Informationsguts (Größe, Komplexität, eingesetzte Medien, ...), von den verwendeten Werkzeugen (Autorensysteme) und den Kenntnissen des Autors. Mit dem professionellen Einsatz von multimedialen und interaktiven Elementen (Ton, Video, Animationen, Simulationen, ...) sind sehr hohe Produktionskosten verbunden. Die Produktionskosten sind sogenannte "Sunk Costs", das sind Fixkosten, die am Anfang für die Herstellung des Guts aufgebracht werden müssen, und die danach unwiederbringlich verloren sind.
- **Lagerungskosten.** Die Kosten für die Speicherung von Informationsgütern hängen von der Größe der Informationsgüter, dem Medium, auf dem sie gespeichert werden und der Dauer der Speicherung ab. Speichermedien sind heute in großen Mengen zu sehr geringen Preisen verfügbar, daher spielen die Speicherkosten in der Regel keine große Rolle.
- **Reproduktionskosten.** Die Reproduktionskosten von Informationsgütern hängen hauptsächlich von technischen Aspekten ab. In der Regel sind sie verglichen mit den Produktionskosten vernachlässigbar klein.

Informationsgüter haben Eigenschaften wie Öffentliche Güter.

Rein öffentliche Güter, wie zum Beispiel öffentliche Sicherheit, besitzen, im Gegensatz zu privaten Gütern, die Eigenschaften der Nicht-Rivalität und der Nicht-Ausschließbarkeit. Nicht-Rivalität bedeutet, daß der Konsum durch eine Person nicht die Verfügbarkeit für andere Personen verringert. Nicht-Ausschließbarkeit bedeutet, daß eine Person nicht andere Personen vom Konsum des Guts ausschließen kann.

Bei Informationsgütern treten diese Eigenschaften in ähnlicher Form auf. Durch die geringen Reproduktionskosten besteht bei der Nutzung von Informationsgütern in der Regel keine Rivalität. Beispielsweise wird durch das Abspielen einer Videokassette der Film darauf nicht zerstört und steht somit anderen Personen weiter zur

Verfügung. Die Ausschließbarkeit ist von den rechtlichen Rahmenbedingungen und deren Überwachung abhängig. Ohne rechtlichen Eigentumsschutz (zum Beispiel Urheberrechte) könnte jeder, der technisch in der Lage ist, eine Kopie des Informationsguts herstellen und diese weitergeben, womit keine Ausschließbarkeit möglich wäre.

Für den Informationsbroker der Virtuellen Universität sind die betrachteten Informationsgüter Objekte im Internet. Diese Objekte beinhalten zusammengehörige Informationen und sind durch einen Verweis (URL) im Internet auffindbar. Diese Objekte können beispielsweise sein:

- Eine einzelne Web-Seite. Zum Beispiel die Web-Seite einer Privatperson oder eines Lehrenden.
- Ein Dokument, das online betrachtet oder ausgedruckt werden kann. Zum Beispiel Unterlagen zu einer Lehrveranstaltung.
- Ein Programm, das online verwendet werden kann. Zum Beispiel ein online Multiple-choice Test.
- Multimedia-Objekte, wie Ton, Video und Animationen. Zum Beispiel der Live-Mitschnitt eines Vortrags.
- Eine Anzahl inhaltlich und organisatorisch zusammengehöriger Web-Seiten, die durch Verweise miteinander verbunden sind. Zum Beispiel die Web-Seiten eines Universitätsinstituts.

8.1.2 Netzwerke und Netzwerkeffekte

Katz und Shapiro [Katz und Shapiro, 1985] beginnen ihren bekannten Artikel *Network Externalities, Competition, and Compatibility* mit:

There are many products for which the utility that a user derives from consumption of the good increases with the number of other agents consuming the good.

Sie begründen diese Aussage in der Folge mit positiven Konsumexternalitäten, die durch direkte und indirekte Effekte ausgelöst werden. Als Beispiel für direkte Effekte beschreiben Katz und Shapiro, daß der Nutzen des Kaufs eines Telefons direkt von der Anzahl der Teilnehmer des Telefonnetzwerks abhängt. Für indirekte Effekte geben sie das Beispiel des Kaufs eines PCs. Ein Käufer wird sich überlegen, daß die Auswahl der verfügbaren Software für ein bestimmtes System von der Anzahl der anderen Personen abhängt, die eine ähnliche Hardware gekauft haben. Damit kauft er

das System mit den meisten Benutzern und erhöht dadurch die Nachfrage nach Software, und damit auch indirekt die Nachfrage nach dem von ihm gewählten System. Für alle Effekte, egal ob direkt oder indirekt, ist die Größe des Netzwerks ausschlaggebend.

Economides betrachtet in seinem Übersichtsaufsatz *The Economics of Networks* [Economides, 1996] unter anderem verschiedene Arten von Netzwerken und beschreibt das Entstehen von positiven Konsum- und Produktionsexternalitäten. Der Begriff Netzwerk wird oft für physische Netzwerke, wie Eisenbahnnetzwerke oder Telekommunikationsnetzwerke verwendet. Netzwerke sind dadurch gekennzeichnet, daß je größer das Netzwerk wird (zum Beispiel je mehr Teilnehmer ein Telekommunikationsnetzwerk verwenden), desto interessanter wird es für andere Kunden an diesem Netzwerk teilzunehmen. Netzwerke mit diesen Eigenschaften, nämlich daß der Nutzen einer Einheit eines Gutes (Teilnahme am Netzwerk) mit der Anzahl der verkauften Einheiten (Größe des Netzwerkes) steigt, besitzen positive Konsumexternalitäten, eine Form von Netzwerkexternalitäten. Genauer betrachtet ergibt sich der erwartete Nutzen einer Einheit eines Gutes durch die erwartete Anzahl der in Zukunft verkauften Einheiten dieses Gutes [Economides, 1996].

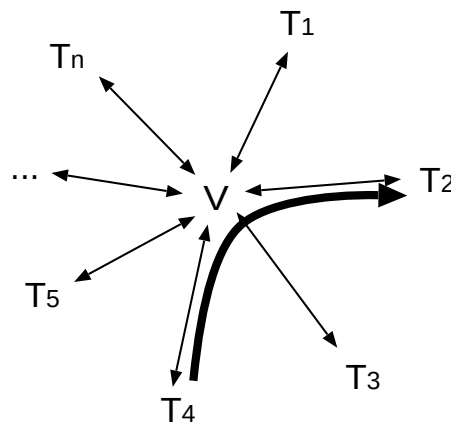


Abbildung 8.1: Ein Komplementärgut in einem Netzwerk mit Sterntopologie

Der Hauptgrund für das Entstehen von Netzwerkexternalitäten ist die Komplementarität von Teilen eines Netzwerks [Economides, 1996]. Ein Beispiel für so ein Komplementärgut ist in Abbildung 8.1 dargestellt. Es handelt sich um ein Netzwerk mit einfacher Sterntopologie, beispielsweise ein lokales Telefonnetzwerk mit mehreren Teilnehmern T_i und einer Vermittlung V . Ein Anruf von Teilnehmer T_4 an Teilnehmer T_2 ist als dicker Pfeil dargestellt. Der Anruf besteht aus zwei komplementären

Teilen des Netzwerks (Komplementärgütern), nämlich den Verbindungen T_4V und VT_2 . Ein Telefonnetzwerk ist ein sogenanntes *Two-way Network* [Economides, 1996], das bedeutet, ein Anruf von T_4 an T_2 ist ein anderes Gut als ein Anruf von T_2 an T_4 . Es existieren in diesem Netzwerk, mit n Teilnehmern, $n(n-1)$ mögliche zusammengesetzte Güter. Jeder Teilnehmer kann jeden anderen erreichen, ein Telefonat eines Teilnehmers mit sich selbst macht keinen Sinn. Ein zusätzlicher Teilnehmer T_{n+1} erzeugt direkte Externalitäten für alle bisherigen Teilnehmer von je einem möglichen Gut, in Summe also $2n$ neue Güter.

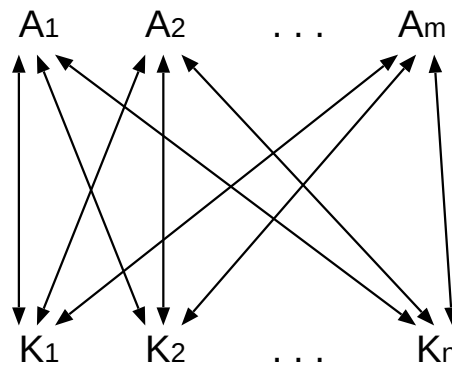


Abbildung 8.2: Mögliche zusammengesetzte Güter aus zwei Komplementärgütern

Abbildung 8.2 zeigt ein Netzwerk mit zwei Arten von Komplementärgütern, Typ A und Typ K. Jeweils ein Gut vom Typ A ergibt mit einem Gut vom Typ K ein zusammengesetztes Gut, wobei alle Güter $A_i, i = 1, \dots, m$ mit allen Gütern $K_j, j = 1, \dots, n$ kompatibel sind. Unter der Annahme, daß die Reihenfolge der Zusammensetzung des entstehenden Gutes unerheblich ist, es sich also um ein *One-Way Network* [Economides, 1996] handelt, bei dem AK und KA äquivalent sind, ergeben sich $m \times n$ mögliche zusammengesetzte Güter. Konsumexternalitäten entstehen in diesem Netzwerk folgendermaßen: Wird ein neues Gut vom Typ K hinzugefügt, also n um eins erhöht, entstehen m neue zusammengesetzte Güter. Analog gilt dies für neue Güter vom Typ A.

Economides [Economides, 1996] beschreibt Finanzmärkte als *One-Way Network* mit zwei komplementären Gütern. Die Topologie entspricht Abbildung 8.2. Ein Gut ist die Bereitschaft, etwas zu einem bestimmten Preis zu verkaufen (Angebot), und das zweite Gut ist die Bereitschaft, es zu diesem Preis zu kaufen (Gegenangebot). Gemeinsam ergibt dieses Güterpaar ein zusammengesetztes Gut, die Transaktion. Eine steigende Größe des Finanzmarkts erhöht den erwarteten Nutzen der Marktteilnehmer. Mehr Teilnehmer reduzieren die Varianz des erwarteten Marktpreises, womit

sich der erwartete Nutzen von risikoaversen Teilnehmern erhöht. Gleichzeitig steigt mit der Teilnehmerzahl die Liquidität des Marktes. Durch diese beiden Eigenschaften weisen Finanzmärkte Netzwerkexternalitäten auf.

8.1.3 Der Einfluß eines Brokers auf Netzwerkeffekte

Analog zu Finanzmärkten können Informationsmärkte gesehen werden. Es wird wieder die Topologie, dargestellt in Abbildung 8.2 auf Seite 97, mit den Anbietern A und den Konsumenten K verwendet. Die Komplementärgüter sind das Angebot eines Informationsguts, das einen bestimmten Informationsbedarf decken kann (A_i), und die Nachfrage der Konsumenten, die diesen Informationsbedarf haben (K_j). Diese Güter gemeinsam ergeben wieder ein zusammengesetztes Gut, nämlich den Verkauf eines Informationsguts AK .

Zusätzlich angebotene Informationsgüter erhöhen die Auswahl für jeden Konsumenten. Damit steigt die Chance, daß ein Informationsgut zur Deckung seines Informationsbedarfs angeboten wird, beziehungsweise steigt die Auswahl, falls mehrere konkurrierende Güter angeboten werden. Diese Effekte vergrößern den erwarteten Nutzen der Konsumenten durch die Teilnahme am Markt und ziehen dadurch zusätzliche Konsumenten an.

Für den Anbieter ist die Anzahl der potentiellen Konsumenten ausschlaggebend. Mehr Konsumenten führen zu einer höheren Chance, daß das angebotene Informationsgut benötigt wird, was zu einer Erhöhung des erwarteten Nutzens der Anbieter führt. Diese Erhöhung führt dazu, daß zusätzliche Anbieter dem Netzwerk beitreten, und daß dadurch die Auswahl für Konsumenten größer wird. Durch diese Eigenschaften weist ein Informationsmarkt ebenfalls direkte und indirekte Netzwerkexternalitäten aus der Größe des Marktes auf.

Auf einem realen Informationsmarkt mit vielen Teilnehmern ist es sehr unwahrscheinlich, daß zum Beispiel ein Konsument alle möglichen Anbieter kennt, oder daß er alle Anbieter mit vernachlässigbar geringen Suchkosten finden kann. Angenommen er kennt nur m' der vorhandenen m Anbieter, dann erscheint ihm der Markt kleiner, als er in Wirklichkeit ist. Damit ist sein erwarteter Nutzen durch den Markt geringer als bei Kenntnis aller Anbieter. Sucht der Konsument nach neuen Anbietern, reduzieren die Suchkosten seinen Gesamtnutzen aus einer möglichen Transaktion. Dieser Effekt führt, wegen fehlender Information über die anderen Marktteilnehmer, zu einer Abschwächung der Wirkung von Netzwerkeffekten aus der Größe des Marktes.

Angenommen für diesen Informationsmarkt (siehe Abbildung 8.2 auf Seite 97) existiert ein einziger Broker, der seine Vermittlungsleistung anbietet. Im Broker sind $k \leq m$ Anbieter mit ihren Informationsgütern registriert, und diese können ohne Kosten (oder mit vernachlässigbar geringen Kosten) für die Konsumenten abgefragt werden.

Ein Konsument, der $k' \leq k$ dieser Anbieter noch nicht kennt, kann seine Auswahl, und damit seinen erwarteten Nutzen, durch die Teilnahme am Broker um k' auf $m' + k' \leq m$ Anbieter erhöhen. Ist der erwartete Nutzen groß genug, ist es für ihn sinnvoll die Leistungen des Brokers in Anspruch zu nehmen.

Anbieter verwenden den Broker, um sich selbst bekannt zu machen, falls sie sich dadurch eine Erhöhung ihres Nutzens erwarten. Erwartet der Anbieter, daß $n' \leq n$ ihm unbekannte Konsumenten den Broker zur Auswahl von Informationsgütern benutzen, dann steigt sein erwarteter Nutzen aus der Teilnahme am Broker um den Anteil dieser n' potentiellen Konsumenten.

Der Broker bildet einen Teilmarkt des Gesamtmarkts. In diesem Teilmarkt wirken die Netzwerkeffekte aufgrund der besseren Information (geringere Suchkosten) stärker, als im Gesamtmarkt ohne Broker. Das bedeutet, daß das Netzwerk des Brokers schneller wächst als der Gesamtmarkt. Sind dem Wachstum des Brokers keine Grenzen gesetzt (konstante oder sogar fallende Grenzkosten), umfaßt der Broker nach einiger Zeit den gesamten Markt. Da damit im Gesamtmarkt die stärkeren Netzwerkeffekte des Brokers wirken, wird sich der Markt zusätzlich ausdehnen, was zu höheren Nutzen für alle Marktteilnehmer führt.

Zusammengefaßt hängt der Nutzen eines Brokers, und damit die Nachfrage nach seiner Vermittlungsleistung von folgenden Aspekten ab:

- **Die Eigenschaften des Marktes.** Je unübersichtlicher der Markt ist, und je höher dadurch die Suchkosten sind, desto stärker werden die Netzwerkeffekte aus der Größe des Marktes gedämpft, und desto mehr Nutzen kann die Informationsfunktion eines Brokers bieten.
- **Die Eigenschaften des Informationskanals des Brokers.** Je übersichtlicher und vollständiger der Broker Konsumenten über Angebote informiert, desto kleiner werden die Suchkosten, und damit werden die Netzwerkeffekte und der entstehende Nutzen erhöht.

Für das Wachstum des Netzwerks des Informationsbrokers sind die Erwartungen der Marktteilnehmer über den Nutzen durch die Teilnahme am Broker ausschlaggebend. Da Marktteilnehmer ohne Zusatzinformation den genauen Nutzen des Brokers erst feststellen können, nachdem sie

ihn verwendet haben, zählt für die erste Teilnahme am Broker die Erwartung über den Nutzen. Katz und Shapiro [Katz und Shapiro, 1985] gehen bei ihren Modellen zwar immer von einem Gleichgewichtszustand mit erfüllten Erwartungen aus, dennoch betonen sie in ihren Folgerungen, daß die Erwartungsbildung der Marktteilnehmer in Märkten mit Netzwerk-externalitäten außerordentlich wichtig ist. Mehr noch, die Erwartungen legen fest, welcher von verschiedenen möglichen Gleichgewichtszuständen (Netzwerkgrößen) erreicht wird. Dadurch bekommt die Möglichkeit zur Beeinflussung der Erwartungen der Marktteilnehmer, beispielsweise durch Werbung und andere Maßnahmen, eine sehr große Bedeutung für den Informationsbroker.

Die für den Broker ökonomisch sinnvolle Größe hängt stark von seiner Kostenstruktur ab. Die Kosten für die technische Infrastruktur des Informationsbrokers (Server, Datenbanken, Netzwerk) können in den meisten Fällen als fixe beziehungsweise sprungfixe Kosten angesehen werden, da sie auch ohne vollständige Nutzung der Ressourcen anfallen. Der Großteil der variablen Kosten ist direkt von der Anzahl und den Eigenschaften der im Broker vertretenen Informationsprodukte abhängig. Das wichtigste Beispiel sind die Kosten für die laufende Aktualisierung der Informationen (Metadaten), die zu einem Informationsprodukt im Broker gespeichert werden.

Damit sind die wichtigsten Punkte für die Umsetzung des Informationsbrokers die Übersichtlichkeit des Angebots, seine Kostenstruktur und das Management der Erwartungen der Marktteilnehmer. Im folgenden Abschnitt wird die Umsetzung des Informationsbrokers für die Virtuelle Universität durch das Analyse Pattern *Virtuelle Bibliothek* beschrieben.

8.2 Umsetzung des Informationsbrokers

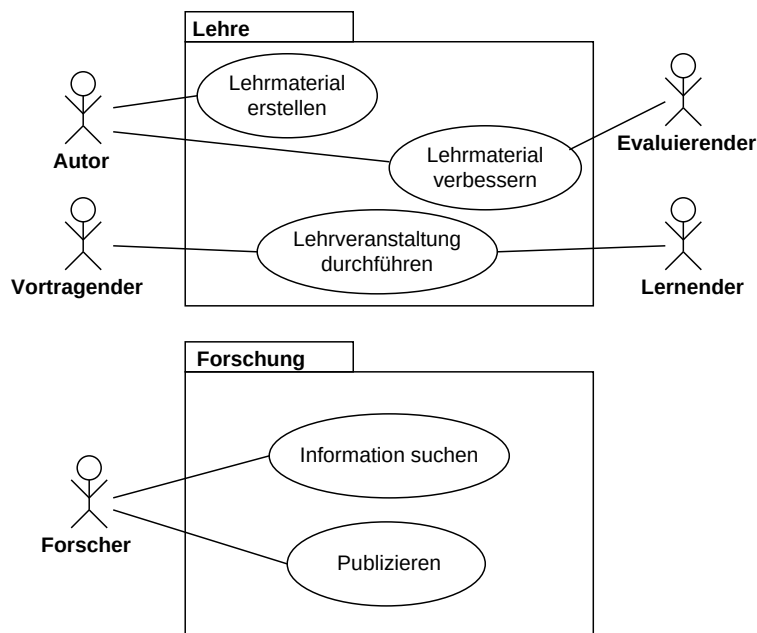
In diesem Abschnitt wird beschrieben, wie das Analyse Pattern *Virtuelle Bibliothek* für den Informationsbroker des VU-Projekts eingesetzt wird. Es werden die benötigten Informationsgüter identifiziert. Danach wird gezeigt, wie das Analyse Pattern *Virtuelle Bibliothek* die konkreten Aufgaben eines Informationsbrokers erfüllt. Abgeschlossen wird der Abschnitt mit einer Beschreibung des für das VU-Projekt eingesetzten Prototyps des Informationsbrokers.

8.2.1 Identifikation der Informationsgüter

Für die Virtuelle Universität muß definiert werden, welche Informationsgüter durch den Informationsbroker vermittelt werden sollen. Dazu

müssen die Aufgabenbereiche untersucht werden, die durch den Informationsbroker unterstützt werden sollen. Der Informationsbroker der Virtuellen Universität, in weiterer Folge kurz VU genannt, hat als Aufgabe, Lehre und Forschung zu unterstützen. Um herauszufinden, welche Informationsgüter in Frage kommen, müssen die unmittelbaren Zielgruppen des Informationsbrokers, ihre möglichen Rollen und ihre Aktivitäten identifiziert werden.

Use Cases:



Rollenhierarchie:

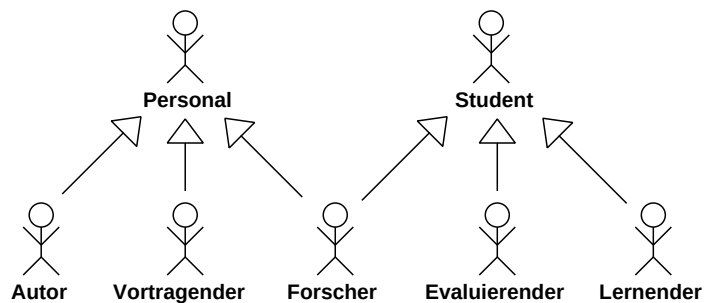


Abbildung 8.3: Use Case Diagramm: Typische Rollen des Personals und der Studenten in Lehre und Forschung

Identifikation von Zielgruppen, Rollen und Aktivitäten. Die Zielgruppen eines Informationsbrokers zur Unterstützung des Lehr- und Forschungsprozesses an einer Universität sind leicht zu erkennen. Grob vereinfacht gibt es an einer Universität wissenschaftliches Personal, das Lehrveranstaltungen abhält, und Studenten, die diese Lehrveranstaltungen besuchen. Diese zwei Zielgruppen nehmen im Lernprozeß verschiedene Rollen ein. Im Use Case Diagramm in Abbildung 8.3 sind typische Rollen des wissenschaftlichen Personals und der Studenten mit ihren Tätigkeiten dargestellt. Das wissenschaftliche Personal erstellt Lehrmaterial für Lehrveranstaltungen, führt Lehrveranstaltungen mit Studenten durch und verbessert die Lehrunterlagen aufgrund des studentischen Feedbacks. Außerdem forschen das wissenschaftliche Personal und die Studenten. Diese Aktivitäten werden in Tabelle 8.1 kurz beschrieben.

Gruppen von Informationsgütern. Um die beschriebenen Aktivitäten zu unterstützen, ergeben sich verschiedene Gruppen von gleichartigen Informationsgütern, die zur Unterstützung der Aktivitäten notwendig sind. Für das VU-Projekt wurden die Lehrveranstaltungen (mit den dazugehörigen Lehrunterlagen) als zentrale Informationsgüter identifiziert.

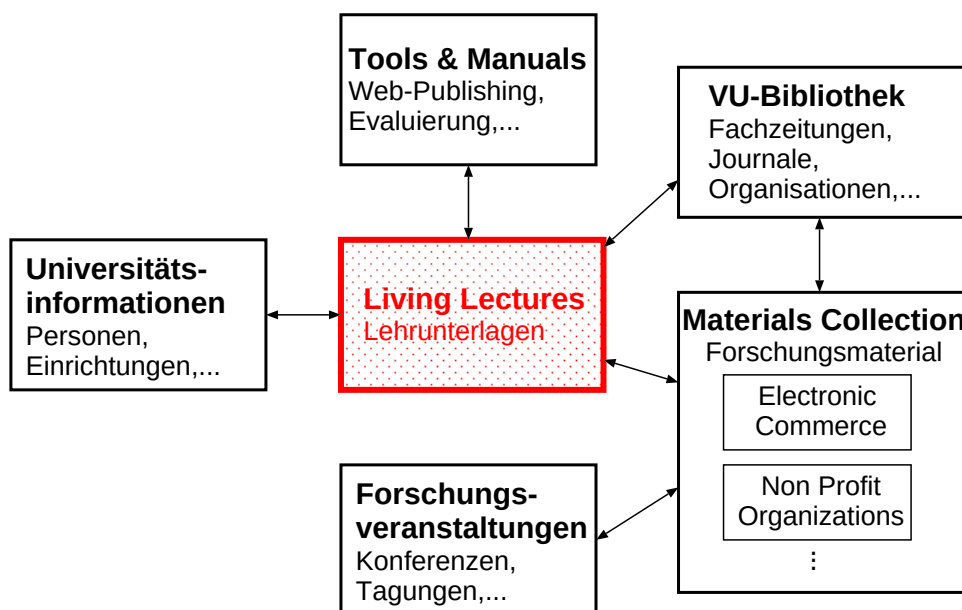


Abbildung 8.4: Inhalte des VU Informationsbroker

Im Mittelpunkt von Abbildung 8.4 steht daher die Gruppe der Lehrveranstaltungen, in der VU *Living Lectures* genannt. Diese Unterlagen werden von einem Autor erstellt, von mehreren Vortragenden (potentiell andere Personen als der Autor) verwendet und von vielen Lernenden genutzt. Der

Aktivität	Rollen	Beschreibung
Lehrmaterial erstellen	Autor	Der Autor muß Quellen für die Lehrveranstaltung sammeln und daraus Lehrmaterialien zusammenstellen. Ein wichtiger Teil dieser Lehrmaterialien sind Unterlagen für die Lernenden (Handouts, Aufgaben, Beispiele, ...).
Lehrveranstaltung durchführen	Vortragender, Lernender	Neben der Vermittlung der Lehrinhalte fällt für den Vortragenden auch der organisatorische Aufwand für die Verteilung von Unterlagen an. (Zum Beispiel: Wo und wie bekommt der Lernende die Unterlagen?) Der Lernende benötigt Informationen zur Lehrveranstaltung, bevor er entscheiden kann, ob er diese Lehrveranstaltung besuchen soll. Besucht der Lernende eine Lehrveranstaltung, benötigt er die passenden Unterlagen.
Lehrmaterial verbessern	Evaluiender, Autor	Während oder nachdem eine Lehrveranstaltung durchgeführt wird, kann das Lehrmaterial vom Autor verbessert werden. Beispielsweise können Studenten die Rolle von Evaluierenden übernehmen und aufgrund ihrer Verwendung der Materialien Feedback für Verbesserungsmöglichkeiten geben.
Information suchen	Forscher	Der Forscher sucht und sammelt Informationsquellen (Literaturverweise, Artikel, Informationen im Internet, ...) zu den Themen, an denen er gerade arbeitet.
Publizieren	Forscher	Forscher veröffentlichen Forschungsergebnisse (Bücher, wissenschaftliche Artikel, Diskussionspapiere, ...)

Tabelle 8.1: Beschreibung der Aktivitäten

Name *Living Lectures* soll andeuten, daß es sich bei dem Lehrmaterial nicht um starre Information, wie bei gedruckten Skripten oder Büchern handelt, sondern daß die online Unterlagen einem laufenden Weiterentwicklungsprozeß unterliegen, also lebendig sind. Um die Lehrveranstaltungen sind weitere Gruppen von Informationsobjekten organisiert. Diese Gruppen ergeben sich aus der Analyse der zu unterstützenden Aktivitäten.

Lehrmaterial muß in einer für das Internet geeigneten Form erstellt werden. Anleitungen dazu werden im Bereich *Tools & Manuals* gesammelt und den Autoren zur Verfügung gestellt. Weiters benötigt der Autor für die Inhalte des Lehrmaterials aktuelle Informationen. Quellen dafür (Fachzeitschriften, Publikationen von Organisationen, ...) werden in der *VU-Bibliothek* gesammelt. Sind die Unterlagen erstellt, und wird die Lehrveranstaltung abgehalten, benötigt der Lernende die passenden Unterlagen sowie organisatorische Informationen (zum Beispiel die E-Mail Adresse des Vortragenden, Öffnungszeiten der Bibliothek, ...). Die Quellen für Informationen rund um die Universität werden im Bereich *Universitätsinformationen* angeboten. Schließlich werden noch für die universitäre Forschung der Bereich *Materials Collection* mit Forschungsmaterial und der Bereich *Forschungsveranstaltungen* bereitgestellt.

Zur Unterstützung der Forschung werden für das VU-Projekt im Forschungsbereich *Materials Collection* noch spezialisierte Kataloge (zum Beispiel zum Thema Electronic Commerce) angeboten. Der Broker stellt Forschungsgruppen die Funktionalität für die Verwaltung von Katalogen zur Verfügung, die diese autonom verwalten.

Die Eintragung von Informationsgütern in die Kataloge des Brokers erfolgt freiwillig. Für Lehrveranstaltungen geschieht dies in der Regel durch den Vortragenden (meist gleichzeitig der Autor). In die anderen Kataloge (*Tools & Manuals*, *VU-Bibliothek*, ...) trägt ein Benutzer oft fremde Informationsgüter ein. Da diese Eintragung mit einem gewissen Aufwand verbunden ist, werden nur Informationsgüter eingetragen, die der Benutzer sehr interessant und nützlich findet. Dadurch wird schon bei der Eintragung eine Auswahl getroffen, die in weiterer Folge auch als eine explizite Empfehlung durch den Eintragenden betrachtet werden kann.

8.2.2 Der Informationsbroker als eine Virtuelle Bibliothek

Das Analyse Pattern *Virtuelle Bibliothek* verwendet seine eigene Sprache zur Beschreibung seines abstrakten Konzepts. Damit es auf ein reales Problem angewandt werden kann, muß zuerst geklärt werden, welches reale Objekt mit jedem abstrakten Begriff im Pattern gemeint ist.

Die abstrakte Beschreibung des Zwecks (*Intent*) des Analyse Patterns *Virtuelle Bibliothek* (siehe Abschnitt 5.3) lautet:

Einheitlicher und effizienter Zugang zu verteilten Informationsquellen und Diensten mit zentraler Suchmöglichkeit.

Die verteilten Informationsquellen und Dienste im Pattern stellen für den Informationsbroker die Informationsgüter dar. Die im Pattern beschriebene, zentrale Suchmöglichkeit ist der Informationskanal des Brokers. Folgende im Pattern beschriebene Kräfte (*Forces*) sind für den Broker wichtig:

- Informationsobjekte werden oft verändert. Das bedeutet konkret, daß Informationsgüter, wie Lehrveranstaltungsunterlagen oft angepaßt oder aktualisiert werden.
- Informationsobjekte in einer Vielzahl von Formaten müssen unterstützt werden. Da jeder sein eigenes Autorenwerkzeug verwenden möchte, und unterschiedliche Formate im Internet üblich sind, tritt diese Kraft bei den Informationsgütern des VU-Brokers auf.

Weiters müssen die konkreten Auswirkungen der beschriebenen Konsequenzen (*Consequences*) beachtet werden. Die negativen Konsequenzen des Patterns scheinen auf den ersten Blick kein unüberwindbares Problem für die Verwendung des Patterns zur Umsetzung des Informationsbrokers darzustellen. Die schwierige Durchsetzung einer gemeinsamen *Corporate Identity* für alle Informationsobjekte in der Virtuellen Bibliothek stellt für den Broker des Projekts kein Problem dar, da es sich nur um eine Vermittlungsleistung handelt, und die Informationsobjekte die *Corporate Identity* des jeweiligen Anbieters behalten sollen. Das Problem mit ungültigen Verweisen stellt ein größeres Hindernis dar, für das noch eine Lösung gefunden werden muß.

Im Abschnitt *Solution* wird beschrieben, daß die Virtuelle Bibliothek auf dem Analyse Pattern *Pinwand mit strukturierten Nachrichten* (siehe Abschnitt 5.2) aufbaut, daher müssen auch die Konsequenzen dieses Patterns beachtet werden. Als Schwierigkeit wird der Umgang mit unterschiedlichen Typen von Nachrichten auf einer Pinwand beschrieben. Das bedeutet, daß Einträge mit unterschiedlich aufgebauten Metadaten in der Virtuellen Bibliothek schwer zu verwalten sind. In Abschnitt 8.2.1 wurden mehrere unterschiedliche Typen von Gütern für den VU-Broker identifiziert. Um dieses Problem zu lösen, wird entschieden, Informationsobjekte, die so unterschiedlich sind, daß sie verschiedene Metadaten benötigen, in unterschiedlichen virtuellen Bibliotheken zu sammeln. Diese virtuellen Bibliotheken stellen unterschiedliche Produktkataloge des Brokers dar (siehe Abbildung 8.4, Seite 102).

8.2.3 Design und Implementierung des Informationsbrokers

In der Diplomarbeit *Software Patterns: Pinwände* [Hahsler, 1997] wurde eine Implementierung des Patterns *Virtuelle Bibliothek* beschrieben. Diese Implementierung wird verwendet, um für jede Gruppe der benötigten Informationsgüter (siehe Abbildung 8.4, Seite 102) einen online Produktkatalog zu verwalten. Da alle Kataloge ähnlich aufgebaut sind, wird hier stellvertretend für die verschiedenen Gruppen nur der Katalog für den zentralen Bereich für Lehrveranstaltungen *Living Lectures* beschrieben. Es wird der Aufbau der Metadaten (Einträge in den Katalogen), sowie das verwendete Interface beschrieben.

Aufbau der Metadaten. Abbildung 8.5 zeigt ein Beispiel für die Metadaten, die intern für eine Lehrveranstaltung als Eintrag im Katalog des Brokers verwaltet werden. Diese Metadaten sind als Attribut-Wert Liste ($\langle \text{Attribut} \rangle := \{ \langle \text{Wert} \rangle \}$) gespeichert. Die Attribute unterteilen sich in:

- Allgemeine Attribute, die in jedem Katalog vorkommen. Diese Attribute stimmen großteils mit den Elementen des *Dublin Core Element Set V1.0* [Weibel *et al.*, 1998] zur Beschreibung von elektronischen Quellen überein.
 1. CATEGORY - Zuordnung des Informationsguts zu dem im Katalog verwendeten Klassifikationsschema. Bei Lehrveranstaltungen wird die Zuordnung der Lehrveranstaltung zu Fächern im Studienplan der Studienrichtung Betriebswirtschaft an der Wirtschaftsuniversität Wien [WU, 1998] verwendet. (Dublin Core: *Subject & Keywords*)
 2. EMAIL - E-Mail Adresse des Zuständigen für das Informationsgut. (Dublin Core: *Author or Creator*)
 3. KEYWORDS - Schlagworte zu dem Informationsgut. (Dublin Core: *Subject & Keywords*)
 4. NAME - Bezeichnung des Informationsguts. (Dublin Core: *Title*)
 5. URL - Verweis auf das Informationsgut im Internet. (Dublin Core: *Resource Identifier*)
- Spezielle Attribute für den Katalog Lehrveranstaltungen wurden anhand des Vorlesungsverzeichnisses der Wirtschaftsuniversität Wien [WU, 1999] identifiziert.
 1. description - Beschreibung der Inhalte der Lehrveranstaltung.

```

CATEGORY := {Betriebswirtschaftslehre/Informationswirtschaft}
DATE := {Wed Jun 4 00:00:00 1997}
EMAIL := {Michael.Hahsler@wu-wien.ac.at}
ID := {wu01_4a}
KEYWORDS := {C++, Java, objektorientiertes Programmieren}
LAST_MODIFIED := {Tue Jul 11 10:44:49 2000}
MD5 := {de374ac38321369b9aa60f52f2454d7b+10241}
NAME := {Einführung in das Objektorientierte Programmieren mit C++}
PAGE_EXTRACT := {
title= Home-Page C++ Arbeitsgemeinschaft
meta= C++ course, Vienna, Austria, WU-Wien, Homepage
meta= Wirtschaftsuniversitaet, lectures, teaching, C++
meta= Lehrunterlagen, course material, object oriented
meta= Gottfried Rudorfer, Michael Hahsler
h1= Arbeitsgemeinschaft
h1= Objektorientiertes Programmieren
h2= Aktuelles:
a= <a href="http://aif.wu-wien.ac.at/usr/geyers/archive/iworg/">
  Informationsmanagement in Organisationen II</a>
a= <a href="http://wwwai.wu-wien.ac.at/sbwl/">SBWL
  Informationswirtschaft</a>
h2= Folien, Literatur, Material:
...
}
Studienabschnitt := {2}
URL := {http://miss.wu-wien.ac.at/usr/cppag/}
description := {Die Arbeitsgemeinschaft Objektorientiertes
Programmieren in C++ und Java ist eine ...}
hits := {78.088}
language := {Deutsch}
lvtype := {AG}
mark_hot := {HOT}
org_unit := {Informationswirtschaft}
teacher := {Hahsler Michael}

```

Abbildung 8.5: Metadaten einer Lehrveranstaltung

2. language - Unterrichtssprache.
 3. lvtype - Art der Lehrveranstaltung (Vorlesung, Seminar, Übung, ...)
 4. org_unit - Organisationseinheit, zu der die Lehrveranstaltung zugeordnet ist (Institut).
 5. teacher - Name des Vortragenden.
 6. studienabschnitt - Stellung der Lehrveranstaltung im Studium (Studienabschnitte im Studienplan).
- Attribute für die Verwaltung der Einträge im Katalog stammen von der verwendeten Implementierung, der Virtuellen Bibliothek [Hahsler, 1997].
 1. DATE - Zeitpunkt der Eintragung des Informationsguts in den Broker.
 2. ID - Durch den Broker vergebener, eindeutiger Schlüssel für das Informationsgut.
 3. LAST_MODIFIED - Datum der letzten Änderung der ersten Web-Seite des Informationsguts (aus den Angaben im HTTP-Protokoll [Fielding *et al.*, 1997] durch den jeweiligen Web-Server).
 4. MD5 - Quersumme und Länge (in Zeichen) des Inhalts der Startseite des Informationsguts für die Erkennung von Änderungen. Zur Berechnung der Quersumme wird der *MD5 Message Digest-Algorithmus* [Rivest, 1992] verwendet.
 5. PAGE_EXTRACT - Automatisch aus dem Informationsgut extrahierte Phrasen (Titel, Metadaten, Überschriften, Verweise, ... aus der Startseite).
 6. hits - Wird intern für die Verwendungsstatistik des Informationsguts verwendet.

Umsetzung des Interfaces. Abbildung 8.6 zeigt die Startseite des Katalogs für Lehrveranstaltungen. Diese Startseite ist das Inhaltsverzeichnis eines Produktkatalogs des Brokers.

Die Hauptelemente sind die Kopfzeile mit den Navigationselementen, die Suchmaske und das Verzeichnis aller vorhandenen Kategorien. Im Navigationsteil befindet sich auch die Funktion "Add URL", um einen Eintrag für eine neue Lehrveranstaltungen samt Metadaten (Bezeichnung, Hersteller, Fachgebiet, ...) in den Katalog einzutragen. Mit der Suchmaske können die Metadaten aller eingetragenen Lehrveranstaltungen nach Schlagworten durchsucht werden. Das Verzeichnis der Kategorien dient für die fachgebietsweise Abfrage des Inhalts und als Orientierungshilfe, welche In-

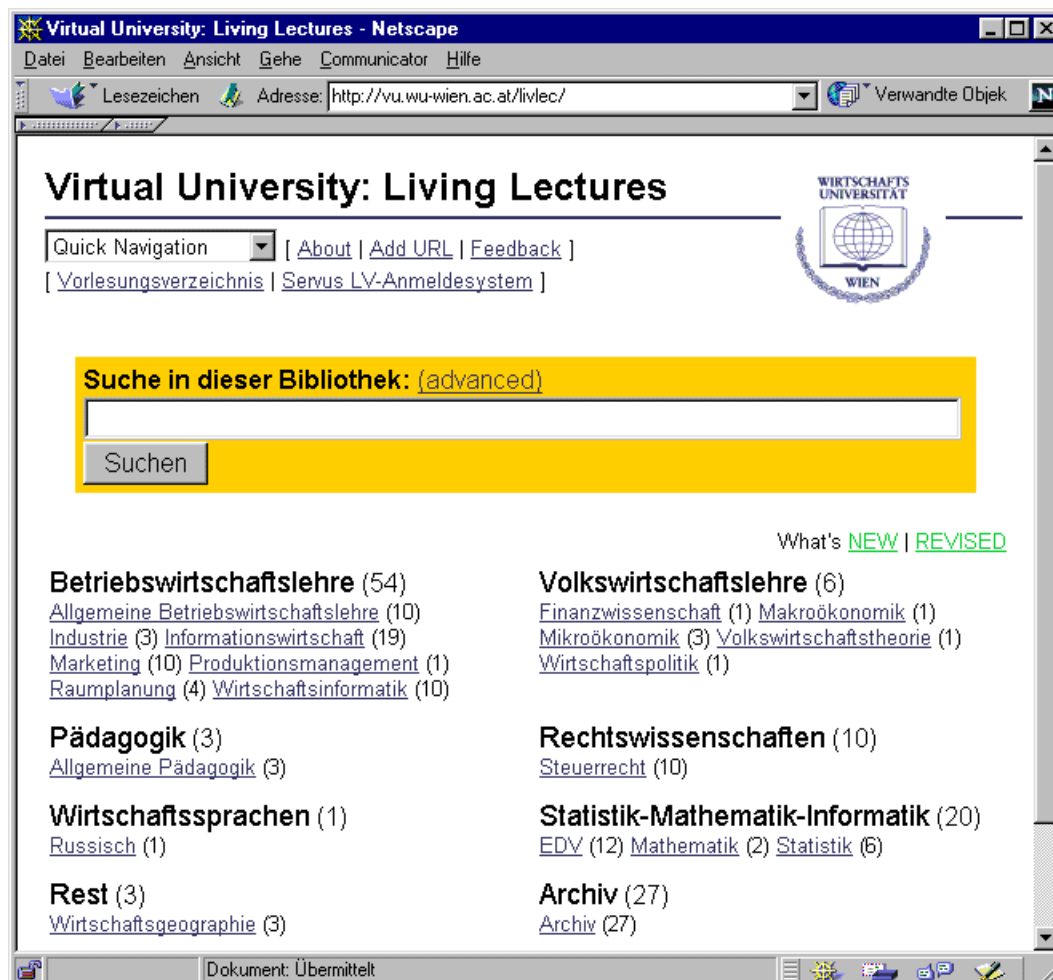


Abbildung 8.6: Startseite des Produktkatalogs für Lehrveranstaltungen

halte im Broker zu finden sind. Wie schon für das Analyse Pattern *Pinwand mit strukturierten Nachrichten* (siehe Abschnitt 5.2) beschrieben, existieren für viele Anwendungsbereiche bereits Klassifikationssysteme, die als Kategorien verwendet werden können. Für die Fachgebiete im Katalog für *Living Lectures* wird die an der Wirtschaftsuniversität Wien übliche Einteilung verwendet (Studienplan der Studienrichtung Betriebswirtschaft [WU, 1998]).

Abbildung 8.7 zeigt das Ergebnis einer Abfrage des Produktkatalogs für Lehrveranstaltungen. Dieses Ergebnis stellt die Antwort des Brokers auf die Anfrage durch einen Konsumenten dar. Es handelt sich um eine Liste von Lehrveranstaltungen, die den eingegebenen Suchkriterien des Konsumenten entsprechen. Im oberen Bereich befindet sich die Suchmaske mit den Suchkriterien. Hier hat der Benutzer die Möglichkeit, die Suche durch zusätzliche Schlagworte weiter einzuschränken. In der Liste im unteren Teil wird für alle gefundenen Einträge ein Teil der Metadaten (Vortragender, Bezeichnung, Kategorie, ...) angezeigt. Die Bezeichnung ist mit einem Hyperlink auf die tatsächlichen Unterlagen beim Anbieter unterlegt.

In Abbildung 8.8 ist die Sicht des Benutzers auf die Metadaten einer Lehrveranstaltung im Broker dargestellt. Diese Sicht ist ein Ausschnitt der vorhandenen Metadaten, anhand der der Konsument entscheiden können muß, ob das Informationsgut für ihn interessant ist oder nicht. Diese Sicht auf die Metadaten für Lehrveranstaltungen beinhaltet die Bezeichnung der Veranstaltung, den Verweis (URL) auf die Unterlagen (beim Anbieter), den Namen des Vortragenden mit E-Mail Adresse, organisatorische Informationen, die Zuordnung zu einem Fach (Kategorie), Lehrinhalte und Schlagworte.

Der Informationsbroker des VU-Projekts verwaltet neben dem eben beschriebenen Katalog für Lehrunterlagen noch weitere Kataloge (siehe Abbildung 8.4, Seite 102). Diese einzelnen Kataloge sind sehr ähnlich und unterscheiden sich nur durch den Aufbau der Metadaten für die Informationsobjekte und geringfügig durch das verwendete Interface. Über die Internet-Seite der *Virtuellen Universität der Wirtschaftsuniversität Wien* <http://vu.wu-wien.ac.at> können alle Kataloge verwendet werden.

8.3 Konsequenzen der Verwendung der Virtuellen Bibliothek

Durch die Verwendung des Analyse Patterns *Virtuelle Bibliothek* wird ein zentrales, durchsuchbares Verzeichnis für den Informationsbroker reali-

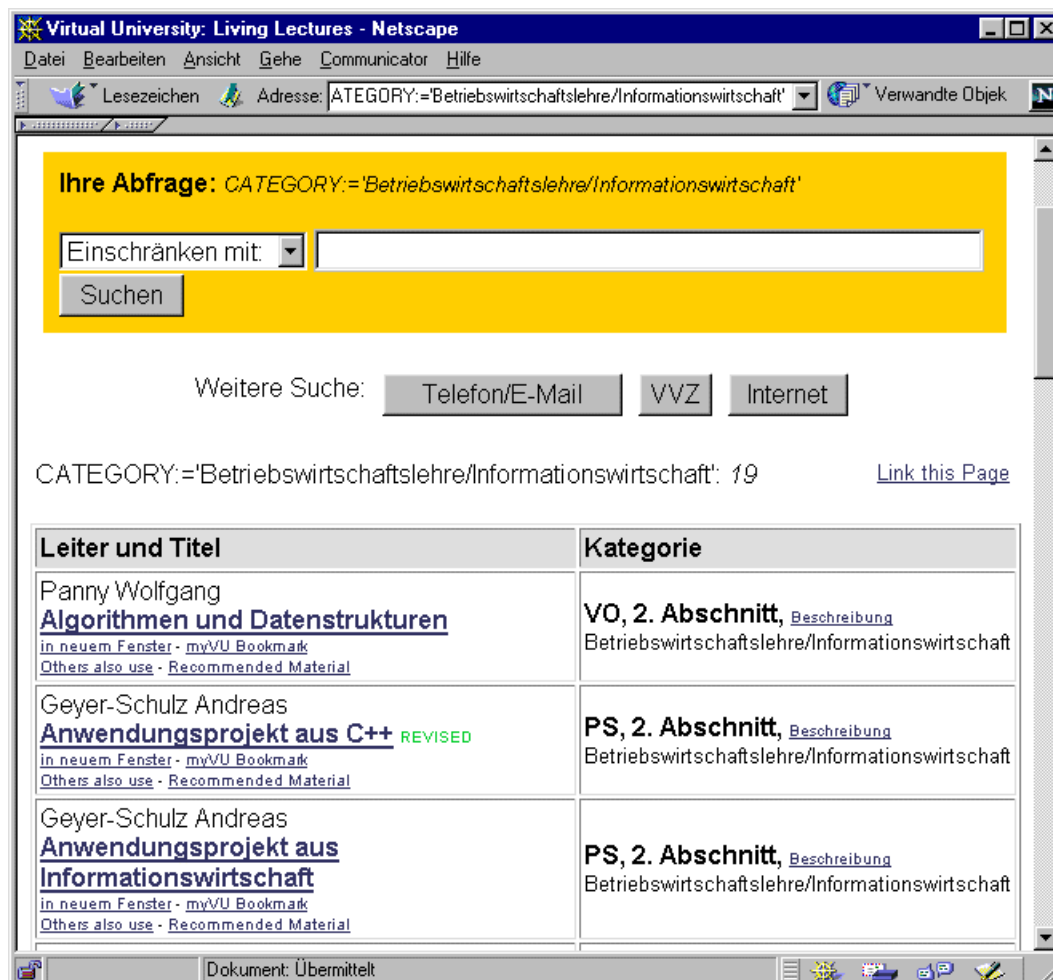


Abbildung 8.7: Ergebnis einer Abfrage des Katalogs

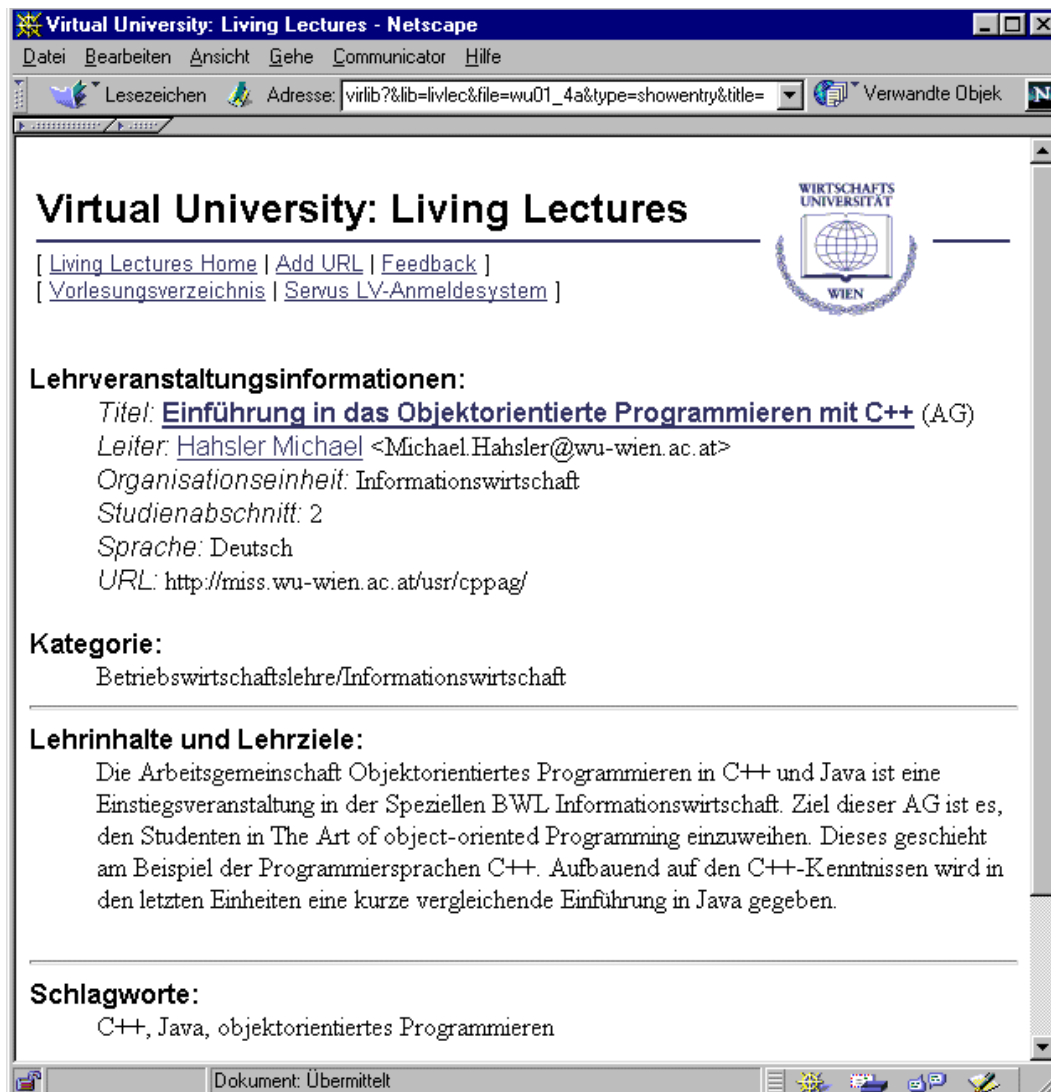


Abbildung 8.8: Sicht des Benutzers auf die Metadaten einer Lehrveranstaltung

siert. Durch Analyse Patterns werden aber nicht nur technische Probleme gelöst, sondern ihre Stärke liegt in der Berücksichtigung von organisatorischen und sozialen Konsequenzen. Solche Konsequenzen, die im Analyse Pattern bereits unter *Consequences* angesprochen wurden, treten für den Katalog *Living Lectures* in folgender Form auf:

- Durch die Verteilung können Informationsgüter jederzeit, ohne Beschränkung verändert werden. Dadurch können zum Beispiel die Unterlagen zu Lehrveranstaltungen vom Lehrenden jederzeit verbessert oder erweitert werden. Wird ein System mit einer zentralen Datenbank eingesetzt, wäre dies nur schwieriger möglich.
- Der Ersteller von Lehrmaterial (in der Regel ein Lehrender) behält die volle Kontrolle über sein Produkt. Er übergibt dem Broker nur die Metadaten mit dem Verweis auf das Material, das Material selbst bleibt aber in seinem Bereich. Diese Eigenschaft verringert Widerstände der Lehrenden, die die Kontrolle über von ihnen freiwillig erstellte Unterlagen behalten wollen. Der Lehrende kann jederzeit selbst entscheiden, ob er den Zugang zu seinen Unterlagen beschränkt oder kostenpflichtig machen will. Damit werden die von Noble [Noble, 1998] und Margolis [Margolis, 1998] beschriebenen sozialen Gefahren von Informationstechnologie für Universitäten (Ausbeutung und Abbau von Lehrenden, Universitäten als reine Lehrinstitutionen ohne Forschung, soziale Verarmung der Studenten, ...) berücksichtigt und damit Ängste bei den Beteiligten abgebaut.
- Durch die Unterstützung von allen Formaten, die im Internet möglich sind, wird den Lehrenden nicht vorgeschrieben, wie und mit welchem Werkzeug sie ihre Lehrmaterialien erstellen müssen. Ein Beispiel für diese Problematik für Lehrinstitutionen beschreibt Turoff [Turoff, 1999]. Er meint, daß Versuche, Lehrende bei der Entwicklung von online Lehrmaterialien in Hinblick auf Aussehen, Struktur oder verwendete Werkzeuge einzuschränken, Kreativität ersticken, und damit zu Widerständen bei den Lehrenden führen werden. Die freie Wahlmöglichkeit der bevorzugten Werkzeuge und Medien ist daher wichtig für die Akzeptanz und damit für den langfristigen Erfolg eines Systems, wie dem Informationsbroker des VU-Projekts.

Quantifiziert kann die Akzeptanz des Systems durch die Anzahl der Personen werden, die freiwillig am VU-Projekt mitarbeiten oder es unterstützen. Derzeit, Stand Oktober 2000, sind das:

- Mehr als 15 Personen arbeiten freiwillig oder im Rahmen ihrer Tätigkeit an der Universität als Bibliothekare zur Wartung der Inhalte des Informationsbrokers mit.

- 62 Lehrende (8 externe Lektoren oder Gastprofessoren und 54 Mitarbeiter der Universität) sind im Informationsbroker mit Informationen und Unterlagen zu 119 Lehrveranstaltungen (entspricht ungefähr 6,5% der insgesamt pro Semester angebotenen Lehrveranstaltungen) vertreten. Damit verwenden mehr als 17% der 317 derzeit an der Wirtschaftsuniversität beschäftigten lehrenden Personen (Assistenten und Professoren) den Informationsbroker, um Unterlagen anzubieten, und das, obwohl es dazu keinerlei finanziellen oder organisatorischen Anreiz seitens der Universität gibt.
- Es bestehen mehrere Kooperationen mit Verwaltungs- und Forschungseinrichtungen der Wirtschaftsuniversität Wien. Beispielsweise werden Teile der Internetpräsenz der Universität durch den VU-Informationsbroker verwaltet, und der Informationsbroker wird von 3 Abteilungen der Universität für die Sammlung von Forschungsmaterial benutzt.
- Und nicht zuletzt haben weit über 600 Personen freiwillig Informationen in den Informationsbroker eingegeben, und dabei ihren Namen beziehungsweise ihre E-Mail Adresse angegeben.

Durch das Analyse Pattern *Virtuelle Bibliothek* wurden einige Punkte unzureichend gelöst, oder gar nicht angesprochen. Folgende Beispiele wurden bereits in den Konsequenzen des Patterns (siehe Abschnitt 5.3) beschrieben:

- Das organisatorische Problem mit ungültigen Verweisen nach der Löschung oder Verschiebung von Informationsgütern, durch zum Beispiel eine Reorganisation der Web-Seiten eines Informationsanbieters. Die Verweise in den Metadaten der Einträge im Broker müssen daher regelmäßig manuell überprüft werden. Dadurch entstehen dem Betreiber des Informationsbrokers Kosten, die mit der Größe des Systems (der Anzahl der im Broker vertretenen Informationsprodukte) proportional ansteigen. Wächst der Broker aufgrund von Netzwerkeffekten (siehe Abschnitt 8.1.2) stark, stellt dies den Betreiber vor ein enormes Kostenproblem.
- Wie kann die Akzeptanz zur freiwilligen Verwendung des Informationsbrokers durch zusätzlichen Nutzen für Anbieter und Konsumenten weiter erhöht werden? Dies wird im Pattern *Virtuelle Bibliothek* nicht behandelt.

Im folgenden Kapitel werden diese Fragen behandelt, und es wird gezeigt, welchen Einfluß Transaktionskosten und die Verwendung von aktiven Agenten auf den Informationsbroker haben.

Kapitel 9

Erweiterung des Informationsbrokers durch das Analyse Pattern Virtuelle Bibliothek mit aktiven Agenten

Im Kapitel 8 wurde das Analyse Pattern *Virtuelle Bibliothek* verwendet, um für das VU-Projekt einen Informationsbroker zu realisieren. Dabei blieben jedoch folgende Fragen ungelöst:

- Wie können ungültige Verweise in einem stark wachsenden Informationsbroker behandelt werden?
- Wovon ist die Akzeptanz des Informationsbrokers abhängig? Wie kann die Akzeptanz, und damit die freiwillige Mitarbeit von Lehrenden und Studierenden verstärkt werden?

Diese Fragen werden im Analyse Pattern *Virtuelle Bibliothek mit aktiven Agenten* behandelt. Durch das Analyse Pattern sollen folgende Kräfte (*Forces*) in ein stabiles Gleichgewicht gebracht werden:

- Steigende Anzahl von verteilten Informationsobjekten oder Wachstum der Größe der Informationsobjekte, wobei Veränderungen von Informationsobjekten schnell erkannt werden müssen.
- Transaktionskosten sind von zentraler Bedeutung für die Wirtschaftlichkeit, die Verwendbarkeit und die Akzeptanz des Systems.
- Flexibilität bei der Erweiterung. Neue Funktionen zur Reduktion von Transaktionskosten müssen schnell und einfach integrierbar sein, ohne den Rest des Systems zu gefährden.

Der abstrakte Zweck (*Intent*) des Patterns wird beschrieben als:

Reduktion von Transaktionskosten bei der Sammlung, Klassifikation, Verwaltung, Suche und der Verwendung von verteilten Informationsquellen und Diensten.

In diesem Kapitel wird dargestellt, wie aktive Agenten des Analyse Patterns *Virtuelle Bibliothek mit aktiven Agenten* für den Informationsbroker der VU angewandt werden. Der Einsatz dieser Agenten reduziert die variablen Betriebskosten des Brokers, wodurch eine größere Anzahl von Informationsgütern effizient verwaltet werden können. Weiters werden Agenten verwendet, um die Transaktionskosten der Benutzer (Anbieter und Konsumenten von Informationsgütern) zu reduzieren, beziehungsweise so zu verändern, daß ihr Nutzen, und damit auch die Verwendung und Akzeptanz des Systems, erhöht wird.

In Abschnitt 9.1 wird die Struktur der Transaktionskosten auf Informationsmärkten analysiert. Dabei wird gezeigt, wie die Anzahl der Anbieter und Konsumenten, die einen Informationsbroker verwenden, direkt von der erwarteten Reduktion ihrer Transaktionskosten durch den Broker abhängt.

In Abschnitt 9.2 wird eine dreischichtige Architektur zur Erweiterung des in Kapitel 8 vorgestellten Informationsbrokers dargestellt. In dieser Architektur werden aktive Agenten in allen Schichten dazu verwendet, die Transaktionskosten der Anbieter und Konsumenten sowie die Betriebskosten des Brokers zu reduzieren.

In den Abschnitten 9.3 und 9.4 werden einige aktive Agenten für die Architektur des Informationsbrokers vorgestellt, und ihre Eigenschaften werden beschrieben.

Abgeschlossen wird das Kapitel mit einer Diskussion der Konsequenzen der Verwendung von aktiven Agenten, dabei wird analysiert, wie sie die Betriebskosten des Brokers sowie die Transaktionskosten der Anbieter und Konsumenten reduzieren.

9.1 Analyse der Transaktionskosten auf Informationsmärkten

Auf Informationsmärkten fallen, wie auf allen Märkten, Kosten für Transaktionen an. Diese sogenannten Transaktionskosten sind wichtig, um herauszufinden welche Arten von aktiven Agenten für einen Broker auf einem Informationsmarkt eingesetzt werden sollen, damit er den Nutzen

der Marktteilnehmer erhöhen kann. Deshalb werden in diesem Abschnitt zuerst Begriffe aus der Transaktionskostentheorie, wie Motivations- und Koordinationskosten, erklärt. Danach werden die Transaktionskosten auf einem Informationsmarkt untersucht, und der Einfluß eines Brokers analysiert.

9.1.1 Transaktionskosten

Ronald Coase wurde 1991 der Nobel-Preis für die Entdeckung der Bedeutung von Transaktionskosten für die institutionelle Struktur der Wirtschaft verliehen. In *The Nature of the Firm* [Coase, 1937] begründet er die Bildung von Organisationen auf einem Markt durch die Bestrebung Transaktionskosten zu minimieren. Er beschreibt Transaktionskosten auf einem Markt, als die Kosten dafür, daß der Marktpreis als Koordinationsmechanismus verwendet wird. Er meint damit die Kosten, den relevanten Preis zu finden, und die Kosten einen Vertrag für die Transaktion auszuhandeln und abzuschließen. Milgrom und Roberts [Milgrom und Roberts, 1992, S.25–30] beschreiben die Herkunft und die Eigenschaften von Transaktionskosten. Sie teilen Transaktionskosten in Kosten für Koordination und Kosten für Motivation ein:

1. **Koordinationskosten** entstehen dadurch, daß Aktivitäten aufeinander abgestimmt werden müssen. Wird ein Markt zur Koordination von Aktivitäten verwendet, treten beispielsweise folgende Koordinationskosten auf:
 - Kosten, die entstehen, um Anbieter und Konsumenten auf einem Markt zusammen zu bringen, und dadurch eine Transaktion zu ermöglichen.
 - Kosten für Marktforschung, um Preise festzulegen oder um die Konsumentenwünsche zu erheben.
 - Werbekosten, um ein Gut bei potentiellen Käufern bekannt zu machen.
 - Suchkosten der Käufer nach Anbietern und dem Angebot mit dem besten Preis.
2. **Motivationskosten** spiegeln die Anreizstruktur bei Transaktionen wider. Folgende Probleme treten dabei auf:
 - Unvollständige Information und Informationsasymmetrien. Es handelt sich um Situationen, in denen ein Teilnehmer nicht über die nötigen Informationen verfügt, um festzustellen, ob die

Transaktion unter den gewählten Bedingungen akzeptabel ist, oder ob der Transaktionspartner die Verpflichtungen der Transaktion auch erfüllen wird.

- **Unvollkommene Verpflichtung.** Dies ist das Problem, daß sich ein Teilnehmer nicht verpflichten kann, an einer für beide Seiten vorteilhaften Transaktion teilzunehmen, da er die Teilnahme später eventuell rückgängig machen möchte. Ein Grund dafür ist beispielsweise Angst vor opportunistischem Verhalten des Transaktionspartners bei transaktionsspezifischen Investitionen. Das bedeutet, muß ein Partner eine Investition für die Transaktion tätigen, die irreversible Kosten ("Sunk Costs") verursacht und für andere Transaktionen schlecht verwendbar ist, kann der zweite Transaktionspartner danach opportunistisch handeln und versuchen, bessere Konditionen für sich auszuverhandeln. Deshalb wird der erste Partner aus Angst vor diesem Verhalten auf die Transaktion verzichten, und eine Chance für eine vorteilhafte Transaktion wird vergeben.

Transaktionskosten entstehen nicht nur in Zusammenhang mit einem durchgeführten Verkauf, sondern auch für abgebrochene oder nicht zustande gekommene Transaktionen. Es gibt viele Gründe für den Abbruch einer Transaktion, beispielsweise wenn ein Partner seinen Teil der Abmachung nicht einhält. Nicht zustande gekommene Transaktionen entstehen zum Beispiel, wenn kein geeigneter Transaktionspartner gefunden wird, oder ein Partner sich nicht verpflichten kann oder will.

Die Höhe der Transaktionskosten hängt von der Natur der Transaktion und der Art, wie die Transaktion organisiert ist, ab. Folgende Attribute von Transaktionen spielen dabei eine wichtige Rolle [Milgrom und Roberts, 1992, S.30–33]:

- **Transaktionsspezifische Investitionen (Asset Specificity).** Es gibt Transaktionen, für die bedeutende, irreversible Investitionen ("Sunk Costs") notwendig sind. Ist diese Investition nicht oder nur schlecht für Transaktionen mit anderen Partnern einsetzbar, handelt es sich um eine transaktionsspezifische Investition. Ein Beispiel ist das Errichten einer Fertigungsstraße durch einen Zulieferer, die spezielle Teile erzeugt, die nur ein Abnehmer benötigt. Ein anderes Beispiel ist das Erlernen eines speziellen Autorenwerkzeugs für die Erstellung von Lehrmaterial.

Wird für eine Transaktion eine transaktionsspezifische Investition benötigt, muß sich der Partner mit der Investition gegen opportunistisches Verhalten durch den Transaktionspartner (späteres Verhan-

deln um bessere Konditionen, beziehungsweise die frühzeitige Beendigung der Transaktion) schützen. Dies kann beispielsweise durch einen Vertrag geschehen.

- **Häufigkeit und Dauer (Frequency and Duration).** Bei Transaktionen, die zwischen den selben Transaktionspartnern nicht regelmäßig auftreten, oder bei kurzen Transaktionen werden aus Kostengründen oft standardisierte Verträge verwendet. Mögliche Streitigkeiten werden vor Gericht verhandelt. Je öfter Transaktionen zwischen den selben Partnern stattfinden, desto wahrscheinlicher wird es, daß spezielle, kostengünstigere Verfahren zur Abwicklung der Transaktionen entwickelt und angewandt werden. Weiters besteht bei wiederholten, oder länger dauernden Transaktionen auch die Möglichkeit, den Transaktionspartner für Fehlverhalten mit eigenem unkooperativen Verhalten zu sanktionieren. Dadurch wird die Notwendigkeit für formale Mechanismen zur Einhaltung der Abmachungen reduziert, und es werden Kosten eingespart.
- **Unsicherheit und Komplexität (Uncertainty and Complexity).** Bei Unsicherheit über die Umstände, unter denen eine Transaktion durchgeführt werden wird, oder bei komplexen Transaktionen, ist es unökonomisch im Voraus genau zu planen, wie die Transaktion in allen Einzelheiten ablaufen muß. In diesem Fall werden oft nur Rechte und Pflichten der Transaktionspartner, sowie ein Verfahren zur Schlichtung von Streitigkeiten, festgelegt.
- **Schwierigkeit der Leistungsfeststellung (Difficulty of Performance Measurement).** Es ist schwierig ein Anreizsystem für die optimale Durchführung von Transaktionen zu schaffen, falls die genaue Messung, wie gut die Leistung bei einer Transaktion erbracht wird, nicht oder nur schwer möglich ist. Oft werden daher Transaktionen so vereinbart, daß entweder die Leistungsfeststellung einfach ist, oder daß gar keine Feststellung notwendig ist, da schlechte Leistung nur dem schlecht Leistenden schadet. Beispielsweise müssen bei vielen Taxiunternehmen die Taxifahrer ihr Fahrzeug selbst kaufen. Damit muß sich das Taxiunternehmen mit der Feststellung, wer für unvorhergesehene Kosten des Fahrzeugs verantwortlich ist, nicht beschäftigen, da solche Kosten aufgrund von schlechter Wartung oder achtlosem Umgang mit dem Fahrzeug (Reparaturen, kurze Haltbarkeit des Fahrzeugs) dem dafür Verantwortlichen, nämlich dem Taxilenker, zufallen.
- **Zusammenhang mit anderen Transaktionen (Connectedness to other Transactions).** Sind mehrere Transaktionen voneinan-

der abhängig, wird es notwendig diese zu koordinieren. Koordination kann durch eine zentrale Planung (zum Beispiel Standardisierung) erfolgen. Weiters ist es möglich, den Koordinationsaufwand zu reduzieren, indem man die Anzahl der zu koordinierenden Partner verringert.

Im folgenden werden Transaktionskosten als Teil der Kostenstruktur von Informationsmärkten betrachtet.

9.1.2 Kostenstruktur eines Informationsmarktes

Auf einem Informationsmarkt werden Informationsgüter zwischen Anbietern und Konsumenten gehandelt. Die Gesamtkosten KG , die bei einem Informationsgut während seiner Lebenszeit anfallen, werden in Formel 9.1 nach ihrer Herkunft aufgeteilt. n ist die Gesamtanzahl der abgeschlossenen Transaktionen (Verkäufe) für ein Informationsgut. KP sind die einmal auftretenden Produktionskosten, KL sind die Kosten für die Speicherung, KR sind die für jede abgeschlossene Transaktion auftretenden Reproduktionskosten des Informationsguts, und KT ist die Summe aller Transaktionskosten.

$$KG = KP + KL + n \times KR + KT \quad (9.1)$$

Die Produktions-, Lagerungs- und Reproduktionskosten von Informationsgütern sind hauptsächlich technologieabhängig und wurden bereits in Abschnitt 8.1.1, bei der Definition von Informationsgütern, beschrieben. Da sie nicht von der Art der Transaktion, also auch nicht vom Vorhandensein eines Brokers am Informationsmarkt abhängen, werden sie in den folgenden Betrachtungen als fix angesehen. Im Unterschied dazu hängen die Transaktionskosten von der Art, wie die Transaktionen organisiert sind, ab. Die Veränderungen der Transaktionen durch einen Informationsbroker werden im folgenden untersucht.

9.1.3 Der Einfluß eines Brokers auf Transaktionskosten

Auf einem Informationsmarkt ohne Informationsbroker werden Informationsgüter direkt zwischen Anbietern und Konsumenten gehandelt. Es gibt keine zentrale Plattform für Informationen über verfügbare Güter, Anbieter und Konsumenten. Die gesamten Transaktionskosten (KT) lassen sich

nach den Rollen der Transaktionsteilnehmer aufteilen. Diese Aufteilung ist in Formel 9.2 dargestellt. Die Menge der Anbieter wird mit A , und die Menge der Konsumenten wird mit K bezeichnet. Den einzelnen Anbietern $a \in A$ entstehen jeweils die Transaktionskosten in der Höhe von KT_a . Einem Konsumenten $k \in K$ entstehen die Transaktionskosten KT_k .

$$KT = \sum_{a \in A} KT_a + \sum_{k \in K} KT_k \quad (9.2)$$

Die Transaktionskosten für Anbieter und Konsument setzen sich aus Koordinations- und Motivationskosten zusammen. Tabelle 9.1 zeigt Beispiele dieser Kostenarten für Anbieter und Konsumenten. Koordinationskosten resultieren hauptsächlich aus Kosten für notwendige Handlungen, wie Marktforschung oder aufgewandte Zeit zum Produktvergleich. Motivationskosten spiegeln sich in nicht durchgeführten Transaktionen, oder in teurem Schutzverhalten gegen opportunistisches Verhalten des Transaktionspartners wider.

	Koordinationskosten	Motivationskosten
Anbieter	Marktforschung, Werbung, ...	Absicherung gegen Zahlungsunfähigkeit oder Betrug durch den Konsumenten, ...
Konsument	Suche nach Anbietern, Vergleich von alternativen Gütern, ...	Verzicht auf eine Transaktion aufgrund fehlenden Vertrauens in den Anbieter, ...

Tabelle 9.1: Beispiele für Transaktionskosten von Anbietern und Konsumenten

Bei Informationsmärkten ohne Broker gehen die Kosten der direkten Geschäftsanbahnung vollständig in die Transaktionskosten der Anbieter und Konsumenten ein. Auf einem Informationsmarkt mit Informationsbroker stellt der Broker eine zentrale Informationsplattform (einen Produktkatalog) für Anbieter und Konsument zur Verfügung. Der Handel erfolgt weiter direkt zwischen Anbieter und Konsument, die Geschäftsanbahnung (Informationen über Anbieter, verfügbare Güter, Konsumentenwünsche, ...) wird jedoch als Dienstleistung vom Broker bereitgestellt. Dadurch werden Transaktionskosten für Anbieter und Konsumenten reduziert.

Teilt man die Gesamtkosten für alle Transaktionen auf einem Informationsmarkt mit Broker (KT') auf die Beteiligten auf, ergibt sich die Kostenstruktur in Formel 9.3. Die Kosten für die Transaktionen werden von den Anbietern ($\sum_{a \in A} KT'_a$), von den Konsumenten ($\sum_{k \in K} KT'_k$) und vom Broker (K_b) getragen.

$$KT' = \sum_{a \in A} KT'_a + \sum_{k \in K} KT'_k + K_b \quad (9.3)$$

Der Broker reduziert durch einen neuen Informationskanal, nämlich durch den zentralen Produktkatalog, die Transaktionskosten für die Geschäftsanbahnung der Marktteilnehmer. Zusätzlich hat er die Möglichkeit, die Transaktionskosten für Anbieter und Konsument durch die Gestaltung von weiteren Dienstleistungen (zum Beispiel: Marktforschungsdaten oder Bonitätsinformationen) zu verändern. In Tabelle 9.2 werden Beispiele zur Veränderung der Transaktionskosten durch den Broker zusammengefaßt.

Die Kosten des Brokers (K_b) entstehen durch das Erstellen, Anbieten und Aktualisieren des Produktkatalogs, dem Betreiben der dazu notwendigen Infrastruktur und durch das Anbieten von weiteren Dienstleistungen, um die Transaktionskosten für Anbieter und Konsument zusätzlich zu beeinflussen. Weiteres entstehen dem Broker Kosten für Marketing, um seine Leistungen zu bewerben. Die gesamten Kosten können als Vermittlungsgebühr, beziehungsweise Gebühr für einzelne Dienstleistungen, verrechnet werden.

Im folgenden wird analysiert, unter welchen Umständen die Marktteilnehmer den Broker verwenden, und wie hoch die Gebühr des Brokers zur Deckung seiner Kosten sein kann. Für diese Analyse werden einige Annahmen getroffen:

1. Alle Marktteilnehmer handeln vollkommen rational und sind risikoneutral.
2. Die Marktteilnehmer kennen ihre Transaktionskosten bevor sie den Broker verwenden (KT_a beziehungsweise KT_k), und sie besitzen Vorinformationen, aus denen sie Erwartungen über ihre Transaktionskosten durch die Teilnahme am Broker ($E(KT'_a)$ beziehungsweise $E(KT'_k)$) bilden. Diese Vorinformationen sind beispielsweise Informationen über den Broker durch andere Marktteilnehmer, durch Werbung oder Experten.
3. Im Gleichgewichtszustand, wenn die Marktteilnehmer den Broker regelmäßig verwenden, gilt, daß die Erwartungen über die Transaktionskosten erfüllt sind.

	Reduktion der Transaktionskosten	Erhöhung der Transaktionskosten
Anbieter	• Mehr Nachfrager werden durch das geordnete einfacher vergleichbare Angebot durch den Broker angezogen. Dadurch sinken die Kosten zum Auffinden eines Transaktionspartners. • Dienstleistungen des Brokers sind günstiger, als die Leistungen selbst zu erbringen, beispielsweise verbesserte Marktforschungsdaten, gezielte Werbung, oder genauere Bonitätsinformation über Konsumenten.	• Der Anbieter muß Informationen über seine Güter zusätzlich an den Broker in einer geeigneten, standardisierten Form übergeben und aktuell halten.
Konsument	• Die Suche nach Anbietern und der Vergleich von alternativen Gütern im Broker ist leichter, da die Produktinformationen im Katalog standardisiert sind. • Der Broker kann zusätzliche Informationen zur Verfügung stellen, beispielsweise Informationen über Güter, die zu den gekauften passen, oder Informationen über die Zuverlässigkeit von Anbietern.	• Das Auffinden von Anbietern, die nicht im Broker aufscheinen, erfordert eine zusätzliche Suche oder ein besser geeignetes Gut wird nicht gefunden.

Tabelle 9.2: Beispiele für die Veränderung der erwarteten Transaktionskosten durch einen Broker

Damit ein Anbieter a sein Gut über den Broker anbietet, müssen dadurch seine erwarteten Transaktionskosten geringer sein als ohne Broker. Beispielsweise muß er erwarten, daß für die Anbahnung des Geschäfts weniger Kosten anfallen werden, oder daß weniger Geschäfte aufgrund von fehlender Information abgebrochen werden. In Formel 9.4 ist die Teilmenge $A' \subset A$ der Anbieter dargestellt, die den Broker aufgrund der erwarteten Kostenvorteile verwenden werden.

$$A' = \{a \mid a \in A \wedge (E(KT'_a) < KT_a)\} \quad (9.4)$$

Damit ein Konsument k den Broker verwendet, müssen seine erwarteten Transaktionskosten ebenfalls geringer als ohne Broker sein. Dies ist der Fall, wenn sich der Konsument aus der Verwendung des Brokers beispielsweise eine kürzere Suchzeit nach benötigten Produkten oder mehr Sicherheit gegen opportunistisches Verhalten durch Anbieter erwartet. In Formel 9.5 ist die Teilmenge $K' \subset K$ der Konsumenten dargestellt, die den Broker verwenden werden.

$$K' = \{k \mid k \in K \wedge (E(KT'_k) < KT_k)\} \quad (9.5)$$

Im Gleichgewichtszustand mit erfüllten Erwartungen ($E(KT'_a) = KT_a$ und $E(KT'_k) = KT_k$), das bedeutet bei wiederholter Verwendung des Brokers durch die selben Marktteilnehmer, ist der maximale Ertrag, den der Broker durch Gebühren direkt von Anbietern und Konsumenten erzielen kann, durch die Einsparung, die er den Marktteilnehmern ermöglicht, begrenzt. Der maximal erzielbare Ertrag $E_{b,max}$ wird in Formel 9.6 dargestellt.

$$E_{b,max} \leq \sum_{a \in A'} (KT_a - KT'_a) + \sum_{k \in K'} (KT_k - KT'_k) \quad (9.6)$$

Wieviel der Broker von diesem Ertrag tatsächlich für sich realisieren kann, ist von zwei Faktoren abhängig. Erstens, wie genau der Broker die Kosteneinsparungen der Anbieter und Konsumenten für jede Transaktion weiterverrechnen kann. Und zweitens, verringert mögliche Konkurrenz durch andere Broker die realisierbaren Erträge.

$$K_b = KF_b + \sum_{m \in M} (KE_m + KW_m) \quad (9.7)$$

Die Kostenstruktur des Brokers entscheidet, ab welchem Ertrag kosten-deckend gearbeitet werden kann. In Formel 9.7 wird die Zusammensetzung der Kosten eines Brokers K_b dargestellt. M ist die Menge der im Informationsbroker eingetragenen Gütern, und m bezeichnet ein einzelnes Gut. Neben den fixen, beziehungsweise sprungfixen Kosten (KF_b) für die Infrastruktur des Brokers (Kosten für Speicherung der Beschreibungen der Informationsgüter, sowie für die Kommunikationsinfrastruktur), sind folgende variable Kosten (abhängig von der Größe M) für den Broker bestimmend:

- Einmalige Kosten für die Eintragung eines zusätzlichen Informationsguts in den Broker (KE). Damit ein Informationsgut vom Broker vermittelt werden kann, muß eine genaue Beschreibung des Guts in den Broker eingegeben werden. Diese Eintragung stellt für den Anbieter Transaktionskosten dar. Die Überprüfung, ob der Eintrag korrekt und vollständig ist, wird durch einen Mitarbeiter des Brokers durchgeführt, und dadurch entstehen einmalige Kosten für den Broker.
- Laufende Kosten für die Wartung der einzelnen Einträge (KW). Damit der Informationsbroker den Konsumenten aktuelle Beschreibungen der Informationsgüter anbieten kann, müssen die im Broker eingetragenen Informationsgüter regelmäßig auf Änderungen und Verfügbarkeit überprüft werden. Diese regelmäßigen Überprüfungen verursachen dem Broker Kosten, die direkt von der Anzahl der eingetragenen Informationsobjekte abhängen.

In den folgenden Abschnitten werden aktive Agenten zur Reduktion der variablen Kosten des Informationsbrokers, sowie der Transaktionskosten der Marktteilnehmer vorgestellt.

9.2 Architektur des Informationsbrokers

In diesem Abschnitt wird der Aufbau des Brokersystems mit aktiven Agenten (siehe Analyse Pattern in Abschnitt 5.4) für das VU-Projekt beschrieben. Abbildung 9.1 zeigt, wie um den Kernbereich des Brokers, den Produktkatalogen, aktive Komponenten (Agenten) organisiert sind, um durch ihre Dienste die Transaktionskosten der Benutzer und die Betriebskosten des Brokers zu verringern. Die Agenten sind durch dicke Kästchen dargestellt, und Pfeile werden verwendet, um Aktionen darzustellen. Pfeile ohne Bezeichnung bedeuten eine einfache Informationsabfrage.

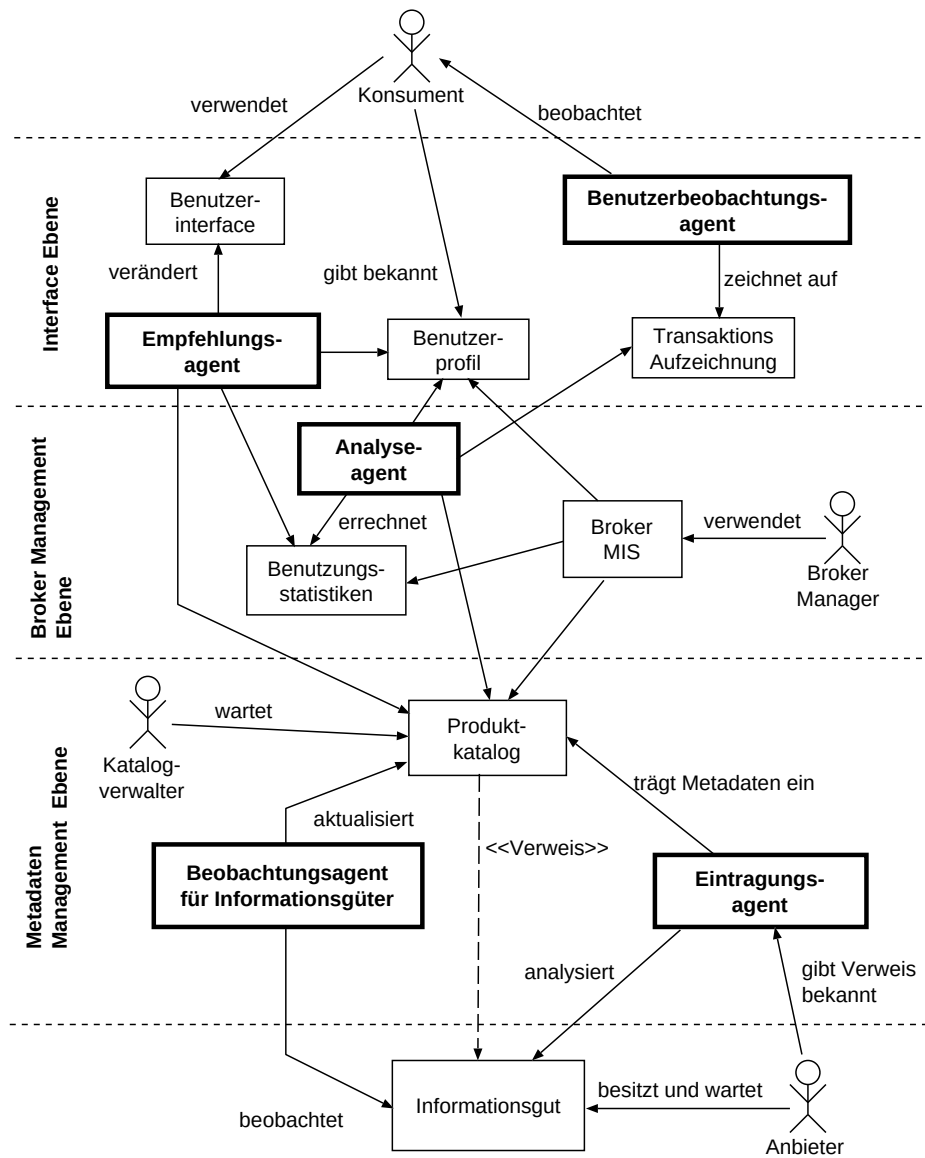


Abbildung 9.1: Struktur des Informationsbrokers der VU mit aktiven Agenten

Die Agenten sollen dem System folgende Eigenschaften verleihen:

- Variable Kosten des Brokers senken, wodurch mehr Informationsgüter verwaltet werden können.
- Transaktionskosten für Anbieter reduzieren, um die freiwillige Verwendung zu erhöhen.
- Transaktionskosten für Konsumenten verringern, um die Akzeptanz des Systems zu steigern.

Das Brokersystem (siehe Abbildung 9.1) besteht aus drei Ebenen. Der Metadaten Management Ebene, der Broker Management Ebene und der Interface Ebene. Diese Ebenen und der Zweck der verwendeten Agenten wird kurz beschrieben.

9.2.1 Metadaten Management Ebene

Auf der Metadaten Management Ebene werden die Produktkataloge des Brokers verwaltet. Katalogverwalter führen notwendige Anpassungen von Katalogeinträgen (Metadaten) durch. Bei einer großen Anzahl von Einträgen entstehen durch diese Verwaltungstätigkeit hohe Kosten. Um diese Kosten zu verringern, werden für diese Ebene aktive Agenten eingesetzt. Diese sind Eintragungsagenten zur Unterstützung der Anbieter und Beobachtungsagenten für Informationsgüter zur Unterstützung der Katalogverwalter.

Der Anbieter (Autor, Vortragender, ...) besitzt die Kontrolle über seine Güter (Lehrunterlagen, Forschungsergebnisse, ...), da sie sich in seinem Web-Bereich befinden. Damit seine Informationsgüter über den Broker vermittelt werden können, muß der Anbieter eine standardisierte Produktbeschreibung (Metadaten) für den Produktkatalog bereitstellen. Dabei wird er durch Eintragungsagenten unterstützt. Eintragungsagenten analysieren Informationsgüter und versuchen automatisch, die für den Produktkatalog notwendigen Metadaten zu extrahieren. Im Idealfall müssen diese Informationen vom Anbieter nur noch überprüft werden. Dieser Dienst reduziert die Eintragungskosten für den Anbieter.

Die Beobachtungsagenten führen Routineüberprüfungen für Katalogeinträge automatisch durch. Beispielsweise testen sie, ob das Informationsgut unter dem in den Metadaten angegebenen Verweis (URL) erreichbar ist, und kennzeichnen ungültige Verweise für den Verwalter. Weiters können Beobachtungsagenten aktuelle Informationen aus den Informationsgütern extrahieren, und damit die Metadaten automatisch aktualisieren. Konkrete Agenten der Metadaten Management Ebene werden in Abschnitt 9.3 beschrieben.

9.2.2 Broker Management Ebene

Die Broker Management Ebene dient zur Analyse der Verwendung des Brokersystems. Durch diese Analyse können Entscheidungen für die weitere Entwicklung des Brokers getroffen werden. Solche Entscheidungen können beispielsweise die Erweiterung der Produktkataloge, Veränderung des Benutzerinterfaces oder die Veränderung der Preispolitik des Brokers sein. Der Broker Manager verwendet das *Broker MIS (Management Information System)*, um Benutzungsdaten, Informationen über Gruppen von Konsumenten und die Struktur der Produktkataloge zu erheben und zu analysieren.

9.2.3 Interface Ebene

Auf der Interface Ebene kann der Konsument (zum Beispiel Lernender oder Forscher bei der Informationssuche) die Produktkataloge des Brokers durchsuchen. Um die Suche zu vereinfachen, werden Beobachtungs- und Empfehlungsagenten verwendet. Die Beobachtungsagenten zeichnen die Transaktionen, die Konsumenten über den Broker vermittelt bekommen, auf. Anhand dieser Aufzeichnungen und dem Benutzerprofil des Konsumenten verändert der Empfehlungsagent das Benutzerinterface, um dem Konsument Suchkosten zu ersparen. Konkrete Agenten auf der Interface Ebene werden in Abschnitt 9.4 entwickelt.

9.3 Agenten der Metadaten Management Ebene

Diese Agenten dienen dem Aufbau und der Verwaltung der Produktkataloge auf der Metadaten Management Ebene des Informationsbrokers (siehe Abbildung 9.1, Seite 126). Die Eintragungsagenten reduzieren die Transaktionskosten, die dem Anbieter bei der Eintragung eines Informationsguts in den Broker entstehen. Die Beobachtungsagenten für Informationsgüter reduzieren die Kosten des Brokers für die Aktualisierung der Einträge im Katalog durch Routineprüfungen. In den folgenden Abschnitten werden Beispiele für solche aktiven Agenten vorgestellt.

9.3.1 Eintragungsagenten

Ziel der Eintragungsagenten ist, den Aufwand zur Eintragung der notwendigen Metadaten in den Produktkatalog für den Anbieter zu mini-

mieren. Im Idealfall gibt der Anbieter nur den Verweis (URL) zu einem Informationsgut bekannt, und bekommt vom Eintragungsagenten ein bereits ausgefülltes Formular. Der Anbieter muß dann nur noch die Angaben überprüfen. Dies erspart Arbeitszeit beim Ausfüllen langer Formulare, da statt dem Auswählen in Menüs und dem Eingeben von Text nur noch die Überprüfung, beziehungsweise die Korrektur der vorgeschlagenen Angaben, anfällt.

Eintragungsagenten mit einfachen Klassifikationsregeln. Derzeit verwendet der Eintragungsagent des VU-Brokers nur sehr einfache Klassifikationsregeln, welche die Strukturen der *Hypertext Markup Language (HTML)* [Raggett *et al.*, 1999], sowie Namenskonventionen der Adressierung im Internet verwendet. Beispielsweise wird der Name des Informationsguts aus dem Inhalt des TITLE-Tagpaars der Startseite des Informationsguts extrahiert. Aus dem *Uniform Resource Locator (URL)* [Berners-Lee *et al.*, 1994] wird versucht, Informationen über die Herkunft eines Informationsguts (wichtig zum Beispiel für regionale Zeitungen) zu erkennen. Dazu wird die in ISO 3166 [ISO, 1997] spezifizierte Länderkennung im URL verwendet. Abbildung 9.2 zeigt Klassifikationsregeln zur Erkennung von europäischen Ländern aus dem URL. Jede Regel beinhaltet als Bedingung einen regulären Ausdruck (*Perl Regular Expressions* [Wall *et al.*, 1996, S.58–76]) zur Erkennung der länderspezifischen Endung des URLs, und als Ergebnis das dazugehörige Land (Deutsch und Englisch).

Weitere einfache Klassifikationsregeln, die auf die Semantik von HTML (Inhalt von Tags wie ADDRESS oder META), beziehungsweise auf die Analyse des URLs abstellen, können einfach implementiert werden. Problematisch ist jedoch, daß der korrekte Einsatz von Tags bei HTML für die Darstellung einer Web-Seite nicht unbedingt notwendig ist. Dadurch werden Tags häufig falsch eingesetzt, was zu fehlerhaften Ergebnissen der Agenten führen kann.

Eintragungsagenten für komplexere Klassifikationsprobleme. Für komplexere Klassifikationsprobleme, wie zum Beispiel der Zuordnung von Lehrunterlagen zu einem Fach, reichen einfache Klassifikationsregeln nicht aus. Für diese Probleme wird ein flexibles System benötigt, daß Klassifikationsregeln lernen und anpassen kann. Abbildung 9.3 zeigt einen Ansatz, wie durch maschinelles Lernen der Einsatz von manueller Klassifikation schrittweise reduziert werden könnte (siehe auch [Geyer-Schulz und Hahsler, 2000]). Das Klassifikationssystem besitzt drei Zustände. Im ersten Zustand wird manuelle Klassifikation

```

\ .ad$ := {Europa/Andorra | Europe/Andorra}
\ .al$ := {Europa/Albanien | Europe/Albania}
\ .at$ := {Europa/Österreich | Europe/Austria}
\ .be$ := {Europa/Belgien | Europe/Belgium}
\ .bg$ := {Europa/Bulgarien | Europe/Bulgaria}
\ .by$ := {Europa/Weißrussland | Europe/Belarus}
\ .ch$ := {Europa/Schweiz | Europe/Switzerland}
\ .cy$ := {Europa/Zypern | Europe/Cyprus}
\ .de$ := {Europa/Deutschland | Europe/Germany}
\ .dk$ := {Europa/Dänemark | Europe/Denmark}
\ .ee$ := {Europa/Estland | Europe/Estonia}
\ .es$ := {Europa/Spanien | Europe/Spain}
\ .fi$ := {Europa/Finnland | Europe/Finland}
\ .fr$ := {Europa/Frankreich | Europe/France}
\ .gr$ := {Europa/Griechenland | Europe/Greece}
\ .hu$ := {Europa/Ungarn | Europe/Hungary}
\ .ie$ := {Europa/Irland | Europe/Ireland}
\ .is$ := {Europa/Island | Europe/Iceland}
\ .li$ := {Europa/Liechtenstein | Europe/Liechtenstein}
\ .lt$ := {Europa/Litauen | Europe/Lithuania}
\ .lu$ := {Europa/Luxemburg | Europe/Luxembourg}
\ .lv$ := {Europa/Lettland | Europe/Latvia}
\ .md$ := {Europa/Moldawien | Europe/Moldova}
\ .mt$ := {Europa/Malta | Europe/Malta}
\ .nl$ := {Europa/Niederlande | Europe/Netherlands}
\ .pl$ := {Europa/Polen | Europe/Poland}
\ .pt$ := {Europa/Portugal | Europe/Portugal}
\ .ro$ := {Europa/Rumänien | Europe/Romania}
\ .se$ := {Europa/Schweden | Europe/Sweden}
\ .si$ := {Europa/Slowenien | Europe/Slovenia}
\ .sk$ := {Europa/Slowakei | Europe/Slovakia}
\ .tr$ := {Europa/Türkei | Europe/Turkey}
\ .ua$ := {Europa/Ukraine | Europe/Ukraine}

```

Abbildung 9.2: Beispiele für Klassifikationsregeln zur Erkennung des Landes aus dem URL

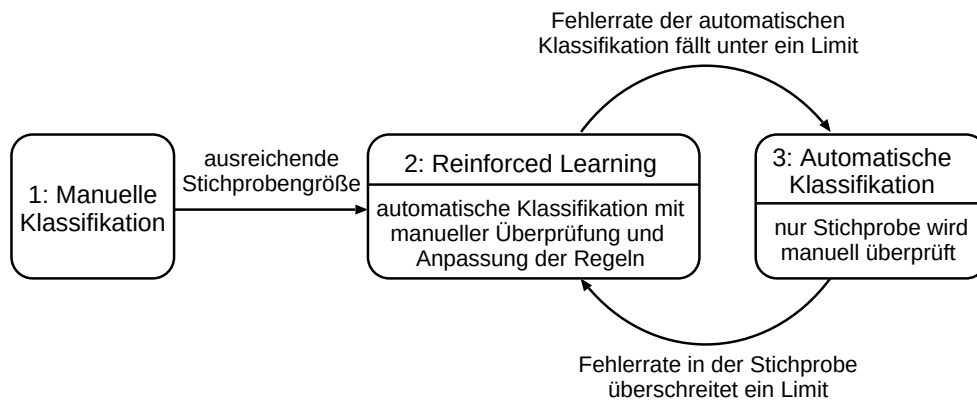


Abbildung 9.3: Von manueller zu automatischer Klassifikation

durchgeführt, das heißt, Informationsgüter werden durch eine Person in ein oder mehrere Klassifikationsschemata eingeordnet. Nachdem durch manuelle Klassifikation eine ausreichend große Anzahl von Gütern klassifiziert wurde, können zum Beispiel unter Verwendung des *Vector Space Models* (siehe Literatur zu *Information Retrieval* [van Rijsbergen, 1983], [Salton und MacGill, 1983]) Cluster gebildet werden (für einen Überblick zu Clusteranalyse siehe [Jain *et al.*, 1999]). Diese Cluster werden gemeinsam mit Zuordnungsregeln (zum Beispiel: Wenn sich das Informationsgut innerhalb einer bestimmten Entfernung von Cluster C1 oder von Cluster C5 befindet, dann ist es der Klasse K2.8 zuzuordnen.) in Zustand 2 verwendet, um neue Informationsgüter automatisch einzuordnen. Das Ergebnis dieser vorläufigen automatischen Klassifikation wird manuell überprüft, und die Cluster und Regeln werden durch dieses Feedback schrittweise angepaßt (*Supervised Learning* beziehungsweise *Reinforced Learning* siehe [Sutton, 1992]). Nachdem die Fehlerrate der automatischen Klassifikation einen Grenzwert unterschritten hat, kann in Zustand 3 (automatische Klassifikation) gewechselt werden. In diesem Zustand muß nur noch eine Stichprobe manuell überprüft werden. Erst bei Auftreten einer zu hohen Fehlerrate wird wieder in den Zustand 2 gewechselt, um Cluster und Regeln anzupassen.

Die Abschätzung einer ausreichenden Stichprobengröße für den Übergang von Zustand 1 auf 2 hängt von der Verteilung innerhalb der Grundgesamtheit der Informationsgüter ab. Da die Verteilung, beziehungsweise die Parameter der Verteilung potentiell für jedes Klassifikationsproblem unterschiedlich sind, muß diese Abschätzung für jedes Problem einzeln getroffen werden. Entschärft wird das Problem jedoch dadurch, daß im Zustand 2 noch immer eine vollständige manuelle Überprüfung stattfindet und daher bei einer Unterschätzung der benötigten Stichprobe keine Fehlzuordnungen auftreten können. Die Festlegung der zulässigen Fehlerraten für den

Übergang von Zustand 2 zu Zustand 3 und umgekehrt ist ebenfalls problematisch. Sie hängen hauptsächlich von der maximal tolerierten Fehlerrate des Klassifikationssystems ab. Wird das System ausschließlich zur Erstellung von Klassifikationsvorschlägen eingesetzt, wechselt es nie in den 3. Zustand. Tatsächlich würde der hier beschriebene Eintragungsagent immer im Zustand 2 arbeiten, da er nur Klassifikationsvorschläge generiert, die dann durch den Anbieter überprüft werden.

Dieses lernfähige System könnte beispielsweise verwendet werden, um automatische Vorschläge für die Zuordnung einer Lehrveranstaltung zu einem bestimmten Fach abzugeben.

9.3.2 Beobachtungsagenten für Informationsgüter

Beobachtungsagenten führen Routineprüfungen der Informationsgüter durch. Ihre wichtigste Aufgabe ist die Überprüfung, ob das Informationsgut unter dem in den Metadaten gespeicherten URL erreichbar ist. Eine Möglichkeit dies festzustellen ist, in regelmäßigen Abständen die Startseiten aller Informationsgüter von den jeweiligen Servern abzurufen. Verschiedene Varianten eines solchen Beobachtungsagenten für Informationsgüter werden kurz beschrieben.

Beobachtungsagent zur Wartung der Verweise. Das *Hypertext Transfer Protocol (HTTP)* [Fielding *et al.*, 1997] regelt den Ablauf für die Übertragung von Web-Seiten. Für Probleme sind in HTTP Fehlercodes vorgesehen. Die für den Beobachtungsagenten wichtigen Fehlercodes fallen in die Klasse 4xx (*Client Error*). Der Code 404 (*Not Found*) ist der am häufigsten auftretende Fehlercode. Er bedeutet, daß unter dem angegebenen URL keine Web-Seite gefunden werden konnte, dabei wird keinerlei Auskunft gegeben, ob die Seite verschoben wurde oder nicht mehr existiert. Es besteht für den Server zwar die Möglichkeit, durch andere Fehlercodes aus Klasse 3xx beziehungsweise 4xx (zum Beispiel 301 *Moved Permanently* oder 410 *Gone*) mehr Informationen über den Verbleib einer Seite weiterzugeben, dies benötigt aber eine spezielle Konfiguration und ständige Wartung des Servers und ist deshalb nicht weit verbreitet.

Trifft der Beobachtungsagent auf einen Eintrag mit einem nicht erreichbaren URL, markiert er diesen Eintrag für den Katalogverwalter. Dieser braucht danach nur noch alle markierten Einträge überprüfen und richtigstellen.

Bei einer durch die Abteilung für Informationswirtschaft der Wirtschaftsuniversität Wien durchgeführten Überprüfung von 420301 HTML-Seiten

von privaten Benutzern (Oktober und November 2000, begonnen bei der Seite <http://exai3.wu-wien.ac.at/~koch/>) traten 79945 URLs auf, für die ein Fehlercode zurückgeliefert wurde. Dies bedeutet eine Fehlerrate von ungefähr 19%. Im Gegensatz dazu sind am 12. November in der VU nur 238 von 9131 URLs nicht erreichbar oder fehlerhaft. Dies entspricht einer Fehlerrate von 2,61%. Diese deutlich geringere Fehlerrate ist auf die aktive Unterstützung der Wartung der Verweise durch Beobachtungsagenten zurückzuführen.

Beobachtungsagent für automatische Reparatur der Verweise.

Einfache Probleme, wie zum Beispiel der Änderung der Endung eines Dateinamens in einem URL von `htm` auf `html` treten häufig beim Wechsel von einem Webserver unter *Microsoft Windows* (Endung mit 3 Buchstaben) auf einen Server unter *Unix* (Endung mit beliebig vielen Buchstaben) auf. Solche Probleme können von Beobachtungsagenten automatisch behoben werden. Ein einfacher Reparaturagent benötigt dazu eine Liste von Ersetzungsregeln für Teile des URLs, die beispielsweise aufgrund der Erfahrung des Katalogverwalters erstellt werden kann. Die Liste von Ersetzungsregeln, die für die VU verwendet wird, ist in Abbildung 9.4 dargestellt. Die Ersetzungsregeln bestehen aus einem regulären Ausdruck (aus der Programmiersprache Perl, siehe [Wall *et al.*, 1996, S.58–76]) zum Auffinden von bestimmten Teilen einer URL und aus der Zeichenkette durch die der auf den regulären Ausdruck passende Teil des URLs ersetzt werden soll (Syntax einer Ersetzungsregel in der Liste: `<regulärer Ausdruck> := {Zeichenkette}`). Die Regeln 1–3 entfernen die oft für Startseiten verwendeten Seitennamen (beispielsweise `index.htm` oder `index.html`), dadurch wird der Web-Server aufgefordert die Standardstartseite zu übermitteln. Die Regeln 4 und 5 verändern die Endung von `htm` auf `html` und umgekehrt. Die letzte Regel löscht das Zeichen `~` aus dem URL. Dieses Zeichen wird bei einigen Web-Servern als Abkürzung für Benutzerseiten verwendet, bei anderen wird es weggelassen.

Im folgenden wird die Wirksamkeit dieser einfachen Regeln zur Reparatur von korrupten URLs in Einträgen untersucht. Am 8. September 2000 waren über 8500 Informationsgüter in den Katalogen der VU registriert. Bei der automatischen Überprüfung der URLs wurde für 179 Informationsgüter vom jeweiligen Server der Fehlercode 404 (*Not Found*) zurückgegeben. Die Ergebnisse der Anwendung einzelner Regeln, beziehungsweise der gesamten Regelbasis, auf die URLs dieser Einträge, sind in Tabelle 9.3 zusammengefaßt. In der ersten Spalte wird die verwendete Regel angegeben. Die zweite Spalte zeigt, wie oft die Regel auf die URLs angewendet werden konnte. Und in der letzten Spalte werden die durch die Regel reparierten URLs, absolut und relativ zu den 179 fehlerhaften URLs, darge-

```

1 index\.htm.*$ := {}
2 welcome\.htm.*$ := {}
3 home\.htm.*$ := {}
4 \.htm$ := {.html}
5 \.html$ := {.htm}
6 \/\/~ := {/}

```

Abbildung 9.4: Ersetzungsregeln zur automatischen Richtigstellung von typischen Änderungen von URLs

Agent verwendet	Bedingung erfüllt	Reparierte URLs	
Regel 1	31	17	9,5%
Regel 2	1	1	0,6%
Regel 3	8	2	1,1%
Regel 4	56	4	2,2%
Regel 5	61	4	2,2%
Regel 6	12	2	1,1%
alle Regeln	169	23	12,8%

Tabelle 9.3: Ergebnisse der Anwendung der Regeln des einfachen Reparaturagent auf 179 fehlerhafte URLs pro Regel und für alle Regeln gemeinsam. Da mehrere Regeln zu einer Reparatur führen können, ergibt sich für alle Regeln weniger als die Summe der einzelnen Regeln

stellt. Dabei ist zu beachten, daß ein fehlerhafter Verweis durch mehrere verschiedene Regeln repariert werden kann. Ändert sich der Dateiname in einem URL von `index.html` auf `index.htm`, und ist diese gleichzeitig die Standardseite, die der Web-Server ohne Angabe eines Dateinamens liefert, so kann das Problem von Regeln 1 oder auch von Regel 5 der Regelbasis in Abbildung 9.4 behoben werden. Daher ist die Anzahl der durch alle Regeln reparierten URLs kleiner als die Summe der URLs, die durch die einzelnen Regeln repariert werden.

Der beschriebene Reparaturagent erreichte bei der Anwendung auf die Informationsgüter der VU eine Reparaturrate von 12,8% und bringt damit eine deutliche Kosteneinsparung bei der Wartung der Produktkataloge.

Beobachtungsagenten zur Verfolgung von Änderungen. Eine weitere Art von Beobachtungsagenten kann verwendet werden, um Schlagworte oder Phrasen aus den Seiten eines Informationsguts zu extrahieren. Dazu kann wieder die Struktur von HTML herangezogen werden. Inhalte von Tags zum Hervorheben von Textpassagen, wie H1 für Überschriften 1. Ordnung oder EM für Betonungen, weisen auf wichtige Inhalte hin und können in den Metadaten des Eintrags als automatisch extrahierte Schlagworte gespeichert werden. Dies verbessert die Suchergebnisse im Broker bei unvollständig beschlagworteten Informationsgütern. Außerdem ändert sich dieser Teil der Metadaten bei regelmäßiger Aktualisierung automatisch mit Veränderungen des Informationsguts mit. Dies ist wichtig, da sich Informationen im Internet häufig verändern und eine manuelle Anpassung der Metadaten der Einträge in großen Katalogen aus Kostengründen nicht möglich ist. Brewington und Cybenko [Brewington und Cybenko, 2000] untersuchten die Änderungshäufigkeit von Informationen im Internet. Sie beobachteten, daß bei 20% der Web-Seiten ihrer Stichprobe die letzte Änderung vor weniger als 12 Tagen stattgefunden hat. Dies weist auf eine hohe Änderungshäufigkeit hin. Bei der Untersuchung eines Informationsguts kann festgestellt werden, ob sich der Inhalt seit der letzten Untersuchung verändert hat. Einträge für veränderte Informationsgüter können für Konsumenten gekennzeichnet werden. Dadurch können Konsumenten erkennen, ob ein Informationsgut, das sie bereits kennen, aufgrund der Veränderung trotzdem angesehen werden soll (siehe [Geyer-Schulz *et al.*, 1999]). Derzeit wird für das Erkennen von Veränderungen eine Quersumme (der Algorithmus MD5 [Rivest, 1992]) über die Startseite des Informationsguts verwendet. Ändert sich die Quersumme zwischen zwei Untersuchungen des selben Informationsguts, hat sich die Startseite verändert. Diese Methode benötigt wenig Speicherplatz, da statt der gesamten Seite nur die Quersumme gespeichert werden muß. Jedoch ist es dadurch nicht möglich das Ausmaß der Veränderung zu erkennen, es kann also nicht unterschied-

den werden, ob nur eine minimale oder eine massive Änderung des Informationsguts stattgefunden hat. Soll das Ausmaß ermittelbar sein, müßten andere Methoden (wie zum Beispiel die Speicherung der gesamten Seite) eingesetzt werden.

Weitere spezialisierte Beobachtungsagenten. Es sind auch weitere spezialisierte Beobachtungsagenten möglich. Ein Beispiel dafür ist ein für die VU entwickelte Prototyp mit dem Namen *NewsWire* (siehe <http://vu.wu-wien.ac.at/newswire/>). Dieser spezialisierte Agent extrahiert Schlagzeilen aus den im Katalog *VU-Bibliothek* enthaltenen Informationsdiensten (Zeitungen, Magazine, online Nachrichtendienste, ...), und macht sie dadurch zentral durchsuchbar. Abbildung 9.5 zeigt das Suchergebnis für das Wort "Intel" in den vom Beobachtungsagent automatisch extrahierten Schlagzeilen. Die einzelnen Schlagzeilen sind mit einem Verweis auf den jeweiligen Artikel unterlegt. Da im Katalog nur Metadaten verwaltet werden, und daher keine lokalen Kopien der Artikel angelegt werden, zeigen die Verweise auf die Artikel bei den Anbietern. Durch diese Verteilung von Information besitzt das System, wie schon im Analyse Pattern *Virtuelle Bibliothek* (siehe Abschnitt 5.3) erwähnt, nur schwache Konsistenz. Derzeit werden die Schlagzeilen nur einmal pro Tag extrahiert. Da Informationsdienste rund um die Welt, also in vielen verschiedenen Zeitzonen, beobachtet werden, entsteht für einige eine große Verzögerung, beziehungsweise werden nicht mehr vorhandene Artikel angezeigt. Zur besseren Synchronisierung des Systems mit den Zeitpunkten, an denen neue Nachrichten publiziert werden, müßte ein adaptives System eingesetzt werden, das für jeden Informationsdienst die notwendige Aktualisierungshäufigkeit, sowie die optimalen Aktualisierungszeitpunkte selbständig lernt.

9.4 Agenten der Interface und Broker Management Ebene

Neben den bisher beschriebenen Agenten der Metadaten Management Ebene werden in der Struktur des Informationsbrokers der VU (siehe Abbildung 9.1, Seite 126) auch eine Reihe von Agenten auf der Interface und der Broker Management Ebene dargestellt. Diese Agenten verringern die Transaktionskosten der Konsumenten (zum Beispiel Suchkosten) in den Katalogen des Brokers durch intelligente Anpassung des Benutzerinterfaces. Die dazu verwendeten Beobachtungs-, Analyse- und Empfehlungsagenten bilden gemeinsam ein sogenanntes *Recommender System*.

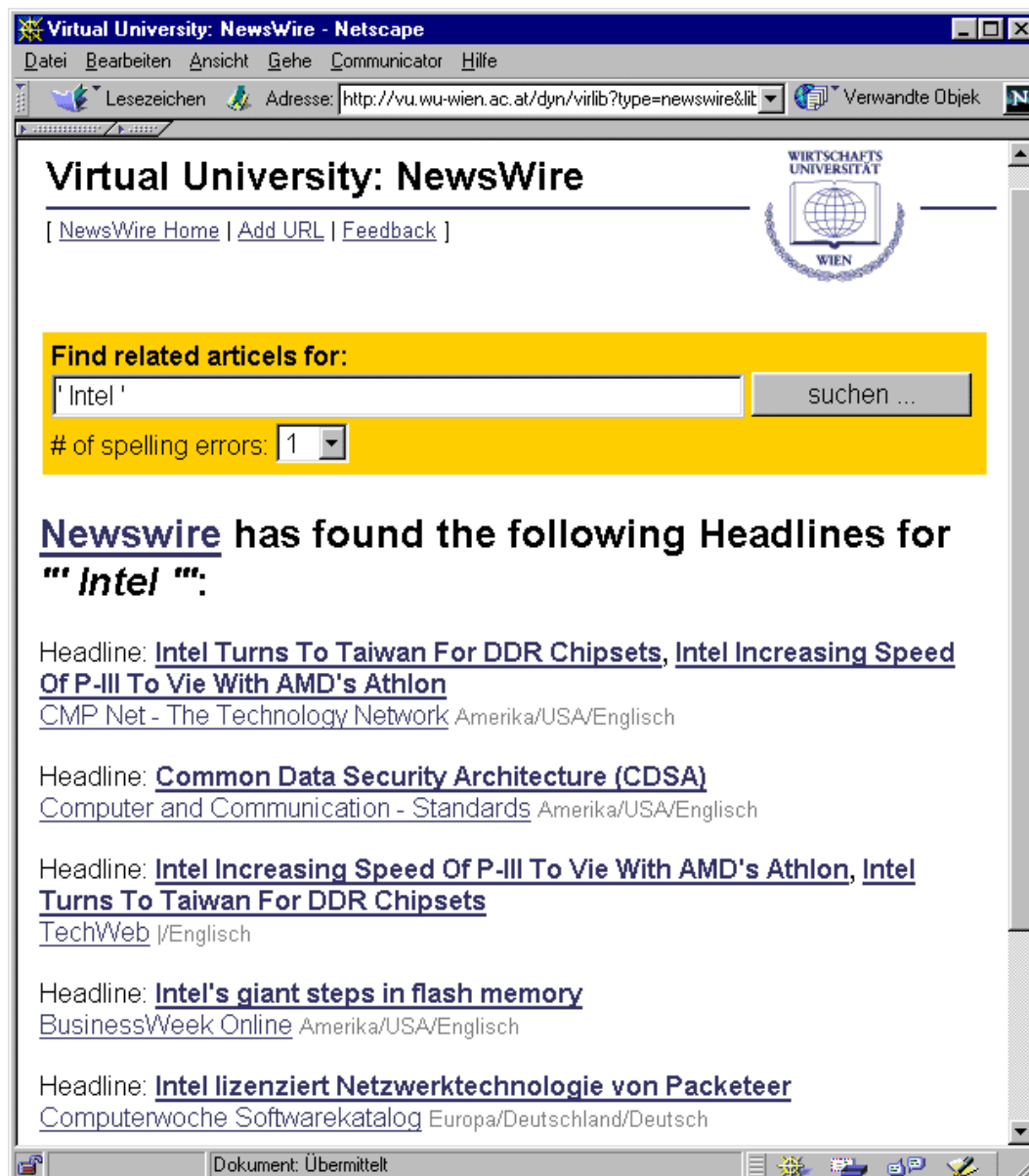


Abbildung 9.5: Suchergebnis in Schlagzeilen von verteilten Informationsdiensten, extrahiert durch einen spezialisierten Beobachtungsagenten

In einem typischen Recommendersystem liefern erfahrene Benutzer Empfehlungen als Eingabe, das System aggregiert diese und leitet sie an die richtigen Benutzer weiter [Resnick und Varian, 1997]. Ein Beispiel für ein nicht automatisiertes Recommendersystem ist der Gourmetführer *Gault Millau Guide Österreich* [Gault und Millau, 2000] für österreichische Restaurants (siehe auch <http://www.gaultmillau.co.at/>). Jedes Jahr werden in einer neuen Ausgabe die besten Restaurants, in Kategorien von Frische der Zutaten bis zu Kreativität der Küche, bewertet. Zusätzlich werden durchschnittliche Preise genannt, das Restaurant wird verbal beschrieben, und aus den erhobenen Eigenschaften der Restaurants werden Empfehlungen erstellt. Durch dieses Buch ist es zum Beispiel möglich, für eine Geschäftsreise ein ausgezeichnetes Restaurant mit italienischer Küche in der Stadt Salzburg zu finden, ohne die Restaurantszene in Salzburg vorher zu kennen. Diese Empfehlungen sind der Nutzen des Gourmetführers als Recommendersystems.

Für den Informationsbroker der VU soll ein Recommendersystem Informationsobjekte vorschlagen können, und damit soll die Suchzeit für die Benutzer verkürzt werden. Die dazu verwendbaren Informationen sollen aus der ständigen Beobachtung aller Benutzer gewonnen werden. Um die dazu nötigen Agenten zu entwickeln, muß zuerst der Anwendungsbereich (*Domain Space*) des benötigten Recommendersystems analysiert werden. Der Anwendungsbereich besteht aus folgenden zwei Teilbereichen (siehe Resnick und Varian [Resnick und Varian, 1997]):

1. Eigenschaften der evaluierten Gegenstände über die Empfehlungen erstellt werden. Diese Eigenschaften unterteilen sich in:
 - Die Art des Gegenstands. Bei den Gegenständen der Evaluierung handelt es sich um die Informationsgüter im Informationsbroker der VU. Dies sind die Informationsobjekte, die hinter den Einträgen der Metadaten in den Produktkatalogen stehen.
 - Anzahl. Die Anzahl der Informationsgüter hängt von der Größe des Informationsbrokers ab, und kann daher nicht genau vorhergesagt werden. Derzeit (November 2000) verwaltet der Broker mehr als 9000 Einträge. Durch diese hohe Anzahl kommen für das VU-Projekt nur automatisierbare Verfahren zur Erstellung von Empfehlungen in Frage.
 - Lebenszeit. Die Lebenszeit der Informationsgüter im Informationsbroker der VU ist sehr unterschiedlich. Beispielsweise sind Lehrunterlagen zwischen einem Semester für ein aktuelles Forschungsseminar und mehreren Jahren bei Grundlagenvorlesungen einsetzbar. Aufgrund der hohen Produktionskosten von In-

formationsgütern (bestehend aus statischen Web-Seiten, dynamischem Inhalt, ...), ist im allgemeinen mit einer relativ langen Lebensdauer zu rechnen. Die Web-Präsenzen von Organisationen, wie beispielsweise von Universitäten oder großen Unternehmen, können sogar noch viel länger unter der selben Internet-Adresse bestehen. Dies liegt daran, daß Internet-Adressen in einigen Bereichen schon einen ähnlichen Stellenwert besitzen, wie die Postanschrift. Im Zeitraum von einem Jahr (August 1999 – Juli 2000) wurden aus den Katalogen des VU-Systems weniger als 5% der über 4844 betrachteten Informationsgüter gelöscht. Eine genauere Analyse (siehe Anhang A.1) ergab eine durchschnittliche Verweildauer von Informationsgütern im Broker von 12,1 Jahren. Diese lange Lebensdauer von Informationsgütern bedeutet, daß es wirtschaftlich vertretbar ist, mehr Aufwand für ein einzelnes Gut im Brokersystem (bei der Erhebung von Empfehlungen, sowie bei der Aggregation) zu betreiben, als dies bei kurzlebigen Gütern der Fall wäre.

Eine weitere Eigenschaft von Informationen im Internet ist ihre hohe Änderungshäufigkeit (siehe Brewington und Cybenko [Brewington und Cybenko, 2000]). Daher muß das System die Möglichkeit besitzen, sich den Veränderungen mit ausreichender Geschwindigkeit anzupassen.

- **Kosten Struktur.** Die Frage, wie hohe Kosten durch schlechte oder fehlende Empfehlungen durch das Recommendersystem entstehen, kann schwer quantifiziert werden, da der Informationsbroker der VU ausschließlich Unterlagen im Internet, die großteils frei verfügbar sind, vermittelt. Durch nicht optimale Empfehlungen entsteht dem Benutzer daher kein direkter finanzieller Schaden, sondern es werden lediglich seine Suchkosten erhöht. Damit können rein automatisch erzeugte Empfehlungen, auch ohne vorherige manuelle Überprüfung, direkt an die Benutzer weitergegeben werden.
2. **Eigenschaften der Benutzer, die Empfehlungen abgeben und empfangen, sowie der Beobachtungen selbst.**
- **Erzeuger und Konsumenten von Empfehlungen.** Da das VU-System zwischen Experten und anderen Benutzern nicht unterscheidet, können alle Benutzer Empfehlungen abgeben und empfangen. Dieses Vorgehen entspricht beispielsweise einem Restaurantführer, für den jeder Kunde eines Restaurants gebeten wird, eine Bewertung abzugeben, und die Zusammenfassung aller Bewertungen die Basis für Empfehlungen bildet.

- Dichte der Empfehlungen. Da die anonymen Benutzer vom System ständig beobachtet werden, und jede Interaktion mit dem Brokersystem aufgezeichnet wird, entstehen viele Beobachtungen für die Errechnung der Vermittlungshäufigkeiten für Informationsgüter. Die Vermittlungshäufigkeit von Informationsgütern wird in Anhang A.2 analysiert. Weiters ist für die Empfehlungen die Länge der Session, das heißt die Anzahl der Informationsgüter, die gemeinsam benutzt werden, wichtig. Je länger die Sessions sind, desto mehr Informationsprodukte können miteinander in Verbindung gebracht werden. Im Beobachtungszeitraum von Mai 2000 – Juli 2000 ergab sich eine durchschnittliche Sessionlänge von 2,23 Vermittlungen. Die Verteilung der Sessionlängen wird in Anhang A.3 analysiert.
- Heterogenität der Benutzer. Beispielsweise sind in der VU einige Lehrveranstaltungen an Vorkenntnisse gebunden. Diese Lehrveranstaltungen vorzuschlagen, ist nur für Benutzer sinnvoll, die diese Vorkenntnisse besitzen. Vorkenntnisse können zum Beispiel in Form von Selbsteinschätzungen erhoben, im Benutzerprofil gespeichert, und für genauere Vorschläge ausgenutzt werden. Solange der Benutzer in der Selbsteinschätzung Nutzen sieht, wird er bereit sein, Information über sich selbst preiszugeben. Selbsteinschätzung wird im anonymen VU-Grundsystem nicht eingesetzt, und ist damit außerhalb des Rahmens dieser Arbeit. In *myVU* (<http://myvu.wu-wien.ac.at>), dem personalisierten Teil der VU, wird diese Möglichkeit untersucht.

Neben diesen Eigenschaften sind auch soziale Auswirkungen der Einführung eines Recommendersystems wichtig. Resnick und Varian [Resnick und Varian, 1997] beschreiben einige solcher sozialen Probleme. Im folgenden wird dargestellt, wie diese Probleme im VU-Recommendere behandelt werden:

- Trittbrettfahrer. In einem Recommendersystem ist es oft möglich, Vorschläge zu verwenden, aber selbst keine Empfehlungen abzugeben. Der Recommender der VU beobachtet ständig alle Interaktionen mit den Benutzern. Dadurch ist keine Verwendung des Systems möglich, ohne gleichzeitig implizit Empfehlungen abzugeben.
- Verzerrte Empfehlungen. Die Möglichkeit, daß jeder Empfehlungen abgeben kann, kann auch für Manipulationen des Systems mißbraucht werden. Im VU-System ist dieser Mißbrauch möglich, er kann jedoch durch die Filterung von Sessions mit wiederholter Verwendung des gleichen Gutes zumindest teilweise leicht korrigiert werden.

- Privatsphäre. Benutzer des Internets wollen in der Regel soviel Anonymität, wie möglich. Das in dieser Arbeit beschriebene Grundsystem der VU arbeitet anonym. Erst die personalisierte Version *myVU* verwendet Pseudonyme zur Wiedererkennung von Benutzern bei gleichzeitiger Wahrung ihrer Privatsphäre.
- Glaubwürdigkeit. Damit Recommendersysteme akzeptiert werden, müssen sie glaubwürdig sein. Unabhängigkeit von Sponsoren und Verbindung zu glaubwürdigen Organisationen helfen in diesem Zusammenhang. Die VU ist von der Wirtschaftsuniversität Wien und weiteren bekannten Organisationen als Forschungsprojekt finanziert und wird daher derzeit unabhängig von kommerziellen Zwängen betrieben.

Weiters spielt die Offenlegung der Art, wie die Empfehlungen zustandekommen eine Rolle. Dies setzt aber die oft unrealistische Annahme voraus, daß die Benutzer die Komplexität des Recommendersystem leicht und schnell verstehen können. Manber et al. [Manber *et al.*, 2000] beschreiben ihre Erfahrungen mit der Personalisierung des Internetportals *Yahoo!*, und kommen zum Schluß, daß durchschnittliche Benutzer das Konzept von adaptiven Systemen nicht verstehen.

- Rückkopplungseffekte. Wenige frühe Empfehlungen können zu selbstverstärkenden Rückkopplungseffekten führen. Ein Gut wird empfohlen, daher verwendet, und im VU-System wird diese Verwendung wiederum als Empfehlung gewertet. Um diesen Effekten entgegenzuwirken, müssen spezielle Maßnahmen ergriffen werden. Eine Möglichkeit wäre das Ziehen (zufällig oder nach einer bestimmten Regel) eines Gutes und die prominente Platzierung dieses Gutes. Dies wird derzeit in *myVU* untersucht.

Nach der Beschreibung der Eigenschaften des Recommendersystems werden die für die VU verwendeten Agenten kurz beschrieben.

9.4.1 Benutzerbeobachtungsagenten

Von Benutzerbeobachtungsagenten werden im Recommendersystem Interaktionen zwischen Benutzern und dem System aufgezeichnet. Aus diesen Aufzeichnungen werden von den anderen Agenten Vorschläge erstellt. Um festzulegen welche Art von Aktionen durch die Benutzerbeobachtungsagenten aufgezeichnet werden sollen und können, müssen folgende Punkte beachtet werden:

1. Art der Erhebung einer Empfehlung. Empfehlungen können explizit von Benutzern in das System eingegeben werden. Beispielsweise können Konsumenten nach der Verwendung eines Informationsguts über ihre Meinung zum Nutzen dieses Gutes befragt werden. Weiters können Empfehlungen durch Beobachtung des Verhaltens von Benutzern erzeugt werden. Beispielsweise kann die Wiederkaufsrate, das heißt ein Konsument verwendet ein bestimmtes Informationsgut öfters, als Indiz dafür gewertet werden, daß dieses Gut dem Konsumenten Nutzen bringt. Dies setzt natürlich, wie Recommendersysteme allgemein, rational handelnde Benutzer voraus.
2. Inhalt einer Empfehlung. Empfehlungen können in unterschiedlicher Form abgegeben werden. Das Ergebnis einer Befragung kann beispielsweise die Zufriedenheit des Benutzers mit einem Gut auf einer Skala von 1 bis 5 sein. Möglich ist auch die Eingabe eines kurzen Texts, der Erfahrungen mit dem Gut beschreibt.
3. Anonymität. Wie viel Information hat das Recommendersystem über seine Benutzer? Der Spielraum geht hier von vollständiger Anonymität bis zur persönlichen Identifizierung. Die möglichen Stufen sind:
 - Vollständige Anonymität. Es werden nur die einzelnen Transaktionen mitgeschrieben. Dies entspricht dem Standardfall durch das zustandslose HTTP-Protokoll [Fielding *et al.*, 1997].
 - Anonymität mit Sessions. Eine Session ist die Gesamtheit aller von einem Benutzer gemeinsam getätigten Transaktionen. Dies entspricht einem Warenkorb, wie er bei einem Supermarkteinkauf entsteht. Kommt derselbe Benutzer später nochmals, kann er nicht als derselbe Benutzer identifiziert werden.
 - Pseudoanonyme Benutzer. Durch die Verwendung eines Pseudonyms können Sessions des selben Benutzers zusammengeführt werden, ohne daß der Benutzer persönlich bekannt ist.
 - Pseudoanonyme Benutzer mit Attributen. Zusätzlich zum Pseudonym werden Eigenschaften des Benutzers (Benutzerprofil) gespeichert. Beispielsweise Geschlecht, Alter, Beruf oder Erfahrungen. Diese Informationen können für die Bildung von Gruppen mit ähnlichen Profilen genutzt werden. Dadurch können genauere Empfehlungen erstellt werden.
 - Persönlich Identifiziert. Der Benutzer ist persönlich durch zum Beispiel Name, Anschrift oder Kreditkartennummer bekannt. Erst durch diese Informationen kann der Broker Dienstleistungen, wie die Abwicklung von Zahlungen, anbieten.

Für die VU werden explizite Empfehlungen und Beobachtung verwendet. Den Inhalt einer expliziten Empfehlungen stellt die Eintragung eines gefundenen Informationsguts (zum Beispiel ein online Wörterbuch) durch einen Benutzer in einen Katalog des Brokers dar. Von rational handelnden Benutzern werden Informationsgüter eingetragen, von denen sie sich den größten Nutzen erwarten. Dadurch maximieren sie die Differenz zwischen Aufwand und dem erwarteten Nutzen durch die Eintragung. Nach der Eintragung wird derzeit nur beobachtetes Kaufverhalten der Benutzer verwendet. Als Kauf wird dabei die Auswahl eines Informationsguts in einem Katalog des Brokers betrachtet.

Derzeit werden folgende Stufen von Anonymität in der VU unterstützt:

- Vollständige Anonymität. Für Benutzer, die bei ihrem Web-Browser HTTP-Cookies, eine Möglichkeit durch den Server Informationen im Browser des Benutzers zwischenspeichern, deaktiviert haben.
- Anonymität mit Sessions. Dies ist der Standardfall.
- Pseudoanonyme Benutzer mit Attributen. Für Benutzer, die die personalisierte Version der VU, *myVU*, verwenden.

9.4.2 Analyseagenten

Die Aufgabe der Analyseagenten ist die Aggregation der von den Beobachtungsagenten gesammelten Daten. Aggregation bedeutet die Verdichtung von mehreren Empfehlungen und/oder verschiedener Arten von Empfehlungen zu einer Datenbasis, aus der Vorschläge generiert werden können. Die Möglichkeiten zur Aggregation hängen natürlich stark von den Eigenschaften der vorhandenen Empfehlungen, und daher von der Art der Erhebung, ab. Verschiedene Arten der Aggregation sind beispielsweise: eine Stimme pro Person, gewogene und diskontierte Stimmen, oder personalisierte Gewichtung für Stimmen. Verwendet man, wie im Fall der VU, beobachtetes Verhalten, ist der Grad der Anonymität für die Art der Aggregation ausschlaggebend.

- Bei vollständiger Anonymität der Benutzer ohne Sessions kann nur die Verwendungshäufigkeit als Schätzer für Wiederverwendung und damit dem Nutzen eines Gutes verwendet werden.
- Mit Sessions können Warenkörbe, und damit bedingte Wahrscheinlichkeiten für die Kreuzverwendung von mehreren Gütern, errechnet werden. Es können Güter vorgeschlagen werden, die zu einem gerade betrachteten passen.

- Pseudoanonyme Benutzer lassen eine Beobachtung der gesamten Kaufgeschichte zu. Dadurch wird die Errechnung einer Wiederkaufsrate als Maß für den persönlichen Nutzen eines Gutes möglich. Außerdem entsteht eine weitere Dimension für die Art der Aggregation durch Personalisierung. Damit kann die Aggregation auf verschiedenen Stufen (beispielsweise Gruppenebene oder global) geschehen.

Derzeit werden für die VU alle drei beschriebenen Arten der Aggregation eingesetzt, an der Personalisierung (*myVU*) wird derzeit gearbeitet.

9.4.3 Empfehlungsagenten

Die Empfehlungsagenten verwenden die von den Analyseagenten erstellte Datenbasis zur Veränderung des Benutzerinterfaces. Diese Veränderung reicht von:

- der einfachen Anzeige einer Einschätzung (zum Beispiel Rating mit 1–5 Sternen, wie bei Hotels) neben dem Eintrag eines Gutes,
- über die Sortierung von Suchergebnissen nach einer Maßzahl (zum Beispiel Verwendungshäufigkeit oder Wiederkaufsrate),
- bis zur Ausblendung von Gütern, die nach einer bestimmten Maßzahl nicht gut genug abschneiden (Informationsfilterung).

Eine weitere Möglichkeit ist, für ein Informationsgut passende weitere Informationsgüter vorzuschlagen. Dabei kann es sich um ähnliche (konkurrierende) Güter, beziehungsweise ergänzende (komplementäre) Güter handeln. Beispiele für konkurrierende Güter sind mehrere Unterlagen zur selben Lehrveranstaltung. Beispiele für komplementäre Güter, die beim Verfassen einer Arbeit in einer Fremdsprache benötigt werden, sind ein Tutorial für ein Textverarbeitungsprogramm und ein online Wörterbuch.

Abbildung 9.6 zeigt Beispiele für die Verwendung von Empfehlungen im Benutzerinterface des VU-Brokers. Es handelt sich um eine Liste von online Wörterbüchern. Der Balken unter dem Namen jedes Informationsguts gibt die Verwendungshäufigkeit relativ zum meistverwendeten Gut in der Liste an. Der Vorschlag besteht in der Reihung der Informationsgüter nach ihrer relativen Verwendungshäufigkeit. Dieser Vorschlag basiert auf der Annahme, daß häufiger verwendete Informationsgüter tendenziell auch öfter wiederverwendet werden, und daß Wiederverwendung auf höheren Nutzen durch die Benutzer schließen läßt.

Virtual University: Tools & Manuals - Netscape

Datei Bearbeiten Ansicht Gehe Communicator Hilfe

Lesezeichen Adresse: chlagwerke/Wörterbücher';+wörterbücher&sortby=hits Verwandte Objek

Virtual University: Tools & Manuals

[Tools & Manuals Home | Add URL | Feedback]

WIRTSCHAFTS UNIVERSITÄT WIEN

Ihre Abfrage: CATEGORY:='Büro/Nachschlagwerke/Wörterbücher' + wörterbücher

Einschränken mit:

Suchen

> Einen Link auf diese Seite erzeugen: [Link this Page](#)

> Suche nach **Büro Nachschlagwerke Wörterbücher wörterbücher** in: [gesamte VU](#) | [WU-Web](#) | [gesamtes Web](#)

25 Einträge gefunden. Sortierung: [A-Z](#) | [Usage](#) | [Score](#)

Bezeichnung	Kategorie
LEO: German <--> English (100%) - Score: 20 - Englisch in neuem Fenster - myVU Bookmark - Others also use	Beschreibung Büro/Nachschlagwerke/Wörterbücher
Wörterbücher, Lexika, Abkürzungen META (32%) - Score: 82 - Deutsch in neuem Fenster - myVU Bookmark - Others also use	Beschreibung Büro/Nachschlagwerke/Lexika Büro/Nachschlagwerke/Wörterbücher
LEO - Collection of Web-Dictionaries META (17%) - Score: 30 - Englisch in neuem Fenster - myVU Bookmark - Others also use	Beschreibung Büro/Nachschlagwerke/Wörterbücher
Sammlung von Web-Wörterbüchern META (14%) - Score: 67 - Deutsch in neuem Fenster - myVU Bookmark - Others also use	Beschreibung Büro/Nachschlagwerke/Wörterbücher
German <--> English Dictionary (13%) - Score: 20 - Deutsch in neuem Fenster - myVU Bookmark - Others also use	Beschreibung Büro/Nachschlagwerke/Wörterbücher

Dokument: Übermittelt

Abbildung 9.6: Verwendung von Recommendations im Benutzerinterface

Rechts neben dem Balken wird mit der Bezeichnung *Score* eine Maßzahl angegeben, wie gut der jeweilige Eintrag auf die Abfrage paßt. Eine gängige Annahme des *Vector Space Model* ist, daß mit einem Vergleich von in einem Dokument (Informationsgut) enthaltenen Termen mit den abgefragten Termen, die Ähnlichkeit von Dokument und Abfrage bestimmt werden kann (siehe [Gross und Harris, 1985, Seite 13–22]). Diese Maßzahl ist derzeit aus Effizienzgründen einfach als die ungewichtete Anzahl der Vorkommnisse jedes abgefragten Terms in den Metadaten des jeweiligen Guts implementiert.

In der Zeile unter dem Balken befindet sich ein Verweis mit dem Namen "*Others also use*", der zu einer Liste von gemeinsam mit dem jeweiligen Informationsgut verwendeten Gütern führt. Diese Liste ist ein Vorschlag von ähnlichen oder komplementären Informationsgütern, die aus der Analyse von Sessions (Warenkörbe) gewonnen wird. Dieser Vorschlag basiert auf der Idee, daß zwischen oft gemeinsam benutzten Informationsgütern ein Zusammenhang besteht.

9.5 Konsequenzen der Verwendung von aktiven Agenten

Durch die Einführung von aktiven Agenten des Analyse Patterns *Virtuelle Bibliothek mit aktiven Agenten* werden die Betriebskosten des Informationsbrokers, sowie die Transaktionskosten der Benutzer folgendermaßen verändert:

- Reduktion der Betriebskosten des Brokers. Die Beobachtungsagenten für Informationsgüter (siehe Abschnitt 9.3.2) reduzieren die Betriebskosten des Brokers, durch die automatische Überprüfung der Links in den Einträgen. Es müssen nur noch die als fehlerhaft markierten Einträge überprüft werden. Dies reduziert den Überprüfungsaufwand derzeit (26. November 2000) von 9187 Einträge in allen Produktkatalogen der VU auf 233 markierte Einträge, also um mehr als 97%. Dies bedeutet, daß die Wartung des Inhaltes des gesamten VU-Informationsbrokers zur Zeit von 9 Personen freiwillig nebenbei durchgeführt werden kann, wobei jede Person in der Regel weniger als 30 Minuten pro Monat für die Reparatur unerreichbarer Einträge, sowie für die Bearbeitung neuer Einträge, aufwendet.

Weiters führen Reparaturagenten zu einer weiteren Kostenreduktion. Wie in Abschnitt 9.3.2 beschrieben, konnte der experimentelle

Reparaturagent der VU mit einfachen Regeln mehr als 12% der un-erreichbaren Links automatisch reparieren.

In Abschnitt 8.1.3 wurde ein Informationsbroker als Netzwerk in einem Informationsmarkt betrachtet. Durch niedrigere Betriebskosten kann ein Informationsbroker effizient ein größeres Netzwerk unterstützen, und die Größe des Netzwerks wiederum ist ein wichtiger Faktor für den möglichen Nutzen des Informationsbrokers für die Benutzer. Damit haben die Betriebskosten auch auf den möglichen Nutzen des Informationsbroker einen indirekten Einfluß.

- Reduktion der Transaktionskosten der Anbieter. Eintragungsagenten (siehe Abschnitt 9.3.1) reduzieren die Kosten der Anbieter für die Eintragung eines Informationsguts in den Informationsbroker. Beispielsweise werden durch die Verwendung der beschriebenen, einfachen Klassifikationsregeln bei der Eintragung von Informationsgütern in den Produktkatalog *VU-Bibliothek* nach der Angabe des URLs bereits 4 (Name, Art, Sprache und Region) der 7 notwendigen Felder vorausgefüllt. Dadurch daß diese 4 Felder nur noch überprüft werden müssen, wird der Zeitaufwand für die Eintragung erheblich verringert. Wird davon ausgegangen, daß potentielle Anbieter die Entscheidung über die Eintragung ihres Informationsguts von der Differenz zwischen dem erwarteten Nutzen durch die Teilnahme am Broker und den Eintragungskosten abhängig machen, kann aufgrund der Reduzierung der Eintragungskosten ein schnelleres Wachstum des Angebots im Informationsbrokers erreicht werden.
- Reduktion der Transaktionskosten der Konsumenten. Die Agenten des Recommendersystems der VU reduzieren die Transaktionskosten für die Benutzer. Die Empfehlungsagenten erleichtern den Konsumenten die Auswahl geeigneter Informationsgüter im Broker.

Um den Nutzen zu demonstrieren, wird hier ein spezieller Recommender für den Produktkatalog *Living Lectures* (Lehrveranstaltungen) betrachtet. Dieser Recommender verwendet die in Abschnitt 9.4.3 kurz beschriebene Funktion *"Others also use"*, um einem Studierenden automatisch weitere Unterlagen für Lehrveranstaltungen vorzuschlagen. Ein Ausschnitt des Vorschlags für die Lehrveranstaltung *Einführung in das objektorientierte Programmieren mit C++* ist in Abbildung 9.7 dargestellt. Nach Analyse des gesamten Vorschlags durch den Lehrveranstaltungsleiter paßten 9 der 11 automatisch vorgeschlagenen Unterlagen sehr gut zu den Inhalten der Lehrveranstaltung.

Durch die Verwendung des Recommenders entfällt das Aktualisieren und Warten einer Liste mit online Unterlagen für den Lehrenden.

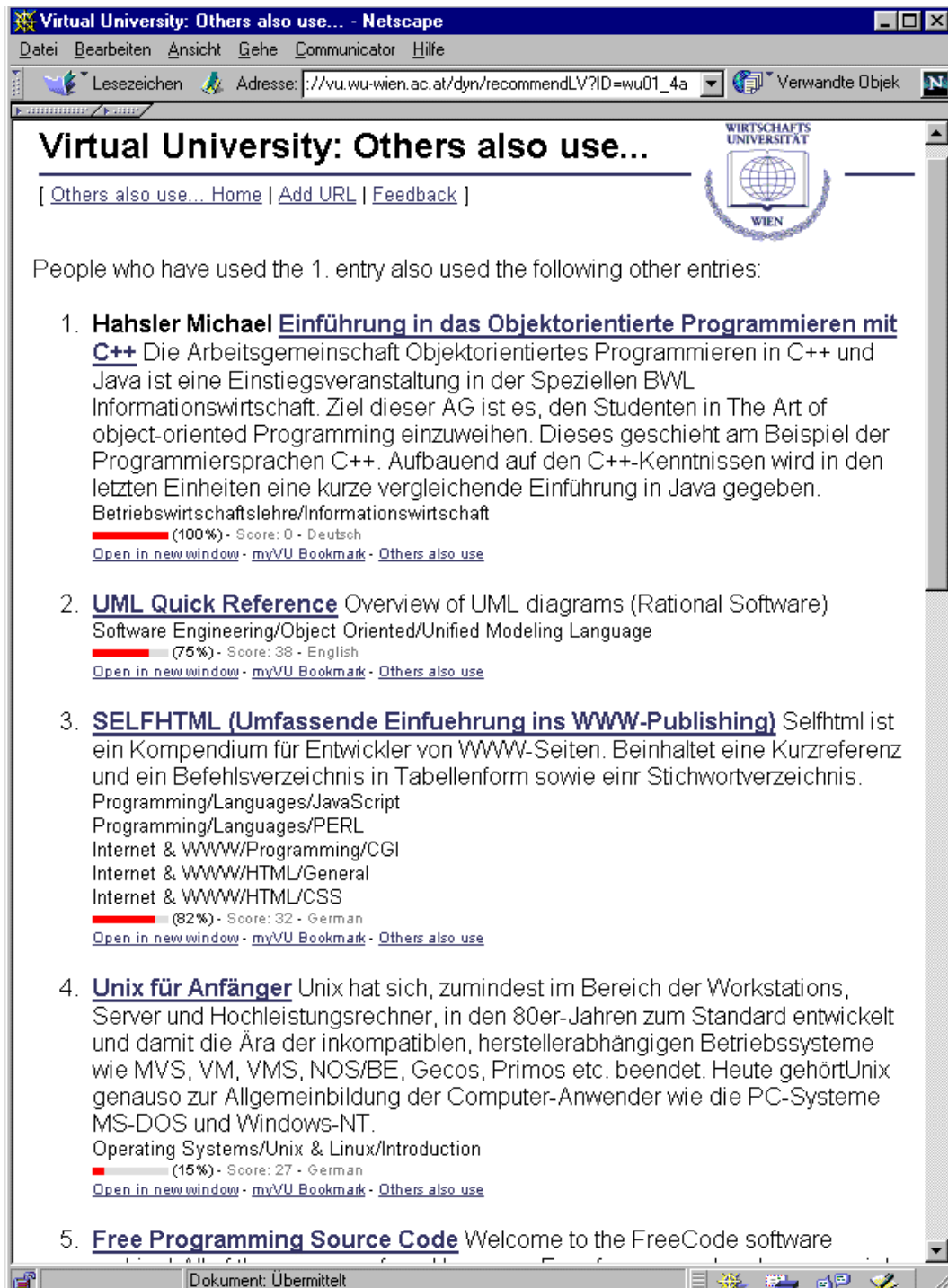


Abbildung 9.7: Beispiel für die automatische Zusammenstellung von empfohlenen Materials für die Lehrveranstaltung "Einführung in das objektorientierte Programmieren mit C++"

Außerdem spiegelt diese Liste die Verwendung der Unterlagen durch die Studenten wider, dadurch bekommt der Lehrende Feedback, ob die Studenten die von ihm vorgeschlagenen Unterlagen verwenden, oder andere bevorzugen. Interessant ist dabei auch noch, daß diese Liste auch Unterlagen enthalten kann, die der Lehrende noch gar nicht kennt. Dies passiert dadurch, daß ein anderer Benutzer (zum Beispiel ein Lehrender, der eine Lehrveranstaltung zu einem ähnlichen Themenbereich anbietet) Unterlagen in den VU-Broker eintragen kann, und sie damit allen zur Verfügung stehen.

Im folgenden Kapitel werden die Vorteile der Verwendung von Analyse Patterns anhand dieser Konsequenzen gemeinsam mit den Ergebnissen aus Kapitel 8 und einer Untersuchung der Implementierung des Informationsbrokers nachgewiesen.

Kapitel 10

Ergebnisse

In diesem Kapitel werden die Effekte des Einsatzes der Analyse Patterns für Informationsmanagement (siehe Kapitel 5), die in der Case Study in Teil II dieser Arbeit aufgetreten sind, analysiert und zusammengefaßt.

In Abschnitt 10.1 wird die Wiederverwendung für zwei Teile der Software, die für das Projekt Virtuelle Universität entwickelt wurden, untersucht. Anhand des wiederverwendeten Programmcodes werden, mit Hilfe des *Constructive Cost Model (COCOMO)* [Boehm, 1981, Boehm *et al.*, 2000], die Einsparung quantifiziert. Damit wird der wirtschaftliche Nutzen des Einsatzes von Analyse Patterns, in Form von Zeit- und Kosteneinsparungen in der Softwareentwicklung, nachgewiesen.

In Abschnitt 10.2 wird untersucht, welche Auswirkungen die Berücksichtigung von sozialen Aspekten durch den Einsatz von Analyse Patterns auf die Softwareentwicklung hat. Dazu wird die Akzeptanz des Informationsbrokers des VU-Projekts, der auf den Analyse Patterns aus Kapitel 5 basiert, analysiert. Da das VU-Projekt auf freiwilliger Mitarbeit der Betroffenen (Lehrende, Studierende, ...) basiert, werden zur Untersuchung der Akzeptanz die Motivation zur freiwilligen Mitarbeit, das Wachstum des Informationsbrokers und die Verwendung des Systems herangezogen.

Abschließend werden die Ergebnisse dieser Arbeit in Abschnitt 10.3 zusammengefaßt.

10.1 Einsparungen bei der Softwareentwicklung durch Analysepatterns

Zur Untersuchung der Einsparungen bei der Softwareentwicklung wird die Wiederverwendung auf Codeebene (*Code Reuse*) und auf der Ebene des

Designs für Teile des in der Case Study in Teil II dieser Arbeit beschriebenen Systems herangezogen. Dazu wird vom Grundsystem des Informationsbrokers ausgegangen, wie es für die Virtuelle Universität eingesetzt und im Analyse Pattern *Virtuelle Bibliothek* (siehe Abschnitt 5.3) beschrieben wird. In 10.1.2 wird die Wiederverwendung bei der Einrichtung eines neuen Katalogs des Informationsbrokers betrachtet, und in 10.1.3 wird die Wiederverwendung zur Entwicklung einer neuen Applikation, der Digitalen Bibliothek, untersucht.

10.1.1 Vorgehensweise

Bei den untersuchten Anwendungen handelt es sich um zwei Teilprojekte der VU, die Erweiterung des Informationsbrokers mit einem neuen Katalog für Informationen zu wissenschaftlichen Konferenzen und die Digitale Bibliothek, eine neue Applikation, die auf dem Informationsbroker aufbaut. Beide untersuchten Anwendungen sind in der Skriptsprache PERL [Wall *et al.*, 1996] implementiert, und haben ein Web-Interface auf Basis von HTML [Raggett *et al.*, 1999], in dem für die dynamischen Teile PERL-Code eingebettet ist.

Zur Quantifizierung des Ausmaßes der Wiederverwendung bei den Erweiterungen oder Veränderungen des Grundsystems werden *Source Lines of Code (SLOC)* [Boehm *et al.*, 2000, S.14–16] verwendet. Für SLOC werden nur Zeilen mit ausführbaren Befehlen und Deklarationen gezählt. Kommentare und Leerzeilen werden für SLOC nicht berücksichtigt.

Die Eckdaten der Teilprojekte sind in Tabelle 10.1 zusammengefaßt. Die angegebene Größe des Projekts, gemessen in SLOC, beinhaltet alle Teile, die für das fertige Projekt verwendet werden, also auch wiederverwendete Module. Für die Teilprojekte der VU werden sehr kleine Teams, in der Regel nur ein Entwickler, eingesetzt. Nur bei größeren Teilprojekten, wie der Digitalen Bibliothek, arbeiten zwei oder mehrere Entwickler gemeinsam. Schließlich wird für die betrachteten Projekte noch der tatsächliche eingesetzte Aufwand in Mannmonaten angegeben.

Projekt	Größe in SLOC	Teamgröße	Aufwand in MM
Neuer Katalog	4021	1	0,5
Digitale Bibliothek	7616	2	6

Tabelle 10.1: Eckdaten der untersuchten Projekte

Für die Untersuchung wird zwischen folgenden zwei Arten von Wiederverwendung (*Reuse*) unterschieden (siehe auch [Boehm *et al.*, 2000, S.19–25]):

- **Black-Box Reuse.** Ein bereits bestehendes Modul wird unverändert in einem Softwareprodukt verwendet. Diese Wiederverwendungsart wird auch *Code Reuse* genannt.
- **White-Box Reuse.** Ein bereits bestehendes Modul wird adaptiert, um mit einem neuen Softwareprodukt verwendet zu werden.

In Anhang B befinden sich Listen mit den einzelnen Dateien (Module) der untersuchten Projekte. In diesen Listen wird zu jeder Datei die Größe in SLOC angegeben, und ob sie für das Projekt vom Grundsystem unverändert übernommen wurde (Black-Box Reuse), angepaßt werden mußte (White-Box Reuse), oder neu entwickelt wurde.

In den Listen wurden die SLOC für die in PERL implementierten Teile direkt gezählt. Um die HTML-Teile des betrachteten Systems ebenfalls zu berücksichtigen, wurde nur der in den dynamisch erzeugten HTML-Seiten enthaltene PERL-Code in Form von SLOC gezählt. Dadurch können diese SLOC für die Analyse einfach zu denen aus den reinen PERL-Teilen addiert werden. Damit wird aber gleichzeitig der Anteil des Aufwandes, der durch die Anpassung und Erstellung der statischen Teile der HTML-Seiten entsteht (grafisches Design, konsistente Gestaltung von mehreren Seiten, ...), nicht berücksichtigt. Dies führt dazu, daß die Projektgröße und das Wiederverwendungspotentials in SLOC unterschätzt werden, und damit die hier angestellten Schätzungen der Kosteneinsparung eher konservativ sind.

Mit den Informationen aus den Listen wird die Wiederverwendung von Code und Design analysiert.

Code Reuse. Die Wiederverwendung auf Codeebene wird als Verhältnis des Aufwands, der als Black-Box wiederverwendeten Module, zum Gesamtaufwand des neuen Projekts errechnet (siehe Formel 10.1). Der Aufwand kann dazu beispielsweise direkt als SLOC den Listen in Anhang B entnommen werden.

$$Code\ Reuse = \frac{Black-Box\ Reuse}{Total} \quad (10.1)$$

Um den Wert dieser Wiederverwendung von SLOC in Mannmonate umzurechnen, wird das *Constructive Cost Model (COCOMO)* [Boehm, 1981, Boehm *et al.*, 2000] verwendet. Dieses Modell wird normalerweise dafür eingesetzt, um aus der geschätzten Größe eines Projekts in *Delivered Source Instructions (DSI)* (entspricht SLOC aus [Boehm *et al.*, 2000]) den

Aufwand für die Entwicklung des Softwareprodukts in Mannmonaten zu schätzen. Hier wird es aber eingesetzt, um aus der Größe eines fertigen Projekts die Einsparungen aufgrund von Reuse zu schätzen.

Für das einfachste Modell (*Basic COCOMO* [Boehm, 1981, S.74–113]) benötigt man nur:

- Die Einordnung des Projekts nach den Entwicklungsbedingungen und den Anforderungen an die Software. Boehm bezeichnet dies als *Mode of Software Development*.
- Und die Größe des Projekts in KDSI (1000 DSI).

Die hier betrachteten Projekte werden nach dem *Mode of Software Development* als "Organic" klassifiziert, da sie von einem sehr kleinen Team (1–2 Entwickler) realisiert wurden, und die technischen Spezifikationen und Anforderungen vor Projektbeginn nicht vollständig fixiert waren (siehe [Boehm, 1981, S.78–79]). Damit ergibt sich für das einfache Modell die Gleichung 10.2 (siehe [Boehm, 1981, S.75]). *MM* bezeichnet den benötigten Aufwand in Mannmonaten, und *KDSI* sind die *Delivered Source Instructions* in 1000 Programmzeilen. 2,4 und 1,05 sind die Parameter des Modells im *Mode* "Organic", die von Boehm auf Basis der von ihm erhobenen Projektdaten geschätzt wurden.

$$MM = 2,4 \times KDSI^{1,05} \quad (10.2)$$

Der Aufwand des Gesamtprojekts, der ohne Wiederverwendung entstanden wäre, und der Wert der in Form von Black-Box Reuse (Code Reuse) wiederverwendeten Teile werden mit Formel 10.2 in Mannmonate (MM) umgerechnet. Die Einsparungen durch die Wiederverwendung sind dadurch direkt in Mannmonaten ersichtlich. Daraus kann mit Formel 10.1 die Aufwandsersparnis in Prozent gegenüber einer Neuentwicklung ohne Code Reuse errechnet werden. Diese Aufwandsersparnis liegt aufgrund der nicht linearen Beziehung zwischen MM und SLOC im COCOMO immer etwas unter der reinen Wiederverwendungsrate in SLOC.

Design Reuse. Zur Abschätzung des Werts der Wiederverwendung auf Designebene, wird neben Black-Box auch White-Box Reuse einbezogen. Es wird von der Annahme ausgegangen, daß bereits bestehende Module, die nur adaptiert werden, keinen (oder sehr wenig) zusätzlichen Designaufwand erfordern, neue Module dagegen, neben dem Programmieraufwand, auch den vollen Aufwand für das Design bedeuten.

Um abzuschätzen, wie groß der Anteil des Designs am Gesamtaufwand eines Projektes ist, wird die durchschnittliche Aufteilung des Aufwands auf die einzelnen Phasen nach COCOMO verwendet. Da die hier untersuchten Projekte eine Größe zwischen circa 4000 und 7500 SLOC (siehe Tabelle 10.1 für die Eckdaten) haben, werden die Projekte nach COCOMO als "Intermediate" klassifiziert. Für "Intermediate" Projekte im *Mode* "Organic", entsteht die Aufwandsaufteilung in Tabelle 10.2 [Boehm, 1981, S.64–65].

Phase	Product Design	Detailed Design	Code	Integration
Aufwand	16%	25%	40%	19%

Tabelle 10.2: Aufteilung des Aufwands nach COCOMO für "Intermediate" Projekt im *Mode* "Organic"

Aus Tabelle 10.2 ergibt sich, daß der gesamte Designaufwand (*Product Design* + *Detailed Design*) im Durchschnitt insgesamt 41% des Gesamtaufwands ausmacht. Somit kann der Designaufwand aus dem Gesamtaufwand herausgerechnet werden, und der Designaufwand für das Gesamtprojekt ohne Wiederverwendung kann mit dem Designaufwand, der durch die Wiederverwendung (Black-Box + White-Box Reuse) eingespart wurde, verglichen werden. Daraus lassen sich die Einsparungen aufgrund von Design Reuse schätzen.

In den folgenden Abschnitten werden die realisierten Einsparungen durch Wiederverwendung für zwei Teilprojekte des VU-Projekts untersucht.

10.1.2 Ein zusätzlicher Katalog des Informationsbrokers

Anhand der Einrichtung des Katalogs für Forschungsveranstaltungen (siehe Abbildung 8.4, Seite 102) wird der Anpassungsaufwand für die Erstellung eines zusätzlichen Katalogs des Informationsbrokers untersucht.

Dieser Katalog wurde für eine Verwaltungseinrichtung der Wirtschaftsuniversität Wien eingerichtet und enthält aktuelle Informationen zu internationalen Konferenzen (siehe <http://vu.wu-wien.ac.at/events>). Folgende Veränderungen des Grundsystems mußten dazu vorgenommen werden (siehe Liste in Anhang B.1):

1. Das Interface wurde geringfügig verändert, und dem Aussehen der Web-Seiten der Universität angepaßt (Dateien 2 und 36).

2. Es müssen für die Informationsgüter in diesem Katalog (Konferenzen) zusätzliche Metadaten verwaltet werden. Zum Beispiel der Konferenzort und das Datum, wann die Konferenz beginnt und wann sie endet (Dateien 15, 16 und 24–28).
3. Der Katalog benötigt Zusatzfunktionen, wie die Sortierung der Konferenzen nach dem Datum, an dem sie stattfinden, und die automatische Archivierung von Einträgen zu Konferenzen, nachdem sie stattgefunden haben (Dateien 37–39).

	Einheit	Total	Black-Box Reuse	Code Reuse
PERL	SLOC	3743	3502	93,56%
HTML	SLOC	278	192	69,06%
Summe	SLOC	4021	3694	91,67%
Summe	MM	10,35	9,46	91,40%

Tabelle 10.3: Wiederverwendung von Code bei der Einrichtung des Katalogs für Forschungsveranstaltungen

Tabelle 10.3 zeigt das Ergebnis der Untersuchung der Wiederverwendung von Code für den neuen Katalog. Es konnten mehr als 91,67% des Programmcodes (3694 SLOC) des Grundsystems unverändert wiederverwendet werden. Die Umrechnung des Aufwands in Mannmonate zeigt, daß allein durch Code Reuse der Entwicklungsaufwand von 10,35 MM um 9,46 MM, also um 91,40% gegenüber einer Neuentwicklung, reduziert werden konnte.

Aufwand	Anteil	Einheit	Total	Black + White-Box Reuse
Gesamt	100%	SLOC	4021	3974
Gesamt	100%	MM	10,35	10,22
Design	41%	MM	4,24	4,19

Tabelle 10.4: Wiederverwendung von Design bei der Einrichtung des Katalogs für Forschungsveranstaltungen

Tabelle 10.4 zeigt den Designaufwand für den neuen Katalog. Wäre das Projekt vollkommen neu entwickelt worden, wären 4,24 MM für das Design (41% des Gesamtaufwandes) angefallen. Durch Wiederverwendung auf der Designebene konnte der notwendige Designaufwand für das Projekt von 4,24 MM um 4,19 MM auf nur 0,05 MM reduziert werden. Das bedeutet eine geschätzte Wiederverwendungsrate von Design in der Höhe von 98,82%.

Die hohen Wiederverwendungsraten im Bereich Code und Design Reuse dieses Projekts können damit begründet werden, daß der Informationsbroker für die Verwaltung von Katalogen entwickelt wurde, und für das betrachtete System nur eine minimale Erweiterung notwendig war. Aus diesem Grund wird im nächsten Abschnitt die Wiederverwendung zur Erstellung einer gänzlich neuen Applikation untersucht.

10.1.3 Eine neue Applikation: Die Digitale Bibliothek

Die Digitale Bibliothek ist eine neue Applikation, die auf Teilen des Designs und des Codes des Grundsystems des Informationsbrokers aufbaut (siehe <http://diglib.wu-wien.ac.at>). Statt Metadaten mit Verweisen, wie sie im Informationsbroker verwaltet werden, muß die Digitale Bibliothek die gesamten Informationsgüter (druckbare Dokumente, Programme, ...) mitsamt Beschreibung (Metadaten) zentral speichern und verwalten.

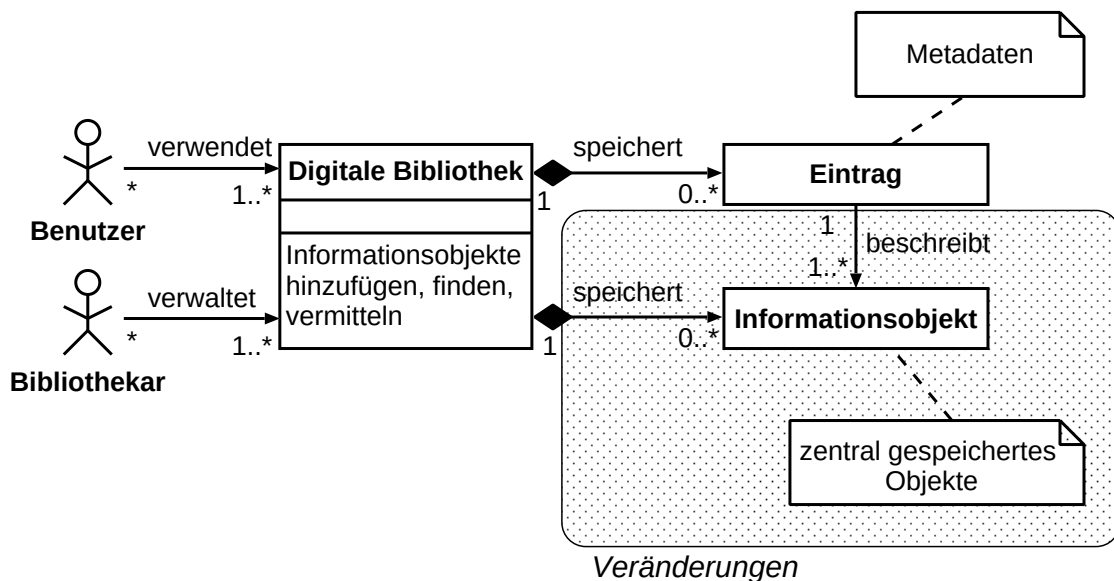


Abbildung 10.1: Klassendiagramm der Digitalen Bibliothek

Um die Beschreibungen der Informationsgüter (analog zu Karteikarten in einer Bibliothek) zu verwalten, wurde auf das Design des Informationsbrokers zurückgegriffen. In Abbildung 10.1 ist das Klassendiagramm der Digitalen Bibliothek dargestellt. Im Vergleich zu dem Klassendiagramm der Virtuellen Bibliothek (siehe Abbildung 5.8 in Abschnitt 5.3) ist die einzige Veränderung, daß die Informationsobjekte nicht mehr verteilt sind, sondern zentral gespeichert werden. Die Realisierung dieser Veränderung war aber aus folgenden Gründen eine komplexe Aufgabe:

- Informationsobjekte verschiedenen Typs (Lehrmaterial, wissenschaftliche Arbeiten, Programme, ...) müssen verwaltet und gespeichert werden. Damit mußten polymorphe Einträge implementiert werden (siehe Abschnitt 5.2).
- Informationsobjekte können in verschiedenen Formen gleichzeitig vorkommen (in HTML, als druckbare Dokumente, in komprimierter Form, ...). Daher mußte ein Informationsobjekt als Container mehrerer Versionen der selben Information entwickelt werden.

Bei der Entwicklung der zentralen Speicherung der Informationsgüter konnte auf die Infrastruktur des Informationsbrokers (Module) zurückgegriffen werden. Folgende Änderungen und Erweiterungen mußten durchgeführt werden (siehe Liste in Anhang B.2):

1. Das Aussehen für die Digitale Bibliothek wurde adaptiert (Datei 2).
2. Die Metadaten zur Beschreibung der Informationsgüter wurden verändert, und mehrere Typen von polymorphen Beschreibungen (siehe Abbildung 5.5, Seite 67) wurden eingeführt (Dateien 16–21, 32–42 und 68–71).
3. Neue Teile für die Speicherung und Verwaltung der Informationsgüter wurden entwickelt (Dateien 43–59, 72–112, 127–132, 134 und 135).

	Einheit	Total	Black-Box Reuse	Code Reuse
PERL	SLOC	6954	3687	53,02%
HTML	SLOC	662	224	33,84%
Summe	SLOC	7616	3911	51,35%
Summe	MM	20,23	10,05	49,68%

Tabelle 10.5: Wiederverwendung von Code bei der Entwicklung der Digitalen Bibliothek

Tabelle 10.5 zeigt die Auswirkungen von Code Reuse auf die Entwicklung der Digitalen Bibliothek. Eine Neuentwicklung ohne Wiederverwendung hätte, auf Basis von COCOMO geschätzt, einen Aufwand von 20,23 MM bedeutet. Alleine durch Wiederverwendung von Code konnte dieser Aufwand auf 10,05 MM ungefähr halbiert werden.

Tabelle 10.6 zeigt die Einsparungen aufgrund von Design Reuse. Von 8,29 MM für das Design des gesamten Systems (41% des mittels COCOMO

Aufwand	Anteil	Einheit	Total	Black + White-Box Reuse
Gesamt	100%	SLOC	7616	4506
Gesamt	100%	MM	20,23	11,66
Design	41%	MM	8,29	4,78

Tabelle 10.6: Wiederverwendung von Design bei der Entwicklung der Digitalen Bibliothek

geschätzten Gesamtaufwandes von 20,23 MM) konnten 4,78 MM wiederverwendet werden. Das bedeutet eine Wiederverwendungsrate für Design von 57,66%.

Aufwand	Anteil	Einheit	ohne Reuse	mit Reuse
Gesamt Black-Box	100%	SLOC	3911	0
Design White-Box	41%	SLOC	244	0
Rest White-Box	59%	SLOC	351	351
Gesamt New	100%	SLOC	3110	3110
Summe	100%	SLOC	7616	3461
Summe	100%	MM	20,23	8,84

Tabelle 10.7: Vergleich des Aufwands mit Reuse und ohne Reuse

Interessant ist auch noch der Vergleich zwischen dem Gesamtaufwand des Projekts und dem Aufwand, der laut COCOMO aufgrund der Einsparungen durch Code und Design Reuse gemeinsam erreicht hätte werden können. Dieser Vergleich ist in Tabelle 10.7 dargestellt. Dazu wird für die Aufwandsschätzung der Entwicklung mit Reuse der gesamte Aufwand für die als Black-Box wiederverwendeten Teile und der Designaufwand (41% des Gesamtaufwandes) für die als White-Box wiederverwendeten Teile herausgerechnet. Im Vergleich zum geschätzten Gesamtaufwand von 20,23 MM ohne Reuse, benötigt die Realisierung des Projekts durch Wiederverwendung nur mehr 8,84 MM, also weniger als die Hälfte. Dies bedeutet eine Gesamteinsparung von 11,39 MM, also fast einem Mannjahr. Die tatsächliche Implementierungszeit für das Projekt (die Digitale Bibliothek ohne der parallel entwickelten Personalisierung) war weniger als 6 Monate. Die Differenz, zu den mittels COCOMO geschätzten 8,84 MM, kann folgende Gründe haben:

1. Der eingesetzte Entwickler ist überdurchschnittlich produktiv, beziehungsweise die Dokumentation des Projekts wurde, aufgrund des sehr kleinen Entwicklerteams, nicht der Industrienorm entsprechend durchgeführt.

2. Auch für die neu entwickelten Teile konnte Design, das im Analyse Pattern *Virtuelle Bibliothek* (siehe 5.3, Page 5.3) vorgeschlagen wird, wiederverwendet werden.
3. Das Analyse Pattern hilft beim Verständnis des Problems, und dadurch wird die Analyse und das Design für die Entwicklung der Digitalen Bibliothek vereinfacht.

Welcher dieser Punkte hauptsächlich für die Differenz verantwortlich ist, kann mit einer Analyse der Projektgröße und der Wiederverwendung von Code alleine nicht beantwortet werden. Dennoch wurden Hinweise gefunden, daß die Differenz nicht ausschließlich auf die Fähigkeiten des Programmierers zurückzuführen sind:

- Bei einer Analyse des Codes der neu entwickelten Teile der Digitalen Bibliothek zeigte sich, daß Design aus dem Analyse Pattern *Virtuelle Bibliothek* übernommen wurde. Beispielsweise wurde das im Analyse Pattern vorgeschlagene Design Pattern *Command Processor* [Sommerlad, 1996] auch für die neuen Teile zur Verwaltung der Informationsobjekte eingesetzt.
- Das Klassendiagramm der Digitalen Bibliothek in Abbildung 10.1 auf Seite 156 weist große Ähnlichkeiten mit dem, im Analyse Pattern *Virtuelle Bibliothek* in Abschnitt 5.3 vorgeschlagenen, Klassendiagramm auf. Die einzige Änderung ist die zentrale Speicherung der Informationsprodukte. Damit können viele Überlegungen aus dem Analyse Pattern auch für die Digitale Bibliothek angewandt werden.

Aus diesen Punkten ergibt sich ein weiteres Einsparungspotential im Bereich des Designs und der Analyse der neuen Teile von 1275 SLOC (41% des Aufwands in SLOC). Das entspricht 3,1 MM, womit sicher auch ein Teil der Differenz zwischen geschätztem und tatsächlichem Entwicklungsaufwand mit dem Einsatz von Analyse Patterns erklärt werden kann.

10.2 Soziale Aspekte der Verwendung von Analyse Patterns

Die Wirkungen der sozialen und organisatorischen Konsequenzen werden hier anhand der Akzeptanz des VU-Informationsbrokers demonstriert. Dazu werden die Dimensionen: Motivation zur freiwilligen Mitarbeit, Wachstum des Informationsbrokers und die Verwendung des Systems betrachtet.

Organisatorisch bietet das Analyse Pattern *Virtuelle Bibliothek* (siehe Abschnitt 5.3) einen zentralen Zugang zu notwendigen Informationen, und eine Möglichkeit gemeinsam diese Informationen zu sammeln. Gleichzeitig bietet es die Eigenschaften, daß die Kontrolle über die Information beim Anbieter der Information bleibt, und daß damit die Information durch den Anbieter jederzeit verändert werden kann. Das bedeutet auch, daß jeder Informationsanbieter die Form, in der er seine Information anbieten will, frei wählen kann. Dies ist ein wichtiger Punkt für die Akzeptanz eines Informationssystems, das auf freiwilliger Mitarbeit basiert. Beispielsweise schreibt Turoff [Turoff, 1999] für den Bereich von online Lehrunterlagen, daß Versuche, Lehrende zu zwingen, Unterlagen nach einem vorgegebenen Schema zu gestalten, Kreativität ersticken und am Widerstand der Lehrenden scheitern werden. Diese Widerstände werden natürlich noch größer, falls man die Lehrenden zwingen möchte, ihre Unterlagen mit einem bestimmten System zu erstellen, und dabei ihre Kontrolle über die erstellten Unterlagen einschränkt.

Motivation zur freiwilligen Mitarbeit. Als Indikator für Motivation wird die Anzahl der Personen, die freiwillig am VU-Projekt mitarbeiten, beziehungsweise, die das Projekt unterstützen, benutzt. Derzeit, Stand Oktober 2000, sind das:

- Mehr als 15 Personen arbeiten freiwillig oder im Rahmen ihrer Tätigkeit an der Universität als Bibliothekare zur Wartung der Inhalte des Informationsbrokers mit.
- 62 Lehrende (8 externe Lektoren oder Gastprofessoren und 54 Mitarbeiter der Universität) sind im Informationsbroker mit Informationen und Unterlagen zu 119 Lehrveranstaltungen (entspricht ungefähr 6,5% der insgesamt pro Semester angebotenen Lehrveranstaltungen) vertreten. Damit verwenden mehr als 17% der 317 derzeit an der Wirtschaftsuniversität beschäftigten lehrenden Personen (Assistenten und Professoren) den Informationsbroker, um Unterlagen anzubieten, und das, obwohl es dazu keinerlei finanziellen oder organisatorischen Anreiz seitens der Universität gibt.
- Es bestehen mehrere freiwillige Kooperationen mit Verwaltungs- und Forschungseinrichtungen der Wirtschaftsuniversität Wien. Beispielsweise werden Teile der Internetpräsenz der Universität durch den VU-Informationsbroker verwaltet, und der Informationsbroker wird von 3 Abteilungen der Universität für die Sammlung von Forschungsmaterial benutzt.

- Und nicht zuletzt haben weit über 600 Personen freiwillig Informationen in den Informationsbroker eingegeben, und dabei ihren Namen beziehungsweise ihre E-Mail Adresse angegeben.

Wachstum des Informationsbrokers. Abbildung 10.2 zeigt das Wachstum der Anzahl der freiwillig in den Informationsbroker eingetragenen Informationsgüter seit Beginn des VU-Projekts, Mitte 1997. Derzeit enthält der Informationsbroker weit über 8000 Einträge, die von Benutzern freiwillig eingetragen wurden.

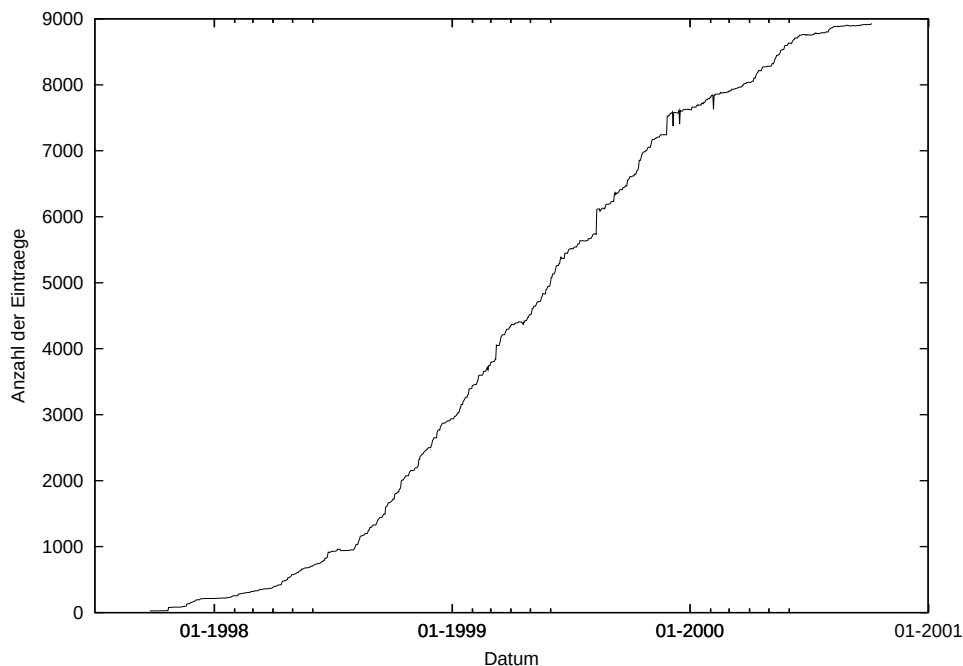


Abbildung 10.2: Anzahl der Informationsgüter im VU-Informationsbrokers

Wie im Abschnitt 8.1.3 über Netzwerkeffekte beschrieben, hängt das Wachstum eines Informationsbrokers auf Informationsmärkten unter anderem davon ab, wie übersichtlich er die Benutzer (Konsumenten) über die angebotenen Informationsgüter informiert. Der Rückgang des Wachstums seit Ende 1999 läßt darauf schließen, daß das listenorientierte Benutzerinterface aufgrund der vielen Informationsgüter bereits unübersichtlich wird, und dadurch die Größeneffekte des Brokers bereits abgeschwächt werden. Die Einführung des Recommendersystems (siehe Abschnitt 9.4) soll dieser Entwicklung Rechnung tragen.

Verwendung des Informationsbrokers. In Abbildung 10.3 ist die Anzahl der Vermittlungen von Informationsgütern durch den Informations-

broker pro Tag dargestellt. Es ist ein stetiger Aufwärtstrend zu erkennen, sowie starke Schwankungen zwischen Spitzentagen und schwächeren Tagen, zum Beispiel Wochenende beziehungsweise Feiertagen.

Um den stetigen Aufwärtstrend besser zu erkennen, wird in Abbildung 10.4 der gleitende Durchschnitt der Vermittlungen pro Tag dargestellt. Das gewählte Zeitfenster für den gleitenden Durchschnitt ist 30 Tage, also circa 1 Monat. Deutlich zu erkennen sind Saisonalitäten durch die vorlesungsfreie Zeit an der Universität. Dies sind besonders die Sommerferien (Juli–September), Weihnachten (Ende Dezember), Semesterferien (Februar) und die Osterferien (Ende April).

Damit die dargestellte Anzahl der Vermittlungen durch den Informationsbroker mit anderen online Informationssystemen verglichen werden kann, müssen die für die Analyse von Transaktionslogs von Web-Servern üblichen Maßzahlen verwendet werden. Zur Analyse des Transaktionslogs des für den Broker eingesetzten Web-Servers wird das frei verfügbare Produkt *Webalizer* in der Version 1.3 eingesetzt (siehe <http://www.mrunix.net/webalizer/>). Dieses Produkt errechnet übliche Maßzahlen, wie beispielsweise *Hits* und *Pageviews*. Hits sind definiert als Anzahl der Einträge im Transaktionslog eines Web-Servers, als Pageviews wird die Anzahl der von einem Server angeforderten HTML-Seiten bezeichnet. Anfang Oktober 2000 wurde eine durchschnittliche Anzahl von circa 920 Vermittlungen pro Tag vom Informationsbroker mitgeschrieben. Die Analyse des Transaktionslogs des für den Broker eingesetzten Web-Servers hat in dieser Zeit pro Tag circa 5000 Pageviews und 10000 Hits ergeben.

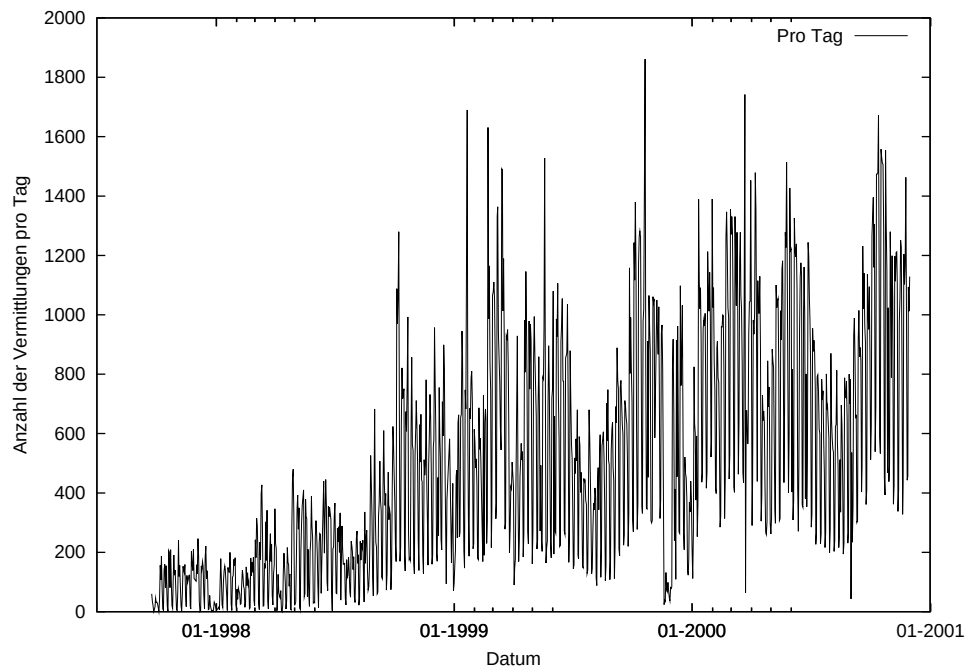


Abbildung 10.3: Vermittlungen des VU-Informationsbrokers pro Tag

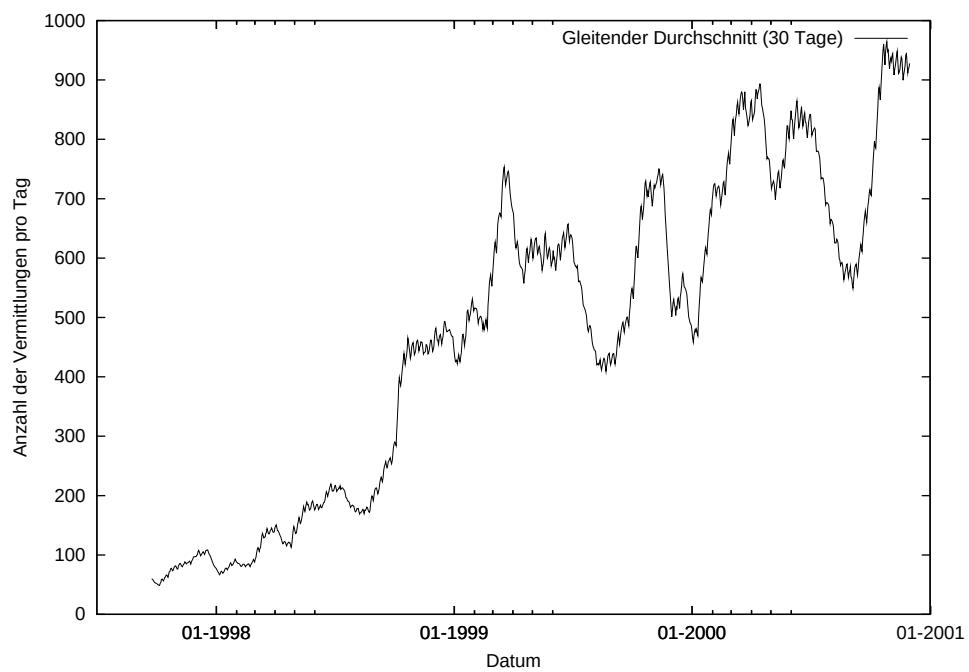


Abbildung 10.4: Trend der Vermittlungen des VU-Informationsbrokers

10.3 Zusammenfassung

In Teil I dieser Arbeit wurde der in der Literatur bisher noch nicht ausreichend beschriebene Einsatz von Patterns in der Analysephase untersucht. Diese Untersuchung ergab, daß Analyse Patterns den Großteil des Softwareentwicklungsprozesses (Analyse, Design bis zur Implementierung) unterstützen, und dadurch die benötigte Entwicklungszeit verkürzen und gleichzeitig die Akzeptanz des entstehenden Systems erhöhen.

Diese Vorteile konnten in der Arbeit anhand einer Case Study in Teil II nachgewiesen werden. Die Case Study beschreibt die Entwicklung des Informationsbrokers des Projekts *Virtuelle Universität der Wirtschaftsuniversität Wien*, der auf dem in dieser Arbeit entwickelten Analyse Patterns für Informationsmanagement basiert.

Die Untersuchung der Entwicklungszeit für eine Erweiterung des Informationsbrokers um einen neuen Katalog ergab, daß dabei durch die Verwendung des Analyse Patterns geschätzte 91,40% des Aufwandes (9,46 Mannmonate) durch Wiederverwendung eingespart werden konnten. Für ein weiteres Teilprojekt, die Realisierung einer Digitalen Bibliothek, konnten durch das Analyse Pattern, obwohl es für Virtuelle Bibliotheken erstellt wurde, noch immer geschätzte 11,39 Mannmonate von insgesamt 20,23 Mannmonaten eingespart werden.

Die erhöhte Akzeptanz wurde anhand der freiwilligen Mitarbeit am Projekt sowie der Verwendung des Informationsbrokers festgestellt. Obwohl andere Systeme durch soziale Probleme nicht erfolgreich sind (siehe Abschnitt 7.2), wurden bisher freiwillig Lehrunterlagen zu 119 Lehrveranstaltungen (entspricht ungefähr 6,5% des derzeitigen Lehrangebots der Wirtschaftsuniversität Wien) in den Informationsbroker des Projekts eingetragen. Weiters bestehen mehrere freiwillige Kooperationen mit Einrichtungen der Universität und über 600 Personen haben freiwillig Informationen in den Informationsbroker eingegeben. Die Verwendung des Informationsbrokers stieg seit Projektbeginn kontinuierlich an und beträgt derzeit (November 2000) über 27000 Vermittlungen pro Monat, das entspricht circa 300000 Hits pro Monat.

Diese Ergebnisse sind vielversprechend, und bestätigen, daß Wiederverwendung durch den Einsatz von Analyse Patterns, gemeinsam mit Design Patterns, AntiPatterns und andere Techniken, ein enormes Potential zur Reduktion von Entwicklungskosten, zur Erhöhung der Softwarequalität und zur Verbesserung der Akzeptanz bieten. Dieses Potential wird in Zukunft immer besser ausgenutzt werden können, da bereits moderne Vorgehensmodelle zur Softwareentwicklung, wie zum Beispiel *The Unified Software Development Process* [Jacobson *et al.*, 1999b, S.71], die Verwendung von Patterns vorsehen.

Anhang A

Analysen zum Informationsbroker der Virtuellen Universität

In diesem Teil des Anhangs werden Analysen zu Informationsgütern und der Verwendung durch Benutzer präsentiert. Diese Analysen dienen zur Begründung von Designentscheidungen für den Informationsbroker, der in der Case Study in Teil II der Arbeit beschrieben wird.

A.1 Verweildauer von Informationsgütern im Broker

In diesem Abschnitt wird die durchschnittliche Verweildauer (Lebensdauer) von Informationsgütern in den Katalogen des Brokers der Virtuellen Universität untersucht. Diese Verweildauer beeinflusst unter anderem das Design des Brokers. Beispielsweise bedeutet eine sehr lange Verweildauer eines Informationsguts im Broker, daß es wirtschaftlich gerechtfertigt ist, mehr Aufwand bei der Klassifikation zu betreiben, und daß damit auch manuelle Klassifikation in Frage kommt. Bei sehr kurzen Verweildauern müssen hingegen automatische Klassifikationsverfahren eingesetzt werden.

Die Lebensdauer wird hier definiert als Zeitraum zwischen der Eintragung eines Informationsguts in einen Katalog des Brokersystems bis zu seiner Löschung. Die Gründe für die Löschungen werden hier nicht betrachtet, Gründe können zum Beispiel sein: das Informationsgut verschwindet aus dem Internet, oder das Informationsgut wird durch ein Neues ersetzt.

Für die Analyse wird die Hasardrate herangezogen (siehe [Lancaster, 1990], [Allison, 1998]). Die Hasardfunktion $h(t)$ für kontinuierliche Daten wird in Formel A.1 definiert. Sie beschreibt das Risiko (die bedingte Wahrscheinlichkeit P), daß für ein betrachtetes Individuum in einem sehr kleinen Intervall um den Zeitpunkt t ein bestimmtes Ereignis eintritt. Die Bedingung $T \geq t$ bedeutet, daß nur Individuen betrachtet werden, für die das Ereignis bis zum Zeitpunkt t noch nicht eingetreten ist.

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{P\{t \leq T < t + \Delta t \mid T \geq t\}}{\Delta t} \quad (\text{A.1})$$

Zur Schätzung der durchschnittlichen Verweildauer von Informationsgütern in den einzelnen Katalogen des Brokers wird das diskrete Modell für die Hasardrate verwendet. Die Hasardrate für diskrete Zeit entspricht der Wahrscheinlichkeit $P(t + \Delta t)$, daß ein Ereignis (Löschung eines Informationsguts) im Zeitraum t bis $t + \Delta t$ eintritt. Diese Wahrscheinlichkeit kann, wie in Formel A.2 dargestellt, als Verhältnis der aufgetretenen Ereignisse (Löschungen) $d(\Delta t)$ zu der dem Risiko ausgesetzten Population Q geschätzt werden.

$$\hat{h}(t) = P(t + \Delta t) = \frac{d(\Delta t)}{Q} \quad (\text{A.2})$$

Als Kehrwert der geschätzten durchschnittlichen Hasardrate $1/\hat{h}(t)$ ergibt sich die geschätzte durchschnittliche Lebensdauer zum Zeitpunkt t .

Für die Schätzung werden folgende Annahmen getroffen:

1. Die Löschungen sind voneinander unabhängige, zufällige Ereignisse, wobei die Wahrscheinlichkeit, daß ein Ereignis in einem sehr kleinen Zeitintervall auftritt, positiv und kleiner als 1 ist.
2. Die durchschnittliche Hasardrate wird über die Zeit als konstant angenommen. Das bedeutet, die Hasardrate ist nicht von der Vorgeschichte, also beispielsweise von der Zeit, die sich die Informationsgüter im Broker befunden haben, abhängig. Sie ist lediglich von der Länge des betrachteten Zeitraums bestimmt. Die Annahme, daß die Hasardrate mit dem Alter eines Informationsguts steigt, erscheint logisch, da mit fortschreitendem Alter die Wahrscheinlichkeit, daß zum Beispiel die Information veraltet, oder der Autor die Lust an der Wartung verliert, steigt. Jedoch entspricht der betrachtete Zeitraum (Verweildauer im Broker) nicht dem gesamten Lebenszyklus

des Informationsguts, sondern nur einem Ausschnitt. Güter können zu jedem Zeitpunkt in ihrem Lebenszyklus in den Broker eingetragen oder auch wieder aus dem Broker entfernt werden. Daher wird für die Schätzung angenommen, daß die durchschnittliche Hasardrate in einem bestimmten Katalog konstant ist. Dies bedeutet eine stationäre Verteilung für die Auftrittswahrscheinlichkeit einer Löschung.

Für die Schätzung werden Daten aus dem Beobachtungszeitraum eines Jahres (August 1999 – Juli 2000) verwendet. Die Daten und Ergebnisse für die einzelnen Kataloge des Brokers sind in Tabelle A.1 zusammengefaßt. Q ist die Größe der betrachteten Population. Die Population besteht aus den Informationsgütern, die zu Beginn des Beobachtungszeitraums t eingetragen, und damit dem Risiko der Löschung ausgesetzt, sind. Während des Betrachtungszeitraums neu hinzugekommene Informationsgüter wurden aus den Daten entfernt, dadurch enthalten die Daten nur noch den zu untersuchenden Sterbeprozess. Die Anzahl der in der betrachteten Periode Δt gelöschten Informationsgüter aus der betrachteten Population wird mit $d(\Delta t)$ bezeichnet. Als gelöscht wurden alle Informationsgüter gewertet, die in diesem Zeitraum manuell durch den Katalogverwalter gelöscht wurden, plus alle Güter, die am Ende der Periode durch die Beobachtungsagenten (siehe Abschnitt 9.3.2, Seite 132) als unerreichbar markiert waren, da diese durch den Katalogverwalter gelöscht werden sollten.

	$Q(t)$	$d(\Delta t)$	$\hat{h}(t)$	$1/\hat{h}(t)$
Living Lectures	106	24	22,64%	4,4 Jahre
Tools & Manuals	472	87	18,43%	5,4 Jahre
Materials Coll.	828	109	13,16%	7,6 Jahre
VU-Bibliothek	3257	180	5,53%	18,1 Jahre
Universitätsinfo.	181	1	0,55%	181,8 Jahre
Summe	4844	401	8,28%	12,1 Jahre

Tabelle A.1: Anzahl der Informationsgüter und Löschungen während 12 Monaten in Katalogen des VU-Brokers

Eine weitere Methode zur Modellierung der Verweildauer von Informationsgütern im Broker stellen Warteschlangenmodelle dar (siehe [Gross und Harris, 1985] [Taha, 1982, Kap.15]). Das gängigste stochastische Warteschlangenmodell nimmt an, daß die Intervallzeiten (hier Verweildauer im Broker) exponential verteilt sind, und daher die Eintrittsrates der Ereignisse (hier Löschungen) einer Poisson-Verteilung folgt [Gross und Harris, 1985, S.20–27]. Mit diesen Annahmen kommt man zum gleichen Ergebnis wie bei der Verwendung der Hasardrate. Die Ver-

weildauer im Broker ist dann mit dem Parameter $1/\lambda = 1/h(t)$ exponential verteilt, und die Anzahl der Löschungen entspricht einer Poisson-Verteilung mit dem Mittelwert und der Varianz λt . Die Wahrscheinlichkeit $P_n(t)$, daß während der Periode t n Ereignisse auftreten, ist in Formel A.3 dargestellt.

$$P_n(t) = \frac{(\lambda t)^n e^{-\lambda t}}{n!}, \quad n = 0, 1, 2, \dots \quad (\text{A.3})$$

Die geschätzte Lebensdauer von Informationsgütern im Broker ist stark vom betrachteten Katalog abhängig. Dies ist hauptsächlich auf die unterschiedlichen Eigenschaften der dort gesammelten Informationsgüter zurückzuführen. Beispielsweise werden die Lehrveranstaltungen im Katalog *Living Lectures* von vielen Vortragenden jedes Semester neu angelegt, um die alten Unterlagen weiter zur Verfügung zu haben. Dadurch entsteht die durchschnittliche Lebensdauer von circa 4,4 Jahren. Die *VU-Bibliothek* beinhaltet hauptsächlich bekannte Zeitschriften, Organisationen (zum Beispiel Universitäten) und große Unternehmen. Diese Web-Sites sind mit ca. 13 Jahren durchschnittlich geschätzter Lebenszeit im Broker deutlich langlebiger, als die Güter in anderen Katalogen. Der Katalog *Universitätsinformationen* ist eine Abbildung der Organisationsstruktur der Universität (Web-Seiten von Instituten, Serviceeinrichtungen, ...). Die lange geschätzte Lebensdauer resultiert aus den relativ wenigen Änderungen der Organisationsstruktur im betrachteten Zeitraum.

A.2 Vermittlungshäufigkeit von Informationsgütern

In diesem Abschnitt wird die Vermittlungshäufigkeit (Anzahl der Vermittlungen) von Informationsgütern in den einzelnen Katalogen des Brokers analysiert, da diese Information für die Weiterentwicklung des Systems interessant sind. Beispielsweise können damit Entscheidungen für den Ausbau bestimmter Kataloge, oder die Notwendigkeit von Werbemaßnahmen, begründet werden.

In der Lagerverwaltung wird die ABC-Analyse für die selektive Bestandssteuerung eingesetzt [Tresine, 1982, S.438–441]. Die Produkte werden dabei nach den Geldwert, dem gebundenen Kapital im Lager (*Menge* $\times$ *Wert*), in Klassen eingeteilt. Die Klasse A besteht aus den Produkten mit den höchsten Geldwerten. Sie repräsentiert typischerweise 70–80% des gesamten gebundenen Kapitals im Lager, beinhaltet aber nur 15–20% der Produkte. Der Klasse B werden die Produkte mit geringerem Geldwert zugeordnet. Sie repräsentiert 20–25% des Lagerwerts, und beinhaltet 20–25% der Produkte. In Klasse C befinden sich die Produkte mit dem geringsten Geldwert. Typischerweise stellen diese Produkte nur 5–10% des gesamten Lagerwerts dar, obwohl sie 60–65% der verschiedenen Produkte sind. Der Sinn der Einteilung der Produkte in Klassen ist es, einen passenden Grad an Kontrolle für die einzelnen Produkte zu verwenden. Beispielsweise wird argumentiert, daß Produkte der Klasse A die größte Aufmerksamkeit verdienen, da sie zwar wenige Produkte sind, aber das meiste Kapital (große Menge und/oder hoher Preis) binden. Kann der Lagerbestand an Klasse A Produkten reduziert werden, entsteht dadurch eine signifikante Reduktion des gebundenen Kapitals.

Die ABC-Analyse kann auch zur Einteilung der Informationsgüter im Broker herangezogen werden. Eingeteilt werden die Güter in Klassen nach der Anzahl der Vermittlungen. Informationsgüter mit hoher Vermittlungsrate verdienen mehr Aufmerksamkeit, da sie mehr zur Benutzung des Brokers beitragen.

Gemessen werden die Vermittlungen im Broker als gleitender Durchschnitt mit einem Zeitfenster von einem Monat (30 Tagen). Errechnet wird dieser gleitende Durchschnitt $\theta(t)$ für jedes Informationsgut einmal täglich aus dem Durchschnitt des Vortages $\theta(t-1)$ diskontiert um den Diskontierungsfaktor δ plus den Vermittlungen des Tages $v(t)$ (siehe Formel A.4). Der Diskontierungsfaktor wird aus der Fenstergröße n durch $\delta = \frac{n-1}{n}$ errechnet.

$$\theta(t) = \delta\theta(t-1) + v(t) \quad (\text{A.4})$$

Für die Analyse werden die Informationsgüter nach der Anzahl ihrer Vermittlungen (Stand: 28. Juli 2000) gereiht. Das Ergebnis dieser Reihung für die 121 Informationsgüter im Katalog *Living Lectures* ist in Abbildung A.1 dargestellt. Stellt man die Vermittlungen kumuliert dar, entsteht die aus der ABC-Analyse bekannte Kurve in Abbildung A.2. Als Klassengrenzen für die Klasse A 20% der Güter und für die Klasse B weitere 20% der Güter festgelegt. Die Klasse C umfaßt die restlichen 60%. Betrachtet werden die Vermittlungen, die durch die Informationsgüter in den einzelnen Klassen verursacht werden.

Tabelle A.3 zeigt, aufgeschlüsselt nach Katalog, den prozentuellen Anteil der Vermittlungen, den die Informationsgüter in den Klassen A, B und C verursachen. In den Abbildungen A.4 bis A.7 sind die kumulierten Vermittlungen für die weiteren Kataloge des Brokers dargestellt. Die Prozentsätze in der Tabelle und die Verläufe der Kurven sind von vielen verschiedenen Faktoren abhängig. Einige davon sind: die Größe des Katalogs, die Wahl der Kategorien und die Art der Informationsgüter. Für eine Quantifizierung der Einflüsse dieser Faktoren müssen weitere Untersuchungen durchgeführt werden.

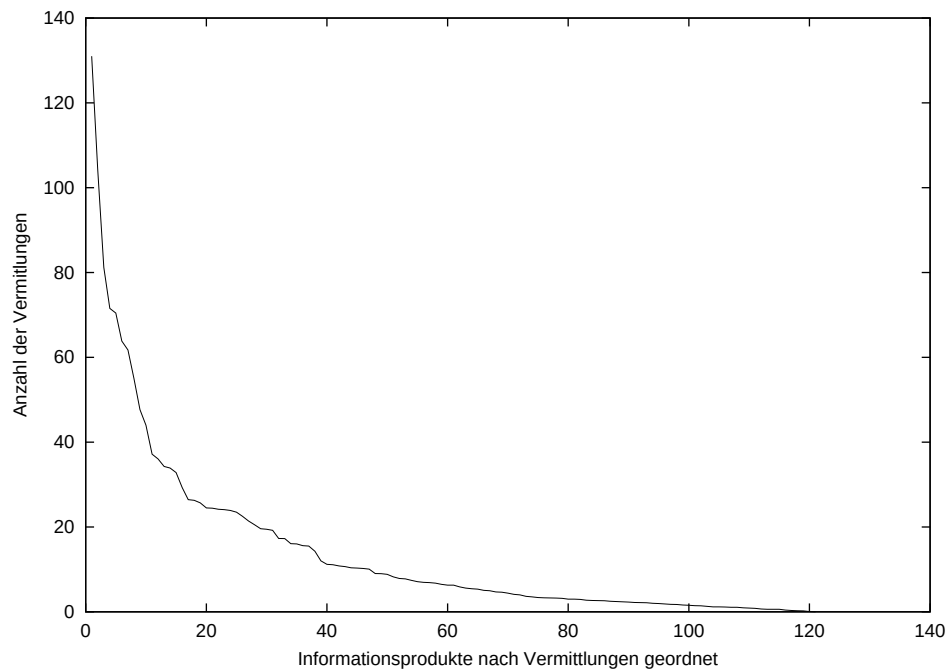


Abbildung A.1: Analyse der Vermittlungen für Living Lectures

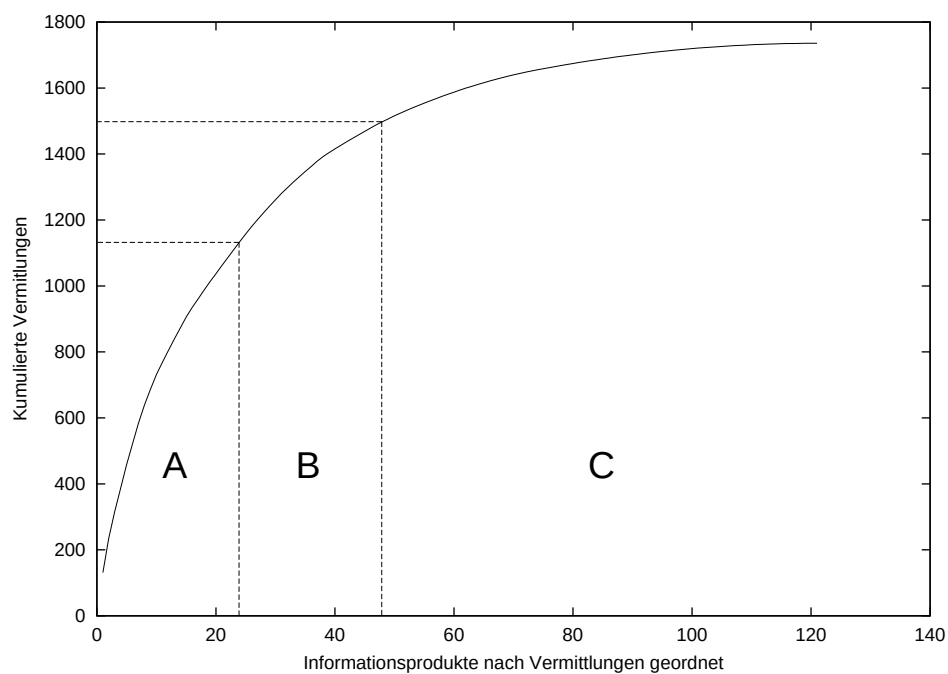


Abbildung A.2: Analyse der kumulierten Vermittlungen für Living Lectures mit der Einteilung in die Klassen A, B und C

	Klasse A	Klasse B	Klasse C
VU-Bibliothek	93,9%	4,8%	1,3%
Tools & Manuals	79,3%	13,0%	7,7%
Materials Coll.	74,4%	21,4%	4,2%
Living Lectures	65,3%	21,0%	13,7%
Universitätsinfo.	53,0%	29,9%	23,1%

Abbildung A.3: Anteil der Informationsgüter der Klassen A, B und C an der Gesamtzahl der Vermittlungen

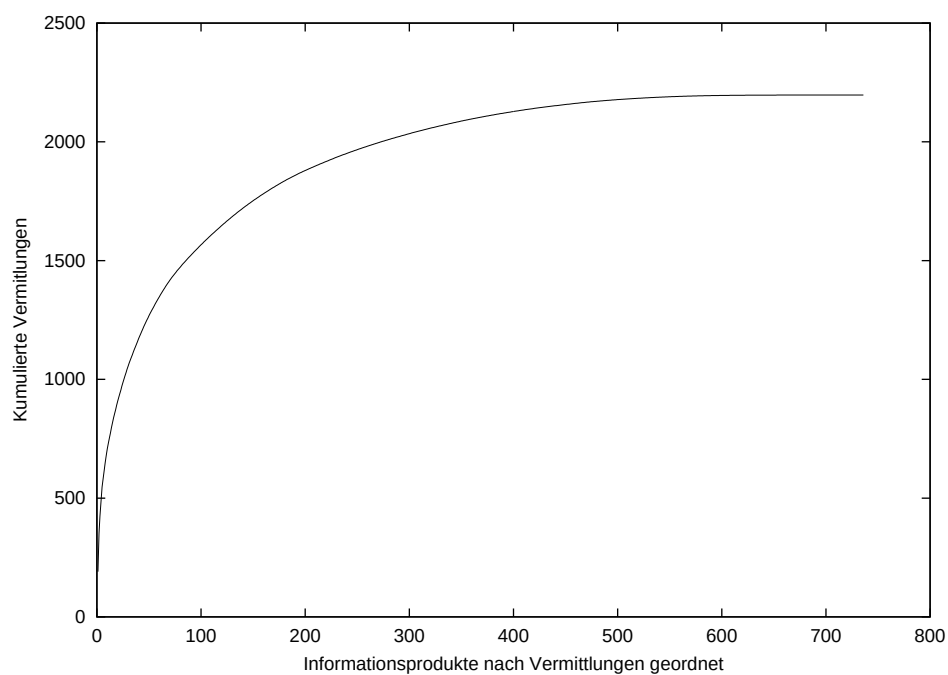


Abbildung A.4: Analyse der kumulierten Vermittlungen für Tools & Manuals

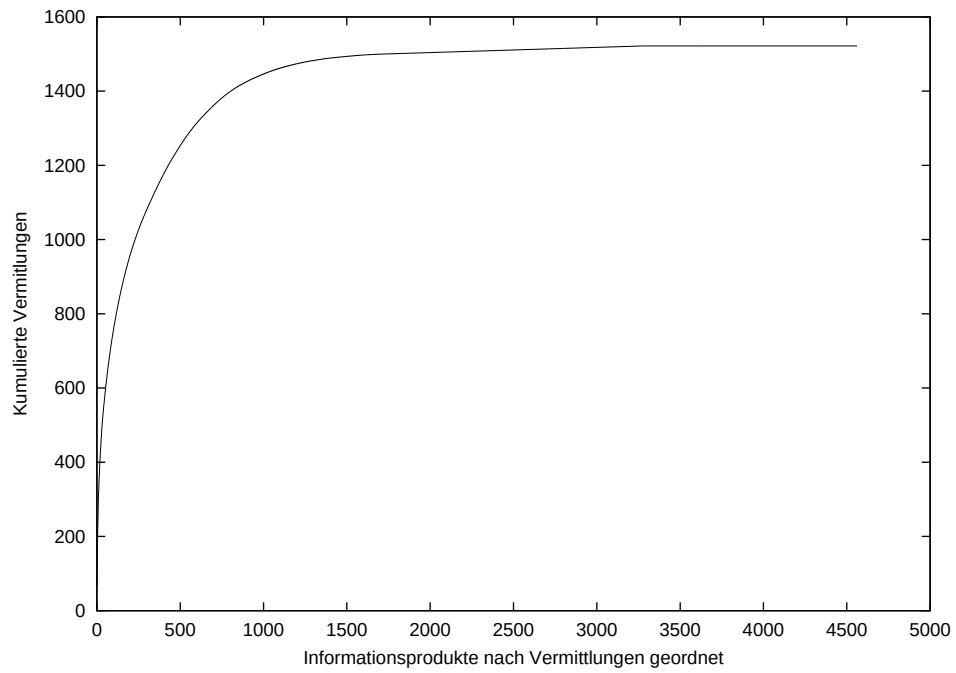


Abbildung A.5: Analyse der kumulierten Vermittlungen für VU-Bibliothek

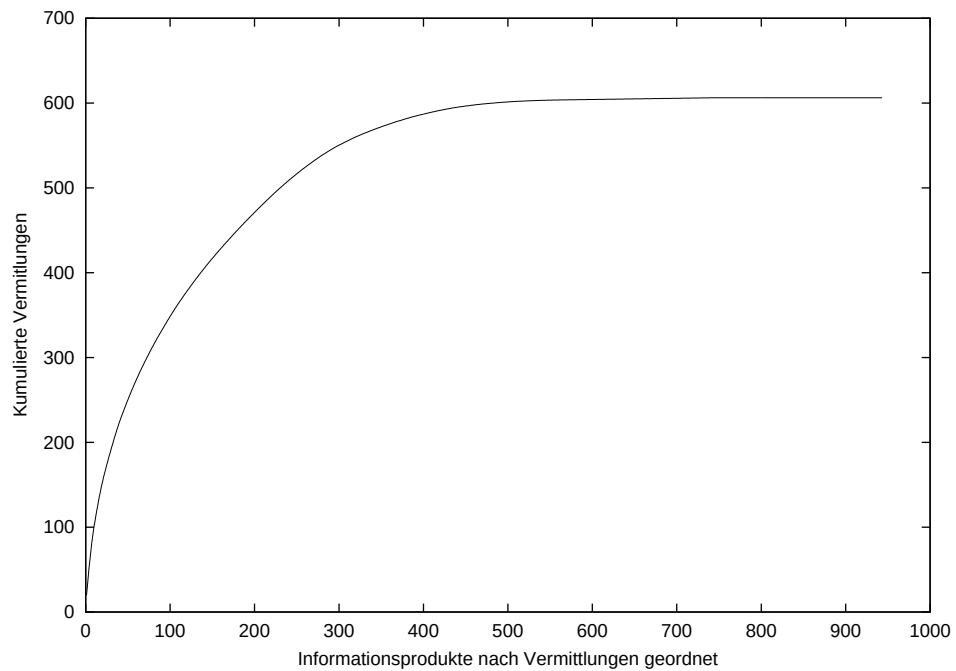


Abbildung A.6: Analyse der kumulierten Vermittlungen für Materials Collection

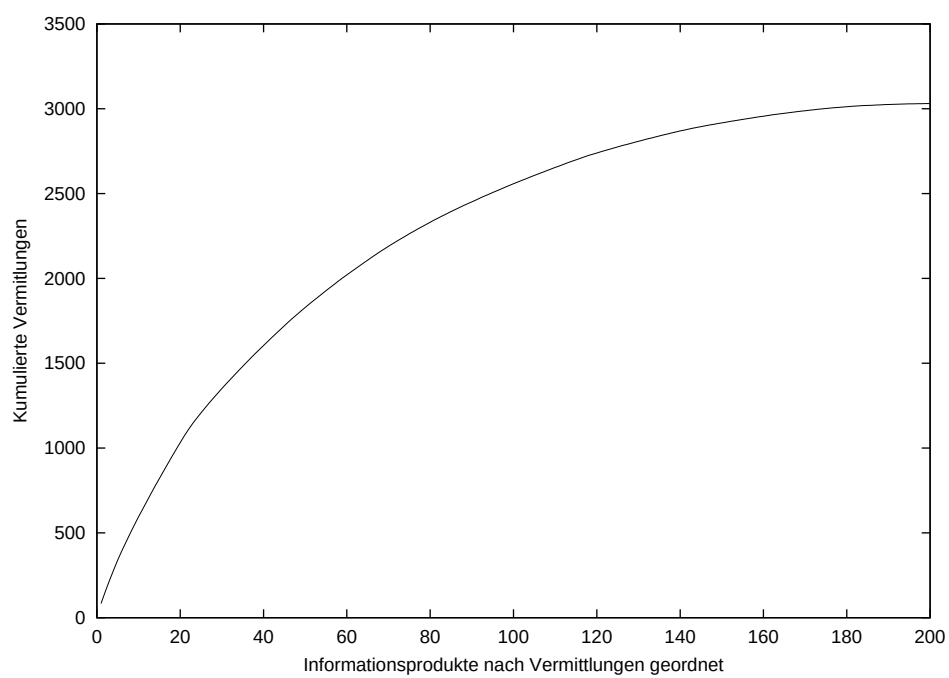


Abbildung A.7: Analyse der kumulierten Vermittlungen für Universitätsinformationen

Diese Analyse zeigt, welche Informationsgüter für viele Kunden interessant sind. Genau wie aus der ABC-Analyse im Lagermanagement, lassen sich daraus Schlüsse für den Grad der ökonomisch sinnvollen Kontrolle der Informationsgüter ziehen. Beispiele sind:

- Bei den Einträgen welcher Informationsgüter lohnen sich Werbeeinschaltungen, wie Bannerwerbung.
- Hilfe bei der Entscheidung, in welche Richtung sich das Sortiment des Brokers weiterentwickeln soll. Soll verstärkt auf Güter gesetzt werden, die ähnlich, wie die bereits in Klasse A vorhandenen, sind? Oder sollen Maßnahmen zur Verbesserung der Nutzung der Güter der Klasse C gesetzt werden?
- Abstufung des Servicegrads im Katalog des Brokers, um Kosten für die Aktualisierung zu sparen. Beispielsweise könnten Einträge für Güter der Klasse A täglich, Güter der Klasse B wöchentlich, und Güter der Klasse C nur monatlich aktualisiert werden. Bei einer Klassenverteilung zwischen von 20/20/60 ergibt das eine Reduktion der Anzahl der notwendigen Aktualisierungen von circa 75% und damit auch eine entsprechende Reduktion der dafür anfallenden Kosten.

A.3 Analyse der Sessionlängen

Es wurden die Sessions von 3 Monaten (Mai 2000 – Juli 2000) betrachtet. Sessions werden mithilfe von HTTP-Cookies als Sessiontoken bestimmt. Der Web-Browser des Benutzers bekommt beim ersten Seitenaufruf aus dem Broker-System ein Sessiontoken. Ab diesem Zeitpunkt markiert der Web-Server jeden weiteren *Request* (Seitenaufruf) durch diesen Web-Browser im Server-Log mit dem Sessiontoken. Die Benutzersession ist hier als Zeitraum vom ersten bis zum letzten Auftreten desselben Sessiontokens im Server-Log definiert. Sessiontokens sind an die Laufzeit des Web-Browsers gebunden. Wird der Browser beendet, wird auch das Sessiontoken gelöscht. Diese Methode wurde der Analyse von Sessions mithilfe von IP-Adressen vorgezogen, da an der Universität viele Studenten öffentliche PCs benutzen und für Studenten zuhause dynamisch IP-Adressen vergeben werden. Dadurch können durch IP-Adressen die Sessions verschiedener Studenten nicht unterschieden werden.

Im betrachteten Zeitraum wurden im Server-Log 697920 Requests (Hits) mitgeschrieben. Diese enthalten 23313 Sessions mit 49700 Vermittlungen. Damit beträgt die durchschnittliche Sessionlänge ca. 2,227 Vermittlungen. Aufgrund der verwendeten Technologie (HTTP-Cookies) konnten 6266 Vermittlungen (entspricht circa 2800 Sessions bei der vorher errechneten durchschnittlichen Sessionlänge) nicht zu Sessions zusammengefaßt werden. Diese können daher für die Auswertung der Sessionlängen nicht berücksichtigt werden.

Abbildungen A.8 und A.9 (sowie die dazugehörigen Daten in Tabellen A.10 und A.11) zeigen die absoluten Häufigkeiten der Sessionlängen gemessen als die Anzahl der Vermittlungen, beziehungsweise als Zeitdauer der Session. In Abbildung A.8 wurden nur Sessions mit mindestens einer Vermittlung betrachtet, 15 ist die Restklasse, die alle Sessions mit 15 und mehr Vermittlungen beinhaltet. Abbildung A.9 gibt die Anzahl der Sessions nach ihrer Dauer in Minuten an. Die Klassen sind abgeschlossene, ganze Minuten. 60 ist die Restklasse.

Die Häufigkeit der beobachteten Sessions nimmt mit der Sessionlänge, ausgedrückt in Anzahl der Vermittlungen als auch in Minuten, stark ab. Sessions mit nur einer Vermittlung sind sehr häufig. Dies kann viele verschiedene Gründe haben, beispielsweise:

- Der Benutzer war mit dem als erstes vermittelten Informationsgut zufrieden.
- Der Benutzer weiß bereits aus eigener Erfahrung, welches Informationsgut er verwenden will, und muß nicht erst mehrere vergleichen.

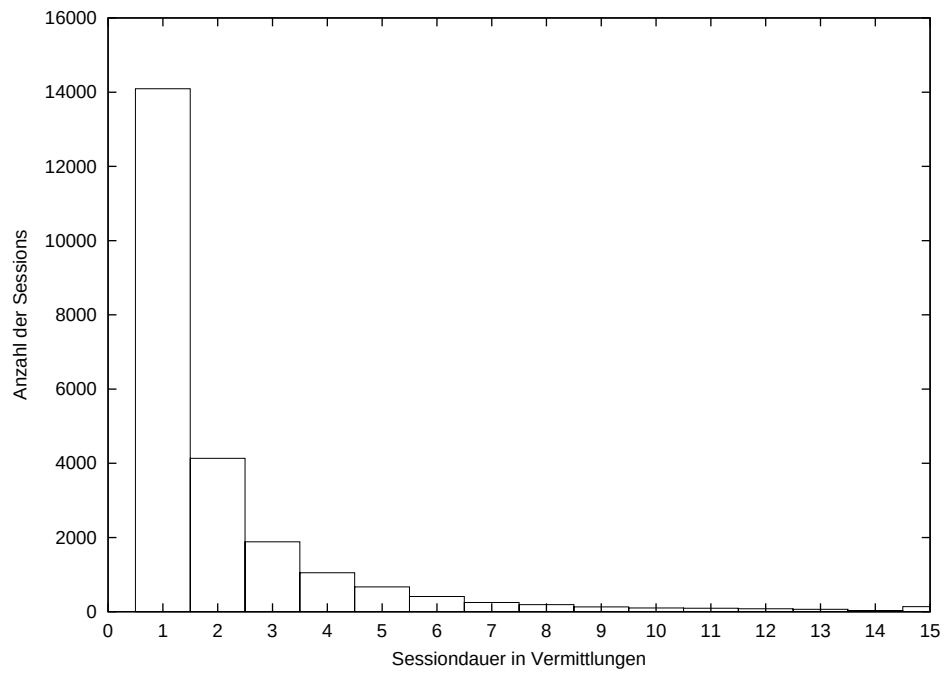


Abbildung A.8: Sessionlänge in Anzahl der Vermittlungen

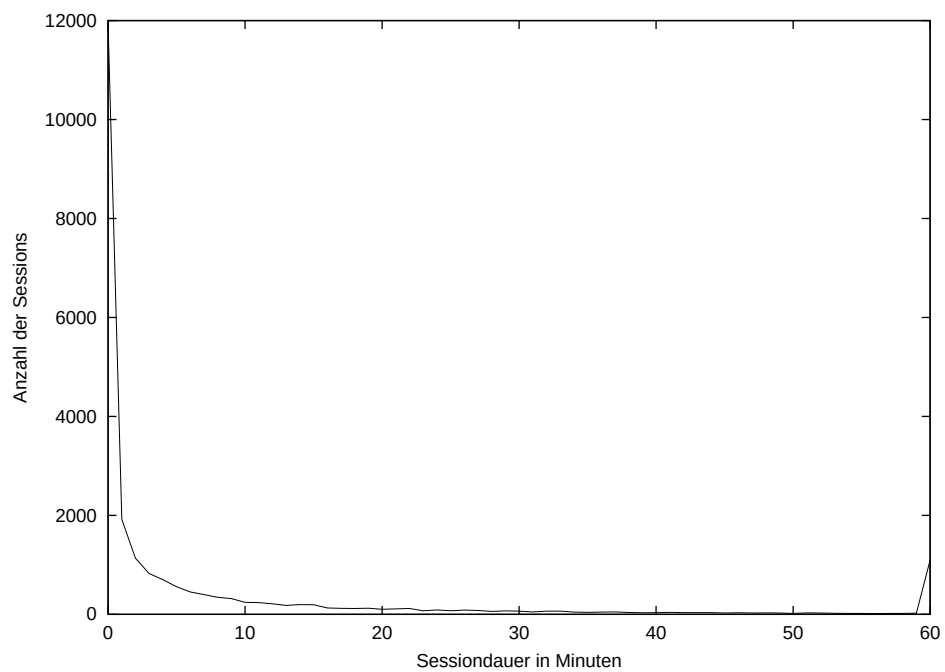


Abbildung A.9: Sessiondauer in ganzen Minuten

- Der Benutzer benötigt oft nur ein Informationsgut zur selben Zeit. Kommt er wieder, beginnt eine neue Session.
- Ein potentieller Konsument versucht das System zum ersten Mal, ist nach der ersten Vermittlung enttäuscht und verwendet deshalb das System nicht weiter.

Um diese Gründe zu analysieren können Benutzerbefragungen oder Personalisierung eingesetzt werden.

Dauer in Vermittlungen	Anzahl Sessions
1	14093
2	4133
3	1883
4	1051
5	670
6	408
7	249
8	192
9	130
10	101
11	95
12	78
13	63
14	32
Restklasse 15	135

Abbildung A.10: Sessionlänge in Anzahl der Vermittlungen

Dauer in Min.	Anzahl Sessions	Dauer in Min.	Anzahl Sessions
0	11917	31	49
1	1919	32	66
2	1135	33	65
3	826	34	47
4	700	35	41
5	559	36	45
6	453	37	49
7	400	38	39
8	345	39	32
9	317	40	34
10	243	41	40
11	236	42	35
12	214	43	34
13	181	44	34
14	196	45	28
15	193	46	31
16	130	47	26
17	120	48	30
18	119	49	28
19	125	50	19
20	103	51	29
21	111	52	28
22	119	53	21
23	72	54	20
24	89	55	19
25	72	56	16
26	86	57	20
27	79	58	19
28	58	59	25
29	70	Restklasse 60	1096
30	65		

Abbildung A.11: Sessionlänge in ganzen Minuten

Anhang B

Untersuchung des Code Reuse

Dieser Teil des Anhangs enthält die Daten zur Untersuchung der Wiederverwendung von Code des Grundsystems des Informationsbrokers für die Einrichtung eines neuen Katalogs des Brokers und zur Entwicklung einer neuen Applikation, der Digitalen Bibliothek.

Es handelt sich jeweils um eine Liste aller Dateien, die für das neue System verwendet wurden, wobei zu jeder Datei in der Liste folgende Angaben gemacht werden:

1. Eine fortlaufende Nummer für die Datei, um sich auf einzelne Dateien im Text beziehen zu können.
2. Der Dateiname.
3. Die verwendete Programmiersprache. Dabei wird unterschieden in:
 - PERL. In dieser Skriptsprache sind die meisten Teile des VU-Projekts implementiert [Wall *et al.*, 1996].
 - HTML. Für statische Web-Seiten und Vorlagen von dynamischen Seiten [Raggett *et al.*, 1999]. Gegenüber von Standard HTML-Code, sind diese Dateien etwas komplexer, da Befehle und Variablen (aus PERL) in die dynamischen Teile eingebettet sind.
4. Die Art der Wiederverwendung:
 - Black-Box. Bedeutet, daß diese Datei unverändert vom Grundsystem des Informationsbrokers übernommen wurde.
 - White-Box. Bedeutet, daß nur Anpassungen stattgefunden haben, die Struktur und Funktionalität (das Design) der Datei ist aber unverändert geblieben. Ein Beispiel ist die Änderung von Feldern in HTML-Formularen, um das Interface des Systems an eine andere Struktur von Metadaten anzupassen.

- Neu. Bezeichnet Dateien, die für das betrachtete Projekt neu implementiert werden mußten. Für diese Dateien mußte vor der Implementierung auch ein Design entwickelt werden. Beispielsweise kann dies eine neue Funktion des Informationsbrokers sein.

5. Die Anzahl der Zeilen der Datei (ohne Leerzeilen).
6. Die Größe der Datei gemessen in SLOC. Für HTML-Dateien, in die PERL-Code eingebettet ist, werden die Zeilen mit PERL-Code als SLOC gezählt. Die so ermittelten SLOC können damit für die Analyse einfach zu den SLOC des PERL-Codes addiert werden. Da reiner HTML-Code auch Aufwand darstellt und auch wiederverwendet werden kann, bedeutet diese Vorgehensweise, daß alle auf diesen Daten beruhenden Schätzungen konservativ sind, das heißt, das Wiederverwendungspotential wird unterschätzt. Das ungefähre Ausmaß dieser Unterschätzung kann aus dem Vergleich der Zeilen und der SLOC bei den HTML-Seiten ersehen werden.

Am Ende jeder Liste befindet sich die Summe der Zeilen, beziehungsweise SLOC, aufgeteilt nach Programmiersprache und Art der Wiederverwendung.

B.1 Liste der Dateien für die Einrichtung des Katalogs für Forschungsveranstaltungen

Nr.	Dateiname	Sprache	Reuse-Art	Zeilen	SLOC
Statischer HTML-Bereich					
1	admin.html	HTML	Black-Box	56	2
2	welcome.html	HTML	White-Box	182	9
Dynamischer HTML-Bereich					
3	admin.html	HTML	Black-Box	127	49
4	cats_change.html	HTML	Black-Box	64	13
5	feedback.html	HTML	Black-Box	55	12
6	feedback.line	HTML	Black-Box	15	8
7	footadmin.design	PERL	Black-Box	23	23
8	headadmin.design	PERL	Black-Box	20	20
9	library.html	HTML	Black-Box	70	14
10	ok.html	HTML	Black-Box	55	11

11	pwdnew.html	HTML	Black-Box	63	12
12	error.html	HTML	Black-Box	53	11
13	autoconfig.html	HTML	Black-Box	102	15
14	navigation.line	HTML	Black-Box	22	11
15	review.html	HTML	White-Box	177	21
16	library.line	HTML	White-Box	13	11
17	pwdnew.html	HTML	Black-Box	37	8
18	lib_stats.html	HTML	Black-Box	19	6
19	ok.html	HTML	Black-Box	18	5
20	error.html	HTML	Black-Box	22	5
21	warning.html	HTML	Black-Box	20	4
22	feedback_ok.html	HTML	Black-Box	17	3
23	new_ok.html	HTML	Black-Box	17	3
24	library.html	HTML	White-Box	22	7
25	library.line	HTML	White-Box	21	17
26	library_no.html	HTML	White-Box	32	8
27	feedback.html	HTML	White-Box	34	6
28	input.html	HTML	White-Box	134	7
Scripts					
29	stat	PERL	Black-Box	39	39
30	lib_stats	PERL	Black-Box	10	10
31	catscheck	PERL	Black-Box	60	56
32	catsget	PERL	Black-Box	32	32
33	catsput	PERL	Black-Box	37	37
34	catsrepair	PERL	Black-Box	41	39
35	catsshow	PERL	Black-Box	30	29
36	table	PERL	White-Box	113	110
37	doindex	PERL	White-Box	42	42
38	rank	PERL	White-Box	42	42
39	moveold	PERL	Neu	47	47
Module					
40	auto.pl	PERL	Black-Box	535	531
41	setaction.pl	PERL	Black-Box	35	35
42	index.pl	PERL	Black-Box	366	366
43	id.pl	PERL	Black-Box	87	85
44	statistics.pl	PERL	Black-Box	36	36
45	token.pl	PERL	Black-Box	56	56
46	Extractor.pm	PERL	Black-Box	233	226
47	changepage.pl	PERL	Black-Box	56	56
48	md5.pl	PERL	Black-Box	54	54
49	style.pl	PERL	Black-Box	95	95
50	makeglobal.pl	PERL	Black-Box	35	35
51	stuff.pm	PERL	Black-Box	386	380

52	httpIO.pm	PERL	Black-Box	230	229
53	fileIO.pm	PERL	Black-Box	139	133
CGI Programme					
54	global.pl	PERL	Black-Box	31	31
55	token.js	PERL	Black-Box	71	70
56	virlib	PERL	Black-Box	753	733
57	quickmed	PERL	Black-Box	66	66

Summe der SLOC nach Sprache und Reuse-Art:

Reuse-Art	Zeilen	SLOC
HTML_Black-Box	832	192
HTML_TOTAL	1447	278
HTML_White-Box	615	86
PERL_Black-Box	3556	3502
PERL_Neu	47	47
PERL_TOTAL	3800	3743
PERL_White-Box	197	194

B.2 Liste der Dateien für die Digitale Bibliothek

Nr.	Dateiname	Sprache	Reuse-Art	Zeilen	SLOC
Statischer HTML-Bereich					
1	admin.html	HTML	Black-Box	56	3
2	welcome.html	HTML	White-Box	106	4
Dynamischer HTML-Bereich					
3	admin.html	HTML	Black-Box	145	55
4	cats_change.html	HTML	Black-Box	65	14
5	ok.html	HTML	Black-Box	56	12
6	feedback.html	HTML	Black-Box	56	13
7	footadmin.design	PERL	Black-Box	23	23
8	headadmin.design	PERL	Black-Box	21	21
9	error.html	HTML	Black-Box	54	12
10	pwdnew.html	HTML	Black-Box	64	13
11	feedback.line	HTML	Black-Box	15	8
12	feedback_no.html	HTML	Black-Box	54	10

13	library_no.html	HTML	Black-Box	52	11
14	autoconfig.html	HTML	Black-Box	103	16
15	cats_repair.html	HTML	Black-Box	81	16
16	library.html	HTML	White-Box	64	15
17	library.line	HTML	White-Box	33	18
18	review_Diplomarbeit	HTML	White-Box	379	29
19	review_Dissertation	HTML	White-Box	364	30
20	review_Proseminar.	HTML	White-Box	166	30
21	review_Seminar.htm	HTML	White-Box	166	30
22	USER_ADMIN.html	HTML	Neu	50	11
23	ok.html	HTML	Black-Box	17	5
24	suboknew.html	HTML	Black-Box	20	3
25	warning.html	HTML	Black-Box	20	4
26	error.html	HTML	Black-Box	22	5
27	feedback.html	HTML	Black-Box	34	6
28	lib_stats.html	HTML	Black-Box	26	7
29	library_no.html	HTML	Black-Box	29	5
30	feedback_ok.html	HTML	Black-Box	17	3
31	new_ok.html	HTML	Black-Box	20	3
32	input_Diplomarbeit	HTML	White-Box	287	9
33	library.html	HTML	White-Box	55	11
34	library.line	HTML	White-Box	50	21
35	entry_Diplomarbeit	HTML	White-Box	40	14
36	entry_Seminar.html	HTML	White-Box	48	17
37	entry_Dissertation	HTML	White-Box	42	15
38	entry_Proseminar.h	HTML	White-Box	46	16
39	input_Diplomarbeit	HTML	White-Box	279	9
40	input_Dissertation	HTML	White-Box	283	9
41	input_Proseminar.h	HTML	White-Box	73	9
42	input_Seminar.html	HTML	White-Box	75	9
43	vis_bu_third.jhtml	HTML	Neu	44	8
44	diglib_upload.html	HTML	Neu	31	4
45	input_working_grou	HTML	Neu	33	9
46	DC_TYPE_select.htm	HTML	Neu	47	8
47	diglib_change_page	HTML	Neu	46	7
48	upload.html	HTML	Neu	21	4
49	vis_bo_third.jhtml	HTML	Neu	59	13
50	diglib_upload_page	HTML	Neu	87	8
51	vis_bo_download_on	HTML	Neu	68	12
52	vis_bo_standard.jh	HTML	Neu	12	3
53	vis_bu_download_on	HTML	Neu	38	6
54	vis_vu_third.jhtml	HTML	Neu	19	7
55	vis_bo_edit_one.jh	HTML	Neu	47	15
56	vis_bu_edit_one.jh	HTML	Neu	26	8
57	vis_bu_standard.jh	HTML	Neu	22	10

58	NO_PERM.html	HTML	Neu	18	4
59	diglib_change_url_	HTML	Neu	38	6
Scripts					
60	stat	PERL	Black-Box	81	81
61	table	PERL	Black-Box	166	162
62	catscheck	PERL	Black-Box	60	56
63	catsget	PERL	Black-Box	31	31
64	catsput	PERL	Black-Box	49	48
65	catsrepair	PERL	Black-Box	41	39
66	catsshow	PERL	Black-Box	30	29
67	rank	PERL	Black-Box	42	42
68	review	PERL	White-Box	54	54
69	showentry	PERL	White-Box	19	19
70	changeit	PERL	White-Box	38	37
71	killlist	PERL	White-Box	27	27
72	DC_TYPE_select2	PERL	Neu	50	40
73	DC_TYPE_select1	PERL	Neu	48	39
74	diglib_change	PERL	Neu	70	55
75	diglib_newentry	PERL	Neu	35	30
76	diglib_new1	PERL	Neu	81	58
77	diglib_show_upload	PERL	Neu	59	47
78	show_bucket	PERL	Neu	59	54
79	diglib_upload_main	PERL	Neu	43	33
80	diglib_view	PERL	Neu	84	69
81	diglib_add_object	PERL	Neu	139	102
82	diglib_delete	PERL	Neu	79	55
83	diglib_view_credit	PERL	Neu	166	149
84	diglib_show_change	PERL	Neu	77	58
85	diglib_edit_change	PERL	Neu	49	42
86	diglib_download	PERL	Neu	73	60
87	diglib_edit_mainlo	PERL	Neu	34	31
88	diglib_control_edi	PERL	Neu	49	45
89	diglib_securityinf	PERL	Neu	81	66
90	diglib_edit_add_ob	PERL	Neu	113	100
91	diglib_edit_delete	PERL	Neu	56	46
92	diglib_edit_show_c	PERL	Neu	44	39
93	diglib_edit_change	PERL	Neu	50	43
94	diglib_securityinf	PERL	Neu	62	51
95	get_thumbnail	PERL	Neu	34	29
96	loginout	PERL	Neu	40	40
97	diglib_access_erro	PERL	Neu	20	20
98	loginout_mod	PERL	Neu	42	42
99	diglib_credit_view	PERL	Neu	186	178
100	diglib_url	PERL	Neu	80	58

101	diglib_changeurl	PERL	Neu	82	61
102	useradmin	PERL	Neu	55	55
103	changeuserperm	PERL	Neu	34	34
104	permissionadmin	PERL	Neu	44	44
105	deleteperm	PERL	Neu	24	24
106	createperm	PERL	Neu	24	24
107	oneuseradmin	PERL	Neu	58	58
108	changeoneuserperm	PERL	Neu	34	34
109	editbucket	PERL	Neu	27	27
110	diglib_editbucket	PERL	Neu	42	33
111	diglib_edit_url	PERL	Neu	44	39
112	show_diglib_link	PERL	Neu	43	38

Module

113	auto.pl	PERL	Black-Box	525	521
114	index.pl	PERL	Black-Box	372	370
115	stuff.pm	PERL	Black-Box	387	381
116	fileIO.pm	PERL	Black-Box	117	116
117	id.pl	PERL	Black-Box	86	84
118	statistics.pl	PERL	Black-Box	85	85
119	token.pl	PERL	Black-Box	55	55
120	Extractor.pm	PERL	Black-Box	233	226
121	changepage.pl	PERL	Black-Box	56	56
122	md5.pl	PERL	Black-Box	54	54
123	style.pl	PERL	Black-Box	91	91
124	makeglobal.pl	PERL	Black-Box	35	35
125	httpIO.pm	PERL	Black-Box	227	226
126	fileIO.pm	PERL	Black-Box	123	121
127	Bucket.pm	PERL	Neu	230	229
128	BucketObject.pm	PERL	Neu	242	234
129	BucketOrganizer.pm	PERL	Neu	166	159
130	BucketVisualizer.p	PERL	Neu	137	122
131	DLError.pm	PERL	Neu	22	22
132	BucketIDTranslator	PERL	Neu	51	51

CGI Programme

133	virlib	PERL	Black-Box	754	734
134	global.pl	PERL	White-Box	31	31
135	diglib	PERL	White-Box	136	132

Summe der SLOC nach Sprache und Reuse-Art:

Reuse-Art | Zeilen | SLOC

HTML_Black-Box		1006	224
HTML_Neu		706	143
HTML_TOTAL		4268	662
HTML_White-Box		2556	295
PERL_Black-Box		3744	3687
PERL_Neu		3362	2967
PERL_TOTAL		7411	6954
PERL_White-Box		305	300

Abbildungsverzeichnis

1.1	Traditioneller Lebenszyklus: (a) nach Davis; (b) grobe Einteilung in Analyse, Design und Implementierung; (c) Ergebnisse der Phasen	9
1.2	Objekt-orientierter Lebenszyklus nach Booch	11
1.3	Transformation zwischen Analyse- und Designmodell	18
2.1	Das Zusammenspiel von Problem, Kontext und Lösung für das Pattern: <i>University as a Marketplace</i>	25
3.1	Einsatzbereich von Design Patterns	28
3.2	Vergleich zwischen Patterns (a) und AntiPatterns (b)	35
4.1	Einsatzbereich von Analyse Patterns	45
4.2	Die Verwendung der einzelnen Abschnitte von Analyse Patterns	52
4.3	Business Process Improvement: Rückwirkung von Analyse Patterns auf das reale Problem	57
5.1	Beispiel einer einfachen Pinwand zur Kommunikation innerhalb einer Arbeitsgruppe	60
5.2	Klassendiagramm der Analyse einer einfachen Pinwand	61
5.3	Klassendiagramm des Designs einer einfachen Pinwand	63
5.4	Beispiel eines Formulars für strukturierte Nachrichten	65
5.5	Klassendiagramm einer Hierarchie von polymorphen Nachrichtentypen	67
5.6	Klassendiagramm des Designs der Pinwand für strukturierte Nachrichten	69
5.7	Beispiel eines Inhaltsverzeichnisses für verteilte Kursunterlagen in einem Universitätsnetzwerk	71
5.8	Klassendiagramm der Analyse einer Virtuellen Bibliothek	72
5.9	Kollaborationsdiagramm für die Zusammenarbeit von zwei Agenten zur Unterstützung einer virtuellen Bibliothek	77

8.1	Ein Komplementärgut in einem Netzwerk mit Sterntopologie	96
8.2	Mögliche zusammengesetzte Güter aus zwei Komplementärgütern	97
8.3	Use Case Diagramm: Typische Rollen des Personals und der Studenten in Lehre und Forschung	101
8.4	Inhalte des VU Informationsbroker	102
8.5	Metadaten einer Lehrveranstaltung	107
8.6	Startseite des Produktkatalogs für Lehrveranstaltungen . . .	109
8.7	Ergebnis einer Abfrage des Katalogs	111
8.8	Sicht des Benutzers auf die Metadaten einer Lehrveranstaltung	112
9.1	Struktur des Informationsbrokers der VU mit aktiven Agenten	126
9.2	Beispiele für Klassifikationsregeln zur Erkennung des Landes aus dem URL	130
9.3	Von manueller zu automatischer Klassifikation	131
9.4	Ersetzungsregeln zur automatischen Richtigstellung von typischen Änderungen von URLs	134
9.5	Suchergebnis in Schlagzeilen von verteilten Informationsdiensten, extrahiert durch einen spezialisierten Beobachtungsagenten	137
9.6	Verwendung von Recommendations im Benutzerinterface . .	145
9.7	Beispiel für die automatische Zusammenstellung von empfohlenen Materials für die Lehrveranstaltung "Einführung in das objektorientierte Programmieren mit C++"	148
10.1	Klassendiagramm der Digitalen Bibliothek	156
10.2	Anzahl der Informationsgüter im VU-Informationsbrokers . .	161
10.3	Vermittlungen des VU-Informationsbrokers pro Tag	163
10.4	Trend der Vermittlungen des VU-Informationsbrokers	163
A.1	Analyse der Vermittlungen für Living Lectures	171
A.2	Analyse der kumulierten Vermittlungen für Living Lectures mit der Einteilung in die Klassen A, B und C	171
A.3	Anteil der Informationsgüter der Klassen A, B und C an der Gesamtzahl der Vermittlungen	172
A.4	Analyse der kumulierten Vermittlungen für Tools & Manuals	172
A.5	Analyse der kumulierten Vermittlungen für VU-Bibliothek . .	173
A.6	Analyse der kumulierten Vermittlungen für Materials Collection	173

A.7 Analyse der kumulierten Vermittlungen für Universitätsin- formationen	174
A.8 Sessionlänge in Anzahl der Vermittlungen	177
A.9 Sessiondauer in ganzen Minuten	177
A.10 Sessionlänge in Anzahl der Vermittlungen	179
A.11 Sessionlänge in ganzen Minuten	180

Literaturverzeichnis

- [ACM, 1998] ACM. *The ACM Computing Classification System (CCS)*. Association for Computing Machinery, New York, NY, 1998. <http://www.acm.org/class/>.
- [Alexander *et al.*, 1977] Christopher Alexander, Sara Ishikawa, und Murray Silverstein. *A Pattern Language*, Band 2 in *Center for Environmental Structure Series*. Oxford University Press, New York, NY, 1977.
- [Alexander, 1979] Christopher Alexander. *The Timeless Way of Building*, Band 1 in *Center for Environmental Structure Series*. Oxford University Press, New York, NY, 1979.
- [Ali *et al.*, 1982] Akif Ali, Timo Kunnas, und Timo Laurmaa, Hg. *FinnAPL Idiom Library*. Finnish APL Association (FinnAPL), FinnAPL P.O. Box 1005, SF-00101 Helsinki 10, Finland, 2. Auflage, 1982. ISBN 951-99397-9-2.
- [Allison, 1998] Paul D. Allison. *Survival Analysis using the SAS System: A Practical Guide*. SAS Institute, North Carolina, 1998.
- [Bentley *et al.*, 1997] R. Bentley, W. Appelt, U. Busbach, E. Hinrichs, D. Kerr, S. Sikkil, J. Trevor, und G. Woetzel. Basic Support for Cooperative Work on the World Wide Web. *International Journal of Human-Computer Studies*, 46(6):827–846, Juni 1997.
- [Berners-Lee *et al.*, 1994] T. Berners-Lee, L. Masinter, und M. McCall. Uniform Resource Locators (URL). Request for Comments RFC 1738, Network Working Group, W3C, Dezember 1994. <http://www.w3.org/Addressing/rfc1738.txt>.
- [Boehm *et al.*, 2000] Barry W. Boehm, Chris Abts, A. Winsor Brown, Sunita Chulani, Bradford K. Clark, Ellis Horowitz, Ray Madachy, Donald J. Reifer, und Bert Steece. *Software Cost Estimation with COCOMO II*. Prentice Hall PTR, Upper Saddle River, NJ, 2000.

- [Boehm, 1981] Barry W. Boehm. *Software Engineering Economics*. Prentice-Hall, Englewood Cliffs, NJ, 1981.
- [Boehm, 1988] Barry W. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61–72, 1988.
- [Booch, 1994] Grady Booch. *Object-Oriented Analysis and Design with Applications*. Benjamin/Cummings, Redwood City, CA, 2. Auflage, 1994.
- [Brewington und Cybenko, 2000] Brian E. Brewington und George Cybenko. Keeping up with the changing web. *IEEE Computer*, 33(5):52–58, Mai 2000.
- [Brickley und Guha, 2000] Dan Brickley und R.V. Guha. Resource Description Framework (RDF) Schema Specification 1.0. W3C Candidate Recommendation, W3C, März 2000. <http://www.w3.org/TR/rdf-schema/>.
- [Brown *et al.*, 1998] William J. Brown, Raphael C. Malveau, Hays W. “Skip” McCormick III, und Thomas J. Mowbray. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley, New York, NY, 1998.
- [Brown *et al.*, 1999] William J. Brown, Hays W. “Skip” McCormick III, und Scott W. Thomas. *AntiPatterns and Patterns in Software Configuration Management*. John Wiley, New York, NY, 1999.
- [Budde *et al.*, 1992] Reinhard Budde, Karlheinz Kautz, Karin Kuhlenskamp, und Heinz Züllinghoven. *Prototyping: An Approach to Evolutionary System Development*. Springer-Verlag, Berlin, Germany, 1992.
- [Buschmann *et al.*, 1996] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, und Michael Stal. *Pattern-Oriented Software Architecture, A System of Patterns*. John Wiley & Sons Ltd, Chichester, England, 1996.
- [Buschmann und Meunier, 1995] Frank Buschmann und Regine Meunier. A system of patterns. In Coplien und Schmidt [1995], Seiten 325–344.
- [Cline und Girou, 2000] Marshall Cline und Mike Girou. Enduring business themes. *Communications of the ACM*, 43(5):101–106, Mai 2000.
- [Cline, 1996] Marshall P. Cline. The pros and cons of adopting and applying design patterns in the real world. *Communications of the ACM*, 39(10):47–49, Oktober 1996.

- [Coad *et al.*, 1997] Peter Coad, David North, und Mark Maryfield. *Object Models: Strategies, Patterns, and Application*. Yourdon Press, New Jersey, NJ, 2. Auflage, 1997.
- [Coad und Yourdon, 1991] Peter Coad und Edward Yourdon. *Object-Oriented Design*. Prentice-Hall, Englewood Cliffs, NJ, 1991.
- [Coase, 1937] Ronald H. Coase. The nature of the firm. *Economica*, (4):386–405, 1937.
- [Coker *et al.*, 1970] Jerry Coker, Jimmy Casalw, Gary Campbell, und Jerry Greene. *Patterns for Jazz*. Studio Publications Recordings, Miami, FL, 1970.
- [Coplien und Schmidt, 1995] James O. Coplien und Douglas C. Schmidt, Hg. *Pattern Languages of Program Design*. Addison–Wesley, Reading, MA, 1995.
- [Coplien, 1992] James O. Coplien. *Advanced C++ Programming Styles and Idioms*. Software Patterns Series. Addison-Wesley, Reading, MA, 1992.
- [Coplien, 1995] James O. Coplien. A generative development-process pattern language. In Coplien and Schmidt [1995].
- [Coplien, 2000] James Coplien. C++ idioms. In Harrison *et al.* [2000], Seiten 167–198.
- [Cunningham, 1995] Ward Cunningham. The checks pattern language of information integrity. In Coplien and Schmidt [1995], Seiten 145–156.
- [Dalheimer, 1999] Matthias Kalle Dalheimer. *Programming With Qt*. O'Reilly & Associates, 1999.
- [Davenport, 1993] Thomas H. Davenport. *Process Innovation: Reengineering Work through Information Technology*. Harvard Business School Press, Boston, MA, 1993.
- [Davis, 1983] William S. Davis. *System Analysis and Design*. Addison-Wesley, Reading, MA, 1983.
- [de Champeaux *et al.*, 1992] Dennis de Champeaux, Doug Lea, und Penelope Faure. The process of object-oriented design. *ACM SIGPLAN Notices*, 27(10):45–62, Oktober 1992.
- [Dearing, 1997] Ron Dearing. Dearing report. Report, The National Committee of Inquiry into Higher Education (NCIHE), PO Box 237, Hayes, Middlesex UB3 1HQ, UK, 1997. <http://www.leeds.ac.uk/educol/ncihe/>.

- [DeMarco, 1979] Tom DeMarco. *Structured Analysis and System Specification*. Prentice-Hall, Englewood Cliffs, NJ, 1979.
- [Economides, 1996] Nicholas Economides. The economics of networks. *International Journal of Industrial Organization*, 14(6):673–700, 1996.
- [Ellingsen, 2000] Lars Ellingsen. Guestbook system: Guestserver. Home Page, 2000. <http://www.stud.ntnu.no/~larsell/guestbook/>.
- [Fayad und Cline, 1996] Mohamed Fayad und Marshall P. Cline. Aspects of software adaptability. *Communications of the ACM*, 39(10):58–59, Oktober 1996.
- [Fernandez und Yuan, 1999] Eduardo B. Fernandez und Xiaohong Yuan. An analysis pattern for reservation and use of reusable entities. In Dragos Manolescu und Bobby Woolf, Hg., *Online Proceedings of the PLoP1999 Conference*, Urbana, IL, 1999. <http://st-www.cs.uiuc.edu/~plop/plop99/proceedings/>.
- [Fernandez, 1998] Eduardo B. Fernandez. Building systems using analysis patterns. In *Proceedings of the third international Workshop on Software Architecture*, Seiten 37–40, Orlando, FL, 1998. ACM.
- [Fielding *et al.*, 1997] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, und T. Berners-Lee. Hypertext Transfer Protocol – HTTP/1.1. Request for Comments RFC 2068, Network Working Group, W3C, Jänner 1997. <http://www.w3.org/Protocols/rfc2068/rfc2068>.
- [Foote und Opdyke, 1995] Brian Foote und William F. Opdyke. Lifecycle and refactoring: Patterns that support evolution and reuse. In Coplien und Schmidt [1995], Seiten 239–258.
- [Fowler, 1997] Martin Fowler. *Analysis Patterns: Reusable Object Models*. Object Technology Series. Addison-Wesley, Reading, MA, 1997.
- [Gaertner und Thirion, 1999] Nathalie Gaertner und Bernard Thirion. Grafcet: an analysis pattern for event driven real-time systems. In Dragos Manolescu und Bobby Woolf, Hg., *Online Proceedings of the PLoP1999 Conference*, Urbana, IL, 1999. <http://st-www.cs.uiuc.edu/~plop/plop99/proceedings/>.
- [Gamma *et al.*, 1993] Erich Gamma, Richard Helm, Ralph Johnson, und John Vlissides. Design patterns: Abstraction and reuse in object-oriented designs. In O. M. Nierstrasz, Hg., *ECOOP'93: Object-Oriented Programming - Proc. of the 7th European Conference*, Seiten 406–431, Berlin, Heidelberg, 1993. Springer Verlag.

- [Gamma *et al.*, 1995] Erich Gamma, Richard Helm, Ralph Johnson, und John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Addison-Wesley Publishing Company, New York, NY, 1995.
- [Gardner *et al.*, 1998] Karen M. Gardner, Alexander Rush, Michael K. Crist, Robert Konitzer, und Bobbin Teegarden. *Cognitive Patterns: Problem-Solving Frameworks for Object Technology*. Cambridge University Press, Cambridge, UK, 1998.
- [Gault und Millau, 2000] Henri Gault und Christian Millau. *Gault Millau Guide Österreich 2001*. Heyne, München, 2000.
- [Genesereth, 1998] Michael R. Genesereth. Knowledge Interchange Format. Draft proposed American National Standard (dpANS) NCITS.T2/98-004, National Committee for Information Technology Standards, 1998. <http://logic.stanford.edu/kif/dpans.html>.
- [Geyer-Schulz *et al.*, 1999] Andreas Geyer-Schulz, Michael Hahsler, und Georg Schneider. The virtual university and its embedded agents. *ÖGAI Journal*, 18(1):14–19, 1999.
- [Geyer-Schulz und Hahsler, 2000] Andreas Geyer-Schulz und Michael Hahsler. Automatic labelling of references for information systems. In Reinhold Decker und Wolfgang Gaul, Hg., *Classification and Information Processing at the Turn of the Millennium*, Studies in Classification, Data Analysis, and Knowledge Organization, Seiten 451–459. Springer Verlag, Heidelberg, Germany, 2000.
- [Gillies, 1992] Alan C. Gillies. *Software Quality: Theory and Management*. Chapman & Hall, London, 1992.
- [Goldfedder und Rising, 1996] Brandon Goldfedder und Linda Rising. A training experience with patterns. *Communications of the ACM*, 39(10):60–64, Oktober 1996.
- [Gross und Harris, 1985] Donald Gross und Carl M. Harris. *Fundamentals of Queueing Theory*. John Wiley & Sons, New York, NY, 2. Auflage, 1985.
- [Grossman und Frieder, 1998] David A. Grossman und Ophir Frieder. *Information Retrieval: Algorithms and Heuristics*. Kluwer Academic Publishers, Boston, MA, 1998.
- [Hahsler, 1997] Michael Hahsler. Software Patterns: Pinwände. Diplomarbeit, Wirtschaftsuniversität Wien, Augasse 2-6, A 1090 Wien, Österreich, November 1997.

- [Harrington, 1991] H. James Harrington. *Business Process Improvement: The Breakthrough Strategy for Total Quality, Productivity and Competitiveness*. McGraw-Hill, New York, NY, 1991.
- [Harrison *et al.*, 2000] Neil Harrison, Brian Foote, und Hans Rohnert, Hg. *Pattern Languages of Program Design 4*. Addison-Wesley, Reading, MA, 2000.
- [Hay, 1996] David C. Hay. *Data Model Patterns: Conventions of Thought*. Dorset House Publishing, New York, NY, 1996.
- [Hazemi *et al.*, 1998] Rezy Hazemi, Stephen Hailes, und Steve Wilbur, Hg. *The Digital University: Reinventing the Academy*. Computer Supported Cooperative Work. Springer, London, UK, 1998.
- [Henderson-Sellers und Edwards, 1990] Brian Henderson-Sellers und Julian M. Edwards. The object-oriented systems life cycle. *Communications of the ACM*, 33(9):143–159, September 1990.
- [Hodgins, 2000] Wayne Hodgins. Draft Standard for Learning Object Metadata. W3C Candidate Recommendation IEEE P1484.12/D4.0, IEEE Learning Technology Standards Committee (LTSC), 445 Hoes Lane, P.O. Box 1331, Piscataway, NJ 08855-1331, USA, Februar 2000. <http://ltsc.ieee.org/doc/wg12/LOM.WD4.htm>.
- [ISO, 1997] ISO. Codes for the Representation of Names of Countries and their Subdivisions. ISO 3166, International Organization for Standardization, Geneva, Switzerland, 1997. <http://logic.stanford.edu/kif/dpans.html>.
- [Jacobson *et al.*, 1999a] Ivar Jacobson, Grady Booch, und James Rumbaugh. The unified process. *IEEE Software*, 16(3):96–102, Mai / Juni 1999.
- [Jacobson *et al.*, 1999b] Ivar Jacobson, James Rumbaugh, und Grady Booch. *The Unified Software Development Process*. Object Technology Series. Addison-Wesley, Reading, MA, 1999.
- [Jacobson, 1992] Ivar Jacobson. *Object Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Wokingham, UK, 1992.
- [Jain *et al.*, 1999] A.K. Jain, M.N. Murty, und P.J. Flynn. Data clustering: A review. *ACM Computing Surveys*, 31(3):264–323, September 1999.
- [Johnson, 1992] Ralph E. Johnson. Documenting frameworks using patterns. *ACM SIGPLAN Notices*, 27(10):63–76, Oktober 1992. *OOPSLA '92 Proceedings*.

- [Johnson, 1997] Ralph E. Johnson. Frameworks=(components+patterns): How frameworks compare to other object-oriented reuse techniques. *Communications of the ACM*, 40(10):39–42, Oktober 1997.
- [Kaindl, 1999] Herman Kaindl. Difficulties in the transformation from oo analysis to design. *IEEE Software*, 16(5):94–102, September / Oktober 1999.
- [Kantor und Lapsley, 1986] Brian Kantor und Phil Lapsley. Network News Transfer Protocol. Request for Comments 977, Network Working Group, Februar 1986. <http://www.w3.org/Protocols/rfc977/rfc977>.
- [Katz und Shapiro, 1985] Michael L. Katz und Carl Shapiro. Network externalities, competition, and compatibility. *American Economic Review*, 75(3):424–440, Juni 1985.
- [Kerth, 1995] Norman L. Kerth. Caterpillar’s fate: A pattern language for the transformation from analysis to design. In Coplien and Schmidt [1995], Seiten 293–320.
- [Lancaster, 1990] Tony Lancaster. *The Econometric Analysis of Transition Data*. Cambridge University Press, New York, NY, 1990.
- [Lewis *et al.*, 1991] John A. Lewis, Sallie M. Henry, und Robert S. Kafura, Dennis G. Kafand Schulman. An empirical study of the object-oriented paradigm and software reuse. In Andreas Paepcke, Hg., *Conference proceedings on Object-oriented programming systems, languages, and applications*, Seiten 184–196. ACM Press, November 1991.
- [Malone *et al.*, 1987] Thomas W. Malone, Kenneth R. Grant, Franklyn A. Turbak, Stephen A. Brobst, und Michael D. Cohen. Intelligent Information-Sharing Systems. *Communications of the ACM*, 30(5):390–402, Mai 1987.
- [Manber *et al.*, 2000] Udi Manber, Ash Patel, und John Robinson. Experience with personalization on yahoo! *Communications of the ACM*, 43(8):35–39, August 2000.
- [Margolis, 1998] Michael Margolis. Brave new university. *firstmonday*, 3(5), Mai 1998. http://www.firstmonday.dk/issues/issue3_5/margolis/.
- [Martin *et al.*, 1998] Robert C. Martin, Dirk Riehle, und Frank Buschmann, Hg. *Pattern Languages of Program Design 3*. Addison–Wesley, Reading, MA, 1998.

- [Milgrom und Roberts, 1992] Paul Milgrom und John Roberts. *Economics, organization and management*. Prentice-Hall, Englewood Cliffs, NJ, 1992.
- [Monarchi und Puhr, 1992] David E. Monarchi und Gretchen I. Puhr. A research typology for object-oriented analysis and design. *Communications of the ACM*, 35(9):35–47, September 1992.
- [Noble, 1998] David F. Noble. Digital diploma mills: The automation of higher education. *firstmonday*, 3(1), Jänner 1998. http://www.firstmonday.dk/issues/issue3_1/noble/.
- [NTIS, 1997] NTIS. *North American Industry Classification System (NAICS) - United States*. National Technical Information Service, Springfield, 1997. ISBN-0-934213-57-7.
- [Oki et al., 1992] B. M. Oki, D. Goldberg, D. Nichols, und D. Terry. Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 35(12):61–70, 1992.
- [OMG, 1999] OMG. Unified modeling language. Specification v1.3, Object Management Group, Juni 1999. <http://www.omg.org/technology/uml/>.
- [Opdyke, 1992] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. Doctoral thesis, Department of Computer Science, University of Illinois at Urbana-Champaign, Digital Computer Laboratory, 1304 West Springfield Avenue, Urbana, IL 61801, Juni 1992.
- [Peters, 1981] Lawrence J. Peters. *Software Design: Methods and Techniques*. Yourdon Press, New York, NY, 1981.
- [PLoP, 2000] PLoP. The pattern languages of programs (plop) conference. Home Page, Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL, USA, 2000. <http://st-www.cs.uiuc.edu/~plop/>.
- [Pree, 1995] Wolfgang Pree. *Design Patterns for Object-Oriented Software Development*. ACM Press Books. Addison-Wesley Publishing Company, Wokingham, UK, 1995.
- [Press, 1992] Larry Press. Collective Dynabases. *Communications of the ACM*, 35(6):26–32, Juni 1992.
- [Prosise, 1999] Jeff Prosise. *Programming Windows With MFC*. Microsoft Press, 2. Auflage, 1999.

- [Raggett *et al.*, 1999] Dave Raggett, Arnaud Le Hors, und Ian Jacobs. HTML 4.01 Specification. W3C Recommendation, W3C, Dezember 1999. <http://www.w3.org/TR/html401/>.
- [Resnick und Varian, 1997] Paul Resnick und Hal R. Varian. Recommender systems. *Communications of the ACM*, 40(3):56–58, März 1997.
- [Rivest, 1992] R. Rivest. The MD5 Message-Digest Algorithm. Request for Comments RFC 1321, Network Working Group, W3C, April 1992. <http://theory.lcs.mit.edu/~rivest/Rivest-MD5.txt>.
- [Rogers, 1995] Everett M. Rogers. *Diffusion of Innovations*. The Free Press, New York, NY, 4. Auflage, 1995.
- [Royce, 1970] Walker W. Royce. Managing the development of large software systems: concepts and techniques. *Proc. IEEE WESTCON, Los Angeles*, Seiten 1–9, August 1970. Reprinted in *Proceedings of the Ninth International Conference on Software Engineering*, March 1987, pp. 328–338.
- [Rumbaugh *et al.*, 1991] James Rumbaugh, Michael Blaha, William Premerlani, Frederick Eddy, und William Lorensen. *Object-Oriented Modeling and Design*. Prentice-Hall International Editions, New York, NY, 1991.
- [Russell und Norvig, 1995] Stuart Russell und Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice-Hall, Upper Saddle River, NJ, 1995.
- [Salton und MacGill, 1983] Gerard Salton und Michael J. MacGill. *Introduction to Modern Information Retrieval*. McGraw-Hill, New York, NY, 1983.
- [Schumann und Hagenhoff, 1998] Matthias Schumann und Svenja Hagenhoff. Virtuelle Universität. *Wirtschaftsinformatik*, 40(2):150–152, April 1998.
- [Shapiro und Varian, 1999] Carl Shapiro und Hal R. Varian. *Information Rules: A Strategic Guide to the Network Economy*. Harvard Business School Press, Boston, MA, 1999.
- [Sommerlad, 1996] Peter Sommerlad. Command processor. In Vlissides *et al.* [1996], Seiten 63–74.
- [Sommerville, 1995] Ian Sommerville. *Software Engineering*. International Computer Sciences Series. Addison-Wesley, Harlow, UK, 5. Auflage, 1995.

- [Sutton, 1992] Richard S. Sutton. Introduction: The challenge of reinforcement learning. *Machine Learning*, 8(3/4):225–227, 1992.
- [Taha, 1982] Hamdy A. Taha. *Operations Research. An Introduction*. Macmillian Publishing Co., New York, NY, 3. Auflage, 1982.
- [Tkach und Puttick, 1994] Daniel Tkach und Richard Puttick. *Object Technology in Application Development*. Benjamin/Cummings, Redwood City, CA, 1994.
- [Tresine, 1982] Richard J. Tresine. *Principles of Inventory and Materials Management*. Elsevier Science Publishing Co., New York, NY, 2. Auflage, 1982.
- [Turoff, 1999] Murray Turoff. Education, commerce, communications: The era of competition. *WebNet Journal*, 1(1):22–31, Jänner / März 1999.
- [Tzu, 1983] Sun Tzu. *The Art of War*. Dell Publishing, New York, NY, 1983. Edited by James Clavell.
- [van Rijsbergen, 1983] Cornelis J. van Rijsbergen. *Introduction to Modern Information Retrieval*. McGraw-Hill, New York, NY, 2. Auflage, 1983.
- [Varian, 1999] Hal R. Varian. Markets for information goods. Working Paper HC79.I55V367, Institute for Monetary and Economic Studies, Bank of Japan, Tokyo, Japan, 1999. <http://www.sims.berkeley.edu/~hal/Papers/japan/>.
- [Vlissides *et al.*, 1996] John M. Vlissides, James O. Coplien, und Norman L. Kerth, Hg. *Pattern Languages of Program Design 2*. Addison-Wesley, Reading, MA, 1996.
- [Vlissides, 1998] John Vlissides. *Pattern Hatching: Design Patterns Applied*. Software Patterns Series. Addison-Wesley, New York, NY, 1998.
- [Wall *et al.*, 1996] Larry Wall, Tom Christiansen, und Randal L. Schwartz. *Programming Perl*. O'Reilley & Associates, Sebastopol, CA, 2. Auflage, 1996.
- [Weibel *et al.*, 1998] Stuart Weibel, John Kunze, Carl Lagoze, und Mischa Wolf. Dublin Core Metadata for Resource Discovery. Request for Comments 2413, Network Working Group, W3C, September 1998. <http://www.ietf.org/rfc/rfc2413.txt>.
- [Whitenack, 1995] Bruce Whitenack. Rappel: A requirements-analysis-process pattern language for object-oriented development. In Coplien and Schmidt [1995], Seiten 259–293.

- [WU, 1998] WU. Studienplan Betriebswirtschaft: Studienzweig Betriebswirtschaft. Verordnung der Studienkommission der Studienrichtung Betriebswirtschaft (STZW), Wirtschaftsuniversität Wien, 1090 Wien, Augasse 2-6, Österreich, 1998.
- [WU, 1999] WU. *Vorlesungsverzeichnis Wintersemester 1999/2000*. Servicebetriebe der Hochschülerschaft an der Wirtschaftsuniversität Wien, 1090 Wien, Augasse 2-6, Österreich, 1999.
- [Yourdon, 1989] Edward Yourdon. *Modern Structured Analysis*. Prentice-Hall, Englewood Cliffs, NJ, 1989.
- [Zimmer, 1995] Walter Zimmer. Relationships between design patterns. In Coplien and Schmidt [1995], Seiten 345–364.